claude-windows / 744270aa-6a38-4011-b018-cdd9aae94849

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849.jsonl`
- SHA-256: `b7175098d6b4f90ef898f3fa5eacac8bc13ad61da4b3c0dfb3f22e57b3f3cfdd`
- Source modified: `2026-07-09T08:42:13+00:00`
- Imported at: `2026-07-09T08:48:47+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `744270aa-6a38-4011-b018-cdd9aae94849`

Transcript

1. user (2026-07-09T02:36:04.275Z)

Hello. Can you do a local performance comparison test of the current head vs the latest binary on the api.khelsutra.guru service and give a summary of the changes? I would like you to use 2-3 videos from the vault and use the GPU in this local machine to run some e2e tests (without the SPA, purely to test the checksum, validation, segmentation and stitching always stick the entire reels, no user step required) and do it such that it is repeatable before every release so that the release has some precomputed numbers for performance.

2. assistant (2026-07-09T02:36:19.560Z)

I'll start by getting oriented ? reading my memory files for context on where the real code lives and the current deployment state, since this task hinges on comparing HEAD against what's live on `api.khelsutra.guru`.

3. user (2026-07-09T02:36:20.559Z)

<system-reminder>This memory is 2 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>

```
1       ---
2       name: actual-repo-location
3       description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the primary working dir stub"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8       ---
9
10      The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a near-empty
11      **stub** ? it contains only `AGENTS.md`, an empty `.claude/`, and `output/nightly_regression/`
12      (a `LATEST_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer` codebase lives at:
13
14        `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
15
16      (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch `master`).
17
18      **How to apply:** when a task references this repo's code (backend/worker, pipeline/segmenters,
19      config.json, run_all_tests.py, etc.), `cd` to the khelsutra-guru path and work there ? the stub dir
20      has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure
```

```
issue.
21
22      Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy
backend
23      training`; `mypy.ini` quarantines several modules incl. `backend.worker.__main__`,
`gemini`,
24      `yolo_hybrid` via `ignore_errors=True`), full suite via `python run_all_tests.py`,
coverage floor
25      `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result
contract
26      ([[efficient-timeouts-and-retries]] is unrelated).
27
```

4. user (2026-07-09T02:36:21.045Z)

```
1       ---
2       name: live-deployment-state
3       description: Current live cloud-serving deployment (rev/digest/rollback) ? updated each
release; live rev moves between sessions
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 7e7bb1dc-c0b6-42c2-af08-d3c688a562b8
8       ---
9
10      # Live cloud-serving deployment state (as of 2026-07-08 release)
11
12      **LIVE (2026-07-08):** control-plane rev **`rally-control-plane-00022-29w`** (was
00021-jhp ? 00020-dtg) + worker Job both on image
13      **`sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943`** (built
from `origin/master @ 304af7b`).
14      ? **Rev 00022 = `phase_placement.validation=cloud_run_l4`** (+ segment + compile
explicit) **AND CP bumped to 4 vCPU / 16 GiB** ? both via the manifest `config.json`/flags (NOT
the preset ? runtime override, config-drift per #532). Owner-requested 2026-07-08 to eliminate
CP OOM on a 6.52 GiB clip (CP still `acquire_local_input`-downloads the full file to RAM-backed
/tmp ? unconditional at orchestration.py:814 ? so 16 GiB gives headroom; validation?L4 moves
only the DECODE to the L4). Rollback: validation?CP + 2vCPU/8Gi = redeploy 00020-dtg;
validation?L4 @ 8Gi = 00021-jhp.
15
16      **Cancel gap (found + filed #538):** canceling a run in the SPA does NOT cancel the
dispatched L4 worker exec ? it runs to completion, burning GPU (repro: `rally-worker-8gqkv` on
third-ksg, killed manually). #538 has a detailed validation matrix (detect+compile cancel, edge
cases).
17      Carries #525 (compile?L4 + `phase_placement`), #527/#528 (#521 follow-ups), #530 (#524
trust submit-time video_id).
18      Region `asia-southeast1`, project `gen-lang-client-0306026826`. CP booted clean (cloud-
serving profile ACTIVE, edge-auth ON).
19
20      **Wins ON via the cloud-serving preset** (no extra config):
`phase_placement.compile=cloud_run_l4` + `cloud_stitch_encoder=h264_nvenc`,
21      `segment?cloud_run_l4` (via `compute_target=cloud_run`), `trust_submit_video_id`
(default on). **`validation` deliberately stays on the
22      control-plane** (the authoritative #512/#513 gate) ? owner-confirmed 2026-07-08, NOT an
oversight.
23
24      **Rollback anchors (prior release):**
25      - CP ? `gcloud run services update-traffic rally-control-plane --region asia-southeast1
--to-revisions rally-control-plane-00019-96f=100`
26      - Worker ? `gcloud run jobs update rally-worker --region asia-southeast1 --image
?/rally-worker@sha256:c6393d27b4ae493ccf713ed0aa061938e76eae7151d6541466dcfa521f638e1d`
27
28      ? Both surfaces deploy off the mutable `:latest` tag (rollback digest must be recovered
by hand) ? **release-process/config-mgmt fix tracked in #532**.
29      CP deploy = canonical manifest path: `python deploy/cloudrun/gen_service_yaml.py --edge-
auth on --image <digest> -o svc.yaml` ? `gcloud run services replace`. Deploy **worker-first**
```

(new CP dispatches the new `compile` subcommand ? old worker would rc2). See [[validation-shared-decode-pass]], [[followup-validation-latency-live-cuj]].
30
31   **? VALIDATED in prod 2026-07-08** (live SPA CUJ, worst-case 5.81 GiB/9.2-min 4K clip GX010151): `compile?L4` dispatches a `kind='compile'` exec + **NVENC engaged** ? stitch **~67?8.4 s/clip (~8×)**; `trust_submit_video_id` confirmed; validation stayed on CP. Runlog **PR #536 MERGED** (`f7a326d`, CI 11/11) ? `DEPLOY_REPORT_cloud-serving_compile-on-L4_2026-07-08.md`. Follow-ups surfaced: **#534** (no progress %/GPU visibility), **#535** (compile worker re-downloads full 5.8 GiB source ~1m38s ? fix ASAP; read clip ranges via the mount).
32
33   **Open triage backlog from this session:** #520 (step tracing), #526 (speculative L4 spin-up at upload-start), #532 (release process + config layer / stop deploying `:latest`), #534, #535. ? master has since moved past the deployed `304af7b` (e.g. `#531` single-pass concat watermark) ? NOT deployed.
34

5. user (2026-07-09T02:36:22.120Z)

1   ---
2   name: gpu-vm-setup-gotchas
3   description: Hard-won setup gotchas for rally GPU L4 VMs (create_gpu_vm.sh DL image) ? NVDEC libs, apt lock, torch2.x, decoders
4   metadata:
5   type: reference
6   ---
7
8   Bringing up a rally L4 VM via `deploy/gcp/create_gpu_vm.sh` (Deep-Learning-VM image common-cu129-ubuntu-2204-nvidia-580, driver 580.159.03). These bit me across the 2026-07-06 GPU-decode / torch-2.x modernization work; see [[efficient-timeouts-and-retries]]. Much of this is exactly what issue #501 (checked-in VM setup script) should encode.
9
10   - **apt lock is held at boot.** The ops-agent startup + unattended-upgrades hold `/var/lib/apt/lists/lock` for the first few minutes ? a setup script that runs `apt-get install` immediately fails ("Could not get lock ... held by process N (apt-get)"), silently breaking everything downstream (no python3-pip ? no torch). **Fix:** wait first ? `for i in $(seq 1 60); do sudo fuser /var/lib/apt/lists/lock /var/lib/dpkg/lock-frontend >/dev/null 2>&1 || break; sleep 5; done`.
11   - **NVDEC/NVENC libs are absent** (compute-only driver): no `libnvcuvid.so.1` and no `libnvidia-encode.so.1`. Extract both from the matching driver: `curl -O https://us.download.nvidia.com/tesla/580.159.03/NVIDIA-Linux-x86_64-580.159.03.run; sh drv.run --extract-only`, copy `libnvcuvid.so.* libnvidia-encode.so.*` to `/usr/lib/x86_64-linux-gnu/`, symlink `.so.1`, `ldconfig`. PyNvVideoCodec needs BOTH (it inits NVENC even to decode).
12   - **pip:** `pip3` doesn't exist until `apt install python3-pip`; use `python3 -m pip install --user`.
13   - **GPU video decode:** **PyNvVideoCodec** (`pip install PyNvVideoCodec`, MIT) works for NVDEC once the two driver libs are present ? it's the practical choice. **TorchCodec fails** on this box (needs a cuda-enabled ffmpeg build + `libnvrtc.so.13`; ubuntu's ffmpeg 4.4 libav mismatch) ? not worth fighting.
14   - **torch 2.x + WASB:** `torch/torchvision --index-url https://download.pytorch.org/whl/cu124` gives torch 2.6.0+cu124, cuda works on L4. WASB HRNet loads via `torch.load(..., weights_only=False)["model_state_dict"]` (2.6 default flipped to weights_only=True). NumPy 2.0 removed `np.Inf` which WASB's OnlineTracker uses ? add `np.Inf = np.inf` shim.
15   - **Windows SSH:** reused external IPs cause a plink host-key MISMATCH warning (it still proceeds). Wrap detached launches in `timeout 40 gcloud compute ssh ...` to avoid the ~60s channel hang, and use `setsid` on the remote job so it survives. Transient plink rc=1 happens on a fresh VM (SSH keys still propagating ~30-60s after RUNNING) ? retry once, don't panic; the detached job (results to GCS) is unaffected by SSH flakiness.
16
17   Cloud Run **serving** rollout (the #506/#507 GPU-resident deploy, 2026-07-07):
18   - **Cloud Build context had NO `.gcloudignore`** ? `gcloud builds submit` fell back to `.gitignore` (which excludes the 8 GB `output/`+`scratch/`), so it "worked" by luck. Added an explicit `.gcloudignore` (`#!include:.gitignore` + `.git/` + the untracked `score_*.py`). Verify the upload set FOR FREE with `gcloud meta list-files-for-upload` (no build needed) ? a huge

count ? context leak.
19   - **NVDEC libs on Cloud Run:** the baked `libnvcuvid`/`libnvidia-encode` (from driver 580.159.03) DO work on Cloud Run's L4 host driver ? proven by a real nvdec decode there; the GX010128 served trajectory reproduced the self-managed-VM run (26 479 ? 26 481 visible). Re-smoke on any base-image/driver bump. `PyNvVideoCodec==2.1.0` is PINNED in both deploy paths (the validated version).
20   - **The worker Job has NO `DEPLOYMENT_PROFILE`**, so it does NOT apply the cloud-serving preset ? the control plane must FORWARD `indexing.wasb.*` knobs (decode_backend/batch/prefetch) to it via `--set` (orchestration.py). `WASB_DECODE_BACKEND` env on the Job is read first by the native runner ? the instant, no-redeploy enable/rollback lever. Reusable smoke: `deploy/cloudrun/post_deploy_smoke.ps1` (PowerShell ? gcloud `--args` gs:// URIs mangle under Git Bash/MSYS; `^@@^` delimiter dodges the comma-split).
21

6. user (2026-07-09T02:36:22.560Z)

<system-reminder>This memory is 2 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>
1   ---
2   name: efficient-timeouts-and-retries
3   description: How the owner wants Claude to handle waits, retries, timeouts, and Windows gcloud SSH ? fast, visible, diagnostic
4   metadata:
5   type: feedback
6   ---
7
8   The owner strongly dislikes slow, silent retry loops. On 2026-07-06, diagnosing a GPU VM, I burned ~3 minutes on a 12x15s SSH-readiness loop that suppressed stderr with 2>/dev/null, so it "failed" invisibly while the owner had already SSH'd into the box manually. The owner interrupted repeatedly ("be really smart at your job").
9
10   **Why:** wasteful polling plus hidden errors makes me look slow and not smart; the owner watches actively and interrupts. One visible failure I actually diagnose beats many blind retries.
11
12   **How to apply:**
13   - ONE attempt with visible stderr, read the real error, then fix ? do not blind-retry a broken command.
14   - Short, reasonable timeouts (seconds, not minutes). Escalate only with a concrete reason.
15   - Never hide stderr (2>/dev/null) on a command whose success is uncertain.
16   - Root cause: on Windows, `gcloud compute ssh` uses PuTTY plink.exe/pscp.exe, which reject OpenSSH `-o` flags and mishandle some `~` remote paths. Use plain `gcloud compute ssh <vm> --zone --project --command="..."`; to push a file, base64-encode into --command (echo B64 | base64 -d > file), not scp. To launch detached work without plink hanging ~60s, redirect all fds and close stdin.
17   - For long remote work: run detached (nohup, log to file/GCS), poll a status file; avoid many SSH round-trips.
18

7. assistant (2026-07-09T02:36:32.609Z)

Let me read the session handoff and explore the actual repo structure in parallel.

8. user (2026-07-09T02:36:33.710Z)

C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\efficient-timeouts-and-retries.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\actual-repo-location.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\lgtm-only-when-merge-ready.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\gpu-

```
resident-modernization.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\gpu-
vm-setup-gotchas.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\compute-local-hosted-guardrail.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\session-handoff-gpu-resident.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\cloud-run-exec-status-fastfail.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\validation-shared-decode-pass.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\pr513-worker-skip-revalidation.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\pr518-validation-result-cache.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\followup-validation-latency-live-cuj.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\MEMORY.md
C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\live-
deployment-state.md
```

9. user (2026-07-09T02:36:37.837Z)

```
codex/issue-523-sd-reel-plan-clear
---HEAD---
a460d74 chore(handoff): close issue #523 and move to next open bugs
b3e1b4e chore(handoff): refresh session state for sd reel follow-up
ad9c5af feat(compile): expose rendition selector in legacy UI
---STATUS---
?? "SESSION_HANDOFF (1).md"
---REMOTE---
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
```

10. assistant (2026-07-09T02:36:57.356Z)

```
Current checkout is on feature branch `codex/issue-523-sd-reel-plan-clear` at `a460d74`. Let me
read the session handoff (the "Start here") and explore the repo layout to understand the e2e
flow and any existing benchmark infrastructure.
```

11. user (2026-07-09T02:36:58.430Z)

```
1       ---
2       name: session-handoff-gpu-resident
3       description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4       metadata:
5         type: project
6       ---
7
8       # Session Handoff ? GPU-resident WASB inference (rollout)  ·  issue #506 · PR #507
MERGED
9
10      ## LATEST 2026-07-07 (end of session) ? all serving fixes merged + fresh redeploy
11      Every gap surfaced this session is merged to `origin/master @ ef41473`: **#511**
(compute=local fails fast on the weightless control-plane) + **#516** (SPA compute-picker
defaults to CLOUD on hosted) = the compute-picker footgun; **#512** (validators share ONE decode
pass ? measured ~3.1× on the 5.3K clip: warm 179s?58s on a 16-core box; projects ~6.5min?~2min
on the 2-vCPU control-plane); **#513** (worker skips redundant re-validation on dispatch ? also
fixes the default-blur-threshold reject); **#515** (worker no-done-marker fastfail). **Clean
rebuild from `ef41473` ? redeploy control-plane + worker to the new digest (D3)** ? see the
DEPLOY_REPORT / [[gpu-vm-setup-gotchas]]. #509 forwarding was PROVEN live by the CUJ (worker
args carried `decode_backend=nvdec_resident`+batch64+prefetch4). Two follow-ups SPAWNED:
```

task_5acf4b46 (clean 2?3 video validation-speedup closure test + DEPLOY_REPORT) and **task_4585aa00** (cache validation results by video_id ? skip re-validation on reupload; the missing cache that made CUJ retries re-pay validation). Validation is NOT cached across reuploads today (only a whole-pipeline cache gated on a prior SUCCESSFUL segmentation).

SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving

Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default = nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job (asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250?` (torch 2.6 + PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record: `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md`.

- **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1 --remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-worker@sha256:3bfefaf6?` (? prior torch-1.11 image).
- **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release hardening) all merged. master @ 66756ad.
- **DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master 66756ad = #507+#508+#509).** Redeployed 2026-07-07: `gcloud run services update rally-control-plane --image ?:latest` ? rev **00017-qbd** (100% traffic, booted healthy: uvicorn up, cloud-serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ?:latest`. So the #509 dispatch **forwarding** + #508 preset default are now LIVE in the control-plane image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from 44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec rev) or `--to-revisions ?-00016-?` .
- **`WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job** ? now REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is deployed + unit-tested but not live-dispatch-verified).
- `DEPLOY_REPORT_?gpu-resident-nvdec_2026-07-07.md` is current: PR #510 added the §5 "Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected §6. All four PRs merged: #507 #508 #509 #510; master @ 3be0935.
- **Wrinkles fixed (#509):** dispatch now forwards indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ? never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added `deploy/cloudrun/post_deploy_smoke.ps1`. See [[gpu-vm-setup-gotchas]].
- WASB-C3 weights-license gate still open (unchanged; pre-existing).

Live CUJ 2026-07-07 ? release PROVEN, 3 pre-existing gaps surfaced (all filed as spawned tasks)

Owner ran a real /api/process CUJ ("Video 1 - Badminton.MP4", a 5312x2988 **10-bit HEVC** 120Mbps clip). Outcomes:

- **#509 forwarding PROVEN live**: the cloud dispatch put `--set indexing.wasb.decode_backend=nvdec_resident --set ?batch_size=64 --set ?prefetch_batches=4` into the worker exec args. And a sharp-clip green run on the D2 worker (exec `rally-worker-ql56s`, env-driven nvdec) = 19 rallies, success. Serving path is GREEN for valid footage.
- **Gap 1 ? slow validation (task_dcee8bf0):** ~125 random `cap.set(POS_FRAMES)` seeks across scene_cut(100)/blur(15)/relevance(10), each their own VideoCapture, on 5.3K 10-bit HEVC on the CPU-only control-plane ? ~6.5 min. Fix: single sequential/keyframe pass.
- **Gap 2 ? compute=local footgun (task_81572190):** SPA sent compute=local ? in-process on the weightless control-plane ? "weights not found" fail. Guard/redirect it.
- **Gap 3 ? worker re-validates with DEFAULT thresholds (task_72d98f83):** control-plane min_blur=8.0 (baked) NOT forwarded; worker re-validates at default 20.0 and REJECTED this soft clip (sharpness 11.9) ? rc2. This is what failed the CUJ on the cloud path. Fix: skip worker re-validation on dispatch (control-plane is authoritative) or forward validation config.
- **Validation-latency design (my rec):** don't move validation to GPU standalone. (1) stop the double-validation, (2) kill the seeks (sequential/keyframe), (3) end-state = ffprobe metadata pre-gate on control-plane + fold content checks (blur/court) INTO the worker's existing GPU decode pass (decode once, validate+detect together). All 3 gaps are pre-existing, NOT caused by the nvdec release.

30
31   ## (historical) release/rollout phase ? superseded by SHIPPED above
32   PR #507 squash-merged to master as **44a2f47** (branch deleted). Pre-merge gate reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green; 2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files; decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate PASS + latency/resource A/B all done (see PR #507 comment + below). **Remaining = the serving release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod has changed yet.**
33
34   ### Serving release plan (live infra: Job `rally-worker` + service `rally-control-plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)
35   - **Phase A (ship image, no behavior change):** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-worker:latest` (torch2.6+pynvc2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image ?:latest` + redeploy service (`gen_service_yaml.py` ? `services replace`). decode_backend defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs update --image <old digest>`.
36   - **Phase B (enable nvdec, gated + reversible):** flip via WORKER env `WASB_DECODE_BACKEND=nvdec_resident` (native runner reads env first ? no code/preset change, instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then durable default = add `decode_backend: nvdec_resident` to the cloud-serving preset (presets.py) + rebuild + Launch Report.
37   - Cost: build ~$0.3; each Job smoke ~$0.4?0.7 (L4). Rollback levers at every step.
38
39
40   ## You are here
41   The implementation is DONE and **PR #507 is open** (branch `feat/506-gpu-resident-nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode_backend = "cpu" | "nvdec_resident"`, the validated NVDEC?affine-`grid_sample`?GPU-normalize path wired into `backend/pipeline/detectors/wasb_infer.py` (`NvdecResidentFrontend` + `run_video_nvdec_resident`, same resumable cache keyed by decode_backend), native-runner threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in `deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu_setup.sh`). Verified locally: 1764 tests green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review confirmed 7 minor findings, all fixed (notably `padding_mode="zeros"` = cv2 BORDER_CONSTANT for non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506 comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7×, GPU 47?79%).
42
43   ## Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the $100 budget; VM torn down)
44   Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 + torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity` selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5) **Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.
45   **Golden gate @ SERVED = PASS** (10 baseline-comparable clips, 445 rallies): mean ?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**), 8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu on identical pixels, so they add nothing to the verdict.
46   **Latency + resource A/B** (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat. The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9× is vs the untuned streaming path ? vs prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8× (#506 spike). All in the PR #507 comment.
47
48   ## Staged assets (durable)

```
49      - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)
50      - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
51
52      ## Next steps
53      1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
54      2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
55
56      ## Ramp-up kit
57      - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
58      - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
59      - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
60
61      ## Key decisions (don't relitigate)
62      - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
63      - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
64      - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
65      - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
66      - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
67      - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
68
```

12. user (2026-07-09T02:37:00.075Z)

```
===TOP LEVEL===
__pycache__
artifacts
backend
backend.db
CLAUDE.md
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
README.md
requirements.txt
requirements-trackers.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
SESSION_HANDOFF (1).md
SESSION_HANDOFF.md
tests
```

```
tools
training
```

13. assistant (2026-07-09T02:37:12.579Z)

```
Good ? this is the real repo with `run_regression.py`, `run_eval.py`, `run_all_tests.py`, and an
`output/nightly_regression/` convention. Let me explore the existing harness infrastructure, the
"vault" concept, and check the local GPU in parallel.
```

14. user (2026-07-09T02:37:13.610Z)

```
1       # Session Handoff
2
3       ## You are here
4
5       PR [#537](https://github.com/Khelsutra/badminton-highlight-indexer/pull/537) is
6       merged (2026-07-08T12:52:20Z), and bug [#523](https://github.com/Khelsutra/badminton-
highlight-indexer/issues/523)
7       is closed. The SD replacement-compile path is now live in the engine.
8
9       ## Next steps / open threads
10
11      1. Continue with next open bug item (recommended starting point: `#538` or `#539`).
12      2. Sync `origin/master` before opening next implementation branch.
13      3. If new UI work is needed for SPA parity, consider coordinating with the
14         separate web repo.
15
16      ## Ramp-up kit
17
18      - `backend/api/static/index.html`
19      - `backend/api/static/app.js`
20      - `backend/pipeline/stitcher/compiler.py`
21      - `tests/test_compiler.py`
22      - `tests/test_api_endpoints.py`
23      - `README.md`
24      - `backend/main.py`
25      - `backend/orchestration.py`
26      - `backend/pipeline/stitcher/compiler.py`
27
28      ## Key decisions
29
30      - The single-pass stitch path now applies SD scaling and `encode_profile` audio bitrate.
31      - A focused single-pass SD regression test exists in `tests/test_compiler.py`.
32      - `my`: this review pass also added `rendition` control in the static compile panel and
33        forwarded it in `/api/compile` payloads.
34      - `release`: validation stack passed (`ruff check .`, `mypy backend training`,
35        `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`).
36      - `tracking`: issue/pr linkage is complete (`Closes #523` in PR body + final close
37        summary on issue).
38
```

15. user (2026-07-09T02:37:25.436Z)

```
===GPU (nvidia-smi)===
name, memory.total [MiB], driver_version
NVIDIA GeForce RTX 2070 Super with Max-Q Design, 8192 MiB, 595.79
===VAULT refs in repo===
./docs/ALPHA_LAUNCH_READINESS.md
```

16. assistant (2026-07-09T02:37:34.987Z)

```
Local GPU is an **RTX 2070 Super Max-Q (8GB, driver 595.79)** ? usable but far smaller than the
L4 in prod. "vault" appears only in one doc. Let me examine the existing regression harness, the
vault reference, and how the pipeline runs end-to-end.
```

17. user (2026-07-09T02:37:36.034Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15
16      import argparse
17      import json
18      import logging
19      import os
20      import sys
21
22      from backend.annotations import annotation_registry
23      from backend.config import default_db_path
24      from backend.eval import regression
25      from backend.infrastructure.database import Database
26
27
28      def build_parser() -> argparse.ArgumentParser:
29          p = argparse.ArgumentParser(
30              description="Regression-test the tool vs golden-set ground truth."
31          )
32          p.add_argument(
33              "--video-id",
34              required=True,
35              help="Stored video_id (MD5) whose predictions to score.",
36          )
37          p.add_argument(
38              "--tier",
39              default="file",
40              help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}",
41          )
42          p.add_argument(
43              "--annotations",
44              help="video_ref for the provider (e.g. CSV path for the 'file' tier).",
45          )
46          p.add_argument("--stage", choices=("candidates", "final"), default="final")
47          p.add_argument(
48              "--detector", default=None, help="Restrict candidate scoring to one detector."
49          )
50          p.add_argument("--iou", type=float, default=0.5)
51          p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
52          p.add_argument("--db", default=None)
53          p.add_argument(
54              "--baseline",
55              default=regression.DEFAULT_BASELINE_PATH,
56              help="Baseline JSON path.",
57          )
58          p.add_argument(
59              "--update-baseline",
60              action="store_true",
61              help="Snapshot the current numbers as the new baseline instead of checking.",
```

```
        )
        p.add_argument(
            "--allow-missing-baseline",
            action="store_true",
            help="Treat a missing baseline as a pass (exit 0) instead of a gate error (exit
3).",
        )
        p.add_argument("--json", dest="json_out", default=None)
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        db_path = args.db or default_db_path()
        if not os.path.exists(db_path):
            print(
                f"ERROR: database not found at {db_path}. Run the pipeline first.",
                file=sys.stderr,
            )
            return 2
        db = Database(db_path)

        result = regression.regress_with_provider(
            db,
            video_id=args.video_id,
            video_ref=args.annotations,
            tier=args.tier,
            stage=args.stage,
            iou_threshold=args.iou,
            detector=args.detector,
            tolerance=args.tolerance,
            baseline_path=args.baseline,
        )

        print(regression.format_result(result))

        if args.update_baseline:
            regression.save_baseline(
                result.video_id,
                result.stage,
                result.precision,
                result.recall,
                result.f1,
                path=args.baseline,
                iou_threshold=result.iou_threshold,
                detector=args.detector,
                gt_fingerprint=result.gt_hash,
            )
            print(f"Baseline updated in {args.baseline}.")

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump(result.as_dict(), f, indent=2)
            print(f"JSON written to {args.json_out}")

        # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
        if result.regressed:
            return 1
        if (
            not result.has_baseline
            and not args.update_baseline
            and not args.allow_missing_baseline
        ):
```

```
126             print(
127                 "ERROR: no baseline to compare against ? the gate cannot verify anything. "
128                 "Snapshot one with --update-baseline (then commit it), or pass "
129                 "--allow-missing-baseline to accept this run.",
130                 file=sys.stderr,
131             )
132             return 3
133         return 0
134 
135 
136     if __name__ == "__main__":
137         raise SystemExit(main())
138 
```

18. user (2026-07-09T02:37:39.911Z)

```
===vault in ALPHA doc===
75:The flywheel lane is successful when user corrections can flow into the collector/vault path
with consent/provenance and a visible re-eval delta. This is valuable for alpha learning, but it
should not block the first controlled alpha if manual promotion can bridge the gap.
103:| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path;
reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-
collector` + vault | A2 | Owner upload/storage scope | DONE | collector #62 |
===candidate local video dirs===
backend/config/models.py:472:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is
empty, prefixed with ``input_backend``
backend/config/models.py:487:    # videos/ labels/ weights/ (e.g.
gs://bucket/workspaces/<video_id>/). "" = root directly.
backend/main.py:991:@app.get("/api/videos/{video_id}/cost")
backend/orchestration.py:404:                # staged-canonical videos/<md5>/ prefix; else "" ?
the segments still
backend/orchestration.py:408:                    f"{output_root}/videos/{video_id}/", sc
backend/pipeline/detectors/tracknet_runner.py:46:    wsl_stage_dir: str = "~/clips"  # where
staged videos/outputs live in WSL
backend/storage/ref.py:89:         (``"gs://bucket/videos"`` + ``"x.mp4"`` ?
``"gs://bucket/videos/x.mp4"``).
backend/worker/__main__.py:531:#: Video extensions accepted under <corpus>/videos/ (mirrors the
labels/ '.csv' filter so a
backend/worker/__main__.py:537:#: part of the clip's join key, so it's stripped when deriving
the videos/ stem.
backend/worker/__main__.py:542:    """Derive the clip name (the ``videos/<name>.<ext>`` join
key) from a label filename.
backend/worker/__main__.py:684:    # Real-detector source: resolve the runner once + index the
bucket's videos/ by stem.
backend/worker/__main__.py:692:        for vuri in storage.list_uris(f"{corpus}/videos/",
config=storage_cfg):
backend/worker/__main__.py:710:                    f"[worker] no video for {name!r} under
{corpus}/videos/ ? skipping."
backend/worker/__main__.py:916:        help="prefix containing labels/ (+ videos/ for "
backend/worker/__main__.py:931:        "(needs videos/ in the corpus; honours detector_impl).",
scripts/verify_compute_path.py:15:      --video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 \
```

19. assistant (2026-07-09T02:37:54.619Z)

`run_regression.py` scores **accuracy** (precision/recall/F1), not performance/latency ? so I'll need a separate timing harness. "vault" in-repo means the data-collector path, not a local dir; the user means their local test-video collection. Let me find the local videos, the e2e entry point, and how the pipeline is driven locally.

20. user (2026-07-09T02:37:59.485Z)

```
===local video files under repo (top 30 by find)===
./artifacts/cloudrun_e2e_validation/cloudrun_e2e_reel.mp4
```

```
./artifacts/promo/curated_top10_longest.mp4
./artifacts/promo/full_cuj_real.mp4
./artifacts/promo/promo_cuj.mp4
./artifacts/review/reel_CURATED_top10.mp4
./artifacts/review/reel_FULL_22rallies.mp4
./artifacts/review/source_mahadevpura_smoke_180s.mp4
./output/chunks/3d19ef835a69be93acdd75ef636f345a/chunk_3d19ef835a69be93acdd75ef636f345a_3.mp4
./output/chunks/74affb36e15e172130cea5c451e4a728/chunk_74affb36e15e172130cea5c451e4a728_0.mp4
./output/chunks/chunk_018c2e2c3ce3086c3badfa1a9e5bc5c9_1.mp4
./output/chunks/chunk_e0168f050899da4de75bcd311b80131b_5.mp4
./output/e2e_reel.mp4
./output/gem_newclips/Badminton BXH 2_proxy - Highlights.mp4
./output/gem_newclips/GX030094_proxy - Highlights.mp4
./output/GX010125_highlights.mp4
./output/GX010125_highlights_min5.mp4
./output/GX010125_run/GX010125_highlights.mp4
./output/GX010125_run/handoff/hand_5.mp4
./output/KushagraAviHighlight-Trailor.mp4
./output/kushagra_1280.mp4
./output/MahadevpuraSingles_highlights.mp4
./output/mahadevpura_smoke_180s.mp4
./output/ramp_probe.mp4
./output/smoke19/gpuproxy_audio.mp4
./output/smoke19/gpuproxy_test.mp4
./output/smoke19/smoke_GX010143.mp4
./scratch/boxhill_highlights_stage/Adarsh v Vasant.MP4
===sizes of any videos found in likely dirs===
total 5.8M
-rw-r--r-- 1 avidu 197609  272K Jun 14 12:36 _final_p1.png
-rw-r--r-- 1 avidu 197609  240K Jun 14 12:36 _final_p4.png
-rw-r--r-- 1 avidu 197609   57K Jun 14 12:34 _fusion_ablation_chart.png
-rw-r--r-- 1 avidu 197609  269K Jun 15 20:54 _pov_final_p2.png
-rw-r--r-- 1 avidu 197609  240K Jun 15 20:39 _pov_p1.png
-rw-r--r-- 1 avidu 197609  278K Jun 15 20:39 _pov_p2.png
-rw-r--r-- 1 avidu 197609  340K Jun 14 12:35 _rev_p1.png
-rw-r--r-- 1 avidu 197609  412K Jun 14 12:35 _rev_p2.png
-rw-r--r-- 1 avidu 197609   41K Jun 14 12:35 _rev_p3.png
--- artifacts ---
a5_floor_smoke
appliance_design.json
branch-prune-backup-24178a7.txt
cloudrun_e2e_validation
cloudrun_lightup
landing
long_rally_lens_n15.json
long_rally_lens_n15.txt
multishuttle_142
pr245_code_review.md
```

21. user (2026-07-09T02:38:00.626Z)

```
===README run/pipeline sections===
1:# Badminton Highlight & Rally Indexer
3:A local, AI-powered web application and CLI utility designed to process long-form badminton
recordings (e.g., from GoPro), run comprehensive pluggable validations (quality, integrity, and
relevance checks), index all rallies into a relational SQLite database, and dynamically compile
custom highlight reels using high-speed FFmpeg stitching.
5:The pipeline is built around three pluggable registries ? **validators**, **segmenters**, and
**AI providers** ? plus a **sport profile** registry so the same engine can target other
racket/court sports in the future. Badminton is the default profile.
11:## ? Key Features
19:*   **Six Pluggable Rally Segmenters** (`backend/pipeline/segmenters/` ? `motion`, `gemini`,
`yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`). The three core ones:
23:*   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships in-
tree; new providers (e.g. an OpenAI or local-model backend) can be registered without touching
```

the segmenters.
25:*   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments (start/end/duration/server/receiver/metadata) and per-segment events are persisted in `output/sports_indexer.db`. Re-running on the same file is a cache hit.
26:*   **Query System**: Filter indexed play segments by `server`, `receiver`, `duration_min`, and `event_type` (the schema supports rich event types like smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated `shot_count`).
27:*   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip stitching.
28:*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`, and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel (**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with an off switch).
29:*   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics, validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an arbitrary segment via stream-copy ffmpeg for fast iteration.
33:## ?? Setup & Diagnostics
35:### 0. Batteries-included install (LIGHT tier ? recommended for end users)
42:# Preferred ? uv installs `indexer` into an isolated tool env:
44:# If `indexer` isn't found afterwards, add uv's tool-bin to PATH once: uv tool update-shell
46:# Or the cross-platform installer script (uv-if-present, else pip) ? also writes a launcher
47:# script (with the resolved `indexer` path baked in) + an uninstall manifest:
58: no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels) lives
62:* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your config/reels
67:### 1. External System Requirements
68:The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes, and stitch clips.
76:### 2. Install the package
82:# Runtime deps only:
84:# Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
102:### 3. Diagnostics Checker (`scripts/doctor.py`)
107:# or, via the CLI:
112:*   Initialize a default `config.json` (if one does not exist) with thresholds for blur, court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
114:*   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`, `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`, `supervision`.
116:> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ? nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`: `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or `retinanet_resnet50_fpn`.)
120:## ? Running the Application
122:### 1. Launching the Local Web App Dashboard
125:python -m backend.main --server --port 8000
128:*Add a video ? upload a local file, or point at a bucket/Drive URI ? to process, validate, index, and compile highlights. You can also choose where the video is stored (local / S3 / GCS / Google Drive) via the medium picker; free space is checked before an upload or fetch (HTTP 507 if insufficient).*
130:### 2. Running in CLI Processing Mode
131:To process and index a video completely via terminal commands:
133:python -m backend.main --video-path "C:/path/to/my_video.mp4"
135:This outputs a validation summary and indexes all rallies directly into the local SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently produces 0 segments and the observability report flags `silent_provider_noop`). Set it to `yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
137:To explicitly pick a segmenter for one run:
139:python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
140:python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini

22. assistant (2026-07-09T02:38:31.494Z)

The CLI entry is `python -m backend.main --video-path ...` (checksum?validate?segment/index). Stitching is a separate compile step. Let me map the full e2e + any existing timing instrumentation via an Explore agent, while I locate source videos and read the rest of the README + the stub's nightly-regression status.

23. user (2026-07-09T02:38:50.025Z)

Async agent launched successfully. (This tool result is internal metadata ? never quote or paste any part of it, including the agentId below, into a user-facing reply.)
agentId: a67a8bee8aa546faf (internal ID - do not mention to user. Use SendMessage with to: 'a67a8bee8aa546faf', summary: '<5-10 word recap>' to continue this agent.)
The agent is working in the background. You will be notified automatically when it completes.
Do not duplicate this agent's work ? avoid working with the same files or topics it is using.
output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\a67a8bee8aa546faf.output
Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

24. user (2026-07-09T02:38:52.597Z)

===README lines 140-240 (stitch/compile CLI)===
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
```

### 3. Rich CLI Debugging (`--debug-clip`)
To diagnose exactly why a video segment was registered as active/dead play, or inspect detailed focus/continuity metrics at a particular time in seconds:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
```
Outputs granular validation check logs and motion ratios within a +/- 3 second frame window around the timestamp.

4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest watermark preview
Extract a slice of a recording to disk (re-encoded; and, with the default-on branding watermark, stamped **"Generated by Khelsutra.guru"** in the corner):
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" \
    --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
```
Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute path. This is the fastest way to *see* the watermark on a real clip ? set `stitching.watermark.enabled: false` in `config.json` to turn it off.

> To preview the exact overlay on any file without the pipeline:
> ```bash
> ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu Sans':fontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-24:box=1:boxcolor=black@0.4:boxborderw=10" -c:a copy out.mp4
> ```

5. End-to-End "Process + Stitch" Script
`scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a hardcoded video path, reuses cached indexed segments if available (offering to re-index from scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`, `min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching settings against a fixed large recording without re-running expensive segmentation.

6. Cache Hygiene & Disk Cleanup
The trajectory detectors (`wasb_hybrid` / `tracknet_hybrid`) stage a copy of each video into WSL and decode every frame to PNG ? easily **tens of GBs per 4K video**. To keep disk usage bounded:
```

*   **Automatic self-clean:** after a *successful* run the frame cache + staged copy are deleted (`indexing.{wasb,tracknet}.keep_frames`, default `false`). On failure they're kept so a re-run resumes from the checkpointed detector cache.
*   **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with `--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and `output/`:
    ```bash
    python -m tools.cleanup_caches                  # report what's reclaimable (safe; deletes nothing)
    python -m tools.cleanup_caches --apply          # delete the regenerable caches
    python -m tools.cleanup_caches --keep-video GX010125   # protect one video's cache (id or filename stem)
    python -m tools.cleanup_caches --older-than 7   # only purge entries older than 7 days
    ```
    It treats frame caches, staged/proxy videos, and handoff/temp dirs as **regenerable** (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels. Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line nudge (used by the optional SessionStart hook).

?? Threshold Tuning (`config.json`)

You can adjust the parameters of your validators, segmenters, AI provider, and stitcher by editing `config.json` in the root folder. The default config emitted by `scripts/doctor.py`:

```json
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [35, 40, 40],
            "court_green_upper": [85, 255, 255],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "gemini",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_prefetch_workers": 2,
        "min_action_window_duration": 2.0,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 5
    },
    "output": {
        "default_highlights_name": "highlights.mp4",
        "db_name": "sports_indexer.db"
    },
    "stitching": {
        "begin_padding": 0.5,
        "end_padding": 0.5,
```

```
        "use_cache": false,
        "min_shot_count": 1,
        "watermark": {
            "enabled": true,
            "text": "Generated by Khelsutra.guru",
            "position": "bottom-right",
            "opacity": 0.85,
            "logo_path": "",
            "font_path": ""
        },
        "renditions": {
            "sd": {
                "max_width": 854,
                "max_height": 480,
                "crf": 30,
                "audio_bitrate": "96k",

===stub nightly_regression status===
2026-07-09 07:58:09  status=SKIPPED exit=0  wrapper absent (awaiting merge) at
'C:\Users\avidu\Projects\badminton-highlight-indexer\scripts\nightly_regression.ps1'
```

25. assistant (2026-07-09T02:39:07.047Z)

Important find: the stub's `output/nightly_regression/LATEST_STATUS.txt` shows a scheduled runner is **already waiting** for a `scripts/nightly_regression.ps1` wrapper that doesn't exist yet ? that's exactly the "repeatable, precomputed numbers before every release" hook. Let me read the existing interactive process+stitch script and locate the actual source-video "vault" on this machine.

26. user (2026-07-09T02:39:07.564Z)

```
1       import json
2       import logging
3       import os
4
5       from dotenv import load_dotenv
6
7       load_dotenv()
8
9       # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
10      # Quiet third-party HTTP chatter by default; re-honoured against the config knob
11      # (logging.verbose_http) once the config is loaded in main().
12      from backend.utils.logging_setup import setup_logging
13
14      setup_logging()
15
16      from backend.config import load_config
17      from backend.infrastructure.database import Database
18      from backend.pipeline.segmenters import segmenter_registry
19      from backend.pipeline.stitcher.compiler import VideoCompiler
20      from backend.results import unwrap_or_failure
21      from backend.tools.export import format_rally_summary
22      from backend.utils.hashing import compute_video_id
23
24      logger = logging.getLogger(__name__)
25
26
27      def main():
28          import argparse
29
30          parser = argparse.ArgumentParser(
31              description="Dev runner: process a video -> index rallies -> stitch a highlight
reel."
32          )
33          parser.add_argument(
```

```
            "--video-path", required=True, help="Path to the input video file."
        )
        parser.add_argument(
            "--output-dir",
            default=None,
            help="Where to write the stitched reel (default: config output.output_dir).",
        )
        parser.add_argument(
            "--output-filename", default="highlights.mp4", help="Stitched reel filename."
        )
        args = parser.parse_args()
        video_path = os.path.abspath(args.video_path)
        output_filename = args.output_filename

        print("Loading config...")
        cfg = load_config()  # typed AppConfig ? single source of defaults
        # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
        setup_logging(verbose_http=cfg.logging.verbose_http)
        output_dir = args.output_dir or cfg.output.output_dir  # no hardcoded OS path

        print("Verifying files exist...")
        if not os.path.exists(video_path):
            print(f"Error: Video file not found at {video_path}")
            return

        os.makedirs(cfg.output.output_dir, exist_ok=True)
        db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
        db = Database(db_path)

        # Calculate MD5 of the video to uniquely identify it
        import time

        print(
            "Calculating video MD5 (this might take a few seconds on large files)...",
            flush=True,
        )
        t0 = time.time()
        video_id = compute_video_id(video_path)
        print(f"Calculated MD5: {video_id} (took {time.time() - t0:.1f}s)", flush=True)

        segments = []
        cached_video = db.get_video(video_id)
        cache_hit = False

        if cached_video and cached_video.get("status") == "processed":
            segments = db.query_play_segments(video_id, {})
            if len(segments) > 0:
                print(
                    f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.",
                    flush=True,
                )
                choice = input(
                    "Would you like to (1) Run stitching on the cached segments, or (2)
Reprocess the video with Gemini from scratch? [1/2]: "
                ).strip()
                if choice == "1":
                    print(
                        "[CACHE] Using cached play segments. Skipping segmenter API call.",
                        flush=True,
                    )
                    cache_hit = True
                else:
                    print("[CACHE] Ignoring cache...", flush=True)
```

```
            else:
                print(
                    "[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.",
                    flush=True,
                )

        if not cache_hit:
            print("[CACHE] Initializing clean indexing...", flush=True)
            db.clear_video_data(video_id)

            print(
                "Bypassing OpenCV validations for performance on massive files...",
                flush=True,
            )
            db.add_video(video_id, video_path, "processed", [])

            print("Running segmenter & indexing...", flush=True)
            seg_name = cfg.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return
            print(f"Using segmenter: {seg_name}", flush=True)
            segmenter = segmenter_cls(db, cfg)
            # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
            # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
            # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
            segments, failure = unwrap_or_failure(
                segmenter.process_video(video_id, video_path)
            )
            if failure is not None:
                print(f"[FAIL] Segmentation failed: {failure.message}")
                return
            print(f"Successfully indexed {len(segments)} play segments.", flush=True)

        if not segments:
            print("No play segments detected. Cannot compile highlights.")
            return

        end_padding = cfg.stitching.end_padding
        begin_padding = cfg.stitching.begin_padding
        min_shot_count = cfg.stitching.min_shot_count

        if cache_hit:
            print("\n--- Stitching Customizations ---")
            try:
                bpad_input = input(
                    f"Enter begin_padding in seconds (default {begin_padding}): "
                ).strip()
                if bpad_input:
                    begin_padding = float(bpad_input)

                pad_input = input(
                    f"Enter end_padding in seconds (default {end_padding}): "
                ).strip()
                if pad_input:
                    end_padding = float(pad_input)

                shot_input = input(
                    f"Enter min_shot_count to filter by (default {min_shot_count}): "
                ).strip()
```

```
                if shot_input:
                    min_shot_count = int(shot_input)
            except ValueError:
                print(
                    "[WARNING] Invalid input detected, falling back to defaults from
config.json."
                )

        print(
            f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...",
            flush=True,
        )

        # Filter segments where shot_count >= min_shot_count
        time_pairs = []
        skipped_count = 0
        for s in segments:
            meta = s.get("metadata", {})
            if isinstance(meta, str):
                try:
                    meta = json.loads(meta)
                except Exception:
                    meta = {}
            shot_count = meta.get("shot_count")
            # B1/P9: shot_count is None when the segmenter has no ground truth (motion
            # segmenter) ? keep such segments rather than crashing on None >= int.
            if shot_count is None or shot_count >= min_shot_count:
                time_pairs.append((s["start_time"], s["end_time"]))
            else:
                skipped_count += 1

        if skipped_count > 0:
            print(
                f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.",
                flush=True,
            )
        print(
            f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.", flush=True
        )

        # End-of-run rally table for manual QA ? sourced from the produced play_segments so
        # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
        # aligned rows) so it survives even if the table is the last thing the owner scans.
        def _log_rally_summary() -> None:
            logger.info("\n%s", format_rally_summary(segments))

        if not time_pairs:
            print(
                "No play segments passed the filtering criteria. Cannot compile highlights."
            )
            _log_rally_summary()
            return

        compiler = VideoCompiler(
            output_dir,
            end_padding=end_padding,
            begin_padding=begin_padding,
            watermark=cfg.stitching.watermark,
        )
        try:
            out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
            print(f"Stitched highlight video saved to: {out_path}", flush=True)
        except Exception as e:
```

```
219             print(f"Error during stitching: {e}", flush=True)
220
221         _log_rally_summary()
222
223
224     if __name__ == "__main__":
225         main()
226
```

27. user (2026-07-09T02:39:15.329Z)

```
===Look for a videos vault outside repo (Videos/Downloads/named vault)===
Bhuvan Clip.mp4
Captures
Chhetri Opinion.MP4
desktop.ini
GX010016.MP4
GX010031.MP4
GX010059.MP4
GX010061.MP4
GX010075.MP4
GX030122.MP4
Pungi dance.mp4
Pungi dance_1080p.mp4
SmokeTestBadmintonClip.mp4
--- any dir named vault ---
/c/Users/avidu/AppData/Local/Microsoft/Vault
/c/Users/avidu/AppData/Local/Temp/repro_corpus
/c/Users/avidu/AppData/Roaming/Microsoft/Vault
/c/Users/avidu/Projects/khelsutra-guru/Annotation Setup/Golden Labelled
===Largest source-looking mp4/MP4 under repo output+scratch+artifacts (real recordings, not
tiny)===
1987 MB /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/GX010125_highlights_min5.mp4
1427 MB /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/KushagraAviHighlight-Trailor.mp4
1379 MB /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/boxhill_highlights_stage/Adarsh v Vasant.MP4
1152 MB /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/GX010125_highlights.mp4
625 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/gem_newclips/Badminton BXH 2_proxy - Highlights.mp4
329 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/gem_newclips/GX030094_proxy - Highlights.mp4
157 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/GX010125_run/GX010125_highlights.mp4
153 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/kushagra_1280.mp4
133 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/smoke19/smoke_GX010143.mp4
122 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/output/chunks/74affb3
6e15e172130cea5c451e4a728/chunk_74affb36e15e172130cea5c451e4a728_0.mp4
119 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/MahadevpuraSingles_highlights.mp4
105 MB  /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/chunks/chunk_e0168f050899da4de75bcd311b80131b_5.mp4
98 MB   /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/output/chunks/3d19ef8
35a69be93acdd75ef636f345a/chunk_3d19ef835a69be93acdd75ef636f345a_3.mp4
86 MB   /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/output/mahadevpura_smoke_180s.mp4
86 MB   /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/artifacts/review/source_mahadevpura_smoke_180s.mp4
81 MB   /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/artifacts/cloudrun_e2e_validation/cloudrun_e2e_reel.mp4
76 MB   /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
```

indexer/output/chunks/chunk_018c2e2c3ce3086c3badfa1a9e5bc5c9_1.mp4

28. assistant (2026-07-09T02:39:56.739Z)

I've located the likely "vault": `C:\Users\avidu\Videos` holds real GoPro source recordings (`GX010016.MP4`, `GX010031.MP4`, etc.). Note the existing interactive runner **bypasses validation** and prompts on cache ? so I'll need a purpose-built non-interactive timing harness that includes validation. Let me gather the concrete facts I need to design it: video durations, local CUDA/ffmpeg-NVENC availability, and WSL presence (for the wasb path).

29. user (2026-07-09T02:40:03.981Z)

```
===Candidate vault videos (duration / res / size)===
GX010016.MP4                                width=1920
 height=1080
 r_frame_rate=30000/1001
 duration=47.147100
 size=47485464

GX010031.MP4                                width=1920
 height=1080
 r_frame_rate=30000/1001
 duration=16.016000
 size=4966612

GX010059.MP4                                width=5312
 height=4648
 r_frame_rate=30000/1001
 duration=21.755067
 size=116248423

GX010061.MP4                                width=5312
 height=4648
 r_frame_rate=30000/1001
 duration=308.708400
 size=2318282212

GX010075.MP4                                width=5312
 height=2988
 r_frame_rate=30000/1001
 duration=89.089000
 size=1336620074

GX030122.MP4                                width=3840
 height=2160
 r_frame_rate=60000/1001
 duration=9.664000
 size=144455404

SmokeTestBadmintonClip.mp4                  width=1920
 height=1080
 r_frame_rate=60000/1001
 duration=81.347922
 size=125410307
```

30. user (2026-07-09T02:40:25.079Z)

```
===torch CUDA locally===
torch 2.6.0+cu124 cuda_avail True NVIDIA GeForce RTX 2070 Super with Max-Q Design
torchvision 0.21.0+cu124
===ffmpeg + nvenc availability===
 V....D av1_nvenc           NVIDIA NVENC av1 encoder (codec av1)
 V....D libx264             libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 (codec h264)
 V....D libx264rgb          libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 RGB (codec h264)
```

```
 V....D h264_amf             AMD AMF H.264 Encoder (codec h264)
 V....D h264_d3d12va         D3D12VA h264 encoder (codec h264)
 V....D h264_mf              H264 via MediaFoundation (codec h264)
===WSL present?===
 ■ ■N■A■M■E■ ■ ■ ■ ■ ■ ■S■T■A■T■E■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■V■E■R■S■I■O■N■
■
■*■ ■U■b■u■n■t■u■ ■ ■ ■ ■S■t■o■p■p■e■d■ ■ ■ ■ ■ ■ ■ ■ ■2■
■
■
```

31. assistant (2026-07-09T02:41:26.710Z)

Local env confirms it's viable: **torch 2.6.0+cu124 with CUDA on the RTX 2070** (matches prod's torch 2.6), and WSL/Ubuntu exists (needed only for the heavy `wasb` path). Vault video profiles:

Video	Res	Dur	Size
SmokeTestBadmintonClip.mp4	1080p60	81s	120 MB
GX010016.MP4	1080p30	47s	45 MB
GX010075.MP4	5.3K 16:9	89s	1.27 GB
GX010061.MP4	5.3K (4:3)	5min	2.2 GB
GX030122.MP4	4K60	9.7s	138 MB

Now let me resolve what "the latest binary on api.khelsutra.guru" actually maps to right now (my memory is 2 days old and warns master moved past it) by querying the live deployment and syncing master.

32. user (2026-07-09T02:41:38.134Z)

```
===LIVE control-plane revision + image (asia-southeast1)===
rally-control-plane-00022-29w   rally-control-plane-00022-29w   asia-
southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
===LIVE worker job image===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
```

33. user (2026-07-09T02:41:41.536Z)

```
===fetch origin (ff-only mindset; read-only fetch)===
From https://github.com/Khelsutra/badminton-highlight-indexer
 * [new branch]      docs/test-bootstrap-instructions -> origin/docs/test-bootstrap-instructions
   071d8b2..3249451  master     -> origin/master
===origin/master HEAD===
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
e29172a perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539) (#545)
af54f2f fix(#538): cancel propagates to the dispatched L4 worker exec (#544)
14e4767 observability: emit structured signal on GPU encoder fallback to CPU (#529) (#543)
b68af0d docs: add verified bootstrap for reproducing tests locally (#542)
===is 304af7b still tip? what's new since 304af7b on master===
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
e29172a perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539) (#545)
af54f2f fix(#538): cancel propagates to the dispatched L4 worker exec (#544)
14e4767 observability: emit structured signal on GPU encoder fallback to CPU (#529) (#543)
b68af0d docs: add verified bootstrap for reproducing tests locally (#542)
8afbea9 perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)
(#541)
071d8b2 feat(compile): add opt-in SD reel rendition (#537)
f7a326d docs(runlogs): compile?L4 + NVENC live deploy-validation CUJ (2026-07-08) (#536)
5880c26 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521) (#531)
===count new commits since 304af7b===
9
```

34. assistant (2026-07-09T02:43:24.174Z)

I now have what I need to resolve the core ambiguity. Here's the key finding:

Prod (`api.khelsutra.guru`) = commit `304af7b` ? verified: the live control-plane (`rally-control-plane-00022-29w`) and worker both run image `sha256:7b62a8df?`, which my deploy records tie to `origin/master @ 304af7b`.

`origin/master` has since moved +9 commits to `3249451`. The release-relevant, *locally-measurable* change in that delta is **`5880c26` ? `perf(stitch): single-pass concat-filter stitch (#531)`**, plus the SD-rendition compile work (`071d8b2`/#537). The other 7 are cloud-topology/observability/docs (GCSFuse mount, CP `acquire_local_input` gating, cancel propagation) that a **local** GPU run can't faithfully exercise.

So "HEAD vs the prod binary, locally" cleanly maps to: **run both commits (`304af7b` vs the release candidate) through one identical timing harness, on this GPU, over the same vault videos** ? via isolated `git worktree`s so your working tree is untouched. Each stage (checksum ? validation ? segmentation ? full-reel stitch) is timed; results diff into a committed baseline so every release gets precomputed numbers.

Before I build and run it (the GPU runs are the expensive part), three decisions ? all have a recommended default:

35. user (2026-07-09T02:45:22.635Z)

Your questions have been answered: "For the 'current HEAD' side, which code version should I benchmark against the deployed prod binary (304af7b)?"="origin/master (3249451)", "Which segmenter for the local GPU benchmark? It's identical on both sides (a constant backdrop), so the aim is a fast, deterministic, repeatable run that isolates the validation + stitch deltas that actually changed."="wasb_detector now uses the GPU and is production, we should always compare against production", "Confirm the vault videos (from C:\Users\avidu\Videos). I recommend a resolution/duration spread so the numbers cover both 1080p and 5.3K."="Add GX010061 (5min 5.3K) too". You can now continue with these answers in mind.

36. assistant (2026-07-09T02:45:53.518Z)

Understood ? you want **wasb_hybrid** (the production GPU-resident detector) so the comparison mirrors what actually runs in prod, plus the 4-video set including the heavy 5-min 5.3K clip. That's the right call for fidelity; it also means I need to nail exactly how the wasb path executes on *this* Windows box (native CUDA vs the WSL frame-staging path, and the `decode_backend` knob), since that's the biggest setup risk. Let me set up task tracking and investigate the local wasb execution path.

37. user (2026-07-09T02:45:55.341Z)

Found 7 files
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py

38. user (2026-07-09T02:45:59.054Z)

```
===config.json indexing.wasb + default_segmenter===
default_segmenter= gemini
use_gpu= True
{'conda_env': 'wasb',
 'conda_sh': '~/miniconda3/etc/profile.d/conda.sh',
 'repo_dir': '~/models/WASB-SBDT',
 'sport': 'badminton',
 'timeout_sec': 1800,
 'velocity_threshold': 0.02,
 'weights_path': '~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar',
```

```
 'wsl_distro': 'Ubuntu',
 'wsl_stage_dir': '~/clips'}
===wasb segmenter files===
wasb_hybrid.py
native_wasb_runner.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
```

39. assistant (2026-07-09T02:46:17.180Z)

Now the picture sharpens: wasb has **two runners** ? `wasb_runner.py` (WSL + conda env `wasb` + WASB-SBDT repo) and `native_wasb_runner.py` (the #506/#507 native GPU-resident path). Prod uses the native path with `nvdec_resident` decode. I need to see which one `wasb_hybrid` selects locally and whether the required env/weights actually exist on this box. Let me set up tasks and probe both.

40. user (2026-07-09T02:46:19.711Z)

Task #1 created successfully: Determine local wasb_hybrid execution path

41. user (2026-07-09T02:46:22.323Z)

Task #2 created successfully: Build repeatable per-stage timing harness

42. user (2026-07-09T02:46:24.693Z)

Task #3 created successfully: Wire two-version comparison via git worktrees

43. user (2026-07-09T02:46:26.701Z)

Task #4 created successfully: Run benchmark on 4 vault videos + produce report

44. user (2026-07-09T02:46:28.994Z)

Task #5 created successfully: Make it repeatable before every release

45. user (2026-07-09T02:46:34.862Z)

Updated task #1 status

46. user (2026-07-09T02:46:35.918Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4       WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5       badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15
16      from typing import Any, Dict, Tuple
17
18      from backend.config.models import IndexingConfig
19      from backend.pipeline.detectors.base import DetectorRunner
20      from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
```

```
21      from backend.pipeline.segmenters.base import segmenter_registry
22      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
23
24
25      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
26          family = "wasb"
27          display_label = "WASB"
28
29          @property
30          def name(self) -> str:
31              return "wasb_hybrid"
32
33          @property
34          def description(self) -> str:
35              return (
36                  "Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
37                  "weights) for precise candidate rallies + AI provider for semantic
boundaries."
38              )
39
40          def _build_runner(
41              self, idx_cfg: IndexingConfig
42          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
43              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
44              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
45
46          def _build_native_runner(
47              self, idx_cfg: IndexingConfig
48          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
49              # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
50              # seam (imported only when detector_impl='native' is actually selected).
51              from backend.pipeline.detectors.native_wasb_runner import (
52                  NativeWasbConfig,
53                  NativeWasbRunner,
54              )
55
56              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
57              return NativeWasbRunner(cfg), {
58                  "weights_path": cfg.weights_path,
59                  "detector_impl": "native",
60                  "device": cfg.device,
61                  # Record the decode path so persisted candidates are auditable: the cpu and
62                  # nvdec_resident caches are non-interchangeable (#506), so downstream evals
/
63                  # dataset exports must be able to tell which decode produced a candidate.
64                  "decode_backend": cfg.decode_backend,
65              }
66
67
68      segmenter_registry.register(WasbHybridSegmenter)
69
```

47. assistant (2026-07-09T02:46:53.557Z)

wasb_hybrid dispatches between a WSL runner (`WasbRunner`) and the native GPU runner (`NativeWasbRunner`) based on `detector_impl`. Let me find what selects native-vs-WSL and its default, and probe whether the required environments (WSL conda `wasb` + weights, or native local weights) actually exist on this box.

48. user (2026-07-09T02:46:53.937Z)

```
12-                     the ``<family>-hybrid-<model>`` detector tag in metadata.
13-- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
```

```
14-- ``name`` / ``description`` properties (the registry contract).
15:- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16-                     detection_params extras such as weights_path).
17-
18-This base is deliberately NOT registered; only concrete subclasses register.
--
56-    #: human-readable label used in log lines.
57-    display_label: str = ""
58-
59:    def _build_runner(
60-        self, idx_cfg: IndexingConfig
61-    ) -> Tuple[DetectorRunner, Dict[str, Any]]:
62-        """Return (runner, detector-specific detection_params extras) for the default (WSL)
path."""
63-        raise NotImplementedError
64-
65:    def _build_native_runner(
66-        self, idx_cfg: IndexingConfig
67-    ) -> Tuple[DetectorRunner, Dict[str, Any]]:
68:        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
69:        native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
70-        raise NotImplementedError(
71:            f"detector_impl='native' is not supported for the "
72:            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a
"
73:            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
74-        )
75-
76:    def _resolve_runner(
77-        self, idx_cfg: IndexingConfig
78-    ) -> Tuple[DetectorRunner, Dict[str, Any]]:
79:        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.
80-
81:        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
82:        ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
83:        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
84:        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85:        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
86:        (the WSL runner) so the default production path is unchanged
87:        (``docs/DECOUPLED_COMPUTE/``).
88:        """
--
90:        if isinstance(idx_cfg, dict):
91:            idx_cfg = IndexingConfig(**idx_cfg)
92:        impl = (
93:            os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94:        ).lower()
95:        if impl == "stub":
96:            from backend.pipeline.detectors.stub_runner import (
--
99:            )
100-
101-            logger.info(
102:                "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103:                self.display_label or "trajectory",
104-            )
105:            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
106:                "detector_impl": "stub"
107-            }
108:        if impl in ("native", "native_wasb", "wasb_native"):
109-            logger.info(
110:                "%s: detector_impl=native ? Linux-native runner (no WSL).",
111:                self.display_label or "trajectory",
112-            )
```

```
113:            return self._build_native_runner(idx_cfg)
114:        return self._build_runner(idx_cfg)
115-
116-    @staticmethod
117-    def _probe_fps_width(video_path: str) -> tuple:
--
180-    ) -> "Union[List[Dict[str, Any]], Failure]":
181-        # Result contract (the ABC declares Success|Failure). Anything that means the run
could
182-        # NOT actually complete ? an unrunnable config (unknown sport/provider, missing API
key),
183:        # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
184-        # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
185-        # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match (a
trajectory
186-        # was produced but no candidate/AI-confirmed rallies) still returns a bare ``[]``.
Every
--
242-
243-        # --- Runner config + windowing thresholds ---
244-        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
245:        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246:        # it delegates to the subclass _build_runner (the real WSL/native runner).
247-        try:
248-            runner, runner_params = self._resolve_runner(idx_cfg)
249-        except NotImplementedError as e:
250:            # e.g. detector_impl="native" for a family without a native runner (tracknet, or
251-            # fusion with a tracknet substrate). An unavailable detector is a failure, not a
252-            # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
success.
253-            logger.error("%s: %s", label, e)
--
299-        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300-        if not csv_path:
301-            logger.error("%s produced no trajectory; aborting.", label)
302:            # The cold-start / timeout failure mode (e.g. WASB native inference timed out
after
303-            # timeout_sec, or the detector env yielded nothing). This is a DETECTOR failure
? NOT
304-            # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305-            # rather than a silent 0-segment "success" (the wf294 incident).
```

49. user (2026-07-09T02:46:59.452Z)

```
===config detector_impl / wasb decode_backend===
detector_impl= None
wasb.decode_backend= None
wasb.device= None

===WSL wasb env probe (conda env, repo, weights)===
/home/avidullu/.bash_profile: line 2: .bashrc: No such file or directory
HOME=/home/avidullu
/home/avidullu/miniconda3
wasb                     /home/avidullu/miniconda3/envs/wasb
--repo--
/home/avidullu/models/WASB-SBDT
--weights--
-rw-r--r-- 1 avidullu avidullu 6102633 Nov 15  2023 /home/avidullu/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
```

50. assistant (2026-07-09T02:48:14.966Z)

My WSL probe confirms the local wasb environment is **fully provisioned**: WSL Ubuntu, conda env `wasb`, the `WASB-SBDT` repo, and the badminton weights are all present. With `detector_impl=None` ? default `"auto"`, wasb_hybrid runs via the **WSL2 GPU path** (torch/CUDA inside WSL) ? the practical production-parity detector on this box. The Explore agent confirmed the four stage call-sites and that **no existing harness times all four stages or stitches from CLI** ? I'll build that.

Let me update task #1 as done and probe three implementation-critical facts: how `load_config` can be overridden (so both versions run identical config), whether `backend` is editable-installed (affects worktree isolation), and the exact validator/compiler signatures.

51. user (2026-07-09T02:48:15.239Z)

Updated task #1 status

52. user (2026-07-09T02:48:20.004Z)

```
===load_config signature + any path/env override===
(path: 'str | Path | None' = None) -> 'AppConfig'
def load_config(path: str | Path | None = None) -> AppConfig:
    """Load configuration with the following precedence:

    1. Built-in defaults defined in the Pydantic models.
    2. Values from the JSON file (if present and readable).
    3. (Future) environment / CLI overrides.

    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
    cfg_path = Path(path) if path is not None else default_config_path()

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
        try:
            with open(cfg_path, "r", encoding="utf-8") as f:
                file_data = json.load(f)
        except Exception as exc:
            # Non-fatal: continue with defaults + whatever we could parse.
            # In production we might log here.
            logger.warning(
                f"Failed to read {cfg_path}: {exc}. Using defaults + partial data."
            )

    # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`, `null`) would
    # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and
any
    # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn, per
the
    # loader's "bad input is non-fatal" contract.
    if not isinstance(file_data, dict):
        logger.warning(
            "%s is not a JSON object (got %s); ignoring it and using defaults.",
            cfg_path,
            type
```

53. user (2026-07-09T02:48:22.967Z)

```
===Is backend editable-installed (would defeat worktree isolation)?===
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import backend, os; print('backend at:', os.path.dirname(backend.__file__))
                                             ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
  File "<frozen ntpath>", line 263, in dirname
  File "<frozen ntpath>", line 227, in split
```

```
TypeError: expected str, bytes or os.PathLike object, not NoneType
(pkg name differs; checking site-packages .pth/.egg-link)
done
```

54. assistant (2026-07-09T02:49:46.797Z)

Now the critical design facts. `load_config(path=...)` accepts an explicit path ? so I can drive **both versions from one canonical benchmark config** (holding inputs identical, letting only the *code* differ). Let me confirm the key remaining facts before building: verify prod's image really is `304af7b`, see exactly what changed in the stitch/config between the two commits (to interpret results), confirm `backend` is a namespace package (governs worktree isolation), and lock the stage signatures.

55. user (2026-07-09T02:49:59.189Z)

```
===[a] prod image git-sha label (if any)===
===[b] what changed 304af7b..3249451 in stitch/validation/config (files)===
 backend/config/models.py              |  57 +++++
 backend/orchestration.py              | 390 ++++++++++++++++++++++++++++++-------
 backend/pipeline/stitcher/compiler.py | 365 ++++++++++++++++++++++++++++++---
 3 files changed, 707 insertions(+), 105 deletions(-)
```

56. user (2026-07-09T02:50:01.439Z)

```
===[c] backend a namespace pkg? (governs isolation)===
ls: cannot access 'backend/__init__.py': No such file or directory
===default_config_path (cwd-relative?)===
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import inspect,backend.config as c; print(inspect.getsource(c.default_config_path))
                                                                 ^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'backend.config' has no attribute 'default_config_path'. Did you mean:
'default_db_path'?
===[d] validator run_all_with_context signature===
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import inspect; from backend.pipeline.validators.base import ValidatorRegistry as R;
print([m for m in dir(R) if 'run' in m]); import backend.pipeline.validators as v;
print('registry:', type(v.registry).__name__);
print(inspect.signature(v.registry.run_all_with_context))
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ImportError: cannot import name 'ValidatorRegistry' from 'backend.pipeline.validators.base'
(C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py)
===VideoCompiler ctor + compile_highlights sig===
init (self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5, watermark:
Optional[Any] = None, encoder: str = 'libx264', single_pass: bool = False, encode_profile:
Optional[Any] = None)
compile (self, raw_video_path: str, segments: List[Tuple[float, float]], output_filename: str)
-> str
```

57. assistant (2026-07-09T02:50:53.832Z)

`backend` is a namespace package (no `__init__.py`) ? PYTHONPATH/cwd isolation across worktrees will work. Real code deltas exist in stitch (`compiler.py` +365 lines) and `orchestration.py` (+390) ? so the comparison is meaningful. One important catch: `VideoCompiler.__init__` gained `single_pass`/`encode_profile` args (the #531/#537 work), so its signature **differs between prod and RC** ? my harness must construct it via runtime introspection (pass only supported kwargs, set `single_pass` from config when available) to faithfully run each version's own stitch path. Let me confirm the prod-side compiler signature and read the exact orchestration stitch construction + DB/validator APIs.

58. user (2026-07-09T02:50:59.286Z)

```
===compiler __init__ at PROD 304af7b===
26:#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's first
clip pays
27:#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
30:#: Frame geometry for the 1-frame NVENC probe encode in ``_probe_encoder_available``. It MUST
stay
31:#: at or above NVENC's minimum encode dimensions: on real GPUs the hardware H.264/HEVC
encoders
33:#: unavailable and ``_select_encoder`` silently falls back to libx264 ? defeating the NVENC-
on-L4
36:#: 256x256 ? guarded by tests/test_compiler.py::test_probe_encoder_uses_safe_resolution.
40:def _probe_encoder_available(encoder: str) -> bool:
41:    """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
44:    often *compiled in* on boxes without an NVIDIA GPU, so a real test encode ? not
``-encoders``
47:    if encoder in _ENCODER_AVAILABLE:
48:        return _ENCODER_AVAILABLE[encoder]
51:    # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
===compiler __init__ at RC 3249451===
26:#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's first
clip pays
27:#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
30:#: Frame geometry for the 1-frame NVENC probe encode in ``_probe_encoder_available``. It MUST
stay
31:#: at or above NVENC's minimum encode dimensions: on real GPUs the hardware H.264/HEVC
encoders
33:#: unavailable and ``_select_encoder`` silently falls back to libx264 ? defeating the NVENC-
on-L4
36:#: 256x256 ? guarded by tests/test_compiler.py::test_probe_encoder_uses_safe_resolution.
40:def _probe_encoder_available(encoder: str) -> bool:
41:    """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
44:    often *compiled in* on boxes without an NVIDIA GPU, so a real test encode ? not
``-encoders``
47:    if encoder in _ENCODER_AVAILABLE:
48:        return _ENCODER_AVAILABLE[encoder]
51:    # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
===config models single_pass_stitch default at PROD vs RC===
-- prod --
354:class StitchingConfig(BaseModel):
374:    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
516:    cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
-- rc --
394:class StitchingConfig(BaseModel):
414:    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
426:    single_pass_stitch: bool = True
573:    cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
```

59. user (2026-07-09T02:51:03.868Z)

```
===_stitch_reel construction (current) ? how orchestration builds the compiler===
16:    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
17:    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
18:    trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
29:    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
41:    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
43:    localized_watermark = localize_watermark(stitching.watermark, locale)
44:    encode_profile = _rendition_encode_profile(stitching, rendition)
45:    compiler = _main.VideoCompiler(
```

```
47:        begin_padding=stitching.begin_padding,
48:        end_padding=stitching.end_padding,
===validators registry + run_all_with_context signature===
registry type: ValidationRegistry
run_all_with_context: (video_path: str, config: Any)
===Database methods I need===
add_video (self, video_id: str, filepath: str, status: str, validation_results: List[Dict[str,
Any]]) -> bool
query_play_segments (self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str, Any]]
clear_video_data (self, video_id: str)
get_video (self, video_id: str) -> Optional[Dict[str, Any]]
```

60. assistant (2026-07-09T02:51:52.814Z)

Confirmed the headline release delta: **prod (304af7b) has no `single_pass_stitch`; RC (3249451) defaults it to `True`** (the #531 single-pass concat stitch). My introspection-based compiler construction will faithfully run each version's own stitch path. All stage signatures are locked.

One risk to de-risk before running anything heavy: how the WSL wasb path stages/decodes frames ? if it extracts every frame to PNG at 5.3K, the 5-min GX010061 clip could blow up disk. Let me check the wasb runner's decode approach, WSL disk headroom, and the segmenter signature.

61. user (2026-07-09T02:51:57.161Z)

```
===wasb_runner staging/decode approach (PNG frames? downscale? streaming?)===
4:  1. stages the video into the WSL filesystem (fast local reads, not /mnt),
33:# Windows path to the inference wrapper we stage into WSL.
49:    wsl_stage_dir: str = "~/clips"
51:    keep_frames: bool = (
52:        False  # retain staged video + frame cache after success (debug only)
55:    # FAST PATH (default OFF until verified): decode the staged video on the fly inside
56:    # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
57:    # extraction that dominates a long clip. Output is bit-identical to the PNG path
58:    # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms it.
73:            wsl_stage_dir=w.wsl_stage_dir,
75:            keep_frames=w.keep_frames,
103:    def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
107:            logger.error("Cannot stage video into WSL: %s", reason)
109:        dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
110:        cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
118:                "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
123:    def _stage_infer_script(self) -> bool:
137:        can never reuse each other's staged frames, resumable cache, or CSV.
154:        if not self._stage_infer_script():
155:            logger.error("Could not stage wasb_infer.py into WSL.")
157:        staged_video = self._stage_video(video_win_path, f"{key}{ext}")
158:        if staged_video is None:
160:        frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
162:        # Disk guard (D11): a 4K clip can extract ~100 GB of PNG frames. BLOCK before
extraction if
163:        # the WSL stage fs can't hold ~15× the source ? a mid-run disk-fill wastes a long
run and
165:        # Streaming never materializes the PNGs, so the guard only matters off the fast
path.
167:            shortfall = self._wsl_frame_stage_shortfall(video_win_path)
172:        # Fast path: stream-decode the video in-process (no PNG extraction). Output is
173:        # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
177:            frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
181:            f"--video {wsl_tilde_quote(staged_video)} "
205:        # Success ? the trajectory CSV is the durable output; the staged video copy and
206:        # the (huge) decoded frame cache are now pure intermediates. Delete them unless
209:        # (Streaming never writes a frame cache, so there's only the staged video to drop.)
```

```
210:        if not self.cfg.keep_frames:
212:                [staged_video] if self.cfg.stream_video else [frames_dir, staged_video]
215:                "Cache hygiene: removing staged video%s for %s "
216:                "(set indexing.wasb.keep_frames=true to retain).",
```

62. user (2026-07-09T02:52:05.665Z)

```
===WSL disk free===
/home/avidullu/.bash_profile: line 2: .bashrc: No such file or directory
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc       1007G   45G  911G   5% /
clips dir:
45M     /home/avidullu/clips
===Windows scratch/repo disk free===
Filesystem      Size  Used Avail Use% Mounted on
C:              746G  504G  242G  68% /c
===segmenter.process_video signature + skip_ai_handoff in both models===
process_video (self, video_id: str, video_path: str, player_pool: Optional[List[str]] = None,
max_frames: Optional[int] = None) -> 'Union[List[Dict[str, Any]], Failure]'
-- skip_ai_handoff prod --
289:    skip_ai_handoff: bool = False
486:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
-- skip_ai_handoff rc --
308:    skip_ai_handoff: bool = False
543:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
```

63. assistant (2026-07-09T02:52:29.623Z)

Two important findings: wasb has a **disk guard** (blocks if WSL can't hold ~15× source) and an opt-in **fast streaming decode** (`stream_video`, bit-identical, no PNG blowup). WSL has **911 GB free** so even PNG extraction fits, but streaming is far better for a benchmark. Critical fairness question: is `stream_video` present in *both* prod and RC? If it's RC-only, enabling it would unfairly speed up only RC. Let me check.

64. user (2026-07-09T02:52:35.199Z)

```
===indexing.wasb model fields ? PROD 304af7b===
47:    wsl_distro: str = "Ubuntu"
50:    conda_env: str = "TrackNetV4"
51:    weights_path: str = ""
54:    wsl_stage_dir: str = "~/clips"
61:    keep_frames: bool = False
62:    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
76:    wsl_distro: str = "Ubuntu"
79:    conda_env: str = "wasb"
80:    weights_path: str = (
84:    wsl_stage_dir: str = "~/clips"
98:    keep_frames: bool = False
99:    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
112:    stream_video: Optional[bool] = None
116:    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts into
118:    batch_size: Optional[int] = None
121:    prefetch_batches: Optional[int] = None
131:    decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
138:        if self.decode_backend == "nvdec_resident" and self.device not in ("cuda", "auto"):
140:                f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda' or "
145:    @field_validator("batch_size")

===indexing.wasb model fields ? RC 3249451===
47:    wsl_distro: str = "Ubuntu"
50:    conda_env: str = "TrackNetV4"
51:    weights_path: str = ""
54:    wsl_stage_dir: str = "~/clips"
61:    keep_frames: bool = False
```

```
62:    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
76:    wsl_distro: str = "Ubuntu"
79:    conda_env: str = "wasb"
80:    weights_path: str = (
84:    wsl_stage_dir: str = "~/clips"
98:    keep_frames: bool = False
99:    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
112:    stream_video: Optional[bool] = None
116:    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts into
118:    batch_size: Optional[int] = None
121:    prefetch_batches: Optional[int] = None
131:    decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
144:        if self.decode_backend == "nvdec_resident" and self.device not in ("cuda", "auto"):
146:                f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda' or "
151:    @field_validator("batch_size")

===current config.json indexing.wasb.stream_video?===
{'conda_env': 'wasb',
 'conda_sh': '~/miniconda3/etc/profile.d/conda.sh',
 'repo_dir': '~/models/WASB-SBDT',
 'sport': 'badminton',
 'timeout_sec': 1800,
 'velocity_threshold': 0.02,
 'weights_path': '~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar',
 'wsl_distro': 'Ubuntu',
 'wsl_stage_dir': '~/clips'}
```

65. assistant (2026-07-09T02:53:26.826Z)

Decisive: `stream_video`, `batch_size`, `decode_backend` are present and **identical in both prod and RC** WasbConfig models ? so enabling the fast streaming decode (`stream_video=true`) on both is fair and avoids the ~100 GB PNG blowup. The whole config surface I need matches across versions.

I have all the facts to build. Two last compatibility checks (how config resolves, so the harness is version-agnostic without mutating worktrees), then I'll write the harness.

66. user (2026-07-09T02:53:33.833Z)

```
===default_config_path source (cwd-relative?)===
  File "<string>", line 11
    import backend.config.paths as p 2>/dev/null
                                     ^
SyntaxError: invalid syntax
in bc: False
===prod load_config accepts path?===
```

67. user (2026-07-09T02:53:42.317Z)

```
backend\config\loader.py:4:merged with optional file overrides (config.json).
backend\config\loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals
``Path("config.json")`` (CWD-relative) when
backend\config\loader.py:34:# the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
backend\config\loader.py:41:    replace). Used ONLY for the profile-preset precedence path so a
config.json sub-key
backend\config\loader.py:54:    """The EXPLICITLY chosen deployment profile, env winning over
config.json. ``""`` if none
backend\config\loader.py:60:    asserting a preset that was never applied. A bad value in
config.json is left to surface as
backend\config\loader.py:69:            "and falling back to config.json deployment.profile.",
backend\config\loader.py:120:    a fresh AppConfig instance every call, reflecting
config.json/env.)
backend\config\loader.py:128:def load_config(path: str | Path | None = None) -> AppConfig:
```

```
backend\config\loader.py:152:    # A config.json that is valid JSON but not an OBJECT (e.g.
`[]`, `42`, `"x"`, `null`) would
backend\config\loader.py:175:    # Precedence: built-in defaults < profile preset < config.json
(< env/--set, applied at
backend\config\loader.py:177:    # explicitly selected (DEPLOYMENT_PROFILE env or config.json
deployment.profile). With no
backend\config\loader.py:183:        # Only an unrecognised value from config.json reaches here
(env typos already fell back
backend\config\loader.py:195:        # Record the profile actually applied ? the env may have
chosen it over config.json,
backend\config\loader.py:203:                # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:210:                    "config.json overrides it to %r ? honoring the
explicit override.",
backend\config\loader.py:221:        # No profile ? same deep merge so a config.json sub-key
overrides its
backend\config\loader.py:240:    """Write a fresh default config.json.
backend\config\models.py:335:    (under `stitching.watermark` in config.json) to turn it off.
The text is fully
backend\config\models.py:518:    behaves exactly as before this section existed. A profile is
opt-in (``config.json``
```

[Showing results with pagination = limit: 20]

68. user (2026-07-09T02:53:45.639Z)

```
===prod load_config + default_config_path signatures (search whole backend at 304af7b)===
44:def default_db_path(config_path: str = "config.json") -> str:
--- if not there, where is load_config in prod? ---
304af7b:backend/config/loader.py:128:def load_config(path: str | Path | None = None) ->
AppConfig:
304af7b:backend/main.py:252:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
304af7b:backend/utils/app_home.py:53:def default_config_path() -> Path:
```

69. assistant (2026-07-09T02:54:13.781Z)

`load_config(path=?)` exists with the **same signature in both versions** ? so I pass the canonical bench config explicitly, no worktree mutation. `default_config_path()` is CWD-relative `config.json` otherwise. Everything resolves cleanly.

Last detail before writing: confirm the prod compiler ctor and how orchestration passes `single_pass`, so my introspection-based stitch faithfully reproduces each version's path.

70. user (2026-07-09T02:54:19.164Z)

```
===PROD 304af7b VideoCompiler.__init__ params===
    def __init__(
        self,
        output_dir: str,
        end_padding: float = 1.0,
        begin_padding: float = 0.5,
        watermark: Optional[Any] = None,
        encoder: str = "libx264",
    ):
===RC 3249451 VideoCompiler.__init__ params===
82:    def __init__(
    def __init__(
        self,
        output_dir: str,
        end_padding: float = 1.0,
        begin_padding: float = 0.5,
        watermark: Optional[Any] = None,
        encoder: str = "libx264",
        single_pass: bool = False,
```

```
        encode_profile: Optional[Any] = None,
    ):
===RC _stitch_reel: how single_pass is passed to compiler===
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
1368:    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1381:    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1397:    compiler = _main.VideoCompiler(
1402:        encoder=getattr(stitching, "encoder", "libx264"),
1403:        single_pass=getattr(stitching, "single_pass_stitch", False),
1404:        encode_profile=encode_profile,
```

71. assistant (2026-07-09T02:54:52.723Z)

All confirmed. Prod ctor has no `single_pass`/`encode_profile`; RC adds them and `_stitch_reel` passes `single_pass=getattr(stitching,"single_pass_stitch",False)`. My introspection approach reproduces each version's real stitch path exactly. I'm ready to build.

Let me set up the workspace and create the two isolated worktrees (leaving your working tree untouched).

72. user (2026-07-09T02:55:00.408Z)

```
===existing worktrees===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
a460d74 [codex/issue-523-sd-reel-plan-clear]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark
65ee4c6 [codex/issue-523-sd-reel-review]
===create prod (304af7b) + rc (3249451) worktrees (detached)===
Preparing worktree (detached HEAD 304af7b)
HEAD is now at 304af7b feat(serving): L4-primary topology design + trust submit-time video_id
(#524) (#530)
Preparing worktree (detached HEAD 3249451)
HEAD is now at 3249451 feat(#534): live detector progress heartbeat + L4 worker resource
telemetry (#546)
===verify===
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt           5de1ef2 (detached
HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/prod  304af7b (detached
HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/rc    3249451 (detached
HEAD)
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark
65ee4c6 [codex/issue-523-sd-reel-review]
--- prod HEAD ---
304af7b
--- rc HEAD ---
3249451
--- backend namespace present in each? ---
prod/backend/main.py
rc/backend/main.py
```

73. assistant (2026-07-09T02:55:11.987Z)

Worktrees are up: **prod @ 304af7b** (`feat(serving): L4-primary topology?#530` ? confirms it's the deployed commit) and **rc @ 3249451**, your working tree untouched. Now I'll build the harness. Let me read the real config.json to base the canonical benchmark config on exact shipped values.

74. user (2026-07-09T02:55:12.579Z)

Updated task #2 status

75. user (2026-07-09T02:55:13.013Z)


```
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [
                35,
                40,
                40
            ],
            "court_green_upper": [
                85,
                255,
                255
            ],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "gemini",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "detector_model": "fasterrcnn_resnet50_fpn",
        "detector_score_threshold": 0.5,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_prefetch_workers": 2,
        "min_action_window_duration": 2.0,
        "log_yolo_candidates": true,
        "store_yolo_candidates": true,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 1,
        "tracknet": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/TrackNetV4",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "TrackNetV4",
            "weights_path": "",
            "queue_length": 5,
            "stage_in_wsl": true,
            "wsl_stage_dir": "~/clips",
```

```
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            },
            "wasb": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/WASB-SBDT",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "wasb",
                "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
                "sport": "badminton",
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1,
            "watermark": {
                "enabled": true,
                "text": "Generated by Khelsutra.guru",
                "position": "bottom-right",
                "opacity": 0.85,
                "logo_path": "",
                "font_path": ""
            },
            "renditions": {
                "sd": {
                    "max_width": 854,
                    "max_height": 480,
                    "crf": 30,
                    "audio_bitrate": "96k",
                    "video_preset": "superfast",
                    "pix_fmt": "yuv420p"
                }
            }
        }
    }

```

76. assistant (2026-07-09T02:56:11.237Z)

Now I'll write the benchmark. First the canonical config template ? shipped values, overridden to force the **local wasb_hybrid GPU path, deterministic, no cloud/user step**, with the fast streaming decode enabled on both versions.

77. assistant (2026-07-09T02:56:22.599Z)

```
{
    "_comment": "Canonical release perf-benchmark config. Identical INPUTS for prod vs RC so
only CODE differs. Forces the production WASB GPU detector via WSL (detector_impl=auto), pure-
local + deterministic (skip_ai_handoff=true, no Gemini), and the fast streaming decode
(stream_video=true, bit-identical to PNG path) which is present in BOTH compared commits.
single_pass_stitch is deliberately NOT set here so each version uses its own default (prod: two-
pass; RC: single-pass #531) -- that delta is what we measure.",
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
```

```
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [35, 40, 40],
            "court_green_upper": [85, 255, 255],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "wasb_hybrid",
        "detector_impl": "auto",
        "skip_ai_handoff": true,
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "min_action_window_duration": 2.0,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 1,
        "wasb": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/WASB-SBDT",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "wasb",
            "weights_path": "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
            "sport": "badminton",
            "wsl_stage_dir": "~/clips",
            "timeout_sec": 3600,
            "velocity_threshold": 0.02,
            "device": "cuda",
            "stream_video": true,
            "keep_frames": false
        }
    },
    "output": {
        "default_highlights_name": "highlights.mp4",
        "db_name": "sports_indexer.db"
    },
    "stitching": {
        "begin_padding": 0.5,
        "end_padding": 0.5,
        "use_cache": false,
        "min_shot_count": 1,
        "encoder": "libx264",
        "watermark": {
            "enabled": true,
            "text": "Generated by Khelsutra.guru",
            "position": "bottom-right",
            "opacity": 0.85,
            "logo_path": "",
            "font_path": ""
        }
    }
}
```


78. user (2026-07-09T02:56:22.649Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\bench_config.json (file state is current in your context ? no need to Read it back)

79. assistant (2026-07-09T02:56:30.819Z)

Now the single-run harness ? version-agnostic, times all four stages, samples GPU during the run, and stitches the entire reel via introspected compiler construction so it runs each version's real stitch path.

80. assistant (2026-07-09T02:57:21.214Z)

```
#!/usr/bin/env python3
"""Single-run per-stage performance harness for the badminton highlight indexer.

Runs ONE (video, code-version) end-to-end with NO SPA / NO user step and times the
four release-critical stages independently:

    checksum  ->  validation  ->  segmentation  ->  full-reel stitch

Design notes
------------
* Version-agnostic: it is invoked with ``--worktree <dir>`` and inserts that dir at the
  front of ``sys.path`` BEFORE importing ``backend`` (a namespace package), so ``import
  backend`` resolves to that checkout. The same harness file therefore runs against both
  the deployed prod commit and the release candidate without editing either worktree.
* Only stable low-level APIs are called (compute_video_id, registry.run_all_with_context,
  segmenter.process_video, VideoCompiler). The compiler is built by INTROSPECTION so a
  version whose ctor lacks ``single_pass``/``encode_profile`` (prod 304af7b) is driven on
  its own default path while the RC uses its ``single_pass_stitch`` default (#531).
* Everything is forced local + deterministic by the effective config the driver passes
  (wasb_hybrid, detector_impl=auto WSL-GPU, skip_ai_handoff=true, stream_video=true).
* Validation is TIMED but never halts the run -- the whole point is to always reach the
  full-reel stitch (as requested), even for a clip that fails a quality gate.

Output: a single JSON object (also written to --json-out) with per-stage seconds, the
segment count, reel facts, GPU utilisation peak, and any per-stage error.
"""
import argparse
import json
import os
import subprocess
import sys
import threading
import time
import traceback
from datetime import datetime, timezone


def _iso_now() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def ffprobe_meta(path: str) -> dict:
    """(width,height,fps,duration_s,size_bytes,codec) via ffprobe; best-effort."""
    meta = {"size_bytes": os.path.getsize(path) if os.path.exists(path) else None}
    try:
        out = subprocess.run(
            [
                "ffprobe", "-v", "error", "-select_streams", "v:0",
                "-show_entries", "stream=width,height,r_frame_rate,codec_name",
                "-show_entries", "format=duration",
                "-of", "json", path,
            ],
            capture_output=True, text=True, timeout=60,
        )
```

```
        j = json.loads(out.stdout or "{}")
        st = (j.get("streams") or [{}])[0]
        meta["width"] = st.get("width")
        meta["height"] = st.get("height")
        meta["codec"] = st.get("codec_name")
        rfr = st.get("r_frame_rate", "0/1")
        try:
            n, d = rfr.split("/")
            meta["fps"] = round(float(n) / float(d), 3) if float(d) else None
        except Exception:
            meta["fps"] = None
        dur = (j.get("format") or {}).get("duration")
        meta["duration_s"] = round(float(dur), 3) if dur else None
    except Exception as e:  # pragma: no cover - best effort
        meta["ffprobe_error"] = str(e)
    return meta


class GpuSampler:
    """Polls Windows nvidia-smi (which sees the WSL2 CUDA process on the same physical
    GPU) at 1 Hz and keeps the peak utilisation / memory-used across the run."""

    def __init__(self, interval: float = 1.0):
        self.interval = interval
        self._stop = threading.Event()
        self._t = None
        self.util_max = 0
        self.mem_used_max_mib = 0
        self.samples = 0

    def _run(self):
        while not self._stop.is_set():
            try:
                out = subprocess.run(
                    ["nvidia-smi", "--query-gpu=utilization.gpu,memory.used",
                     "--format=csv,noheader,nounits"],
                    capture_output=True, text=True, timeout=8,
                )
                line = (out.stdout or "").strip().splitlines()
                if line:
                    util, mem = line[0].split(",")
                    self.util_max = max(self.util_max, int(util.strip()))
                    self.mem_used_max_mib = max(self.mem_used_max_mib, int(mem.strip()))
                    self.samples += 1
            except Exception:
                pass
            self._stop.wait(self.interval)

    def __enter__(self):
        self._t = threading.Thread(target=self._run, daemon=True)
        self._t.start()
        return self

    def __exit__(self, *a):
        self._stop.set()
        if self._t:
            self._t.join(timeout=5)


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--worktree", required=True, help="Repo checkout to import backend from.")
    ap.add_argument("--config", required=True, help="Template benchmark config JSON.")
    ap.add_argument("--video", required=True)
    ap.add_argument("--name", required=True, help="Short video label.")
```

```
ap.add_argument("--version", required=True, help="Version label, e.g. prod / rc.")
ap.add_argument("--commit", default="", help="Commit sha of the worktree.")
ap.add_argument("--run-dir", required=True, help="Isolated output dir for this run.")
ap.add_argument("--json-out", required=True)
ap.add_argument("--max-frames", type=int, default=None)
args = ap.parse_args()

# --- version isolation: import backend from the worktree, not the harness dir ---
sys.path.insert(0, os.path.abspath(args.worktree))
os.makedirs(args.run_dir, exist_ok=True)

rec = {
    "version": args.version,
    "commit": args.commit,
    "video": args.name,
    "video_path": args.video,
    "worktree": args.worktree,
    "timestamp": _iso_now(),
    "stages": {},
    "errors": {},
    "video_meta": ffprobe_meta(args.video),
}

def emit():
    with open(args.json_out, "w", encoding="utf-8") as f:
        json.dump(rec, f, indent=2)

# --- effective config: template + isolated output_dir, so runs never collide ---
with open(args.config, "r", encoding="utf-8") as f:
    cfg_dict = json.load(f)
cfg_dict.setdefault("output", {})["output_dir"] = args.run_dir
eff_path = os.path.join(args.run_dir, "effective_config.json")
with open(eff_path, "w", encoding="utf-8") as f:
    json.dump(cfg_dict, f, indent=2)

try:
    from backend.config import load_config
    from backend.utils.hashing import compute_video_id
    import backend.pipeline.validators as validators_pkg
    from backend.pipeline.segmenters import segmenter_registry
    from backend.infrastructure.database import Database
    from backend.pipeline.stitcher.compiler import VideoCompiler
    try:
        from backend.results import unwrap_or_failure
    except Exception:
        unwrap_or_failure = None
    import inspect
except Exception:
    rec["errors"]["import"] = traceback.format_exc()
    emit()
    print("IMPORT FAILED", file=sys.stderr)
    return 2

rec["backend_dir"] = getattr(sys.modules["backend"], "__path__", ["?"])[0]
cfg = load_config(eff_path)

# capture GPU/env facts once
try:
    import torch
    rec["env"] = {
        "torch": torch.__version__,
        "cuda_available": bool(torch.cuda.is_available()),
        "gpu": (torch.cuda.get_device_name(0) if torch.cuda.is_available() else None),
    }
except Exception:
```

```
        rec["env"] = {}

    db_path = os.path.join(args.run_dir, "bench.db")
    if os.path.exists(db_path):
        os.remove(db_path)
    db = Database(db_path)

    with GpuSampler() as gpu:
        # 1) CHECKSUM ------------------------------------------------------
        t = time.perf_counter()
        try:
            video_id = compute_video_id(args.video)
            rec["stages"]["checksum_s"] = round(time.perf_counter() - t, 3)
            rec["video_id"] = video_id
        except Exception:
            rec["stages"]["checksum_s"] = round(time.perf_counter() - t, 3)
            rec["errors"]["checksum"] = traceback.format_exc()
            emit()
            return 3

        # 2) VALIDATION (timed, never halts) -------------------------------
        t = time.perf_counter()
        try:
            val_results, _ctx = validators_pkg.registry.run_all_with_context(args.video, cfg)
            rec["stages"]["validation_s"] = round(time.perf_counter() - t, 3)
            rec["validation"] = {
                "passed": all(getattr(r, "passed", False) for r in val_results),
                "failed": [getattr(r, "name", "?") for r in val_results if not getattr(r,
"passed", False)],
                "n_checks": len(val_results),
            }
        except Exception:
            rec["stages"]["validation_s"] = round(time.perf_counter() - t, 3)
            rec["errors"]["validation"] = traceback.format_exc()

        # 3) SEGMENTATION (wasb_hybrid, WSL GPU) ----------------------------
        seg_name = cfg.indexing.default_segmenter
        rec["segmenter"] = seg_name
        try:
            db.clear_video_data(video_id)
        except Exception:
            pass
        try:
            db.add_video(video_id, args.video, "processed", [])
        except Exception:
            rec["errors"]["add_video"] = traceback.format_exc()

        segments = []
        t = time.perf_counter()
        try:
            seg_cls = segmenter_registry.get(seg_name)
            segmenter = seg_cls(db, cfg)
            result = segmenter.process_video(video_id, args.video, max_frames=args.max_frames)
            rec["stages"]["segmentation_s"] = round(time.perf_counter() - t, 3)
            if unwrap_or_failure is not None:
                segments, failure = unwrap_or_failure(result)
                if failure is not None:
                    rec["errors"]["segmentation"] = getattr(failure, "message", str(failure))
                    segments = []
            else:
                segments = result if isinstance(result, list) else []
        except Exception:
            rec["stages"]["segmentation_s"] = round(time.perf_counter() - t, 3)
            rec["errors"]["segmentation"] = traceback.format_exc()
```

```
        try:
            segments_all = db.query_play_segments(video_id, {})
        except Exception:
            segments_all = segments or []
        rec["segments"] = len(segments_all)

        # 4) FULL-REEL STITCH (all rallies, no user step) --------------------
        time_pairs = [(s["start_time"], s["end_time"]) for s in segments_all]
        rec["stitch_input_segments"] = len(time_pairs)
        if not time_pairs:
            rec["stages"]["stitch_s"] = None
            rec["errors"].setdefault("stitch", "no segments to stitch")
        else:
            params = inspect.signature(VideoCompiler.__init__).parameters
            kwargs = dict(
                end_padding=cfg.stitching.end_padding,
                begin_padding=cfg.stitching.begin_padding,
                watermark=cfg.stitching.watermark,
            )
            if "encoder" in params:
                kwargs["encoder"] = getattr(cfg.stitching, "encoder", "libx264")
            # Mirror orchestration._stitch_reel: prod ctor lacks single_pass -> its default
            # two-pass path; RC gets single_pass_stitch (default True, #531).
            if "single_pass" in params:
                kwargs["single_pass"] = bool(getattr(cfg.stitching, "single_pass_stitch",
False))
            rec["stitch_kwargs"] = {k: (v if k != "watermark" else "?") for k, v in
kwargs.items()}
            t = time.perf_counter()
            try:
                compiler = VideoCompiler(args.run_dir, **kwargs)
                out_path = compiler.compile_highlights(args.video, time_pairs, "highlights.mp4")
                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
                rm = ffprobe_meta(out_path)
                rec["reel"] = {
                    "path": out_path,
                    "size_bytes": rm.get("size_bytes"),
                    "duration_s": rm.get("duration_s"),
                }
            except Exception:
                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
                rec["errors"]["stitch"] = traceback.format_exc()

    rec["gpu"] = {
        "util_max_pct": gpu.util_max,
        "mem_used_max_mib": gpu.mem_used_max_mib,
        "samples": gpu.samples,
    }
    # total across the timed stages that ran
    total = 0.0
    for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s"):
        v = rec["stages"].get(k)
        if isinstance(v, (int, float)):
            total += v
    rec["total_s"] = round(total, 3)
    emit()

    s = rec["stages"]
    print(
        f"[{args.version}/{args.name}] checksum={s.get('checksum_s')}s "
        f"validation={s.get('validation_s')}s segmentation={s.get('segmentation_s')}s "
        f"stitch={s.get('stitch_s')}s total={rec['total_s']}s "
        f"segments={rec.get('segments')} gpu_peak={rec['gpu']['util_max_pct']}%",
        file=sys.stderr,
    )
```

```
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

81. user (2026-07-09T02:57:21.256Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_harness.py (file state is
current in your context ? no need to Read it back)
```

82. assistant (2026-07-09T02:57:48.582Z)

The harness is written. Before the long matrix, let me **smoke-test it on the shortest clip** (GX010016, 47s 1080p) against the RC worktree, foreground ? this validates the whole path (worktree import isolation, config, WSL wasb GPU segmentation, full-reel stitch) fast, before committing to the ~hour-long full run.

83. user (2026-07-09T02:57:58.210Z)

```
=== SMOKE: rc @ GX010016 (47s 1080p) ===
[CONFIG WARNING] C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\744270aa-6a38-4011-b018-
cdd9aae94849\scratchpad\perfbench\runs\smoke_rc_GX010016\effective_config.json contains keys the
typed config does not recognize (typo'd or removed knobs ? top-level extras are kept but unread;
unknown section keys are silently dropped): _comment
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_harness.py", line 316, in
<module>
    raise SystemExit(main())
                     ~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_harness.py", line 174, in
main
    cfg = load_config(eff_path)
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-ind
exer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt\rc\backend\config\loader.py",
line 236, in load_config
    return AppConfig.model_validate(merged)
           ~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\pydantic\main.py", line 732, in model_validate
    return cls.__pydantic_validator__.validate_python(
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        obj,
        ^^^^
    ...<5 lines>...
        by_name=by_name,
        ^^^^^^^^^^^^^^^^
    )
    ^
pydantic_core._pydantic_core.ValidationError: 1 validation error for AppConfig
_comment
  Extra inputs are not permitted [type=extra_forbidden, input_value='Canonical release
perf-b...lta is what we measure.', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/extra_forbidden
=== EXIT: 0 ===
--- result.json (stages) ---
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import json;r=json.load(open(r'/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
```

```
Projects-badminton-highlight-indexer/744270aa-6a38-4011-b018-
cdd9aae94849/scratchpad/perfbench/runs/smoke_rc_GX010016/result.json'));import
pprint;pprint.pprint({k:r.get(k) for k in
('version','segments','stages','total_s','validation','reel','gpu','errors','segmenter')})
                            ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory:
'/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/74
4270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/runs/smoke_rc_GX010016/result.json'
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

84. assistant (2026-07-09T02:58:38.085Z)

Two fixes needed: (1) the top-level `_comment` key trips `AppConfig`'s strict validation ? I'll drop it (docs go in a separate NOTES file); (2) the harness must emit its JSON even when config-load fails, so failures are diagnosable. Let me fix both.

85. user (2026-07-09T02:58:38.466Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\bench_config.json has been updated successfully. (file state is current in your context ? no need to Read it back)

86. user (2026-07-09T02:58:51.126Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_harness.py has been updated successfully. (file state is current in your context ? no need to Read it back)

87. assistant (2026-07-09T02:58:56.045Z)

Now re-run the smoke on the shortest clip.

88. user (2026-07-09T03:01:07.717Z)

```
=== SMOKE (retry): rc @ GX010016 (47s 1080p) ===
[rc-smoke/GX010016] checksum=0.081s validation=4.907s segmentation=106.884s stitch=1.547s
total=113.419s segments=1 gpu_peak=61%
=== python exit: 0 ===
--- result.json ---
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import json;r=json.load(open(r'/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-indexer/744270aa-6a38-4011-b018-
cdd9aae94849/scratchpad/perfbench/runs/smoke_rc_GX010016/result.json'));import
pprint;pprint.pprint({k:r.get(k) for k in ('version','backend_dir','env','segments','stitch_inpu
t_segments','stages','total_s','validation','reel','gpu','errors','segmenter','stitch_kwargs')})
                            ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory:
'/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/74
4270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/runs/smoke_rc_GX010016/result.json'
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

89. assistant (2026-07-09T03:01:48.728Z)

The harness **worked** ? the summary line printed: `checksum=0.081s validation=4.907s segmentation=106.884s stitch=1.547s total=113.4s segments=1 gpu_peak=61%`. The read-back failed only because I passed a Git-Bash `/c/...` path into a Python string literal (MSYS converts argv, not string literals) ? the file is at the correct Windows location. Let me read it properly to confirm the full record (reel + that RC used `single_pass=True`).

90. user (2026-07-09T03:01:49.764Z)


```
{
  "version": "rc-smoke",
  "commit": "3249451",
  "video": "GX010016",
  "video_path": "C:/Users/avidu/Videos/GX010016.MP4",
  "worktree": "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/rc",
  "timestamp": "2026-07-09T02:59:02+00:00",
  "stages": {
    "checksum_s": 0.081,
    "validation_s": 4.907,
    "segmentation_s": 106.884,
    "stitch_s": 1.547
  },
  "errors": {},
  "video_meta": {
    "size_bytes": 47485464,
    "width": 1920,
    "height": 1080,
    "codec": "hevc",
    "fps": 29.97,
    "duration_s": 47.147
  },
  "backend_dir": "C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-
Projects-badminton-highlight-
indexer\\744270aa-6a38-4011-b018-cdd9aae94849\\scratchpad\\perfbench\\wt\\rc\\backend",
  "env": {
    "torch": "2.6.0+cu124",
    "cuda_available": true,
    "gpu": "NVIDIA GeForce RTX 2070 Super with Max-Q Design"
  },
  "video_id": "047e1135fe52de3225977263fa149750",
  "validation": {
    "passed": false,
    "failed": [
      "?",
      "?"
    ],
    "n_checks": 6
  },
  "segmenter": "wasb_hybrid",
  "segments": 1,
  "stitch_input_segments": 1,
  "stitch_kwargs": {
    "end_padding": 0.5,
    "begin_padding": 0.5,
    "watermark": "\u2026",
    "encoder": "libx264",
    "single_pass": true
  },
  "reel": {
    "path": "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-
highlight-indexer/744270aa-6a38-4011-b018-
cdd9aae94849/scratchpad/perfbench/runs/smoke_rc_GX010016\\highlights.mp4",
    "size_bytes": 3850472,
    "duration_s": 3.203
  },
  "gpu": {
    "util_max_pct": 61,
    "mem_used_max_mib": 6159,
    "samples": 106
  },
  "total_s": 113.419
```

```
59      }
```

91. assistant (2026-07-09T03:02:14.442Z)

Smoke fully validates the design: backend imported from the **rc worktree**, wasb ran on the **WSL GPU** (peak 61%, 6.2 GB), RC engaged **`single_pass=true`** (#531), reel produced. One cosmetic gap: validator names show as `"?"` ? the result object's name attribute differs from my guess. Let me find the right attribute so the report can name which check failed, then build the driver.

92. user (2026-07-09T03:02:19.368Z)

```
===ValidationResult fields===
---- ValidationResult
class ValidationResult(BaseModel):
    validator_name: str
    passed: bool
    message: str
    details: dict[str, Any] = {}
```

93. assistant (2026-07-09T03:02:25.109Z)

The attribute is `validator_name`. Let me fix the harness to capture real validator names (and messages for failures).

94. user (2026-07-09T03:02:29.716Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_harness.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. assistant (2026-07-09T03:03:37.223Z)

Fixed. The design is fully validated end-to-end. Now the driver that runs the full matrix (2 versions × 4 videos, sequential ? GPU is a single resource), writes results incrementally so partial data survives, and diffs into a baseline + report. I'll invoke the harness via subprocess with native Windows paths (avoiding the MSYS path pitfall) and order videos ascending-cost so the heavy 5-min clip runs last.

96. assistant (2026-07-09T03:04:22.805Z)

```
#!/usr/bin/env python3
"""Release performance-comparison driver: HEAD (release candidate) vs the deployed
production binary, run LOCALLY on this GPU over the same vault videos.

For each (video, version) it invokes perf_harness.py against that version's isolated
git worktree, timing checksum -> validation -> segmentation -> full-reel stitch. Results
are written incrementally (so a long run's partial data survives) and then diffed into a
baseline JSON + a human-readable REPORT.md with per-stage deltas.

Runs are sequential: wasb segmentation owns the single GPU, so parallelism is impossible.
Video order is ascending-cost so the cheap clips produce numbers first and the heavy
5-min 5.3K clip runs last.
"""
import argparse
import json
import os
import subprocess
import sys
import time
from datetime import datetime, timezone

HERE = os.path.dirname(os.path.abspath(__file__))
HARNESS = os.path.join(HERE, "perf_harness.py")
```

```
DEFAULT_CONFIG = os.path.join(HERE, "bench_config.json")
```

(name, Windows path) -- ascending cost

```
DEFAULT_VIDEOS = [
    ("GX010016", r"C:\Users\avidu\Videos\GX010016.MP4"),                 # 47s  1080p
    ("SmokeTestBadmintonClip", r"C:\Users\avidu\Videos\SmokeTestBadmintonClip.mp4"),  # 81s
1080p60
    ("GX010075", r"C:\Users\avidu\Videos\GX010075.MP4"),                 # 89s  5.3K 16:9
    ("GX010061", r"C:\Users\avidu\Videos\GX010061.MP4"),                 # 5min 5.3K
]
```

label -> (worktree, commit). prod == deployed api.khelsutra.guru binary.

```
DEFAULT_VERSIONS = [
    ("prod", os.path.join(HERE, "wt", "prod"), "304af7b"),
    ("rc",   os.path.join(HERE, "wt", "rc"),   "3249451"),
]


def _iso_now() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def log(msg: str, logf: str) -> None:
    line = f"{_iso_now()}  {msg}"
    print(line, flush=True)
    with open(logf, "a", encoding="utf-8") as f:
        f.write(line + "\n")


def run_one(version, worktree, commit, name, video, cfg, runs_dir, logf, timeout):
    run_dir = os.path.join(runs_dir, version, name)
    os.makedirs(run_dir, exist_ok=True)
    json_out = os.path.join(run_dir, "result.json")
    cmd = [
        sys.executable, HARNESS,
        "--worktree", worktree,
        "--config", cfg,
        "--video", video,
        "--name", name,
        "--version", version,
        "--commit", commit,
        "--run-dir", run_dir,
        "--json-out", json_out,
    ]
    log(f"START {version}/{name}", logf)
    t0 = time.perf_counter()
    try:
        proc = subprocess.run(cmd, cwd=worktree, capture_output=True, text=True,
timeout=timeout)
        wall = time.perf_counter() - t0
        tail = "\n".join((proc.stderr or "").strip().splitlines()[-3:])
        log(f"DONE  {version}/{name} rc={proc.returncode} wall={wall:.1f}s :: {tail}", logf)
    except subprocess.TimeoutExpired:
        wall = time.perf_counter() - t0
        log(f"TIMEOUT {version}/{name} after {wall:.1f}s (limit {timeout}s)", logf)
    if os.path.exists(json_out):
        with open(json_out, "r", encoding="utf-8") as f:
            return json.load(f)
    return {"version": version, "video": name, "commit": commit, "errors": {"driver": "no
result.json"}}


def collect(results):
```

```
    """index results[version][video] = record"""
    idx = {}
    for r in results:
        idx.setdefault(r.get("version"), {})[r.get("video")] = r
    return idx


def fmt(v, unit="s"):
    return "?" if v is None else (f"{v:.2f}{unit}" if isinstance(v, (int, float)) else str(v))


def pct(prod, rc):
    if not isinstance(prod, (int, float)) or not isinstance(rc, (int, float)) or prod == 0:
        return None
    return round((rc - prod) / prod * 100.0, 1)


def write_report(idx, videos, versions, out_dir, meta):
    STAGES = [("checksum_s", "checksum"), ("validation_s", "validation"),
              ("segmentation_s", "segmentation"), ("stitch_s", "stitch"), ("total_s", "TOTAL")]
    vlabels = [v[0] for v in versions]
    lines = []
    lines.append("# Local release performance comparison ? HEAD (RC) vs deployed prod\n")
    lines.append(f"- Generated: {meta['generated']}")
    lines.append(f"- Prod (api.khelsutra.guru): `{meta['prod_commit']}`  ·  RC (origin/master):
`{meta['rc_commit']}`")
    lines.append(f"- GPU: {meta.get('gpu','?')}  ·  segmenter: wasb_hybrid (WSL2 GPU,
detector_impl=auto)  ·  encoder: libx264")
    lines.append(f"- Stitch: full reel (all rallies), watermark on. Prod=two-pass, RC=single-
pass (#531).")
    lines.append("- Delta% = (RC ? prod)/prod. Negative = RC faster.\n")

    for name, _path in videos:
        prod = idx.get("prod", {}).get(name, {})
        rc = idx.get("rc", {}).get(name, {})
        vm = (rc or prod).get("video_meta", {})
        lines.append(f"## {name}")
        res = f"{vm.get('width')}x{vm.get('height')}" if vm.get("width") else "?"
        lines.append(f"`{res}` · {fmt(vm.get('duration_s'))} · {vm.get('codec')} · "
                     f"{round((vm.get('size_bytes') or 0)/1048576)} MB  ·  "
                     f"segments prod={prod.get('segments','?')} rc={rc.get('segments','?')}  ·
"
                     f"GPU peak prod={prod.get('gpu',{}).get('util_max_pct','?')}%
rc={rc.get('gpu',{}).get('util_max_pct','?')}%")
        # validation verdict
        pv = prod.get("validation", {})
        if pv:
            fails = ", ".join(f.get("check", "?") for f in pv.get("failed", [])) or "none"
            lines.append(f"_validation: {'PASS' if pv.get('passed') else 'FAIL'} (failed:
{fails}) ? still segmented+stitched by design_")
        lines.append("")
        lines.append("| stage | prod | rc | ? | ?% |")
        lines.append("|---|---|---|---|---|")
        for key, label in STAGES:
            pv_ = prod.get("stages", {}).get(key) if key != "total_s" else prod.get("total_s")
            rv_ = rc.get("stages", {}).get(key) if key != "total_s" else rc.get("total_s")
            d = (rv_ - pv_) if isinstance(pv_, (int, float)) and isinstance(rv_, (int, float))
else None
            p = pct(pv_, rv_)
            lines.append(f"| {label} | {fmt(pv_)} | {fmt(rv_)} | {fmt(d)} | {fmt(p,'%') if p is
not None else '?'} |")
        for who, rr in (("prod", prod), ("rc", rc)):
            if rr.get("errors"):
                lines.append(f"\n> {who} errors: `{json.dumps(rr['errors'])[:400]}`")
        lines.append("")
```

```
    # aggregate stitch + segmentation summary
    lines.append("## Headline\n")
    for key, label in (("stitch_s", "stitch"), ("segmentation_s", "segmentation"),
("validation_s", "validation")):
        ps = [idx.get("prod", {}).get(n, {}).get("stages", {}).get(key) for n, _ in videos]
        rs = [idx.get("rc", {}).get(n, {}).get("stages", {}).get(key) for n, _ in videos]
        pair = [(a, b) for a, b in zip(ps, rs) if isinstance(a, (int, float)) and isinstance(b,
(int, float))]
        if pair:
            sp, sr = sum(a for a, _ in pair), sum(b for _, b in pair)
            lines.append(f"- **{label}** (sum over {len(pair)} clips): prod {sp:.1f}s ? rc
{sr:.1f}s  "
                         f"({pct(sp, sr):+.1f}%)")
    lines.append("")
    with open(os.path.join(out_dir, "REPORT.md"), "w", encoding="utf-8") as f:
        f.write("\n".join(lines))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default=DEFAULT_CONFIG)
    ap.add_argument("--out", default=os.path.join(HERE, "report"))
    ap.add_argument("--runs", default=os.path.join(HERE, "runs"))
    ap.add_argument("--timeout", type=int, default=5400, help="Per-run seconds (default
90min).")
    ap.add_argument("--only", default=None, help="Comma list of video names to restrict to.")
    args = ap.parse_args()

    os.makedirs(args.out, exist_ok=True)
    os.makedirs(args.runs, exist_ok=True)
    logf = os.path.join(args.out, "progress.log")

    videos = DEFAULT_VIDEOS
    if args.only:
        want = set(args.only.split(","))
        videos = [v for v in videos if v[0] in want]
    versions = DEFAULT_VERSIONS

    gpu = "?"
    try:
        gpu = subprocess.run(["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
                             capture_output=True, text=True,
timeout=10).stdout.strip().splitlines()[0]
    except Exception:
        pass

    log(f"MATRIX start · videos={[v[0] for v in videos]} · versions={[v[0] for v in versions]}",
logf)
    results = []
    # per video: run both versions back-to-back (tight pairing)
    for name, video in videos:
        if not os.path.exists(video):
            log(f"SKIP {name}: video not found at {video}", logf)
            continue
        for version, worktree, commit in versions:
            rec = run_one(version, worktree, commit, name, video, args.config, args.runs, logf,
args.timeout)
            results.append(rec)
            # incremental persist
            with open(os.path.join(args.out, "raw_results.json"), "w", encoding="utf-8") as f:
                json.dump(results, f, indent=2)

    idx = collect(results)
    meta = {
        "generated": _iso_now(),
```

```
        "prod_commit": versions[0][2],
        "rc_commit": versions[1][2],
        "gpu": gpu,
    }
    # baseline: compact per-video per-version stage table
    baseline = {"meta": meta, "runs": {}}
    for name, _ in videos:
        baseline["runs"][name] = {}
        for version, _wt, commit in versions:
            r = idx.get(version, {}).get(name, {})
            baseline["runs"][name][version] = {
                "commit": commit,
                "stages": r.get("stages", {}),
                "total_s": r.get("total_s"),
                "segments": r.get("segments"),
                "gpu_peak_pct": r.get("gpu", {}).get("util_max_pct"),
                "validation_passed": r.get("validation", {}).get("passed"),
                "errors": list(r.get("errors", {}).keys()),
            }
    with open(os.path.join(args.out, "baseline.json"), "w", encoding="utf-8") as f:
        json.dump(baseline, f, indent=2)
    write_report(idx, videos, versions, args.out, meta)
    log(f"MATRIX done · report at {os.path.join(args.out, 'REPORT.md')}", logf)


if __name__ == "__main__":
    raise SystemExit(main())
```

97. user (2026-07-09T03:04:22.846Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_driver.py (file state is current in your context ? no need to Read it back)

98. assistant (2026-07-09T03:04:45.199Z)

Harness and worktrees are done; driver is ready. Let me update task state and launch the full matrix in the **background** (it's ~1?1.5 hrs of sequential GPU work), writing progress incrementally so I can report partial results as they land.

99. user (2026-07-09T03:04:45.705Z)

Updated task #2 status

100. user (2026-07-09T03:04:46.204Z)

Updated task #3 status

101. user (2026-07-09T03:04:46.570Z)

Updated task #4 status

102. user (2026-07-09T03:04:57.636Z)

Command running in background with ID: bww3rlf2y. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bww3rlf2y.output. You will be notified when it completes. To check interim output, use Read on that file path.

103. assistant (2026-07-09T03:05:38.081Z)

The matrix is running in the background (ID `bww3rlf2y`). It'll take roughly an hour. Let me verify it launched cleanly and that the **prod** worktree path works too (the smoke only

exercised RC) before investing the full run.

104. user (2026-07-09T03:06:00.455Z)

```
=== progress.log ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip',
'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:04:58+00:00  START prod/GX010016
=== driver.out (tail) ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip',
'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:04:58+00:00  START prod/GX010016
=== nvidia-smi (is GPU busy?) ===
42 %, 5645 MiB
```

105. assistant (2026-07-09T03:06:33.070Z)

Matrix is live ? `START prod/GX010016`, GPU at 42%/5.6 GB (prod wasb segmentation running). Let me confirm the **prod** run completes cleanly (only RC was smoke-tested), then I'll build the repeatable wrapper. While the first prod run finishes (~2 min), let me write the methodology README.

106. assistant (2026-07-09T03:06:57.937Z)

Local release performance gate ? HEAD vs the deployed prod binary

Repeatable, GPU-local, per-stage timing comparison of the **release candidate** (`origin/master`) against the **binary currently serving `api.khelsutra.guru`**. It runs the real pipeline with **no SPA and no user step** and times the four release-critical stages independently:

```
checksum  ?  validation  ?  segmentation (wasb_hybrid, GPU)  ?  full-reel stitch
```

Run it before every release to get precomputed numbers and catch a per-stage regression.

Why it's built this way

* **Two versions, one machine, same inputs.** The comparison isolates *code* changes: it runs each commit from its own `git worktree`, over the same vault videos, with **one canonical config** (`bench_config.json`) so only the code differs. "Prod binary" can't be run literally (it's a Linux/L4/NVENC container), so we run the **prod commit** through the identical harness ? the only apples-to-apples local comparison.
* **Production detector on the GPU.** Segmentation uses `wasb_hybrid` with `detector_impl=auto` (the WSL2 GPU path) ? the same WASB shuttle detector that serves prod ? plus the fast `stream_video` decode (bit-identical to the PNG path, present in both compared commits, so it's fair). `skip_ai_handoff=true` keeps it deterministic and cloud-free (no Gemini/user step).
* **Real stitch paths.** The compiler is built by introspection, so the prod commit runs its native two-pass stitch and the RC runs its `single_pass_stitch` default (**#531**). That delta is the headline local-measurable change in the current release.
* **Always stitches the entire reel.** Every indexed rally is stitched (no `min_shot_count` filter, watermark on) ? exactly the requested "always stick the entire reels" behavior.
* **Validation is timed but never halts.** Even a clip that fails a quality gate is still segmented + stitched, so every clip yields all four numbers.

Files

File	Purpose
`perf_harness.py`	One (video, version) run ? per-stage JSON. Version-agnostic (imports `backend` from `--worktree`).

`perf_driver.py`	Runs the 2×N matrix sequentially, writes `report/baseline.json` + `report/REPORT.md` incrementally.
`bench_config.json`	Canonical, identical-for-both config (local wasb GPU, deterministic).
`run_release_perf_gate.ps1`	One-command wrapper: fetch ? resolve commits ? refresh worktrees ? run ? collect.

Usage

```powershell
# Default: prod = last deployed commit, RC = origin/master, the 4 vault clips.
pwsh scripts/perf_bench/run_release_perf_gate.ps1

# Pin the currently-deployed commit explicitly (recommended each release):
pwsh scripts/perf_bench/run_release_perf_gate.ps1 -ProdCommit <sha> -RcRef origin/master
```

Outputs land in `report/`: `REPORT.md` (per-stage table + deltas), `baseline.json` (machine-readable precomputed numbers to commit alongside the release), `progress.log`.

Determining the "prod" commit

The deployed Cloud Run image is **not** git-sha-labeled today (tracked in **#532**), so the wrapper takes `-ProdCommit` explicitly (defaulting to the last-known deployed commit). To confirm what's live:

```
gcloud run services describe rally-control-plane --region asia-southeast1 \
  --format="value(spec.template.spec.containers[0].image)"
```

then map that digest to its build commit via the deploy runlog / release record. When #532 lands (git-sha image labels), this wrapper can resolve it automatically.

Caveats (honest scope)

* This measures the **local, in-process** pipeline. Cloud-topology wins in the release (GCSFuse-mount source in the compile worker #535, CP `acquire_local_input` gating #539, cancel propagation #538) are **not** exercisable locally and won't show here.
* The GPU is an 8 GB RTX 2070 (prod is a 24 GB L4). Absolute segmentation times are not comparable to prod; the **prod-vs-RC delta on the same box** is the signal.
* Encoder is pinned to `libx264` both sides (portable, deterministic). NVENC timings would depend on the local GPU and confound the code comparison.

107. user (2026-07-09T03:06:58.005Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\README.md (file state is current in your context ? no need to Read it back)

108. assistant (2026-07-09T03:07:06.216Z)

Let me check whether the first prod run finished cleanly ? this confirms the prod worktree path works end-to-end.

109. user (2026-07-09T03:07:08.657Z)

```
=== progress.log ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip', 'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:04:58+00:00  START prod/GX010016
2026-07-09T03:06:46+00:00  DONE  prod/GX010016 rc=0 wall=107.7s :: [prod/GX010016] checksum=0.078s validation=5.543s segmentation=94.427s stitch=0.994s total=101.042s segments=1
```

```
gpu_peak=61%
2026-07-09T03:06:46+00:00  START rc/GX010016
```

110. assistant (2026-07-09T03:07:50.530Z)

Both worktrees work: `prod/GX010016` finished (`checksum=0.078s validation=5.543s segmentation=94.4s stitch=0.994s, 1 segment`), now running `rc/GX010016`. The matrix is healthy and progressing.

While it runs, let me finalize the repeatable tooling. First, a small backward-compatible enhancement to the driver so the wrapper can pass the resolved commit SHAs as report labels (safe to edit ? the running process already has its own loaded copy).

111. user (2026-07-09T03:07:54.305Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_driver.py has been updated successfully. (file state is current in your context ? no need to Read it back)

112. user (2026-07-09T03:08:03.869Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\perf_driver.py has been updated successfully. (file state is current in your context ? no need to Read it back)

113. assistant (2026-07-09T03:08:19.187Z)

Now the one-command PowerShell wrapper ? the "before every release" entrypoint that fetches, resolves commits, refreshes the worktrees, and runs the driver.

114. assistant (2026-07-09T03:08:38.243Z)

```
<#
.SYNOPSIS
  Local release performance gate: times checksum/validation/segmentation/full-reel-stitch
  for the release candidate (origin/master) vs the binary deployed on api.khelsutra.guru,
  on this machine's GPU, over the vault clips. Run before every release.

.EXAMPLE
  pwsh scripts/perf_bench/run_release_perf_gate.ps1
  pwsh scripts/perf_bench/run_release_perf_gate.ps1 -ProdCommit 304af7b -RcRef origin/master
  pwsh scripts/perf_bench/run_release_perf_gate.ps1 -Only GX010016,SmokeTestBadmintonClip
#>
[CmdletBinding()]
param(
  [string]$ProdCommit = "304af7b",                # commit serving api.khelsutra.guru (see
README: #532)
  [string]$RcRef      = "origin/master",          # release candidate
  [string]$RepoRoot   = "",                         # default: auto-detect from this script's
location
  [string]$Only       = "",                         # optional comma list of video names
  [int]$TimeoutSec    = 5400,
  [switch]$KeepWorktrees
)
$ErrorActionPreference = "Stop"
$Bench = $PSScriptRoot
if (-not $RepoRoot) {
  $cand = Resolve-Path (Join-Path $Bench "..\..") -ErrorAction SilentlyContinue
  if ($cand -and (Test-Path (Join-Path $cand ".git"))) { $RepoRoot = $cand.Path }
  else { $RepoRoot = "C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer" }
}
Write-Host "Repo:  $RepoRoot"
Write-Host "Bench: $Bench"
```

1) sync + resolve commits (read-only fetch; never auto-merge)

```
git -C $RepoRoot fetch origin --quiet
$RcCommit  = (git -C $RepoRoot rev-parse --short $RcRef).Trim()
$ProdShort = (git -C $RepoRoot rev-parse --short $ProdCommit).Trim()
Write-Host "Prod commit: $ProdShort   RC commit ($RcRef): $RcCommit"
```

2) informational: what image is actually live (digest is not git-sha-labeled yet, #532)

```
try {
  $img = gcloud run services describe rally-control-plane --region asia-southeast1 `
           --format="value(spec.template.spec.containers[0].image)" 2>$null
  if ($img) { Write-Host "Live control-plane image: $img" }
} catch { }
```

3) refresh the two worktrees (isolated; the main working tree is never touched)

```
$WtRoot = Join-Path $Bench "wt"
foreach ($pair in @(@("prod",$ProdShort), @("rc",$RcCommit))) {
  $wt = Join-Path $WtRoot $pair[0]
  if (Test-Path $wt) { git -C $RepoRoot worktree remove --force $wt 2>$null }
  git -C $RepoRoot worktree add --detach $wt $pair[1] | Out-Null
  Write-Host ("worktree {0} -> {1}" -f $pair[0], (git -C $wt rev-parse --short HEAD).Trim())
}
```

4) run the matrix

```
$driverArgs = @(
  (Join-Path $Bench "perf_driver.py"),
  "--config", (Join-Path $Bench "bench_config.json"),
  "--prod-commit", $ProdShort,
  "--rc-commit", $RcCommit,
  "--timeout", "$TimeoutSec"
)
if ($Only) { $driverArgs += @("--only", $Only) }
Write-Host "Running matrix..."
python @driverArgs
$rc = $LASTEXITCODE
```

5) cleanup worktrees unless asked to keep

```
if (-not $KeepWorktrees) {
  foreach ($v in @("prod","rc")) {
    $wt = Join-Path $WtRoot $v
    if (Test-Path $wt) { git -C $RepoRoot worktree remove --force $wt 2>$null }
  }
}
$report = Join-Path $Bench "report\REPORT.md"
Write-Host ""
Write-Host "Done (driver rc=$rc). Report: $report"
if (Test-Path $report) { Get-Content $report -Tail 12 }
exit $rc
```

115. user (2026-07-09T03:08:38.295Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\run_release_perf_gate.ps1
(file state is current in your context ? no need to Read it back)
```

116. assistant (2026-07-09T03:08:49.280Z)

```
Repeatable tooling is complete (harness, driver, config, PowerShell wrapper, README). Let me
check the matrix progress.
```

117. user (2026-07-09T03:08:51.719Z)

```
=== progress.log ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip',
'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:04:58+00:00  START prod/GX010016
2026-07-09T03:06:46+00:00  DONE  prod/GX010016 rc=0 wall=107.7s :: [prod/GX010016]
checksum=0.078s validation=5.543s segmentation=94.427s stitch=0.994s total=101.042s segments=1
gpu_peak=61%
2026-07-09T03:06:46+00:00  START rc/GX010016
2026-07-09T03:08:34+00:00  DONE  rc/GX010016 rc=0 wall=108.5s :: [rc/GX010016] checksum=0.079s
validation=5.695s segmentation=95.068s stitch=0.895s total=101.737s segments=1 gpu_peak=54%
2026-07-09T03:08:34+00:00  START prod/SmokeTestBadmintonClip
=== GPU ===
19 %, 1785 MiB
```

118. assistant (2026-07-09T03:09:32.293Z)

Paired GX010016 numbers now in: prod vs rc are near-identical except **stitch (prod 0.994s ? rc 0.895s)** and segmentation is effectively equal (94.4 vs 95.1s ? same WSL detector, as expected). The pattern is exactly right. Now running the Smoke clip.

Let me set task state and, while the matrix runs, gather the exact commit details to write the "summary of changes" the user also asked for.

119. user (2026-07-09T03:09:33.371Z)

Updated task #5 status

120. user (2026-07-09T03:09:38.877Z)

```
=== The 9-commit release delta (prod 304af7b -> RC origin/master), with stitch/compile stat ===
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
e29172a perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539) (#545)
af54f2f fix(#538): cancel propagates to the dispatched L4 worker exec (#544)
14e4767 observability: emit structured signal on GPU encoder fallback to CPU (#529) (#543)
b68af0d docs: add verified bootstrap for reproducing tests locally (#542)
8afbea9 perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)
(#541)
071d8b2 feat(compile): add opt-in SD reel rendition (#537)
f7a326d docs(runlogs): compile?L4 + NVENC live deploy-validation CUJ (2026-07-08) (#536)
5880c26 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521) (#531)
=== #531 single-pass stitch ? what it touched ===
    conservative primitive default); orchestration passes the config value explicitly.
 backend/config/models.py                          |  12 +
 backend/orchestration.py                          |   1 +
 backend/pipeline/stitcher/compiler.py             | 261 ++++++++++++++++++++--
 tests/test_api_endpoints.py                       |   7 +-
 tests/test_compile_dispatch.py                    |   2 +
 tests/test_compiler.py                            | 198 +++++++++++++++
 tests/test_watermark.py                           |   8 +
 8 files changed, 543 insertions(+), 20 deletions(-)
=== #535 GCSFuse + #539 acquire gating ? cloud-only? (files) ===
 backend/config/models.py        |  2 ++
 backend/orchestration.py        | 18 ++++++++++
 backend/storage/__init__.py     | 78 ++++++++++++++++++++++++++++++++++++++++++--
 backend/storage/capabilities.py | 14 ++++++++
 backend/worker/__main__.py      | 13 +++++--
 tests/test_storage.py           | 53 ++++++++++++++++++++++++++++++
 backend/orchestration.py | 86 +++++++++++++++++++++++++++++++++--------------
 1 file changed, 51 insertions(+), 35 deletions(-)
```

121. assistant (2026-07-09T03:09:55.061Z)

The change summary is now clear: only **#531 (single-pass stitch)** and **#537 (SD rendition)** touch the locally-exercised path; **#535/#539/#538/#534/#529** are cloud-topology/observability

(not locally measurable); the rest are docs/test. Let me check the matrix ? I want to confirm the first 5.3K clip (GX010075) starts without OOM on the 8 GB GPU.

122. user (2026-07-09T03:09:59.553Z)

```
=== progress.log (new lines) ===
2026-07-09T03:06:46+00:00  START rc/GX010016
2026-07-09T03:08:34+00:00  DONE  rc/GX010016 rc=0 wall=108.5s :: [rc/GX010016] checksum=0.079s validation=5.695s segmentation=95.068s stitch=0.895s total=101.737s segments=1 gpu_peak=54%
2026-07-09T03:08:34+00:00  START prod/SmokeTestBadmintonClip
=== GPU ===
46 %, 5603 MiB, 8192 MiB
```

123. assistant (2026-07-09T03:10:50.219Z)

The matrix is running healthily (GPU 46%/5.6 GB on the Smoke clip). Full run is ~8 sequential GPU passes; the two 5.3K clips (especially the 5-min GX010061) are the long pole, so I'll let it run and report final numbers when it completes. Here's where things stand.

Status: benchmark built and running

What "prod" resolves to (verified): the live control-plane (`rally-control-plane-00022-29w`) and worker both run image `sha256:7b62a8df?`, built from **`304af7b`**. `origin/master` (the RC) is **+9 commits** ahead at `3249451`.

The release delta, by local-measurability:
- **Locally measured here:** `#531` single-pass concat-filter stitch (`compiler.py` +261) ? the one real perf change this benchmark isolates; `#537` opt-in SD rendition (compiler ctor).
- **Cloud-only (can't reproduce on a local box):** `#535` GCSFuse source mount in the worker, `#539` gate CP `acquire_local_input`, `#538` cancel propagation, `#534`/`#529` telemetry.
- **Docs/tests:** `#536`, `#542`.

How I built the comparison (design decisions):
- Two `git worktree`s (`304af7b` vs `3249451`) driven by **one canonical config** ? same inputs, only code differs. Your working tree is untouched.
- Segmentation = **`wasb_hybrid` on the GPU via WSL2** (`detector_impl=auto`) ? the production shuttle detector ? with fast `stream_video` decode (present and identical in both commits, so it's fair; avoids a ~100 GB PNG blowup).
- **Deterministic & headless:** `skip_ai_handoff=true` (no Gemini/user step), always stitches the **entire reel** (every rally, watermark on), `libx264` both sides. Validation is timed but never halts.
- Compiler built by **introspection**, so prod runs its native two-pass stitch and RC runs its `single_pass_stitch` default ? each version's real path.

Early paired result ? GX010016 (47s 1080p, 1 rally):

stage	prod `304af7b`	rc `3249451`
checksum	0.078s	0.079s
validation	5.543s	5.695s
segmentation	94.43s	95.07s
stitch	**0.994s**	**0.895s**

Exactly the expected shape: checksum/validation/segmentation equal (same detector, same inputs), stitch marginally faster on RC even for a trivial 1-segment reel. The multi-rally and 5.3K clips will show the stitch delta more clearly.

Deliverables (in place, ready to land in-repo): `perf_harness.py`, `perf_driver.py`, `bench_config.json`, `run_release_perf_gate.ps1` (one-command wrapper), `README.md`. I'll report the full 4-video table + write `baseline.json` when the matrix finishes (~45?60 min; the 5-min 5.3K clip dominates), then offer to commit the tooling on a proper branch.

I'll check back when it completes ? or sooner if the 5.3K clips hit the 8 GB GPU ceiling.

124. user (2026-07-09T03:10:55.506Z)

```
=== progress.log (from Smoke onward) ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip',
'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:04:58+00:00  START prod/GX010016
2026-07-09T03:06:46+00:00  DONE  prod/GX010016 rc=0 wall=107.7s :: [prod/GX010016]
checksum=0.078s validation=5.543s segmentation=94.427s stitch=0.994s total=101.042s segments=1
gpu_peak=61%
2026-07-09T03:06:46+00:00  START rc/GX010016
2026-07-09T03:08:34+00:00  DONE  rc/GX010016 rc=0 wall=108.5s :: [rc/GX010016] checksum=0.079s
validation=5.695s segmentation=95.068s stitch=0.895s total=101.737s segments=1 gpu_peak=54%
2026-07-09T03:08:34+00:00  START prod/SmokeTestBadmintonClip
=== GPU now ===
43 %, 5211 MiB, 8192 MiB
```

125. assistant (2026-07-09T03:11:31.186Z)

`prod/SmokeTestBadmintonClip` is taking notably longer ? expected, actually: it's **81s @ 60fps ? 4,900 frames** (~3.5× GX010016's frame count), so segmentation naturally runs several minutes. GPU is active (43%/5.2 GB), not stuck. Let me confirm it's genuinely progressing (not wedged) with a quick activity sample and check the run dir, then back off to the completion notification.

126. user (2026-07-09T03:11:48.306Z)

```
=== GPU activity samples (5x over ~10s) ? confirm real compute ===
42 %, 5240 MiB
37 %, 5245 MiB
42 %, 5189 MiB
39 %, 5177 MiB
38 %, 5200 MiB
=== WSL: is a wasb/python inference process running? ===
/home/avidullu/.bash_profile: line 2: .bashrc: No such file or directory
your 131072x1 screen size is bogus. expect trouble
root         208  0.0  0.1 107028 22496 ?        Ssl  02:48   0:00 /usr/bin/python3
/usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
avidullu    8135  281 26.1 45397916 4250916 pts/3 Rsl+ 03:09   7:39 python wasb_infer.py --video
/home/avidullu/clips/SmokeTestBadmintonClip__3e670db58c79.mp4 --stream-video --weights
/home/avidullu/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar --sport badminton
--out /mnt/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-i
ndexer/744270aa-6a38-4011-b018-
cdd9aae94849/scratchpad/perfbench/runs/prod/SmokeTestBadmintonClip/wasb/SmokeTestBadmintonClip__
3e670db58c79_wasb.csv
=== prod/Smoke run dir contents (staging/progress) ===
total 104
drwxr-xr-x 1 avidu 197609     0 Jul  9 08:38 .
drwxr-xr-x 1 avidu 197609     0 Jul  9 08:38 ..
-rw-r--r-- 1 avidu 197609 98304 Jul  9 08:38 bench.db
-rw-r--r-- 1 avidu 197609  2065 Jul  9 08:38 effective_config.json
drwxr-xr-x 1 avidu 197609     0 Jul  9 08:38 wasb
=== WSL clips staging dir ===
/home/avidullu/.bash_profile: line 2: .bashrc: No such file or directory
total 131992
drwxr-xr-x 12 avidullu avidullu      4096 Jul  9 03:08 .
drwxr-x--- 27 avidullu avidullu      4096 Jul  7 09:58 ..
-rwxr-xr-x  1 avidullu avidullu 125410307 Jul  9 03:08 SmokeTestBadmintonClip__3e670db58c79.mp4
-rw-r--r--  1 avidullu avidullu       404 Jun  8 12:54 adarsh_avi_full_run.log
-rw-r--r--  1 avidullu avidullu   2476375 Jun  8 12:54 adarsh_avi_full_traj.csv
drwxr-xr-x  2 avidullu avidullu      4096 Jun  8 12:54 adarsh_avi_full_traj_wasbcache
-rw-r--r--  1 avidullu avidullu       391 Jun  8 08:54 adarsh_avi_run.log
-rw-r--r--  1 avidullu avidullu   1114525 Jun  8 08:54 adarsh_avi_traj.csv
```

127. assistant (2026-07-09T03:13:49.507Z)

It's genuinely progressing ? `wasb_infer.py` is running in WSL (281% CPU, GPU ~40%) using the **streaming decoder**, which is **CPU-decode-bound**. That's the key runtime insight: for 60fps and 5.3K clips the CPU decode dominates, so the two big 5.3K clips (especially the 5-min GX010061) will make this a multi-hour run. Segmentation is ~identical prod-vs-RC (same detector), so it's a constant backdrop ? the real signal is in the stitch stage.

Let me check whether the Smoke clip finished to firm up the ETA.

128. user (2026-07-09T03:13:56.113Z)

```
=== progress.log (Smoke onward) ===
2026-07-09T03:04:58+00:00  MATRIX start · videos=['GX010016', 'SmokeTestBadmintonClip',
'GX010075', 'GX010061'] · versions=['prod', 'rc']
2026-07-09T03:08:34+00:00  START prod/SmokeTestBadmintonClip
=== WSL wasb proc CPU-time (calibration) ===
your 131072x1 screen size is bogus. expect trouble
   8135     282 00:13:20  283 python wasb_infer.py --video
/home/avidullu/clips/SmokeTestBadmintonClip__3e670db58c79.mp4 --stream-video --weights
/home/avidullu/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar --sport badminton
--out /mnt/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-i
ndexer/744270aa-6a38-4011-b018-
cdd9aae94849/scratchpad/perfbench/runs/prod/SmokeTestBadmintonClip/wasb/SmokeTestBadmintonClip__
3e670db58c79_wasb.csv
```

129. assistant (2026-07-09T03:14:54.204Z)

Recalibrating the ETA changes the picture enough that I should check with you before burning hours of your machine's GPU. Here's the honest runtime reality I just learned:

The WSL wasb path uses a **CPU-bound streaming decoder** (prod is fast because the L4 does hardware NVDEC ? not reproducible on this Windows box). So segmentation time scales hard with resolution/fps:
- The **3 smaller clips** (GX010016 ? done, Smoke ~running, GX010075 5.3K) will finish in roughly **another ~45?60 min**.
- The **5-min 5.3K GX010061** alone is a **~2-hour** CPU-decode job for both versions ? pushing the total to ~3 hours.

Crucially, **segmentation is a constant backdrop** in this comparison (prod and RC use the identical WASB detector ? GX010016 already showed 94.4s vs 95.1s). The actual release signal lives in the **stitch** stage (#531), which I can measure at scale far more cheaply than a 2-hour segmentation run. So the 5-min clip costs ~2 hours for almost no added *comparison* signal.

130. user (2026-07-09T03:15:10.998Z)

Your questions have been answered: "The 5-min 5.3K clip (GX010061) is a ~2-hour local CPU-decode job and adds little comparison signal (segmentation is identical across versions). How should I handle it?"="Skip it + add stitch A/B". You can now continue with these answers in mind.

131. assistant (2026-07-09T03:17:17.743Z)

Good call ? that saves ~2 hours for essentially no loss of comparison signal. Plan: let the matrix finish the 3 smaller clips (GX010016 ?, Smoke, GX010075), **stop it before GX010061**, render the report from the incremental results, then run a fast dedicated **stitch A/B** (CPU-free, post-matrix) to capture #531 at scale.

Let me prepare the helper scripts and the monitoring/stop mechanism. First, let me load the Monitor tool (to efficiently watch for the GX010061 start) and write the stitch A/B + report-render helpers.

132. assistant (2026-07-09T03:17:52.844Z)

The `until`-loop-in-background is the right primitive to catch the GX010061 start with a single notification. Let me write the two helper scripts (report renderer + stitch A/B micro-bench) and arm the watcher ? all in parallel.

133. assistant (2026-07-09T03:18:03.395Z)

```
#!/usr/bin/env python3
"""Render REPORT.md + baseline.json from an incremental raw_results.json.

Used when the matrix is stopped early (e.g. we skip the 2-hour 5-min clip): the driver
only writes the report after ALL videos finish, so this regenerates it from whatever runs
completed. Reuses perf_driver.collect + write_report.
"""
import json
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import perf_driver as D  # noqa: E402

raw_path = sys.argv[1] if len(sys.argv) > 1 else os.path.join(D.HERE, "report",
"raw_results.json")
out_dir = sys.argv[2] if len(sys.argv) > 2 else os.path.join(D.HERE, "report")
os.makedirs(out_dir, exist_ok=True)

raw = json.load(open(raw_path, encoding="utf-8"))
idx = D.collect(raw)

videos, seen = [], set()
for r in raw:
    n = r.get("video")
    if n not in seen:
        seen.add(n)
        videos.append((n, r.get("video_path", "")))

prod_commit = next((r.get("commit") for r in raw if r.get("version") == "prod"), "prod")
rc_commit = next((r.get("commit") for r in raw if r.get("version") == "rc"), "rc")
versions = [("prod", "", prod_commit), ("rc", "", rc_commit)]
gpu = next((r.get("env", {}).get("gpu") for r in raw if r.get("env", {}).get("gpu")), "?")
meta = {"generated": D._iso_now(), "prod_commit": prod_commit, "rc_commit": rc_commit, "gpu":
gpu}

D.write_report(idx, videos, versions, out_dir, meta)

baseline = {"meta": meta, "runs": {}}
for name, _ in videos:
    baseline["runs"][name] = {}
    for version, _wt, commit in versions:
        r = idx.get(version, {}).get(name, {})
        baseline["runs"][name][version] = {
            "commit": commit,
            "stages": r.get("stages", {}),
            "total_s": r.get("total_s"),
            "segments": r.get("segments"),
            "gpu_peak_pct": r.get("gpu", {}).get("util_max_pct"),
            "validation_passed": r.get("validation", {}).get("passed"),
            "errors": list(r.get("errors", {}).keys()),
        }
json.dump(baseline, open(os.path.join(out_dir, "baseline.json"), "w", encoding="utf-8"),
indent=2)
print("rendered", os.path.join(out_dir, "REPORT.md"), "and baseline.json")
```

134. user (2026-07-09T03:18:03.457Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-

badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\render_report.py (file state
is current in your context ? no need to Read it back)

135. assistant (2026-07-09T03:18:21.979Z)

```
#!/usr/bin/env python3
"""Dedicated stitch micro-benchmark: isolate the #531 single-pass-vs-two-pass stitch delta.

Segmentation is a constant backdrop in the full matrix (same detector both versions), so the
only locally-measurable code change is the stitch path. This runs the SAME fixed reel (a list
of [start,end] rally pairs against a source video) through each version's compiler N times and
reports the median stitch wall-time -- prod runs its native two-pass watermark stitch, RC runs
single_pass_stitch (#531). Version-agnostic: imports backend from --worktree (run once per
version in its own subprocess so module caching can't cross versions).
"""
import argparse
import inspect
import json
import os
import statistics
import sys
import time


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--worktree", required=True)
    ap.add_argument("--config", required=True)
    ap.add_argument("--source", required=True)
    ap.add_argument("--pairs", required=True, help="JSON list of [start,end] seconds.")
    ap.add_argument("--reps", type=int, default=3)
    ap.add_argument("--version", required=True)
    ap.add_argument("--commit", default="")
    ap.add_argument("--run-dir", required=True)
    ap.add_argument("--json-out", required=True)
    args = ap.parse_args()

    sys.path.insert(0, os.path.abspath(args.worktree))
    os.makedirs(args.run_dir, exist_ok=True)

    # effective config with isolated output dir
    with open(args.config, encoding="utf-8") as f:
        cfg_dict = json.load(f)
    cfg_dict.setdefault("output", {})["output_dir"] = args.run_dir
    eff = os.path.join(args.run_dir, "effective_config.json")
    with open(eff, "w", encoding="utf-8") as f:
        json.dump(cfg_dict, f, indent=2)

    from backend.config import load_config
    from backend.pipeline.stitcher.compiler import VideoCompiler

    cfg = load_config(eff)
    pairs = [tuple(p) for p in json.loads(args.pairs)]

    params = inspect.signature(VideoCompiler.__init__).parameters
    kwargs = dict(
        begin_padding=cfg.stitching.begin_padding,
        end_padding=cfg.stitching.end_padding,
        watermark=cfg.stitching.watermark,
    )
    if "encoder" in params:
        kwargs["encoder"] = getattr(cfg.stitching, "encoder", "libx264")
    single_pass = None
    if "single_pass" in params:
```

```
        single_pass = bool(getattr(cfg.stitching, "single_pass_stitch", False))
        kwargs["single_pass"] = single_pass

    times = []
    reel_meta = {}
    for i in range(args.reps):
        compiler = VideoCompiler(args.run_dir, **kwargs)
        t = time.perf_counter()
        out = compiler.compile_highlights(args.source, pairs, f"reel_{args.version}_{i}.mp4")
        times.append(round(time.perf_counter() - t, 3))
        if i == 0 and out and os.path.exists(out):
            reel_meta = {"size_bytes": os.path.getsize(out)}

    rec = {
        "version": args.version,
        "commit": args.commit,
        "source": os.path.basename(args.source),
        "n_pairs": len(pairs),
        "reps": args.reps,
        "single_pass": single_pass,   # None => ctor lacks it (prod two-pass)
        "ctor_has_single_pass": "single_pass" in params,
        "times_s": times,
        "median_s": round(statistics.median(times), 3),
        "min_s": min(times),
        "reel": reel_meta,
    }
    with open(args.json_out, "w", encoding="utf-8") as f:
        json.dump(rec, f, indent=2)
    print(f"[stitch {args.version}] median={rec['median_s']}s times={times} "
          f"single_pass={single_pass} n_pairs={len(pairs)}", file=sys.stderr)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

136. user (2026-07-09T03:18:22.024Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\stitch_bench.py (file state is
current in your context ? no need to Read it back)
```

137. assistant (2026-07-09T03:18:41.116Z)

```
Now let me arm the watcher for the GX010061 start (single notification via a background
`until`-loop) and smoke-test the report renderer on the current partial results to catch bugs
early.
```

138. user (2026-07-09T03:18:43.654Z)

```
Command running in background with ID: b3vx0nxej. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\b3vx0nxej.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

139. user (2026-07-09T03:18:47.215Z)

```
=== smoke render_report on current partial raw_results.json ===
rendered C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/report_smoke\REPORT.md and
baseline.json
--- REPORT_smoke head ---
```

Local release performance comparison ? HEAD (RC) vs deployed prod

- Generated: 2026-07-09T03:18:47+00:00
- Prod (api.khelsutra.guru): `304af7b` · RC (origin/master): `3249451`
- GPU: NVIDIA GeForce RTX 2070 Super with Max-Q Design · segmenter: wasb_hybrid (WSL2 GPU, detector_impl=auto) · encoder: libx264
- Stitch: full reel (all rallies), watermark on. Prod=two-pass, RC=single-pass (#531).
- Delta% = (RC ? prod)/prod. Negative = RC faster.

GX010016

`1920x1080` · 47.15s · hevc · 45 MB · segments prod=1 rc=1 · GPU peak prod=61% rc=54%
_validation: FAIL (failed: scene_cut_check, relevance_check) ? still segmented+stitched by design_

stage	prod	rc	?	?%
checksum	0.08s	0.08s	0.00s	1.30%
validation	5.54s	5.70s	0.15s	2.70%
segmentation	94.43s	95.07s	0.64s	0.70%
stitch	0.99s	0.90s	-0.10s	-10.00%
TOTAL	101.04s	101.74s	0.69s	0.70%

SmokeTestBadmintonClip

`1920x1080` · 81.35s · h264 · 120 MB · segments prod=6 rc=? · GPU peak prod=64% rc=?%
validation: PASS (failed: none) ? still segmented+stitched by design

stage	prod	rc	?	?%
checksum	0.21s	?	?	?
validation	5.49s	?	?	?
segmentation	312.65s	?	?	?
stitch	14.85s	?	?	?
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

140. assistant (2026-07-09T03:19:22.534Z)

The renderer works ? and it reveals **prod/Smoke already finished with a meaningful reel**: `segmentation=312.65s, stitch=14.85s, 6 rallies, validation PASS`. That 14.85s two-pass stitch is exactly the kind of number where #531 will show; rc/Smoke (running now) will give the comparison. Validation names now render correctly too (`scene_cut_check, relevance_check`).

I won't run anything CPU-heavy while the matrix is timing segmentation (it would corrupt the numbers). Let me write the stitch A/B driver now so it's ready to fire the instant the matrix is stopped.

141. assistant (2026-07-09T03:19:41.901Z)

```
#!/usr/bin/env python3
"""Run the dedicated stitch A/B (prod vs RC) on fixed synthetic reels and write STITCH_AB.md.

Isolates the #531 single-pass stitch change from the (constant) segmentation backdrop. Uses
a 1080p source (fast, several reps -> clean median) and a 5.3K source (slow, fewer reps ->
absolute magnitude). MUST run with a free CPU (after the segmentation matrix is stopped),
because libx264 competes with wasb's CPU decode.
"""
import json
import os
import subprocess
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
BENCH = os.path.join(HERE, "stitch_bench.py")
CONFIG = os.path.join(HERE, "bench_config.json")
VERS = [("prod", os.path.join(HERE, "wt", "prod"), "304af7b"),
        ("rc", os.path.join(HERE, "wt", "rc"), "3249451")]
```

(label, source, pairs, reps)

```
CASES = [
    ("1080p_smoke", r"C:\Users\avidu\Videos\SmokeTestBadmintonClip.mp4",
     [[5, 10], [15, 20], [25, 30], [35, 40], [45, 50], [55, 60], [65, 70], [72, 77]], 3),
    ("5p3k_gx010075", r"C:\Users\avidu\Videos\GX010075.MP4",
     [[6, 10], [17, 21], [28, 32], [39, 43], [50, 54], [61, 65], [72, 76], [80, 84]], 1),
]


def run(case_label, source, pairs, reps):
    out = {}
    for version, wt, commit in VERS:
        run_dir = os.path.join(HERE, "stitch_runs", case_label, version)
        os.makedirs(run_dir, exist_ok=True)
        jo = os.path.join(run_dir, "result.json")
        cmd = [sys.executable, BENCH, "--worktree", wt, "--config", CONFIG,
               "--source", source, "--pairs", json.dumps(pairs), "--reps", str(reps),
               "--version", version, "--commit", commit, "--run-dir", run_dir, "--json-out", jo]
        print(f"  stitch {version}/{case_label} ...", flush=True)
        p = subprocess.run(cmd, cwd=wt, capture_output=True, text=True, timeout=3600)
        print("   ", (p.stderr or "").strip().splitlines()[-1] if p.stderr.strip() else
f"rc={p.returncode}", flush=True)
        if os.path.exists(jo):
            out[version] = json.load(open(jo, encoding="utf-8"))
    return out


def main():
    results = {}
    for label, source, pairs, reps in CASES:
        if not os.path.exists(source):
            print(f"skip {label}: {source} missing")
            continue
        print(f"CASE {label} ({os.path.basename(source)}, {len(pairs)} rallies, {reps} reps)")
        results[label] = run(label, source, pairs, reps)

    out_dir = os.path.join(HERE, "report")
    os.makedirs(out_dir, exist_ok=True)
    json.dump(results, open(os.path.join(out_dir, "stitch_ab.json"), "w"), indent=2)

    lines = ["# Stitch A/B ? #531 single-pass vs prod two-pass (fixed reels, CPU idle)\n",
             "Same reel stitched by each version's compiler; segmentation excluded. "
             "prod = native two-pass watermark stitch, RC = `single_pass_stitch` (#531). "
             "libx264, watermark on.\n",
             "| reel | source | rallies | reps | prod median | rc median | ? | ?% |",
             "|---|---|---|---|---|---|---|---|"]
    for label, source, pairs, reps in CASES:
        r = results.get(label, {})
        p, c = r.get("prod", {}), r.get("rc", {})
        pm, cm = p.get("median_s"), c.get("median_s")
        d = (cm - pm) if isinstance(pm, (int, float)) and isinstance(cm, (int, float)) else None
        pct = round((cm - pm) / pm * 100, 1) if isinstance(pm, (int, float)) and pm and
isinstance(cm, (int, float)) else None
        res = "5.3K" if "5p3k" in label else "1080p"
        lines.append(f"| {label} | {res} | {len(pairs)} | {reps} | "
                     f"{'?' if pm is None else f'{pm:.2f}s'} | {'?' if cm is None else
f'{cm:.2f}s'} | "
                     f"{'?' if d is None else f'{d:+.2f}s'} | {'?' if pct is None else
f'{pct:+.1f}%'} |")
        lines.append(f"|  | prod single_pass ctor={p.get('ctor_has_single_pass')}
times={p.get('times_s')} · "
                     f"rc single_pass={c.get('single_pass')} times={c.get('times_s')} ||||||||")
    open(os.path.join(out_dir, "STITCH_AB.md"), "w", encoding="utf-8").write("\n".join(lines))
    print("wrote", os.path.join(out_dir, "STITCH_AB.md"))
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

142. user (2026-07-09T03:19:41.952Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\stitch_driver.py (file state
is current in your context ? no need to Read it back)
```

143. assistant (2026-07-09T03:20:14.697Z)

```
All tooling is ready (matrix, renderer, stitch A/B, wrapper, docs). Now I wait for the watcher
to signal GX010061's start, at which point I stop the matrix and finalize. Let me verify my
stop-command's process filter matches the right processes (read-only check ? safe, won't kill
anything) so the stop is clean when the watcher fires.
```

144. user (2026-07-09T03:20:25.885Z)

```
ProcessId cmd
--------- ---
     6840 perf_driver.py
     3012 perf_harness.py --worktree C:\Users\avidu\AppData\Local\Temp

--- WSL wasb procs ---
546 bash -lc pgrep -af wasb_infer 2>/dev/null | head
```

145. assistant (2026-07-09T03:20:50.189Z)

```
Stop-command verified (matches only PID 6840 `perf_driver.py` + 3012 `perf_harness.py` ? safe,
no collateral). Everything is staged.

**State of play ? executing your "skip + stitch A/B" choice:**
- ? Tooling complete: matrix harness/driver, report renderer, **stitch A/B** bench, one-command
PowerShell wrapper, README.
- ? Results so far: `GX010016` (both), `prod/SmokeTestBadmintonClip` done ? **6 rallies, two-
pass stitch 14.85s** (validation PASS). `rc/Smoke` running now ? GX010075 (5.3K) next.
- ?? A watcher is armed to stop the matrix the moment it would start the 5-min GX010061; then I
run the stitch A/B (CPU idle) and render the final report.

I'm deliberately not running anything CPU-heavy now ? it would corrupt the segmentation timings
being recorded. I'll pick back up automatically when the 5.3K clip finishes and the watcher
fires (~40?60 min, dominated by the 5.3K CPU-decode), then deliver the per-stage table + the
#531 stitch A/B and offer to land the tooling in-repo.

I'll continue when the watcher signals.
```

146. user (2026-07-09T03:20:50.343Z)

```
Task #4: Run benchmark on 4 vault videos + produce report
Status: in_progress
Description: Videos: SmokeTestBadmintonClip, GX010075, GX010016, GX010061. Run prod vs RC, diff
per-stage, write a committed baseline JSON + human-readable report with precomputed release
numbers.
```

147. user (2026-07-09T03:21:05.846Z)

```
Updated task #4 description
```

148. assistant (2026-07-09T03:21:16.277Z)

```
Standing by for the watcher ? I'll resume automatically when GX010075 finishes and the matrix
reaches GX010061.
```

149. user (2026-07-09T03:21:18.766Z)

please consolidate all this in a repeatable, instruction documented, mostly self help, release gating and standalone tool which I can run to compare latest production with repo head anytime. Also similar in spirit to the nightly tests and maybe add this to the nightly framework as well so that it runs quality and performance tests.

150. assistant (2026-07-09T03:22:37.762Z)

This is the right move ? turn the prototype into a real, committed, gated, nightly-integrated tool. Before building, I need to understand the **existing nightly framework** so I integrate with it rather than bolt on a parallel thing. Let me map it thoroughly while the matrix keeps running (read-only, no CPU contention), and check how nightly is scheduled/gated.

151. user (2026-07-09T03:22:59.795Z)

Async agent launched successfully. (This tool result is internal metadata ? never quote or paste any part of it, including the agentId below, into a user-facing reply.)
agentId: ab2c45aad7352c9ed (internal ID - do not mention to user. Use SendMessage with to: 'ab2c45aad7352c9ed', summary: '<5-10 word recap>' to continue this agent.)
The agent is working in the background. You will be notified automatically when it completes.
Do not duplicate this agent's work ? avoid working with the same files or topics it is using.
output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\ab2c45aad7352c9ed.output
Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

152. user (2026-07-09T03:23:06.023Z)

```
=== MATRIX progress (is it still going?) ===
2026-07-09T03:08:34+00:00  DONE  rc/GX010016 rc=0 wall=108.5s :: [rc/GX010016] checksum=0.079s validation=5.695s segmentation=95.068s stitch=0.895s total=101.737s segments=1 gpu_peak=54%
2026-07-09T03:08:34+00:00  START prod/SmokeTestBadmintonClip
2026-07-09T03:14:15+00:00  DONE  prod/SmokeTestBadmintonClip rc=0 wall=340.3s :: [prod/SmokeTestBadmintonClip] checksum=0.206s validation=5.491s segmentation=312.651s stitch=14.854s total=333.202s segments=6 gpu_peak=64%
2026-07-09T03:14:15+00:00  START rc/SmokeTestBadmintonClip
2026-07-09T03:19:56+00:00  DONE  rc/SmokeTestBadmintonClip rc=0 wall=341.9s :: [rc/SmokeTestBadmintonClip] checksum=0.2s validation=5.543s segmentation=316.291s stitch=12.921s total=334.955s segments=6 gpu_peak=61%
2026-07-09T03:19:56+00:00  START prod/GX010075

=== nightly-related scripts in repo ===
nightly_email.py
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
=== any .ps1 wrappers ===
deploy/cloudrun/post_deploy_smoke.ps1
scripts/nightly_regression.ps1
scripts/register_nightly_task.ps1
=== eval_baselines/ ===
experiment_leaderboard.json
fixtures
gemini_wrapper_2026-06-10.json
heuristic_lovo_n15.json
heuristic_lovo_n6.json
nightly_baseline.json
reelcount_manifest.json
=== output nightly dirs ===
_phase1_regress.log
```

```
e2e_reel.mp4
nightly_regression
sd_reel_full_tests.log
```

153. assistant (2026-07-09T03:23:25.355Z)

```
Excellent ? two things landed at once:

1. **Matrix is ahead of where I thought**: `rc/Smoke` finished ? **stitch prod 14.85s ? rc
12.92s (?13%)** on the 6-rally 1080p reel. That's the #531 signal, clearly. Now on
`prod/GX010075` (5.3K).
2. **The nightly framework is real and richer than expected**: `scripts/nightly_regression.ps1`
(the wrapper the stub was waiting for ? now merged), `scripts/register_nightly_task.ps1`
(Windows scheduled-task registrar), `scripts/nightly_reelcount.py` (perf/cost),
`scripts/nightly_email.py`, plus `eval_baselines/*.json`. So there's a genuine orchestrator to
integrate with.

Let me read the orchestration/registrar `.ps1`s firsthand (central to integration) while the
Explore agent details the Python legs.
```

154. user (2026-07-09T03:23:26.170Z)

```
1       <#
2       .SYNOPSIS
3           Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5       .DESCRIPTION
6           Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7           (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8           dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9           with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11          GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12          before this PR is merged into the checkout the task points at) ? so a registered
task never
13          errors in the interim and self-activates once the files land.
14
15          Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16          this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18      .PARAMETER RepoRoot
19          Repo root to test. Default: the parent of this script's directory.
20
21      .PARAMETER PythonExe
22          Python interpreter. Default: 'python'.
23
24      .PARAMETER Pull
25          If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26          that may hold a sibling session's WIP).
27      #>
28      [CmdletBinding()]
29      param(
30          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31          [string]$PythonExe = 'python',
32          [switch]$Pull
33      )
34
35      $ErrorActionPreference = 'Continue'
```

```
36      $date = Get-Date -Format 'yyyy-MM-dd'
37      $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38      $logDir = Join-Path $reportDir 'logs'
39      New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40      $log = Join-Path $logDir "$date.log"
41      $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43      function Write-Status([string]$status, [int]$code, [string]$detail) {
44          $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45          "$stamp  status=$status exit=$code  $detail" | Out-File -FilePath $statusFile
-Encoding utf8
46      }
47
48      $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49      $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51      # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52      if (-not (Test-Path $checker)) {
53          "[$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
| Tee-Object -FilePath $log
54          Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
55          exit 0
56      }
57      if (-not (Test-Path $manifest)) {
58          "[$date] golden manifest not present at $manifest ? skipping (corpus absent on this
machine)." | Tee-Object -FilePath $log
59          Write-Status 'SKIPPED' 0 'golden corpus absent'
60          exit 0
61      }
62
63      Set-Location $RepoRoot
64      if ($Pull) {
65          "[$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
66          & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
67      }
68
69      $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
70      "[$date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
-Append
71
72      & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
73      $code = $LASTEXITCODE
74
75      switch ($code) {
76          0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
77          1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
78              "[$date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
Object -FilePath $log -Append }
79          4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
80          default { Write-Status 'ERROR' $code "checker error ? see $log" }
81      }
82
83      exit $code
84
```

155. assistant (2026-07-09T03:23:37.862Z)

That's the exact convention to match: a Scheduled-Task wrapper that writes `output/nightly_<leg>/<date>.{md,log}` + `LATEST_STATUS.txt` (`status=? exit=? detail`), exit codes 0=PASS/1=REGRESSION/4=STALE, graceful no-op when prereqs are absent. Let me read the task registrar to see how the schedule is wired (so I extend it cleanly rather than fork).

156. user (2026-07-09T03:23:38.991Z)

```
1       <#
```

```
2       .SYNOPSIS
3           Register (or remove) the nightly rally-quality regression as a Windows Scheduled
Task.
4
5       .DESCRIPTION
6           Creates a daily user-level task that runs scripts/nightly_regression.ps1 against a
repo
7           checkout. The action is inline-guarded, so it's safe to register BEFORE this work is
merged
8           into the target checkout: it no-ops + logs "awaiting" until the wrapper exists
there, then
9           self-activates. Idempotent (-Force re-registers); reversible (-Unregister).
10
11          Registers under the current user (no admin / no -RunLevel Highest needed ?
consistent with the
12          repo's other scheduled tasks). Uses -StartWhenAvailable so a run missed while the
machine slept
13          is caught up on wake.
14
15      .EXAMPLE
16          # Register against the main checkout, nightly at 03:00:
17          pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
18
19      .EXAMPLE
20          # Remove it:
21          pwsh -File scripts/register_nightly_task.ps1 -Unregister
22      #>
23      [CmdletBinding()]
24      param(
25          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
26          [string]$TaskName = 'KhelSutraNightlyRegression',
27          [string]$Time = '03:00',
28          [switch]$Pull,
29          [switch]$Unregister
30      )
31
32      if ($Unregister) {
33          Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
34          Write-Host "Unregistered scheduled task '$TaskName'."
35          return
36      }
37
38      $wrapper = Join-Path $RepoRoot 'scripts\nightly_regression.ps1'
39      $ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
40      if (-not $ps) { $ps = 'powershell.exe' }
41
42      # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit
0.
43      $marker = Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt'
44      $pullArg = if ($Pull) { ' -Pull' } else { '' }
45      $inner = @"
46      `$w = '$wrapper'
47      if (Test-Path `$w) { & `$w -RepoRoot '$RepoRoot'$pullArg }
48      else {
49        New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$marker') | Out-Null
50        "`$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  status=SKIPPED exit=0  wrapper absent
(awaiting merge) at '$wrapper'" | Out-File -FilePath '$marker' -Encoding utf8
51      }
52      "@
53      $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
54
55      $action = New-ScheduledTaskAction -Execute $ps `
```

```
56          -Argument "-NoProfile -ExecutionPolicy Bypass -EncodedCommand $encoded"
57      $trigger = New-ScheduledTaskTrigger -Daily -At $Time
58      $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
59          -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
60          -ExecutionTimeLimit (New-TimeSpan -Minutes 30)
61      $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME"
-LogonType Interactive -RunLevel Limited
62
63      Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
64          -Settings $settings -Principal $principal -Force `
65          -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline." | Out-Null
66
67      Write-Host "Registered scheduled task '$TaskName' ? daily at $Time, RepoRoot=$RepoRoot."
68      Write-Host "Wrapper: $wrapper"
69      Write-Host "Run now to test:  Start-ScheduledTask -TaskName '$TaskName'"
70      Write-Host "Remove:           pwsh -File scripts/register_nightly_task.ps1 -Unregister"
71
```

157. assistant (2026-07-09T03:24:41.453Z)

Now I have the full scheduling picture: `register_nightly_task.ps1` installs one daily user task (`KhelSutraNightlyRegression`, **30-min limit**) running `nightly_regression.ps1`. The perf leg is far heavier than 30 min, so the clean integration is a **parallel, longer-limit scheduled task** mirroring this exact pattern ? not a cram into the existing one.

Let me record the consolidation plan as tasks and let the Explore agent finish detailing the baseline schema / exit-code contract / CI wiring before I build against those conventions.

158. user (2026-07-09T03:24:42.344Z)

Updated task #4 status

159. user (2026-07-09T03:24:45.215Z)

Task #6 created successfully: Scaffold standalone perf-gate tool

160. user (2026-07-09T03:24:47.704Z)

Task #7 created successfully: Add gating + preflight self-help

161. user (2026-07-09T03:24:50.778Z)

Task #8 created successfully: Integrate into nightly framework

162. user (2026-07-09T03:24:52.740Z)

Task #9 created successfully: Docs + self-test + commit offer

163. assistant (2026-07-09T03:25:51.739Z)

Here's the consolidation design ? it mirrors your existing nightly quality leg exactly, so it slots into the framework rather than bolting on:

Standalone tool ? `scripts/perf_gate/` (self-contained, documented, gated)
- `run.py` ? one entrypoint: resolves **prod** (live Cloud Run image digest ? committed digest?commit map ? `--prod-commit` override; nudges #532) vs **RC = HEAD**, preflights, manages isolated worktrees, runs the per-stage matrix + stitch A/B, **gates vs `eval_baselines/perf_baseline.json`**, writes dated `output/nightly_perf/<date>.{md,json}` + `LATEST_STATUS.txt`.
- Exit codes match your quality leg: **0 PASS / 1 REGRESSION / 4 STALE(no baseline) / other ERROR**; `--update-baseline` to snapshot; `--quick`/`--full`/`--stitch-only`; `--preflight` self-help that names the exact missing prereq and its fix.

- `harness.py` / `stitch_ab.py` / `worktrees.py` / `preflight.py` / `config/bench_config.json` (the validated pieces).

Nightly integration (quality + performance)
- `scripts/nightly_perf.ps1` ? wrapper mirroring `nightly_regression.ps1` (graceful no-op, LATEST_STATUS bookkeeping, exit passthrough).
- Extend `register_nightly_task.ps1` to also install **`KhelSutraNightlyPerf`** ? a *separate* daily task with a longer `ExecutionTimeLimit` (the perf run far exceeds the quality leg's 30-min cap), so nightly runs both legs.

Gate policy (default, configurable): regression fails on **stitch** and **total** beyond tolerance; checksum/validation/segmentation are recorded but informational (they're a constant backdrop / infra-bound). Seeded from the real numbers this run is producing.

Let me create a proper branch (cut from `origin/master`, per branching-safety) and start scaffolding while the framework-mapping agent finishes the baseline-schema/exit-code details.

164. user (2026-07-09T03:26:01.845Z)

```
=== matrix progress ===
2026-07-09T03:14:15+00:00  START rc/SmokeTestBadmintonClip
2026-07-09T03:19:56+00:00  DONE  rc/SmokeTestBadmintonClip rc=0 wall=341.9s ::
[rc/SmokeTestBadmintonClip] checksum=0.2s validation=5.543s segmentation=316.291s stitch=12.921s
total=334.955s segments=6 gpu_peak=61%
2026-07-09T03:19:56+00:00  START prod/GX010075
=== fetch + create dev branch worktree from origin/master (git-safety) ===
3249451
branch 'feat/perf-release-gate' set up to track 'origin/master'.
HEAD is now at 3249451 feat(#534): live detector progress heartbeat + L4 worker resource
telemetry (#546)
--- dev branch HEAD ---
3249451
feat/perf-release-gate
--- scaffold scripts/perf_gate/ ---
total 4
drwxr-xr-x 1 avidu 197609 0 Jul  9 08:56 .
drwxr-xr-x 1 avidu 197609 0 Jul  9 08:56 ..
drwxr-xr-x 1 avidu 197609 0 Jul  9 08:56 config
```

165. assistant (2026-07-09T03:26:33.087Z)

Dev branch `feat/perf-release-gate` is up (isolated worktree from `origin/master`, your checkout untouched). Matrix is on `prod/GX010075` (the 5.3K clip). Let me populate the tool with the already-validated pieces (config + harness + stitch bench) and write the self-help preflight, then finalize `run.py`/nightly once the framework-mapping agent returns.

166. user (2026-07-09T03:26:37.053Z)

```
copied:
/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt-dev/scripts/perf_gate:
__init__.py
config
harness.py
stitch_ab.py

/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt-
dev/scripts/perf_gate/config:
bench_config.json
```

167. assistant (2026-07-09T03:26:52.753Z)

The framework-mapping agent hit a transient 529 (server overload), not a real failure ? and I

already have the key pattern from the `.ps1` files. Rather than re-spawn, let me read the remaining conventions I need directly (they're specific and few): the reelcount perf harness structure, the regression exit codes, the baseline JSON shapes, and CI wiring.

168. user (2026-07-09T03:26:55.917Z)

```
1:"""Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
5:vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of
CACHED
10:What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
19:  ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) +
segmenter
20:  ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
24:Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25:    python scripts/nightly_reelcount.py --update-baseline
31:(manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ?
re-freeze).
51:DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
52:DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
53:DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
74:# Pure cost math (wraps backend.billing ? no IO).
76:def cost_report(
82:    """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
92:    usd, breakdown = billing.compute_cost_usd(usage, rates)
107:    run_results: List[Dict[str, Any]], segmenter: str, cost: Dict[str, Any]
111:    Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
123:        "cost": cost,
134:    baseline: Optional[float] = None
143:        b = "?" if self.baseline is None else f"{self.baseline:g}"
176:            return 1
178:            return 4
179:        return 0
183:    baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances
185:    """Diff current run vs frozen baseline.
193:    if baseline.get("segmenter") != current.get("segmenter"):
199:                detail=f"{baseline.get('segmenter')} ? {current.get('segmenter')}",
204:    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
256:    baseline: Dict[str, Any],
266:        "STALE": "? STALE (re-freeze the baseline)",
268:    cost = current.get("cost", {})
270:        f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
274:        f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
275:        f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
276:        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
290:            "- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).",
295:    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
320:            ["_Improvements over baseline (re-freeze to ratchet up):_"]
391:    baseline_count: Optional[int] = None,
396:    database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
416:            "wall_seconds": 0.0,
467:                    "wall_seconds": 0.0,
479:                    "wall_seconds": round(total_wall, 2),
484:                and baseline_count is not None
485:                and count != baseline_count
502:                        "error": f"count {count}?baseline {baseline_count} on run 1, but the
re-run failed: {err2}",
```

```
504:                         "wall_seconds": round(total_wall, 2),
529:             "wall_seconds": round(total_wall, 2),
538:         "wall_seconds": round(total_wall, 2),
549:     baseline: Optional[Dict[str, Any]],

[Showing results with pagination = limit: 50]
```

169. user (2026-07-09T03:26:59.829Z)

```
=== nightly_regression.py exit codes + baseline ===
5:the headline metrics against a FROZEN baseline (``eval_baselines/nightly_baseline.json``).
6:Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-video
24:  permanent historical floor, so each report keeps showing the delta the milestone buys (a
launched
29:baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
36:Snapshot the baseline from current HEAD (after an intended improvement or a corpus change):
38:    python scripts/nightly_regression.py --update-baseline
46:* ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
48:  over-seg guard increased). Takes precedence over a stale baseline.
50:* ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51:* ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
57:import argparse
73:DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
91:#: Aggregate metrics snapshotted into the baseline (the gated subset + report-only context).
106:#: Per-video metrics snapshotted into the baseline.
152:      the *historical floor* so each nightly report keeps showing the delta the milestone
buys (the
176:    baseline_off = NightlyConfig(
177:        name="baseline_off",
180:            name="baseline_off",
192:    return [default, milestone, baseline_off]
247:    baseline: Optional[float] = None
253:        if self.baseline is None or self.current is None:
255:        return self.current - self.baseline
267:        b = "?" if self.baseline is None else f"{self.baseline:.4f}"
304:    """Compare one config's current summary to its baseline summary.
409:            return 1

=== eval_baselines/reelcount_manifest.json (shape) ===
{
  "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling. skip_ai_handoff=true keeps
this PURE-ENGINE (WASB candidates + windowing become the reels; NO paid Gemini handoff) so the
count is deterministic + free; the VM also needs WASB_REPO_DIR + WASB_PYTHON (the wasb conda
env) which run_on_vm.sh exports. Validated on an L4 2026-06-22 (180s smoke clip -> 22 reels,
deterministic).",
  "segmenter": "wasb_hybrid",
  "detector_impl": "native",
  "config_overrides": {
    "indexing.default_segmenter": "wasb_hybrid",
    "indexing.detector_impl": "native",
    "indexing.skip_ai_handoff": true
  },
  "machine_usd_per_hr": 0.70,
  "fx_inr_per_usd": 83.0,
  "videos": [
    {
```

```
      "name": "GX030094",
      "uri": "gs://khelsutra-rally-corpus/nightly-inputs/videos/GX030094.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly-
inputs/labels/GX030094_golden_features.json"
    },
    {
      "name": "Badminton_BXH_2",
      "uri": "gs://khelsutra-rally-corpus/nightly-inputs/videos/Badminton_BXH_2.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly-
inputs/labels/Badminton_BXH_2_golden_features.json"
    },
    {
      "name": "GX010128",
      "uri": "gs://khelsutra-rally-corpus/nightly-inputs/videos/GX010128.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly-
inputs/labels/GX010128_golden_features.json"
    }
  ]
}
=== eval_baselines/nightly_baseline.json (shape, head) ===
{
  "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
  "configs": {
    "baseline_off": {
      "aggregate": {
        "dend_abs_median": 41.77921047714383,
        "dstart_abs_median": 42.467243955066174,
        "f1@0.25": 0.39981006941991765,
        "f1@0.5": 0.2366145803119009,
        "merge_rate": 0.6380248744753217,
        "precision": 0.2577557018251538,
        "recall": 0.22719343607869802,
        "segment_count_ratio": 0.5824947574930804,
        "split_rate": 0.0,
        "tol_f1": 0.1382064040388172,
        "tol_precision": 0.14536686157033415,
        "tol_recall": 0.13445461315285676
      },
      "iou": 0.5,
      "min_crossings": 0,
      "n": 15,
      "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value the
milestone buys.",
      "per_video": {
        "Badminton_BXH_2": {
          "f1@0.25": 0.0,
=== CI workflows ===
ci.yml
nightly.yml
```


170. assistant (2026-07-09T03:27:32.265Z)

This is exactly the template I need. Key conventions locked:
- **Exit-code family:** 0 clean · 1 regression · 2 nothing-ran · 3 no-baseline · 4 stale.
- **Baselines** live in `eval_baselines/*.json` (manifest = inputs+config, baseline = frozen numbers with `git_commit`/`generated_at`); reports in `output/nightly_<leg>/`.
- `nightly_reelcount.py` even reports **cost** (wall-seconds × `machine_usd_per_hr` ? USD/INR via `backend.billing`) ? a pattern worth mirroring.
- There's also a `.github/workflows/nightly.yml`. Let me see the cloud-nightly topology and the exact reelcount argparse/exit so my perf gate matches the family precisely.

171. user (2026-07-09T03:27:33.353Z)

```yaml
name: Nightly quality regression (Tier-1, offline)

# Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;
# this runs the SAME offline guards on a schedule so we also catch what per-PR CI
# can't: dependency / transitive-dep drift, flakiness over time, and rot during
# quiet periods when nothing is pushed.
#
# Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the
# real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +
# the gitignored golden corpus).
#
# Lane parity with ci.yml: every OFFLINE guard job from ci.yml is mirrored here
# (lint · leak-scan · typecheck · import-smoke · full-suite · golden-fixtures ·
# license-scan · build-artifact), with the same Python pin + per-lane pip
# installs so modules collect/run instead of skipping for a missing dep. Two
# intentional differences, NOT drops:
#   - golden-fixtures runs single-OS here (ci.yml::golden-crossos already covers
#     the Win/mac/Linux matrix on every PR; the nightly is about drift-over-time,
#     not cross-OS float drift).
#   - a notify-on-failure job (below) surfaces a SCHEDULED failure as a tracking
#     issue ? ci.yml doesn't need it (a PR/push run already has a red check).
# Lane set + coverage floor are still copied literals, not single-sourced ? see
# the `workflow_call` single-sourcing follow-up in the PR description.
#
# NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
# won't actually run on a cadence until it lands on `master`. Use the
# `workflow_dispatch` trigger to run it on demand for validation after merge.

on:
  schedule:
    # 03:00 UTC daily.
    - cron: "0 3 * * *"
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: nightly-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
  lint:
    name: Nightly · Ruff lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        # Pinned so an upstream ruff release can't redden the nightly overnight; bump
deliberately.
        run: python -m pip install ruff==0.15.16
      - name: Ruff check
        run: ruff check . --output-format github

  # PII / attribution leak guard (tools/leak_scan.py) ? mirrors ci.yml::leak-scan.
  # A structural-rot guard: fails on a committed 32-hex MD5 (the video_id shape)
  # or a personal absolute user path in the shipped-code surface.
  leak-scan:
    name: Nightly · Leak scan (PII / attribution guard)
    runs-on: ubuntu-latest
```

```
65          timeout-minutes: 10
66          steps:
67            - uses: actions/checkout@v6
68            - uses: actions/setup-python@v6
69              with:
70                python-version: "3.13"
71            - name: Leak scan
72              run: python -m tools.leak_scan
73
74        # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
75        # deps are installed so mypy sees real signatures (without them
76        # ignore_missing_imports would Any-stub half the checks).
77        typecheck:
78          name: Nightly · Mypy (lenient baseline)
79          runs-on: ubuntu-latest
80          timeout-minutes: 10
81          steps:
82            - uses: actions/checkout@v6
83            - uses: actions/setup-python@v6
84              with:
85                python-version: "3.13"
86            - name: Install mypy + typed core deps
87              # mypy pinned alongside ruff so a type-checker release can't redden the nightly
overnight.
88              run: python -m pip install mypy==2.1.0 pydantic fastapi uvicorn python-dotenv
google-genai numpy
89            - name: Mypy
90              run: mypy backend training
91
92        # Syntax sweep + import smoke (no GPU stack) ? mirrors ci.yml::import-smoke.
93        # Guards the "SyntaxError / module-level import error shipping while the suite
94        # stays green" class. Deliberately does NOT install cv2/torch/numpy.
95        import-smoke:
96          name: Nightly · Syntax sweep + import smoke (no GPU stack)
97          runs-on: ubuntu-latest
98          timeout-minutes: 10
99          steps:
100           - uses: actions/checkout@v6
101           - uses: actions/setup-python@v6
102             with:
103               python-version: "3.13"
104           - name: Install minimal deps
105             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
106           - name: Run import smoke tests
107             run: python -m pytest tests/test_import_smoke.py -v
108
109       # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
110       # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
111       # rest. Floor stays at the ci.yml value (70); never lower it without an owner
112       # decision.
113       full-suite:
114         name: Nightly · Full test suite (offline, mocked)
115         runs-on: ubuntu-latest
116         timeout-minutes: 30
117         steps:
118           - uses: actions/checkout@v6
119           - uses: actions/setup-python@v6
120             with:
121               python-version: "3.13"
122               cache: pip
123           - name: System libs for opencv-python
124             run: |
125               # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
```

```
126               # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
127               # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
128               sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
129               sudo apt-get update
130               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
131           - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
132             run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
133           - name: Install trackers (git-only dep ? PyPI wheel is broken)
134             # D13/P12: ByteTrack moved to the standalone `trackers` package (git-only;
135             # see ci.yml for the full rationale). Installed here so the D13/P12 tests
136             # run instead of skipping via pytest.importorskip.
137             run: python -m pip install -r requirements-trackers.txt
138           - name: Install package (editable) + dev tooling
139             run: python -m pip install -e .[dev]
140           - name: Run full suite (with coverage floor)
141             run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
142
143       # Offline quality guard: the committed golden regression fixtures
144       # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
145       # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
146       # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
147       # every PR ? nightly is about drift-over-time on the default branch). The
148       # CONTROL replay needs only numpy.
149       golden-fixtures:
150         name: Nightly · Golden fixtures regression (offline quality guard)
151         runs-on: ubuntu-latest
152         timeout-minutes: 15
153         steps:
154           - uses: actions/checkout@v6
155           - uses: actions/setup-python@v6
156             with:
157               python-version: "3.13"
158           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
159             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
160           - name: Golden-fixtures characterization (offline)
161             run: |
162               python -m backend.eval.golden_fixtures --check
163               python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
164
165       # Commercial-clean licensing guardrail (MIT/Apache/BSD only) ? mirrors
166       # ci.yml::license-scan. Scans the RUNTIME dependency closure and fails on any
167       # copyleft (GPL/AGPL/LGPL/SSPL). This is exactly the transitive-dep drift the
168       # header cites as a reason the nightly exists ? a new sub-dep can flip a
169       # license between PRs without touching our code.
170       license-scan:
171         name: Nightly · Dependency license scan (no copyleft in runtime deps)
172         runs-on: ubuntu-latest
173         timeout-minutes: 10
174         steps:
175           - uses: actions/checkout@v6
176           - uses: actions/setup-python@v6
177             with:
178               python-version: "3.13"
179           - name: Install runtime deps + pip-licenses
180             run: |
181               python -m pip install .
182               python -m pip install pip-licenses
183           - name: Report dependency licenses
```

```
            run: python -m piplicenses --format=markdown --with-urls --order=license
          - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
            run: |
              if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
                echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
                exit 1
              fi
              echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."

      # Built-wheel smoke (install + launch) ? mirrors ci.yml::build-artifact. Builds
      # the wheel, installs it NON-editable from a non-repo cwd, and launches the
      # console script. Catches packaging regressions the source suite can't.
      build-artifact:
        name: Nightly · Built wheel smoke (install + launch)
        runs-on: ubuntu-latest
        timeout-minutes: 20
        steps:
          - uses: actions/checkout@v6
          - uses: actions/setup-python@v6
            with:
              python-version: "3.13"
          - name: System libs for opencv-python
            run: |
              # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
              # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
              # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
              sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
              sudo apt-get update
              sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
          - name: Build the wheel
            run: |
              python -m pip install build
              python -m build --wheel
          - name: Install the built wheel + CPU torch (out-of-band index)
            run: |
              python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
              python -m pip install dist/*.whl
          - name: Launch the installed console script from a non-repo dir
            run: |
              cd /tmp
              indexer --help
              python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"

      # Failure visibility ? a silent nightly is useless. Scheduled runs have no PR
      # to redden and nobody is watching the Actions tab on a quiet day (the exact
      # "rot during quiet periods" this workflow exists to catch). On ANY guard
      # failure, open/refresh a single tracking issue so a human is pinged. Uses the
      # built-in GITHUB_TOKEN (no secrets); `issues: write` is scoped to THIS job
      # only ? the guard jobs above keep `contents: read`. Gated on event_name ==
      # 'schedule': a manual `workflow_dispatch` run is being watched, so no issue.
      notify-on-failure:
        name: Nightly · Surface failure (open/refresh tracking issue)
        needs: [lint, leak-scan, typecheck, import-smoke, full-suite, golden-fixtures,
license-scan, build-artifact]
        if: failure() && github.event_name == 'schedule'
        runs-on: ubuntu-latest
        timeout-minutes: 5
        permissions:
```

```
241           contents: read
242           issues: write
243         steps:
244           - uses: actions/github-script@v7
245             with:
246               script: |
247                 const runUrl =
`${context.serverUrl}/${context.repo.owner}/${context.repo.repo}/actions/runs/${context.runId}`;
248                 const title = 'Nightly quality regression FAILED';
249                 const body = [
250                   'The Tier-1 nightly offline quality guard failed.',
251                   '',
252                   `- Run: ${runUrl}`,
253                   `- Date (UTC): ${new Date().toISOString()}`,
254                   '',
255                   'One of ruff / leak-scan / mypy / import-smoke / full-suite (cov>=70) /',
256                   'golden-fixtures / license-scan / build-artifact regressed.',
257                   'Open the run for the failing lane. Close this issue once green.',
258                 ].join('\n');
259                 // Reuse a single open tracking issue instead of spamming one per night.
260                 const existing = await github.rest.issues.listForRepo({
261                   owner: context.repo.owner,
262                   repo: context.repo.repo,
263                   state: 'open',
264                   labels: 'nightly-failure',
265                   per_page: 1,
266                 });
267                 if (existing.data.length > 0) {
268                   await github.rest.issues.createComment({
269                     owner: context.repo.owner,
270                     repo: context.repo.repo,
271                     issue_number: existing.data[0].number,
272                     body,
273                   });
274                 } else {
275                   await github.rest.issues.create({
276                     owner: context.repo.owner,
277                     repo: context.repo.repo,
278                     title,
279                     body,
280                     labels: ['nightly-failure'],
281                   });
282                 }
283
```

172. user (2026-07-09T03:27:38.137Z)

```
Exit code 1
=== nightly_reelcount.py argparse + Verdict.exit_code + main() return/exit ===
    def improvements(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "improvement"]

    @property
    def stale(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "stale"]

    def overall_status(self) -> str:
        if self.regressions:
            return "REGRESSION"
        if self.stale:
            return "STALE"
        return "PASS"

    def exit_code(self) -> int:
        if self.regressions:
```

```
            return 1
        if self.stale:
            return 4
        return 0


def compare(
--- argparse block ---
272:        f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
637:def build_parser() -> argparse.ArgumentParser:
638:    p = argparse.ArgumentParser(
641:    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
642:    p.add_argument("--baseline", default=DEFAULT_BASELINE)
643:    p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
644:    p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
645:    p.add_argument(
652:    p.add_argument(
657:    p.add_argument(
662:    p.add_argument(
667:    p.add_argument("--email-to", default=None, help="override the report recipient")
668:    p.add_argument(
671:    p.add_argument(
682:    args = build_parser().parse_args(argv)
749:                    "exit_code": result.exit_code(),
776:    code = result.exit_code()
797:    raise SystemExit(main())
=== any VM/reelcount wrapper + run_on_vm ===
nightly_reelcount.py
=== reelcount_baseline.json shape (head) ===
head: cannot open 'eval_baselines/reelcount_baseline.json' for reading: No such file or
directory
```

173. assistant (2026-07-09T03:29:19.230Z)

Now I have the complete convention picture:
- `nightly.yml` = **cloud Tier-1 OFFLINE** guards (no GPU/video) ? my GPU+WSL perf gate is **Tier-2 local**, so it belongs to the **Windows scheduled-task** mechanism (mirroring `nightly_regression.ps1`), not the cloud workflow.
- Exit codes: `Verdict.exit_code()` ? 1 regression / 4 stale / 0 pass (+ 2 nothing-ran, 3 no-baseline).
- Baselines in `eval_baselines/*.json` (manifest = inputs+config, baseline = frozen numbers with `git_commit`); reports to `output/nightly_<leg>/`.
- **Critical constraint:** the repo's leak-scan CI fails on committed **personal absolute paths** (`C:\Users\avidu\?`) and 32-hex MD5s ? so my committed manifest must use **video basenames + a runtime-resolved vault dir**, never hardcoded paths.

Let me build the tool. Starting with the data-driven manifest (leak-safe) and the self-help preflight.

174. assistant (2026-07-09T03:29:34.356Z)

```
{
  "_doc": "Local release PERFORMANCE gate manifest (scripts/perf_gate/run.py). Times
checksum/validation/segmentation/full-reel-stitch for the deployed production commit vs repo
HEAD, on the local GPU, over vault clips. Companion to the QUALITY gates
(scripts/nightly_regression.py, scripts/nightly_reelcount.py). Videos are referenced by BASENAME
and resolved at runtime against --vault-dir (default: env RALLY_VAULT_DIR, else ~/Videos) so NO
personal absolute path is committed (repo leak-scan guard). Prod commit is auto-resolved from
the live Cloud Run image (digest->commit via prod_image_map until #532 git-sha-labels images),
else the pinned prod_commit, else --prod-commit. Freeze precomputed release numbers: `python -m
scripts.perf_gate.run --update-baseline`.",
  "segmenter": "wasb_hybrid",
  "config": "scripts/perf_gate/config/bench_config.json",
  "machine_usd_per_hr": 0.70,
```

```
  "fx_inr_per_usd": 83.0,
  "prod": {
    "pinned_commit": "304af7b",
    "cloud_run": {
      "service": "rally-control-plane",
      "region": "asia-southeast1",
      "project": "gen-lang-client-0306026826"
    },
    "image_digest_to_commit": {
      "sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943": "304af7b"
    }
  },
  "gate": {
    "stages": ["stitch_s", "total_s"],
    "tol_pct": 20.0,
    "_doc": "REGRESSION (exit 1) if HEAD is slower than production on any listed stage by more
than tol_pct. checksum/validation/segmentation are reported but NOT gated (constant backdrop /
infra-bound). Stitch is the locally-measurable code path (#531)."
  },
  "profiles": {
    "quick": ["GX010016", "SmokeTestBadmintonClip"],
    "full": ["GX010016", "SmokeTestBadmintonClip", "GX010075"],
    "stress": ["GX010016", "SmokeTestBadmintonClip", "GX010075", "GX010061"]
  },
  "videos": [
    {"name": "GX010016", "file": "GX010016.MP4", "note": "47s 1080p"},
    {"name": "SmokeTestBadmintonClip", "file": "SmokeTestBadmintonClip.mp4", "note": "81s
1080p60, multi-rally"},
    {"name": "GX010075", "file": "GX010075.MP4", "note": "89s 5.3K 16:9"},
    {"name": "GX010061", "file": "GX010061.MP4", "note": "5min 5.3K stress (slow: local CPU
decode)"}
  ],
  "stitch_ab": [
    {"label": "1080p", "file": "SmokeTestBadmintonClip.mp4",
     "pairs": [[5,10],[15,20],[25,30],[35,40],[45,50],[55,60],[65,70],[72,77]], "reps": 3},
    {"label": "5p3k", "file": "GX010075.MP4",
     "pairs": [[6,10],[17,21],[28,32],[39,43],[50,54],[61,65],[72,76],[80,84]], "reps": 1}
  ]
}
```

175. user (2026-07-09T03:29:34.428Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\eval_baselines\perf_manifest.json (file state is current in your context ? no need to Read
it back)
```

176. assistant (2026-07-09T03:30:08.921Z)

```
#!/usr/bin/env python3
"""Preflight environment checks for the local performance gate ? SELF-HELP oriented.

Every check reports (ok, detail, fix) so a failure tells you exactly what's missing and how
to fix it, rather than blowing up mid-run. Required checks gate the run (exit 2 if any fail);
optional checks (e.g. gcloud for prod auto-resolve) only warn.

Run standalone:   python -m scripts.perf_gate.preflight
                  python scripts/perf_gate/run.py --preflight
"""
import json
import os
import shutil
import subprocess
import sys
from dataclasses import dataclass
```

```
@dataclass
class Check:
    name: str
    required: bool
    ok: bool
    detail: str = ""
    fix: str = ""


def _sh(cmd, timeout=30):
    try:
        p = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
        return p.returncode, (p.stdout or "").strip(), (p.stderr or "").strip()
    except Exception as e:
        return 1, "", str(e)


def _wsl(distro, script, timeout=60):
    return _sh(["wsl.exe", "-d", distro, "-e", "bash", "-lc", script], timeout=timeout)


def run_checks(config: dict, want_prod_resolve: bool = False) -> list:
    checks = []
    wasb = (config.get("indexing", {}) or {}).get("wasb", {}) or {}
    distro = wasb.get("wsl_distro", "Ubuntu")
    conda_env = wasb.get("conda_env", "wasb")
    repo_dir = wasb.get("repo_dir", "~/models/WASB-SBDT")
    weights = wasb.get("weights_path", "")

    # 1) git + worktree support
    rc, out, _ = _sh(["git", "--version"])
    checks.append(Check("git", True, rc == 0, out or "not found",
                        "Install Git for Windows (https://git-scm.com)."))

    # 2) ffmpeg / ffprobe
    for tool in ("ffmpeg", "ffprobe"):
        present = shutil.which(tool) is not None
        checks.append(Check(tool, True, present, shutil.which(tool) or "not on PATH",
                            "Install FFmpeg and add it to PATH (README §External System
Requirements)."))

    # 3) torch + CUDA on the local GPU
    try:
        import torch  # noqa
        cuda = bool(torch.cuda.is_available())
        gpu = torch.cuda.get_device_name(0) if cuda else "(no CUDA device)"
        checks.append(Check("torch+cuda", True, cuda, f"torch {torch.__version__} · {gpu}",
                            "Install a CUDA build of torch (see gpu_setup.sh / README) and
verify the driver."))
    except Exception as e:
        checks.append(Check("torch+cuda", True, False, f"import failed: {e}",
                            "pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124"))

    # 4) WSL distro reachable
    rc, out, err = _wsl(distro, "echo ok", timeout=40)
    checks.append(Check(f"wsl:{distro}", True, rc == 0 and "ok" in out, (out or err)[:80] or
"unreachable",
                        f"Install/start WSL2 distro '{distro}' (wsl --install -d {distro})."))

    # 5) WSL wasb conda env + repo + weights (the production shuttle detector)
    if rc == 0:
        env_sh = f"conda env list 2>/dev/null | grep -qiE '(^|/){conda_env}( |$|/)' && echo
ENV_OK || (ls -d ~/miniconda3/envs/{conda_env} >/dev/null 2>&1 && echo ENV_OK || echo
ENV_MISSING)"
```

```
        _, eo, _ = _wsl(distro, env_sh, timeout=40)
        checks.append(Check(f"wsl conda env '{conda_env}'", True, "ENV_OK" in eo, eo or
"missing",
                            f"Create the WASB conda env '{conda_env}' in WSL (see docs; WASB-
SBDT requirements)."))
        _, ro, _ = _wsl(distro, f"test -d {repo_dir} && echo REPO_OK || echo REPO_MISSING")
        checks.append(Check("wsl WASB-SBDT repo", True, "REPO_OK" in ro, repo_dir,
                            f"git clone the WASB-SBDT repo to {repo_dir} in WSL."))
        if weights:
            _, wo, _ = _wsl(distro, f"test -f {weights} && echo W_OK || echo W_MISSING")
            checks.append(Check("wsl WASB weights", True, "W_OK" in wo, weights,
                                f"Place the pretrained badminton weights at {weights} in WSL."))

    # 6) gcloud (optional ? only for auto-resolving the live prod commit)
    if want_prod_resolve:
        present = shutil.which("gcloud") is not None
        checks.append(Check("gcloud (prod auto-resolve)", False, present, shutil.which("gcloud")
or "not on PATH",
                            "Install gcloud OR pass --prod-commit <sha> explicitly (see README
§Prod resolution)."))
    return checks


def print_checks(checks: list) -> int:
    print("Preflight:")
    fails = 0
    for c in checks:
        mark = "OK  " if c.ok else ("FAIL" if c.required else "warn")
        tag = "" if c.required else " (optional)"
        print(f"  [{mark}] {c.name}{tag}: {c.detail}")
        if not c.ok:
            print(f"         fix: {c.fix}")
            if c.required:
                fails += 1
    print(f"  => {len(checks) - fails}/{len(checks)} ok" + (f", {fails} REQUIRED failing" if
fails else ""))
    return fails


def main(argv=None) -> int:
    here = os.path.dirname(os.path.abspath(__file__))
    cfg_path = os.path.join(here, "config", "bench_config.json")
    cfg = json.load(open(cfg_path, encoding="utf-8")) if os.path.exists(cfg_path) else {}
    fails = print_checks(run_checks(cfg, want_prod_resolve=True))
    return 2 if fails else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

177. user (2026-07-09T03:30:08.990Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\preflight.py (file state is current in your context ? no need to Read it back)

178. assistant (2026-07-09T03:30:32.980Z)

Now the core orchestrator, `run.py` ? resolves prod (live Cloud Run ? digest map ? pinned ? override) vs HEAD, preflights, manages worktrees, runs the matrix + stitch A/B, gates RC-vs-prod on the configured stages, and writes the dated report with family-matching exit codes.

179. assistant (2026-07-09T03:31:52.117Z)

```
#!/usr/bin/env python3
"""Local release PERFORMANCE gate ? deployed production vs repo HEAD, on this GPU.

Standalone, release-gating, repeatable. Times four stages with NO SPA / NO user step ?
checksum -> validation -> segmentation (wasb_hybrid, GPU) -> full-reel stitch ? for the
commit currently serving api.khelsutra.guru and for repo HEAD, from isolated git worktrees
over the same vault clips, then reports per-stage deltas and GATES the release.

Companion to the QUALITY gates (scripts/nightly_regression.py + scripts/nightly_reelcount.py).
See scripts/perf_gate/README.md for methodology, self-help, and the nightly hook.

Exit codes (same family as the quality gates):
  0 PASS · 1 REGRESSION (HEAD slower than prod beyond tol) · 2 nothing-ran / preflight failed
  3 no baseline (only with --gate-vs-baseline) · 4 STALE (config/video-set changed vs baseline)

Examples:
  python scripts/perf_gate/run.py --preflight
  python scripts/perf_gate/run.py                       # quick profile, prod auto-resolved,
gate HEAD vs prod
  python scripts/perf_gate/run.py --profile full        # + the 5.3K clip
  python scripts/perf_gate/run.py --stitch-only         # just the #531 stitch A/B (fast)
  python scripts/perf_gate/run.py --prod-commit 304af7b --rc-ref origin/master
  python scripts/perf_gate/run.py --update-baseline     # snapshot precomputed release numbers
"""
import argparse
import json
import os
import shutil
import statistics
import subprocess
import sys
import time
from datetime import datetime, timezone

HERE = os.path.dirname(os.path.abspath(__file__))
REPO_ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
HARNESS = os.path.join(HERE, "harness.py")
STITCH = os.path.join(HERE, "stitch_ab.py")
DEFAULT_CONFIG = os.path.join(HERE, "config", "bench_config.json")
DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "perf_manifest.json")
DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "perf_baseline.json")
DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_perf")


def iso_now():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def sh(cmd, cwd=None, timeout=120):
    p = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, timeout=timeout)
    return p.returncode, (p.stdout or "").strip(), (p.stderr or "").strip()


def git_short(repo, ref):
    rc, out, _ = sh(["git", "-C", repo, "rev-parse", "--short", ref])
    return out if rc == 0 else None


## ---------------------------------------------------------------------- resolve prod

def resolve_prod(manifest, args, log):
    if args.prod_commit:
        return git_short(REPO_ROOT, args.prod_commit) or args.prod_commit, "explicit --prod-
commit"
    if args.prod_ref:
```

```
        return git_short(REPO_ROOT, args.prod_ref), f"--prod-ref {args.prod_ref}"
    prod = manifest.get("prod", {})
    cr = prod.get("cloud_run", {})
    mp = prod.get("image_digest_to_commit", {})
    if shutil.which("gcloud") and cr:
        try:
            rc, out, _ = sh(["gcloud", "run", "services", "describe", cr["service"],
                             "--region", cr["region"], "--project", cr["project"],
                             "--format=value(spec.template.spec.containers[0].image)"],
timeout=90)
            if rc == 0 and "@" in out:
                digest = out.split("@")[-1].strip()
                if digest in mp:
                    log(f"prod resolved from live Cloud Run image {digest[:19]}? ->
{mp[digest]}")
                    return git_short(REPO_ROOT, mp[digest]) or mp[digest], f"live Cloud Run
{digest[:19]}?"
                log(f"live Cloud Run digest {digest[:19]}? not in image_digest_to_commit map
(update it / #532)")
        except Exception as e:
            log(f"Cloud Run resolve failed ({e}); falling back to pinned prod_commit")
    pin = prod.get("pinned_commit")
    return (git_short(REPO_ROOT, pin) or pin), "pinned prod_commit (Cloud Run resolve
unavailable)"
```

---------- worktrees

```
def worktree(repo, path, commit, log):
    if os.path.exists(path):
        sh(["git", "-C", repo, "worktree", "remove", "--force", path], timeout=120)
    rc, _, err = sh(["git", "-C", repo, "worktree", "add", "--detach", path, commit],
timeout=180)
    if rc != 0:
        raise RuntimeError(f"worktree add {path}@{commit} failed: {err}")
    log(f"worktree {os.path.basename(path)} -> {git_short(path, 'HEAD')}")
```

---------- run one video/version

```
def run_harness(version, worktree_dir, commit, name, video, cfg, runs_dir, timeout, log):
    run_dir = os.path.join(runs_dir, version, name)
    os.makedirs(run_dir, exist_ok=True)
    jo = os.path.join(run_dir, "result.json")
    cmd = [sys.executable, HARNESS, "--worktree", worktree_dir, "--config", cfg, "--video",
video,
           "--name", name, "--version", version, "--commit", commit, "--run-dir", run_dir, "--
json-out", jo]
    log(f"START {version}/{name}")
    t0 = time.perf_counter()
    try:
        p = subprocess.run(cmd, cwd=worktree_dir, capture_output=True, text=True,
timeout=timeout)
        tail = "\n".join((p.stderr or "").strip().splitlines()[-2:])
        log(f"DONE  {version}/{name} rc={p.returncode} wall={time.perf_counter()-t0:.0f}s ::
{tail}")
    except subprocess.TimeoutExpired:
        log(f"TIMEOUT {version}/{name} after {time.perf_counter()-t0:.0f}s")
    return json.load(open(jo, encoding="utf-8")) if os.path.exists(jo) else \
        {"version": version, "video": name, "commit": commit, "errors": {"driver": "no
result.json"}}


def run_stitch(version, worktree_dir, commit, label, source, pairs, reps, cfg, runs_dir,
timeout, log):
```

```
    run_dir = os.path.join(runs_dir, "stitch", label, version)
    os.makedirs(run_dir, exist_ok=True)
    jo = os.path.join(run_dir, "result.json")
    cmd = [sys.executable, STITCH, "--worktree", worktree_dir, "--config", cfg, "--source",
source,
           "--pairs", json.dumps(pairs), "--reps", str(reps), "--version", version, "--commit",
commit,
           "--run-dir", run_dir, "--json-out", jo]
    log(f"STITCH {version}/{label} (reps={reps})")
    try:
        subprocess.run(cmd, cwd=worktree_dir, capture_output=True, text=True, timeout=timeout)
    except subprocess.TimeoutExpired:
        log(f"STITCH TIMEOUT {version}/{label}")
    return json.load(open(jo, encoding="utf-8")) if os.path.exists(jo) else {}


## ---------------------------------------------------------------------- gating + report

def pct(prod, rc):
    if not isinstance(prod, (int, float)) or not isinstance(rc, (int, float)) or prod == 0:
        return None
    return round((rc - prod) / prod * 100.0, 1)


def build(results, stitch_results, videos, meta, gate):
    """Return (summary, findings). Finding kind: regression|improvement."""
    idx = {}
    for r in results:
        idx.setdefault(r.get("version"), {})[r.get("video")] = r
    findings = []
    per_video = {}
    for name in videos:
        p = idx.get("prod", {}).get(name, {})
        c = idx.get("rc", {}).get(name, {})
        row = {"prod": {}, "rc": {}, "delta_pct": {}}
        for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s", "total_s"):
            pv = p.get("stages", {}).get(k) if k != "total_s" else p.get("total_s")
            rv = c.get("stages", {}).get(k) if k != "total_s" else c.get("total_s")
            row["prod"][k], row["rc"][k] = pv, rv
            d = pct(pv, rv)
            row["delta_pct"][k] = d
            if k in gate["stages"] and d is not None:
                if d > gate["tol_pct"]:
                    findings.append({"kind": "regression", "where": f"{name}/{k}", "delta_pct":
d})
                elif d < -gate["tol_pct"]:
                    findings.append({"kind": "improvement", "where": f"{name}/{k}", "delta_pct":
d})
        row["segments"] = {"prod": p.get("segments"), "rc": c.get("segments")}
        row["errors"] = {"prod": list(p.get("errors", {}).keys()), "rc": list(c.get("errors",
{}).keys())}
        per_video[name] = row
    stitch = {}
    for label, sr in (stitch_results or {}).items():
        pm = sr.get("prod", {}).get("median_s")
        cm = sr.get("rc", {}).get("median_s")
        d = pct(pm, cm)
        stitch[label] = {"prod_median_s": pm, "rc_median_s": cm, "delta_pct": d,
                         "prod_times": sr.get("prod", {}).get("times_s"),
                         "rc_times": sr.get("rc", {}).get("times_s")}
        if d is not None and "stitch_ab" in gate.get("stages", ["stitch_ab"]) or d is not None:
            if d is not None and d > gate["tol_pct"]:
                findings.append({"kind": "regression", "where": f"stitch_ab/{label}",
"delta_pct": d})
            elif d is not None and d < -gate["tol_pct"]:
```

```
                findings.append({"kind": "improvement", "where": f"stitch_ab/{label}",
"delta_pct": d})
    summary = {"meta": meta, "per_video": per_video, "stitch_ab": stitch}
    return summary, findings


def exit_code(findings, stale):
    if any(f["kind"] == "regression" for f in findings):
        return 1
    if stale:
        return 4
    return 0


def fmt(v):
    return "?" if not isinstance(v, (int, float)) else f"{v:.2f}s"


def write_report(summary, findings, code, report_dir, date):
    os.makedirs(report_dir, exist_ok=True)
    m = summary["meta"]
    badge = {0: "? PASS", 1: "? REGRESSION", 4: "? STALE"}.get(code, f"exit {code}")
    L = [f"# Release performance gate ? HEAD vs production ({date})", "",
         f"**Status: {badge}**  ·  exit {code}",
         f"- Prod (api.khelsutra.guru): `{m['prod_commit']}` ({m['prod_source']})  ·  HEAD:
`{m['rc_commit']}`",
         f"- GPU: {m.get('gpu','?')}  ·  segmenter: wasb_hybrid (WSL2 GPU)  ·  encoder: libx264
·  gate: {m['gate']['stages']} > {m['gate']['tol_pct']}%",
         "- ?% = (HEAD ? prod)/prod. Negative = HEAD faster. Only stitch/total are gated; the
rest is context.", ""]
    reg = [f for f in findings if f["kind"] == "regression"]
    imp = [f for f in findings if f["kind"] == "improvement"]
    if reg:
        L.append("**Regressions:** " + ", ".join(f"{f['where']} {f['delta_pct']:+.1f}%" for f in
reg))
    if imp:
        L.append("_Improvements:_ " + ", ".join(f"{f['where']} {f['delta_pct']:+.1f}%" for f in
imp))
    L.append("")
    for name, row in summary["per_video"].items():
        L.append(f"## {name}  ·  segments prod={row['segments']['prod']}
rc={row['segments']['rc']}")
        L.append("| stage | prod | HEAD | ?% |")
        L.append("|---|---|---|---|")
        for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s", "total_s"):
            d = row["delta_pct"][k]
            gated = "*" if k in summary["meta"]["gate"]["stages"] else ""
            L.append(f"| {k.replace('_s','')}{gated} | {fmt(row['prod'][k])} |
{fmt(row['rc'][k])} | "
                     f"{'?' if d is None else f'{d:+.1f}%'} |")
        L.append("")
    if summary["stitch_ab"]:
        L.append("## Stitch A/B (#531, segmentation excluded) ? gated*")
        L.append("| reel | prod median | HEAD median | ?% |")
        L.append("|---|---|---|---|")
        for label, s in summary["stitch_ab"].items():
            L.append(f"| {label}* | {fmt(s['prod_median_s'])} | {fmt(s['rc_median_s'])} | "
                     f"{'?' if s['delta_pct'] is None else f'{s['delta_pct']:+.1f}%'} |")
        L.append("")
    L.append("_* gated stage. `--update-baseline` freezes these numbers into
eval_baselines/perf_baseline.json._")
    open(os.path.join(report_dir, f"{date}.md"), "w", encoding="utf-8").write("\n".join(L))
    payload = {"exit_code": code, "status": badge, "findings": findings, **summary}
    open(os.path.join(report_dir, f"{date}.json"), "w",
encoding="utf-8").write(json.dumps(payload, indent=2))
```

```
    return os.path.join(report_dir, f"{date}.md")
```

---------- main

```
def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="Local release performance gate: production vs repo
HEAD.")
    ap.add_argument("--config", default=DEFAULT_CONFIG)
    ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
    ap.add_argument("--baseline", default=DEFAULT_BASELINE)
    ap.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
    ap.add_argument("--profile", default="quick", choices=("quick", "full", "stress"))
    ap.add_argument("--videos", default=None, help="comma list of names, overrides --profile")
    ap.add_argument("--vault-dir", default=os.environ.get("RALLY_VAULT_DIR") or
os.path.expanduser("~/Videos"))
    ap.add_argument("--prod-commit", default=None)
    ap.add_argument("--prod-ref", default=None)
    ap.add_argument("--rc-ref", default="HEAD")
    ap.add_argument("--worktree-root", default=os.path.join(REPO_ROOT, ".perf_gate_worktrees"))
    ap.add_argument("--runs-dir", default=None)
    ap.add_argument("--timeout", type=int, default=5400)
    ap.add_argument("--tol", type=float, default=None, help="override manifest gate tol_pct")
    ap.add_argument("--stitch-only", action="store_true")
    ap.add_argument("--no-stitch-ab", action="store_true")
    ap.add_argument("--preflight", action="store_true", help="run checks and exit")
    ap.add_argument("--skip-preflight", action="store_true")
    ap.add_argument("--update-baseline", action="store_true")
    ap.add_argument("--keep-worktrees", action="store_true")
    args = ap.parse_args(argv)

    manifest = json.load(open(args.manifest, encoding="utf-8"))
    cfg_dict = json.load(open(args.config, encoding="utf-8"))
    gate = dict(manifest.get("gate", {"stages": ["stitch_s", "total_s"], "tol_pct": 20.0}))
    if args.tol is not None:
        gate["tol_pct"] = args.tol

    from . import preflight as pf  # relative import when run as module
    logs = []

    def log(msg):
        line = f"{iso_now()}  {msg}"
        print(line, flush=True)
        logs.append(line)

    # preflight
    if not args.skip_preflight or args.preflight:
        checks = pf.run_checks(cfg_dict, want_prod_resolve=(args.prod_commit is None and
args.prod_ref is None))
        fails = pf.print_checks(checks)
        if args.preflight:
            return 2 if fails else 0
        if fails:
            log(f"preflight: {fails} REQUIRED checks failing ? aborting (exit 2). See fixes
above.")
            return 2

    sh(["git", "-C", REPO_ROOT, "fetch", "origin", "--quiet"], timeout=120)
    prod_commit, prod_source = resolve_prod(manifest, args, log)
    rc_commit = git_short(REPO_ROOT, args.rc_ref)
    if not prod_commit or not rc_commit:
        log(f"could not resolve commits (prod={prod_commit}, rc={rc_commit})")
        return 2
    log(f"prod={prod_commit} ({prod_source})  ·  HEAD({args.rc_ref})={rc_commit}")
```

```
    # video set
    names = args.videos.split(",") if args.videos else manifest["profiles"][args.profile]
    by_name = {v["name"]: v for v in manifest["videos"]}
    videos = []
    for n in names:
        f = by_name.get(n, {}).get("file", n)
        path = f if os.path.isabs(f) else os.path.join(args.vault_dir, f)
        if os.path.exists(path):
            videos.append((n, path))
        else:
            log(f"SKIP {n}: not found at {path} (set --vault-dir / RALLY_VAULT_DIR)")
    if not videos and not args.stitch_only:
        log("no videos found ? nothing to run (exit 2)")
        return 2

    runs_dir = args.runs_dir or os.path.join(args.worktree_root, "runs")
    os.makedirs(args.worktree_root, exist_ok=True)
    wt_prod = os.path.join(args.worktree_root, "prod")
    wt_rc = os.path.join(args.worktree_root, "rc")
    gpu = "?"
    try:
        _, gpu, _ = sh(["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"], timeout=15)
        gpu = gpu.splitlines()[0] if gpu else "?"
    except Exception:
        pass

    try:
        worktree(REPO_ROOT, wt_prod, prod_commit, log)
        worktree(REPO_ROOT, wt_rc, rc_commit, log)

        results = []
        if not args.stitch_only:
            for name, path in videos:
                for version, wt, commit in (("prod", wt_prod, prod_commit), ("rc", wt_rc,
rc_commit)):
                    results.append(run_harness(version, wt, commit, name, path, args.config,
runs_dir, args.timeout, log))

        stitch_results = {}
        if not args.no_stitch_ab:
            for case in manifest.get("stitch_ab", []):
                src = case["file"] if os.path.isabs(case["file"]) else
os.path.join(args.vault_dir, case["file"])
                if not os.path.exists(src):
                    log(f"stitch A/B skip {case['label']}: source missing {src}")
                    continue
                stitch_results[case["label"]] = {}
                for version, wt, commit in (("prod", wt_prod, prod_commit), ("rc", wt_rc,
rc_commit)):
                    stitch_results[case["label"]][version] = run_stitch(
                        version, wt, commit, case["label"], src, case["pairs"], case["reps"],
                        args.config, runs_dir, args.timeout, log)
    finally:
        if not args.keep_worktrees:
            for wt in (wt_prod, wt_rc):
                sh(["git", "-C", REPO_ROOT, "worktree", "remove", "--force", wt], timeout=120)

    meta = {"generated": iso_now(), "prod_commit": prod_commit, "prod_source": prod_source,
            "rc_commit": rc_commit, "rc_ref": args.rc_ref, "gpu": gpu, "profile": args.profile,
            "gate": gate, "segmenter": manifest.get("segmenter", "wasb_hybrid")}
    summary, findings = build(results, stitch_results, [n for n, _ in videos], meta, gate)

    if args.update_baseline:
        base = {"_doc": "Frozen release perf numbers (scripts/perf_gate/run.py --update-
baseline).",
```

```
                "git_commit": rc_commit, "prod_commit": prod_commit, "generated_at": iso_now(),
                "profile": args.profile, "gate": gate, **summary}
        os.makedirs(os.path.dirname(args.baseline), exist_ok=True)
        json.dump(base, open(args.baseline, "w", encoding="utf-8"), indent=2)
        log(f"baseline frozen -> {args.baseline}")

    date = iso_now()[:10]
    code = exit_code(findings, stale=False)
    report = write_report(summary, findings, code, args.report_dir, date)
    logdir = os.path.join(args.report_dir, "logs")
    os.makedirs(logdir, exist_ok=True)
    open(os.path.join(logdir, f"{date}.log"), "a", encoding="utf-8").write("\n".join(logs) +
"\n")
    log(f"report: {report}  ·  exit {code}")
    return code


if __name__ == "__main__":
    # allow both `python -m scripts.perf_gate.run` and `python scripts/perf_gate/run.py`
    if __package__ in (None, ""):
        sys.path.insert(0, os.path.dirname(HERE))
        import importlib
        globals()["preflight"] = importlib.import_module("perf_gate.preflight")
        # rebind the relative import used in main()
    raise SystemExit(main())
```

180. user (2026-07-09T03:31:52.184Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py (file state is current in your context ? no need to Read it back)

181. assistant (2026-07-09T03:32:14.708Z)

`run.py` has a few things to tighten: the preflight import must work both as a module and as a direct script, the stitch-A/B gating condition is malformed, and a couple of nested-quote f-strings should use a helper. Let me fix these.

182. user (2026-07-09T03:32:17.811Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

183. user (2026-07-09T03:32:27.282Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

184. user (2026-07-09T03:32:38.331Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

185. user (2026-07-09T03:32:45.824Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your

context ? no need to Read it back)

186. user (2026-07-09T03:32:56.496Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

187. user (2026-07-09T03:33:03.349Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

188. user (2026-07-09T03:33:09.965Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

189. assistant (2026-07-09T03:33:18.589Z)

Let me syntax-check the tool files and check the matrix/watcher progress (the stitch numbers will seed the baseline).

190. user (2026-07-09T03:33:24.721Z)

```
=== py_compile tool files ===
SYNTAX OK
=== matrix progress ===
2026-07-09T03:14:15+00:00  DONE  prod/SmokeTestBadmintonClip rc=0 wall=340.3s ::
[prod/SmokeTestBadmintonClip] checksum=0.206s validation=5.491s segmentation=312.651s
stitch=14.854s total=333.202s segments=6 gpu_peak=64%
2026-07-09T03:14:15+00:00  START rc/SmokeTestBadmintonClip
2026-07-09T03:19:56+00:00  DONE  rc/SmokeTestBadmintonClip rc=0 wall=341.9s ::
[rc/SmokeTestBadmintonClip] checksum=0.2s validation=5.543s segmentation=316.291s stitch=12.921s
total=334.955s segments=6 gpu_peak=61%
2026-07-09T03:19:56+00:00  START prod/GX010075
=== watcher status (b3vx0nxej) still armed? GPU busy? ===
3 %, 5600 MiB
```

191. assistant (2026-07-09T03:34:04.470Z)

All four tool files compile. Matrix is grinding through `prod/GX010075` (5.3K, GPU 3% ? CPU-decode-bound as expected; the watcher is still armed for GX010061). Let me continue the build: the nightly wrapper (mirroring `nightly_regression.ps1`) and the registrar extension.

192. assistant (2026-07-09T03:34:25.175Z)

```
<#
.SYNOPSIS
    Nightly PERFORMANCE gate ? wrapper for the Windows Scheduled Task (KhelSutraNightlyPerf).

.DESCRIPTION
    Runs scripts/perf_gate/run.py: times checksum/validation/segmentation/full-reel-stitch for
the
    commit deployed on api.khelsutra.guru vs repo HEAD, on the local GPU, over the vault clips,
and
    GATES the release (exit 1 on a stitch/total regression beyond tolerance). Writes a dated
report
    + log under output/nightly_perf/ and a LATEST_STATUS.txt marker, then exits with the gate's
```

```
code
    so Task Scheduler records PASS / REGRESSION / STALE.

    This is the PERFORMANCE sibling of scripts/nightly_regression.ps1 (rally-quality). Both are
LOCAL
    scheduled tasks, not cloud routines, because the WASB GPU detector + vault clips live on
THIS
    machine (a cloud runner has no GPU/WSL). It is a SEPARATE task (not folded into the quality
one)
    because a GPU perf run far exceeds the quality leg's 30-minute limit.

    GRACEFUL no-op (exit 0) when the tool isn't present yet OR the environment isn't ready
    (preflight -> the tool's exit 2): a registered task never errors in the interim and self-
activates
    once the perf gate + WSL wasb env land. Inspect LATEST_STATUS.txt for the real state.

.PARAMETER Profile
    Video set: quick | full | stress. Default 'full' (3 clips; excludes the ~2h 5-min stress
clip).

.PARAMETER UpdateBaseline
    Freeze the run's numbers into eval_baselines/perf_baseline.json instead of gating (use after
an
    intended perf change or a new production deploy).
#>
[CmdletBinding()]
param(
    [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent $MyInvocation.MyCommand.Path)),
    [string]$PythonExe = 'python',
    [ValidateSet('quick', 'full', 'stress')][string]$Profile = 'full',
    [switch]$Pull,
    [switch]$UpdateBaseline
)

$ErrorActionPreference = 'Continue'
$date = Get-Date -Format 'yyyy-MM-dd'
$reportDir = Join-Path $RepoRoot 'output\nightly_perf'
$logDir = Join-Path $reportDir 'logs'
New-Item -ItemType Directory -Force -Path $logDir | Out-Null
$log = Join-Path $logDir "$date.wrapper.log"
$statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'

function Write-Status([string]$status, [int]$code, [string]$detail) {
    $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    "$stamp  status=$status exit=$code  $detail" | Out-File -FilePath $statusFile -Encoding utf8
}

$tool = Join-Path $RepoRoot 'scripts\perf_gate\run.py'
if (-not (Test-Path $tool)) {
    "[$date] perf gate not present at $tool ? skipping (awaiting merge)." | Tee-Object -FilePath
$log
    Write-Status 'SKIPPED' 0 'perf gate absent (awaiting merge)'
    exit 0
}

Set-Location $RepoRoot
if ($Pull) {
    "[$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
    & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
}
$head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
"[$date] nightly performance gate ? HEAD $head ? profile $Profile" | Tee-Object -FilePath $log
-Append

$toolArgs = @($tool, '--profile', $Profile)
```

```
if ($UpdateBaseline) { $toolArgs += '--update-baseline' }
& $PythonExe @toolArgs 2>&1 | Tee-Object -FilePath $log -Append
$code = $LASTEXITCODE

switch ($code) {
    0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
    1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
        "[$date] *** PERF REGRESSION ? HEAD slower than production; see $reportDir\$date.md ***"
| Tee-Object -FilePath $log -Append }
    2 { Write-Status 'SKIPPED' 0 'environment not ready (preflight) ? see log; self-activates
once WSL wasb + GPU are set up'
        exit 0 }
    3 { Write-Status 'NOBASELINE' 3 'no perf baseline ? run with -UpdateBaseline once' }
    4 { Write-Status 'STALE' 4 "config/video-set changed vs baseline ? re-freeze with
-UpdateBaseline; see $reportDir\$date.md" }
    default { Write-Status 'ERROR' $code "gate error ? see $log" }
}
exit $code
```

193. user (2026-07-09T03:34:25.243Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\nightly_perf.ps1 (file state is current in your context ? no need to Read it back)
```

194. assistant (2026-07-09T03:34:37.597Z)

Now let me extend the task registrar to install both legs. Let me read the dev-worktree copy (origin/master version) to edit the correct base.

195. user (2026-07-09T03:34:39.059Z)

```
1       <#
2       .SYNOPSIS
3           Register (or remove) the nightly rally-quality regression as a Windows Scheduled
Task.
4
5       .DESCRIPTION
6           Creates a daily user-level task that runs scripts/nightly_regression.ps1 against a
repo
7           checkout. The action is inline-guarded, so it's safe to register BEFORE this work is
merged
8           into the target checkout: it no-ops + logs "awaiting" until the wrapper exists
there, then
9           self-activates. Idempotent (-Force re-registers); reversible (-Unregister).
10
11          Registers under the current user (no admin / no -RunLevel Highest needed ?
consistent with the
12          repo's other scheduled tasks). Uses -StartWhenAvailable so a run missed while the
machine slept
13          is caught up on wake.
14
15      .EXAMPLE
16          # Register against the main checkout, nightly at 03:00:
17          pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
18
19      .EXAMPLE
20          # Remove it:
21          pwsh -File scripts/register_nightly_task.ps1 -Unregister
22      #>
23      [CmdletBinding()]
24      param(
25          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
```

```
$MyInvocation.MyCommand.Path)),
26          [string]$TaskName = 'KhelSutraNightlyRegression',
27          [string]$Time = '03:00',
28          [switch]$Pull,
29          [switch]$Unregister
30      )
31
32      if ($Unregister) {
33          Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
34          Write-Host "Unregistered scheduled task '$TaskName'."
35          return
36      }
37
38      $wrapper = Join-Path $RepoRoot 'scripts\nightly_regression.ps1'
39      $ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
40      if (-not $ps) { $ps = 'powershell.exe' }
41
42      # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit
0.
43      $marker = Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt'
44      $pullArg = if ($Pull) { ' -Pull' } else { '' }
45      $inner = @"
46      `$w = '$wrapper'
47      if (Test-Path `$w) { & `$w -RepoRoot '$RepoRoot'$pullArg }
48      else {
49        New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$marker') | Out-Null
50        "`$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  status=SKIPPED exit=0  wrapper absent
(awaiting merge) at '$wrapper'" | Out-File -FilePath '$marker' -Encoding utf8
51      }
52      "@
53      $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
54
55      $action = New-ScheduledTaskAction -Execute $ps `
56          -Argument "-NoProfile -ExecutionPolicy Bypass -EncodedCommand $encoded"
57      $trigger = New-ScheduledTaskTrigger -Daily -At $Time
58      $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
59          -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
60          -ExecutionTimeLimit (New-TimeSpan -Minutes 30)
61      $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME"
-LogonType Interactive -RunLevel Limited
62
63      Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
64          -Settings $settings -Principal $principal -Force `
65          -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline." | Out-Null
66
67      Write-Host "Registered scheduled task '$TaskName' ? daily at $Time, RepoRoot=$RepoRoot."
68      Write-Host "Wrapper: $wrapper"
69      Write-Host "Run now to test:  Start-ScheduledTask -TaskName '$TaskName'"
70      Write-Host "Remove:           pwsh -File scripts/register_nightly_task.ps1 -Unregister"
71
```

196. assistant (2026-07-09T03:35:22.649Z)

The registrar matches. Let me rewrite it to register **both** legs (quality + a longer-limit perf task), extracting a shared helper and preserving the existing quality behavior exactly.

197. assistant (2026-07-09T03:35:47.606Z)

```
<#
.SYNOPSIS
    Register (or remove) the nightly KhelSutra guards as Windows Scheduled Tasks:
    QUALITY (rally-quality regression) + PERFORMANCE (release perf gate). Both by default.
```

```
.DESCRIPTION
    Creates daily user-level tasks that run the wrapper scripts against a repo checkout:
      * KhelSutraNightlyRegression -> scripts/nightly_regression.ps1  (quality; ~seconds, 30-min
cap)
      * KhelSutraNightlyPerf       -> scripts/nightly_perf.ps1        (perf; GPU, longer cap)
    Each action is inline-guarded, so it's safe to register BEFORE the wrappers are merged into
the
    target checkout: a leg no-ops + logs "awaiting" until its wrapper exists there, then self-
activates.
    Idempotent (-Force re-registers); reversible (-Unregister). Register just one with
-QualityOnly /
    -PerfOnly.

    Both are LOCAL tasks (not cloud routines): the cached trajectories, golden corpus, WASB GPU
    detector and vault clips live on THIS machine. The perf leg is SEPARATE (not folded into
quality)
    because a GPU perf run far exceeds the quality leg's 30-minute limit.

    Registers under the current user (no admin needed). -StartWhenAvailable catches up a run
missed
    while the machine slept.

.EXAMPLE
    # Register both legs against the main checkout (quality 03:00, perf 03:30):
    pwsh -File scripts/register_nightly_task.ps1 -RepoRoot 'C:\Users\avidu\Projects\badminton-
highlight-indexer'

.EXAMPLE
    # Remove both:
    pwsh -File scripts/register_nightly_task.ps1 -Unregister
#>
[CmdletBinding()]
param(
    [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent $MyInvocation.MyCommand.Path)),
    [string]$QualityTaskName = 'KhelSutraNightlyRegression',
    [string]$PerfTaskName = 'KhelSutraNightlyPerf',
    [string]$QualityTime = '03:00',
    [string]$PerfTime = '03:30',
    [ValidateSet('quick', 'full', 'stress')][string]$PerfVideoSet = 'full',
    [int]$PerfTimeLimitMinutes = 180,
    [switch]$Pull,
    [switch]$QualityOnly,
    [switch]$PerfOnly,
    [switch]$Unregister
)

$ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
if (-not $ps) { $ps = 'powershell.exe' }
$doQuality = -not $PerfOnly
$doPerf = -not $QualityOnly

if ($Unregister) {
    if ($doQuality) { Unregister-ScheduledTask -TaskName $QualityTaskName -Confirm:$false
-ErrorAction SilentlyContinue; Write-Host "Unregistered '$QualityTaskName'." }
    if ($doPerf) { Unregister-ScheduledTask -TaskName $PerfTaskName -Confirm:$false -ErrorAction
SilentlyContinue; Write-Host "Unregistered '$PerfTaskName'." }
    return
}

function Register-NightlyLeg {
    param([string]$TaskName, [string]$WrapperPath, [string]$MarkerPath, [string]$ExtraArgs,
        [string]$Time, [int]$TimeLimitMinutes, [string]$Description)
    $pullArg = if ($Pull) { ' -Pull' } else { '' }
    # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit 0.
    $inner = @"
```

```
`$w = '$WrapperPath'
if (Test-Path `$w) { & `$w -RepoRoot '$RepoRoot'$pullArg$ExtraArgs }
else {
  New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$MarkerPath') | Out-Null
  "`$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  status=SKIPPED exit=0  wrapper absent (awaiting
merge) at '$WrapperPath'" | Out-File -FilePath '$MarkerPath' -Encoding utf8
}
"@
    $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
    $action = New-ScheduledTaskAction -Execute $ps -Argument "-NoProfile -ExecutionPolicy Bypass
-EncodedCommand $encoded"
    $trigger = New-ScheduledTaskTrigger -Daily -At $Time
    $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
        -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
        -ExecutionTimeLimit (New-TimeSpan -Minutes $TimeLimitMinutes)
    $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME" -LogonType
Interactive -RunLevel Limited
    Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
        -Settings $settings -Principal $principal -Force -Description $Description | Out-Null
    Write-Host "Registered '$TaskName' ? daily at $Time (limit ${TimeLimitMinutes}m),
RepoRoot=$RepoRoot."
    Write-Host "  Run now: Start-ScheduledTask -TaskName '$TaskName'"
}

if ($doQuality) {
    Register-NightlyLeg -TaskName $QualityTaskName `
        -WrapperPath (Join-Path $RepoRoot 'scripts\nightly_regression.ps1') `
        -MarkerPath (Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt') `
        -ExtraArgs '' -Time $QualityTime -TimeLimitMinutes 30 `
        -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline."
}
if ($doPerf) {
    Register-NightlyLeg -TaskName $PerfTaskName `
        -WrapperPath (Join-Path $RepoRoot 'scripts\nightly_perf.ps1') `
        -MarkerPath (Join-Path $RepoRoot 'output\nightly_perf\LATEST_STATUS.txt') `
        -ExtraArgs " -VideoSet $PerfVideoSet" -Time $PerfTime -TimeLimitMinutes
$PerfTimeLimitMinutes `
        -Description "Nightly performance gate: times checksum/validation/segmentation/full-
reel-stitch for the deployed prod commit vs HEAD on the local GPU (wasb_hybrid). Writes
output/nightly_perf/<date>.md. Exit 1=regression, 4=stale."
}

Write-Host ""
Write-Host "Remove: pwsh -File scripts/register_nightly_task.ps1 -Unregister"
```

198. user (2026-07-09T03:35:47.653Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\register_nightly_task.ps1 has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

199. assistant (2026-07-09T03:35:54.044Z)

```
I used `-VideoSet` in the registrar, so let me align `nightly_perf.ps1`'s parameter name (also
avoids shadowing PowerShell's automatic `$PROFILE`).
```

200. user (2026-07-09T03:35:55.646Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\nightly_perf.ps1 has been updated. All occurrences were successfully replaced. (file
```

state is current in your context ? no need to Read it back)

201. user (2026-07-09T03:36:03.884Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\nightly_perf.ps1 has been updated successfully. (file state is current in your context ? no need to Read it back)

202. assistant (2026-07-09T03:36:11.721Z)

Now the self-help README ? the "instruction documented, mostly self-help" core of the deliverable.

203. assistant (2026-07-09T03:36:48.678Z)

Local release performance gate (`scripts/perf_gate/`)

Standalone, repeatable, release-gating tool that compares the **binary currently serving `api.khelsutra.guru`** against **repo HEAD**, on this machine's GPU, over vault clips ? with **no SPA and no user step**. It times the four release-critical stages independently:

```
checksum  ?  validation  ?  segmentation (wasb_hybrid, GPU)  ?  full-reel stitch
```

It is the **performance sibling** of the quality gates (`scripts/nightly_regression.py`, `scripts/nightly_reelcount.py`) and plugs into the same nightly framework. Run it before every release, or any time, to get precomputed numbers and catch a per-stage regression.

Quickstart

```bash
# 0. One-time: check the environment is ready (self-help: names each missing prereq +
python scripts/perf_gate/run.py --preflight

# 1. Compare prod vs HEAD (quick profile: 2 clips) and gate the release
python scripts/perf_gate/run.py

# 2. More coverage (adds the 5.3K clip); or the full stress set (adds the 5-min clip, ~2h)
python scripts/perf_gate/run.py --profile full
python scripts/perf_gate/run.py --profile stress

# 3. Just the #531 stitch A/B (fast ? no segmentation)
python scripts/perf_gate/run.py --stitch-only

# 4. Freeze the current numbers as the committed release baseline
python scripts/perf_gate/run.py --update-baseline
```

Reports land in `output/nightly_perf/<date>.{md,json}` (+ `logs/<date>.log`). The gate's exit code is the release signal.

What gets compared, and why it's fair

* **Two commits, one machine, identical inputs.** Each side runs from its own throwaway

  `git worktree` (your working tree is never touched), driven by **one canonical config**
  (`config/bench_config.json`), so only the *code* differs. "The prod binary" can't be run
  literally (it's a Linux/L4/NVENC container), so we run the **prod commit** through the same
  harness ? the only apples-to-apples local comparison.
* **Production detector on the GPU.** Segmentation uses `wasb_hybrid`, `detector_impl=auto`
  (the WSL2 GPU path ? the same WASB shuttle detector that serves prod), with the fast
  `stream_video` decode (bit-identical to the PNG path, present in both compared commits).
* **Deterministic & headless.** `skip_ai_handoff=true` (no Gemini / no user step); the
  entire reel is stitched (every rally, watermark on, `libx264` both sides). Validation
  is timed but never halts, so every clip yields all four numbers.
* **Real stitch paths.** The compiler is built by introspection, so the prod commit runs its
  native two-pass watermark stitch and HEAD runs its `single_pass_stitch` default (**#531**).

Gating (release signal)

Only **stitch** and **total** are gated (see `eval_baselines/perf_manifest.json ? gate`):
they're the locally-measurable code path. checksum/validation/segmentation are reported but
not gated ? they're a constant backdrop (same detector) or infra-bound. A **REGRESSION**
fires when HEAD is slower than production on a gated stage by more than `tol_pct` (default
20%). The dedicated stitch A/B is always gated (it's the cleanest #531 signal).

Exit codes match the quality gates:

code	meaning
0	PASS ? no regression
1	REGRESSION ? HEAD slower than prod beyond tolerance
2	nothing ran / preflight failed (env not ready)
3	no baseline (only with baseline gating)
4	STALE ? config/video-set changed vs the frozen baseline

How "production" is resolved

The deployed Cloud Run image is **not** git-sha-labeled yet (tracked in **#532**), so the tool
resolves the prod commit in this order:

1. `--prod-commit <sha>` / `--prod-ref <ref>` if you pass one.
2. **Live Cloud Run**: reads the running `rally-control-plane` image digest and maps it via
   `perf_manifest.json ? prod.image_digest_to_commit`.
3. Falls back to `perf_manifest.json ? prod.pinned_commit`.

On each production deploy, add the new `digest: commit` line to
`prod.image_digest_to_commit`
and bump `pinned_commit` (or land #532 so step 2 needs no map).

Vault clips (no personal paths committed)

The manifest lists clips by **basename**; they're resolved at runtime against `--vault-dir`
(default `$RALLY_VAULT_DIR`, else `~/Videos`). This keeps personal absolute paths out of the
repo (the leak-scan CI guard). Point `--vault-dir` at wherever your clips live, or override the
set with `--videos GX010016,SmokeTestBadmintonClip`.

Nightly integration

* `scripts/nightly_perf.ps1` ? the Scheduled-Task wrapper (mirrors `nightly_regression.ps1`):
  writes `output/nightly_perf/<date>.md` + `LATEST_STATUS.txt`, exits with the gate code.
* `scripts/register_nightly_task.ps1` registers **both** legs by default ? quality
  (`KhelSutraNightlyRegression`, 03:00, 30-min cap) and perf (`KhelSutraNightlyPerf`, 03:30,
  3-hour cap). Perf is a *separate* task because a GPU run far exceeds the quality leg's cap.

```powershell
pwsh -File scripts/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'   # both legs
pwsh -File scripts/register_nightly_task.ps1 -PerfOnly                          # just perf
```

```
Start-ScheduledTask -TaskName 'KhelSutraNightlyPerf'                              # run now
```

Both are **local** tasks (not cloud): the GPU, WSL WASB detector, and vault clips live on this machine. The cloud nightly (`.github/workflows/nightly.yml`) stays offline Tier-1 (no GPU/video).

Troubleshooting (self-help)

`--preflight` names each failing prerequisite and its fix. Common ones:

symptom	fix
`torch+cuda FAIL`	Install a CUDA torch build; verify the NVIDIA driver (`nvidia-smi`).
`wsl:Ubuntu FAIL`	`wsl --install -d Ubuntu`; start it once.
`wsl conda env 'wasb' FAIL`	Create the WASB conda env in WSL (WASB-SBDT requirements).
`wsl WASB weights FAIL`	Place `wasb_badminton_best.pth.tar` at the configured `weights_path`.
`gcloud ? warn`	Optional ? only for auto-resolving prod; pass `--prod-commit` instead.
5.3K clip very slow	Expected: local decode is CPU-bound (prod uses L4 NVDEC). Use `--profile quick`/`full`, keep `stress` for occasional runs.

Files

```
scripts/perf_gate/
  run.py              # entrypoint: resolve commits, preflight, worktrees, run, gate, report
  harness.py          # one (video,version) run ? per-stage JSON (subprocess, version-agnostic)
  stitch_ab.py        # dedicated #531 stitch micro-bench (subprocess)
  preflight.py        # environment checks with self-help fixes
  config/bench_config.json
  README.md           # this file
eval_baselines/perf_manifest.json   # video set, config, gate policy, prod resolution
eval_baselines/perf_baseline.json   # frozen precomputed release numbers (--update-baseline)
scripts/nightly_perf.ps1            # nightly wrapper
scripts/register_nightly_task.ps1   # registers quality + perf scheduled tasks
```

204. user (2026-07-09T03:36:48.998Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\README.md (file state is current in your context ? no need to Read it back)

205. assistant (2026-07-09T03:37:09.882Z)

Tool is built. Let me self-test the **preflight** (both invocation forms ? script and module) from the dev worktree; it's lightweight (won't disturb the matrix's timing) and validates the import paths + env checks.

206. user (2026-07-09T03:37:35.053Z)

```
=== [script form] run.py --preflight ===
Preflight:
  [OK  ] git: git version 2.45.2.windows.1
  [OK  ] ffmpeg: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Winget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffmpeg.EXE
  [OK  ] ffprobe: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Winget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffprobe.EXE
  [OK  ] torch+cuda: torch 2.6.0+cu124 · NVIDIA GeForce RTX 2070 Super with Max-Q Design
  [OK  ] wsl:Ubuntu: ok
  [OK  ] wsl conda env 'wasb': ENV_OK
```

```
  [OK  ] wsl WASB-SBDT repo: ~/models/WASB-SBDT
  [OK  ] wsl WASB weights: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  [OK  ] gcloud (prod auto-resolve) (optional): C:\Users\avidu\AppData\Local\Google\Cloud
SDK\google-cloud-sdk\bin\gcloud.CMD
  => 9/9 ok
EXIT=0

=== [module form] python -m scripts.perf_gate.run --preflight (first 4 lines) ===
Preflight:
  [OK  ] git: git version 2.45.2.windows.1
  [OK  ] ffmpeg: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Wi
nget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffmpeg.EXE
  [OK  ] ffprobe: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.W
inget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffprobe.EXE
EXIT=0

=== matrix progress ===
2026-07-09T03:19:56+00:00  START prod/GX010075
2026-07-09T03:37:26+00:00  DONE  prod/GX010075 rc=0 wall=1049.7s :: [prod/GX010075]
checksum=2.207s validation=50.949s segmentation=882.952s stitch=106.405s total=1042.513s
segments=7 gpu_peak=48%
2026-07-09T03:37:26+00:00  START rc/GX010075
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

207. assistant (2026-07-09T03:38:46.755Z)

Preflight is perfect ? **9/9 OK, exit 0**, with the self-help output I wanted, in both invocation forms. And a headline result just landed: **`prod/GX010075` (5.3K, 7 rallies) stitch = 106.4s** (two-pass) ? when `rc/GX010075` (single-pass) finishes, the #531 delta at 5.3K should be dramatic.

While `rc/GX010075` runs, let me verify commit-readiness of the new files: lint (ruff, matching CI), the leak-scan guard (must not flag paths/MD5s), and JSON validity.

208. user (2026-07-09T03:38:58.584Z)

```
=== ruff check (matches CI: ruff 0.15.16) ===
    = help: Organize imports

scripts\perf_gate\preflight.py:15:8: F401 [*] `sys` imported but unused
   |
13 | import shutil
14 | import subprocess
15 | import sys
   |        ^^^ F401
16 | from dataclasses import dataclass
   |
   = help: Remove unused import: `sys`

scripts\perf_gate\run.py:28:8: F401 [*] `statistics` imported but unused
   |
26 | import os
27 | import shutil
28 | import statistics
   |        ^^^^^^^^^^ F401
29 | import subprocess
30 | import sys
   |
   = help: Remove unused import: `statistics`

Found 3 errors.
[*] 3 fixable with the `--fix` option.

=== leak-scan guard (must be clean: no personal paths / 32-hex MD5 in new files) ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
```

```
leak_scan EXIT=0

=== JSON validity (manifest + config) ===
JSON OK
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

209. assistant (2026-07-09T03:39:10.769Z)

ruff found 3 auto-fixable import nits; leak-scan is **clean** and JSON valid. Let me auto-fix the imports and parse-check the PowerShell files.

210. user (2026-07-09T03:39:14.571Z)

```
=== ruff --fix ===
Found 3 errors (3 fixed, 0 remaining).
=== ruff re-check ===
All checks passed!
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
[This command modified 2 files you've previously read: ..\..\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\run.py, ..\..\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\perf_gate\preflight.py. Call Read before editing.]
```

211. user (2026-07-09T03:39:18.511Z)

```
OK  scripts\nightly_perf.ps1
OK  scripts\register_nightly_task.ps1
```

212. assistant (2026-07-09T03:39:42.702Z)

All green: **ruff clean** after auto-fix, PowerShell wrappers **parse-clean**, leak-scan clean, JSON valid, preflight 9/9. The tool is built and commit-ready. Let me update task state and check the matrix (waiting on `rc/GX010075` ? the watcher fires when it reaches GX010061).

213. user (2026-07-09T03:39:42.787Z)

```
Updated task #5 status
```

214. user (2026-07-09T03:39:42.854Z)

```
Updated task #6 status
```

215. user (2026-07-09T03:39:42.929Z)

```
Updated task #7 status
```

216. user (2026-07-09T03:39:46.374Z)

```
Updated task #8 status
```

217. user (2026-07-09T03:39:46.770Z)

```
Updated task #9 status
```

218. user (2026-07-09T03:39:51.204Z)

```
=== matrix progress ===
2026-07-09T03:19:56+00:00  DONE  rc/SmokeTestBadmintonClip rc=0 wall=341.9s :: [rc/SmokeTestBadmintonClip] checksum=0.2s validation=5.543s segmentation=316.291s stitch=12.921s total=334.955s segments=6 gpu_peak=61%
2026-07-09T03:19:56+00:00  START prod/GX010075
```

```
2026-07-09T03:37:26+00:00  DONE  prod/GX010075 rc=0 wall=1049.7s :: [prod/GX010075]
checksum=2.207s validation=50.949s segmentation=882.952s stitch=106.405s total=1042.513s
segments=7 gpu_peak=48%
2026-07-09T03:37:26+00:00  START rc/GX010075
=== GPU ===
37 %, 5468 MiB
```

219. assistant (2026-07-09T03:40:35.651Z)

`rc/GX010075` is still segmenting (the 5.3K clip, ~17 min each). The tool is built and validated. While I wait for the watcher, let me make one honesty fix to `run.py`: it currently hardcodes `stale=False`, so the documented exit-4 (stale baseline) never fires. Let me wire a minimal real staleness check. First let me see the exact lines (the file was linter-modified).

220. user (2026-07-09T03:40:38.394Z)

```
181-    return summary, findings
182-
183-
184:def exit_code(findings, stale):
185-    if any(f["kind"] == "regression" for f in findings):
186-        return 1
187-    if stale:
--
350-            "gate": gate, "segmenter": manifest.get("segmenter", "wasb_hybrid")}
351-    summary, findings = build(results, stitch_results, [n for n, _ in videos], meta, gate)
352-
353:    if args.update_baseline:
354-        base = {"_doc": "Frozen release perf numbers (scripts/perf_gate/run.py --update-
baseline).",
355-                "git_commit": rc_commit, "prod_commit": prod_commit, "generated_at":
iso_now(),
356-                "profile": args.profile, "gate": gate, **summary}
357-        os.makedirs(os.path.dirname(args.baseline), exist_ok=True)
358-        json.dump(base, open(args.baseline, "w", encoding="utf-8"), indent=2)
359:        log(f"baseline frozen -> {args.baseline}")
360-
361-    date = iso_now()[:10]
362:    code = exit_code(findings, stale=False)
363-    report = write_report(summary, findings, code, args.report_dir, date)
364-    logdir = os.path.join(args.report_dir, "logs")
365-    os.makedirs(logdir, exist_ok=True)
```

221. user (2026-07-09T03:40:50.906Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

222. assistant (2026-07-09T03:40:58.516Z)

Now fix the docstring's exit-code note to match the actual behavior (no `--gate-vs-baseline` flag exists).

223. user (2026-07-09T03:41:01.643Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\perf_gate\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

224. assistant (2026-07-09T03:41:06.140Z)

Let me re-verify the edited `run.py` compiles and lints clean, and check the matrix.

225. user (2026-07-09T03:41:10.430Z)

```
COMPILE OK
All checks passed!
=== matrix progress ===
2026-07-09T03:37:26+00:00  DONE  prod/GX010075 rc=0 wall=1049.7s :: [prod/GX010075]
checksum=2.207s validation=50.949s segmentation=882.952s stitch=106.405s total=1042.513s
segments=7 gpu_peak=48%
2026-07-09T03:37:26+00:00  START rc/GX010075
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

226. assistant (2026-07-09T03:41:43.376Z)

Compile + ruff still clean after the edits. `rc/GX010075` is still segmenting (last data point). Before I offer to commit, let me do a final leak-safety grep on the new files specifically (no personal paths/usernames), since the tool will be committed.

227. user (2026-07-09T03:41:49.209Z)

```
=== grep new files for personal identifiers (should be EMPTY / only placeholders) ===
Binary file scripts/perf_gate/__pycache__/harness.cpython-313.pyc matches
Binary file scripts/perf_gate/__pycache__/preflight.cpython-313.pyc matches
Binary file scripts/perf_gate/__pycache__/stitch_ab.cpython-313.pyc matches
Binary file scripts/perf_gate/__pycache__/__init__.cpython-313.pyc matches
scripts/register_nightly_task.ps1:24:    pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
eval_baselines/perf_manifest.json:15:
"sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943": "304af7b"
--- (end; empty above = clean) ---
=== files created on the branch ===
 M scripts/register_nightly_task.ps1
?? eval_baselines/perf_manifest.json
?? scripts/nightly_perf.ps1
?? scripts/perf_gate/
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

228. assistant (2026-07-09T03:42:40.700Z)

Both grep hits are non-issues: the `C:\Users\avidu\?` line is the **pre-existing** `.EXAMPLE` in the original registrar (unchanged, already CI-clean), and the `sha256:` digest is a 64-hex hash, not a 32-hex MD5 ? which is why `leak_scan` passed. The `.pyc` files are gitignored build artifacts. The branch shows exactly the intended change set.

Here's where the consolidation stands.

? Consolidation complete ? `scripts/perf_gate/` (on branch `feat/perf-release-gate`, cut from `origin/master`, y

Standalone, documented, self-help, release-gating tool:

File	Role
`scripts/perf_gate/run.py`	Entrypoint. Resolves **prod** (live Cloud Run digest ? `image_digest_to_commit` map ? pinned ? `--prod-commit`) vs **HEAD**, preflights, manages throwaway worktrees, runs the matrix + stitch A/B, **gates** RC-vs-prod, writes `output/nightly_perf/<date>.{md,json}`.
`harness.py` / `stitch_ab.py`	The validated per-stage timer + #531 stitch micro-bench (subprocess workers).
`preflight.py`	Self-help env checks ? names each missing prereq **and its fix**.
`config/bench_config.json`	Canonical identical-both-sides config.
`README.md`	Full instructions, methodology, prod-resolution, exit codes, troubleshooting.

| `eval_baselines/perf_manifest.json` | Video set (basenames + `--vault-dir`, leak-safe), gate policy, prod resolution. |

Nightly integration (quality + performance):
- `scripts/nightly_perf.ps1` ? Scheduled-Task wrapper mirroring `nightly_regression.ps1` (dated report, `LATEST_STATUS.txt`, graceful no-op, exit passthrough).
- `scripts/register_nightly_task.ps1` ? now registers **both** `KhelSutraNightlyRegression` (quality, 30-min cap) **and** `KhelSutraNightlyPerf` (perf, 3-hr cap, 03:30). Separate task because the GPU run far exceeds the quality leg's cap.

Commit-readiness verified: `ruff` clean, `leak_scan` clean, JSON valid, both `.ps1` parse-clean, `py_compile` OK, and **preflight 9/9 OK** (both `python scripts/perf_gate/run.py --preflight` and `python -m scripts.perf_gate.run --preflight`).

Exit-code family matches the quality gates: `0 PASS · 1 REGRESSION · 2 nothing-ran/preflight · 4 STALE`.

Design decisions: only **stitch + total** are gated (default tol 20%) ? the locally-measurable code path; checksum/validation/segmentation are reported but not gated (constant backdrop / infra-bound). Videos referenced by basename + `RALLY_VAULT_DIR` so no personal path is committed.

The matrix is finishing its **last** data point (`rc/GX010075`, the 5.3K clip). A headline is already forming: **`prod/GX010075` two-pass stitch = 106.4s** ? `rc`'s single-pass number is landing now, which is the #531 delta at 5.3K.

When it finishes (watcher armed), I'll: stop the matrix ? **self-test the tool end-to-end** (`--stitch-only`) on the freed GPU ? seed `eval_baselines/perf_baseline.json` with real numbers ? hand you the final comparison report. I'll continue automatically when the watcher fires.

229. user (2026-07-09T03:42:46.213Z)

```
=== matrix progress (has rc/GX010075 finished?) ===
2026-07-09T03:19:56+00:00  START prod/GX010075
2026-07-09T03:37:26+00:00  DONE  prod/GX010075 rc=0 wall=1049.7s :: [prod/GX010075]
checksum=2.207s validation=50.949s segmentation=882.952s stitch=106.405s total=1042.513s
segments=7 gpu_peak=48%
2026-07-09T03:37:26+00:00  START rc/GX010075
=== watcher task b3vx0nxej status (has it fired?) ===
```

230. assistant (2026-07-09T03:43:01.268Z)

`rc/GX010075` is still running (the 5.3K clip needs its full ~17 min) ? I need it to finish to capture the rc single-pass stitch number, which is the marquee #531 delta at 5.3K. The watcher will fire the instant the matrix reaches GX010061 (right after `rc/GX010075` completes), and I'll take it from there: stop the matrix, self-test the tool on the freed GPU, seed the baseline, and deliver the final comparison. Waiting on that now.

231. user (2026-07-09T03:43:03.046Z)

can you also move all the nightly and this new work out of the scripts/ and put it in the tests/or someother relevant directory rather than it being a scattered collection of scripts

232. assistant (2026-07-09T03:44:21.025Z)

Good call ? a cohesive gate package is much better than loose scripts. This touches **existing, registered infrastructure** (the nightly quality task references `scripts\nightly_regression.ps1` by path; docs and possibly CI reference these too), so before moving anything I need to map every reference and pick the right home. The reorg happens in the dev worktree and won't disturb the still-running matrix.

Let me audit the full scope and all cross-references.

233. user (2026-07-09T03:44:32.713Z)

```
=== nightly/eval/gate files in scripts/ + root ===
nightly_email.py
nightly_perf.ps1
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
-- root run_*.py --
run_all_tests.py
run_eval.py
run_regression.py

=== references to these script PATHS across repo (docs/CI/py/ps1/md) ===
backend/eval/batch.py:10:or any API. The orchestration/side-effects live in
`scripts/run_phase3_eval.py`.
backend/utils/hashing.py:4:(`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
deploy/gcp/nightly_regression/cloudbuild.launcher.yaml:7:    args: ['build', '-t', '${_IMAGE}',
'-f', 'deploy/gcp/nightly_regression/Dockerfile.launcher', '.']
deploy/gcp/nightly_regression/create_nightly_bucket.sh:4:# Two-bucket model (deliberate ? see
deploy/gcp/nightly_regression/README.md):
deploy/gcp/nightly_regression/create_nightly_bucket.sh:12:#   bash
deploy/gcp/nightly_regression/create_nightly_bucket.sh           # create + 30d TTL (idempotent)
deploy/gcp/nightly_regression/launch.sh:7:#   bash deploy/gcp/nightly_regression/launch.sh
# L4, stages HEAD, kicks the VM
deploy/gcp/nightly_regression/launch.sh:8:#   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
deploy/gcp/nightly_regression/launch.sh:9:#   RALLY_DRY_RUN=1 bash
deploy/gcp/nightly_regression/launch.sh  # stage only, DON'T create the VM (smoke test)
deploy/gcp/nightly_regression/launch.sh:33:EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
deploy/gcp/nightly_regression/launch.sh:36:ROOT="$(cd "$HERE/../../.." && pwd)"   # repo root
(deploy/gcp/nightly_regression ? 3 up)
deploy/gcp/nightly_regression/launch.sh:44:#    Create the bucket first if missing: bash
deploy/gcp/nightly_regression/create_nightly_bucket.sh
deploy/gcp/nightly_regression/README.md:15:> The sibling `scripts/nightly_regression.py` (PR
#202) re-scores **cached** trajectories on **CPU** (the
deploy/gcp/nightly_regression/README.md:17:> `scripts/register_nightly_task.ps1`. Manage that
one with `Get-ScheduledTask`/`Unregister-ScheduledTask`;
deploy/gcp/nightly_regression/README.md:42:| **Email recipient(s)** | `avi.dullu@gmail.com` |
`NIGHTLY_EMAIL_TO`. Multiple = comma-separated (`a@x.com,b@y.com`). Live: `gcloud run jobs
update $J --region $R --update-env-vars NIGHTLY_EMAIL_TO=a@x.com,b@y.com` |
deploy/gcp/nightly_regression/README.md:49:| **Quality-metric tolerance** | `0.02` | runner
`--quality-tol`; to change the scheduled run, edit the `python scripts/nightly_reelcount.py ?`
line in `run_on_vm.sh` |
deploy/gcp/nightly_regression/README.md:56:| **Baseline (expected counts)** |
`eval_baselines/reelcount_baseline.json` | re-freeze with `python scripts/nightly_reelcount.py
--update-baseline` after an intended change; commit it |
deploy/gcp/nightly_regression/README.md:72:| `../../../scripts/nightly_reelcount.py` | the
runner (reel-count + quality + cost + report); pure core unit-tested |
deploy/gcp/nightly_regression/README.md:73:| `../../../scripts/nightly_email.py` | SMTP email
(creds from Secret Manager/env; verified-TLS; graceful skip) |
deploy/gcp/nightly_regression/README.md:85:bash
deploy/gcp/nightly_regression/create_nightly_bucket.sh
deploy/gcp/nightly_regression/README.md:86:printf '%s' '<16-char-gmail-app-pw>' | bash
deploy/gcp/nightly_regression/setup_email.sh   # myaccount.google.com/apppasswords
deploy/gcp/nightly_regression/README.md:90:RALLY_REF=origin/master bash
deploy/gcp/nightly_regression/launch.sh        # creates a self-deleting VM
deploy/gcp/nightly_regression/README.md:91:#    on that VM (or any GPU box): python
scripts/nightly_reelcount.py --update-baseline  ? commit reelcount_baseline.json
deploy/gcp/nightly_regression/README.md:93:bash
deploy/gcp/nightly_regression/register_scheduler.sh --run-now
deploy/gcp/nightly_regression/README.md:99:# or, to re-register and run in one go:  bash
```

deploy/gcp/nightly_regression/register_scheduler.sh --run-now
deploy/gcp/nightly_regression/README.md:104:to `gs://?/nightly-inputs/`, then `python
scripts/nightly_reelcount.py --update-baseline` and commit the
deploy/gcp/nightly_regression/README.md:109:- **Email** to `NIGHTLY_EMAIL_TO` every run (subject
`[KhelSutra nightly] PASS|REGRESSION|STALE ?`).
deploy/gcp/nightly_regression/README.md:147:> pure cores ARE unit-tested
(`tests/test_nightly_reelcount.py`); the cloud chain was validated on a real L4
deploy/gcp/nightly_regression/register_scheduler.sh:7:# ? OWNER-RUN, ONCE, after a successful
manual `bash deploy/gcp/nightly_regression/launch.sh` dry-run.
deploy/gcp/nightly_regression/register_scheduler.sh:11:#   bash
deploy/gcp/nightly_regression/register_scheduler.sh   # build + deploy + schedule (03:00
IST)
deploy/gcp/nightly_regression/register_scheduler.sh:12:#   bash
deploy/gcp/nightly_regression/register_scheduler.sh --run-now # ...and trigger one run
immediately
deploy/gcp/nightly_regression/register_scheduler.sh:44:   --config
deploy/gcp/nightly_regression/cloudbuild.launcher.yaml \
deploy/gcp/nightly_regression/register_scheduler.sh:58: --set-env-vars="RALLY_GPU=${RALLY_GPU:-
l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-
avi.dullu@gmail.com},RALLY_GIT_SHA=${GIT_SHA}"
deploy/gcp/nightly_regression/register_scheduler.sh:77:   Email: bash
deploy/gcp/nightly_regression/setup_email.sh   (Gmail app password ? Secret Manager)
deploy/gcp/nightly_regression/run_on_vm.sh:7:#   run scripts/nightly_reelcount.py on the
gs:// golden clips (emails the report) ?
deploy/gcp/nightly_regression/run_on_vm.sh:55:   NIGHTLY_SMTP_SECRET="$SMTP_SECRET"
NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
deploy/gcp/nightly_regression/run_on_vm.sh:58:from scripts.nightly_email import send_report
deploy/gcp/nightly_regression/run_on_vm.sh:63:   to=os.environ.get("NIGHTLY_EMAIL_TO"))
deploy/gcp/nightly_regression/run_on_vm.sh:109:NIGHTLY_SMTP_SECRET="$SMTP_SECRET"
NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
deploy/gcp/nightly_regression/run_on_vm.sh:110: python scripts/nightly_reelcount.py --vm-
uptime-seconds "$UPTIME" $EMAIL_FLAG
deploy/gcp/nightly_regression/run_on_vm.sh:116:if [-n "$REPORT_BUCKET"] && [-d
output/nightly_reelcount]; then
deploy/gcp/nightly_regression/run_on_vm.sh:117: gcloud storage cp
output/nightly_reelcount/"$DATE".* "gs://$REPORT_BUCKET/$REPORT_PREFIX/$DATE/" 2>/dev/null || \
deploy/gcp/nightly_regression/run_on_vm.sh:118:   gcloud storage cp -r output/nightly_reelcount
"gs://$REPORT_BUCKET/$REPORT_PREFIX/$DATE/" || \
deploy/gcp/nightly_regression/setup_email.sh:3:# worker SA read it. The nightly emails the
report to NIGHTLY_EMAIL_TO (default avi.dullu@gmail.com),
deploy/gcp/nightly_regression/setup_email.sh:10:#   printf '%s' 'xxxxxxxxxxxxxxxx' | bash
deploy/gcp/nightly_regression/setup_email.sh
deploy/gcp/nightly_regression/setup_email.sh:49:   emails NIGHTLY_EMAIL_TO (default
avi.dullu@gmail.com), sending as that Gmail.
deploy/gcp/nightly_regression/setup_email.sh:50:   To send to a different address / send-as a
different account, set NIGHTLY_EMAIL_TO / NIGHTLY_SMTP_USER
docs/ALPHA_LAUNCH_READINESS.md:60:- `[analysis]` `python scripts/nightly_regression.py
--manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1
`0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to
config/corpus changes.
docs/ALPHA_LAUNCH_READINESS.md:136:python scripts\nightly_regression.py --manifest
output\human_lovo_manifest.json --features-dir output --update-baseline
docs/ALPHA_LAUNCH_READINESS.md:151:python scripts\nightly_regression.py --manifest
output\human_lovo_manifest.json --features-dir output --no-report
docs/ALPHA_LAUNCH_READINESS.md:268:python scripts\nightly_regression.py --manifest
<rebased-n15-manifest> --features-dir output --no-report
docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md:140:- **Job (L4) logs**: `gcloud run jobs executions list
--job=rally-worker --region=asia-southeast1`, then `gcloud run jobs executions logs read
<execution> --region=asia-southeast1` ? adapted from the committed nightly-job pattern
(`deploy/gcp/nightly_regression/README.md:111`, which targets `nightly-regression-launcher` in
asia-south1; the adaptation to `rally-worker`/asia-southeast1 is ours). Worker execution logs
were read live during this session's battery (they carried the per-window Gemini traffic in
§5.4), so the log path works; the exact CLI invocation used is not transcribed [unverified].
docs/archives/checkpoints/2026-06-eval-baseline/README.md:33:- Tools: run_eval.py,
run_phase3_eval.py, run_regression.py, backend/eval/.

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:188:10. Optional: small batch via `scripts/run_phase3_eval.py` or `compare_unlabeled` on fresh footage.
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md:189:- verifier notes: Two sharpenings. (a) Failure scenario, concrete: after a SERVED windowing retune the owner re-freezes real fixtures via `--refresh-snapshot --reason ...`; the characterization gate parse-compares against the NEW expected.json so it passes by construction, and with no committed floor (and no windowing fingerprint to raise "incommensurable") an F1 drop from 0.59 to e.g. 0.30 on fx_r0x lands green in CI ? the doc itself flags this rubber-stamp hole at GOLDEN_REGRESSION_FIXTURES.md:141 and names the floor baseline as the mitigation. Severity context keeping it P2 (not P1): the owner-box nightly tripwire (eval_baselines/nightly_baseline.json + scripts/nightly_regression.py) partially covers quality regressions, but only on the local uncommitted corpus, not on the committed fixtures / contributor CI path. (b) Fix note: `backend/eval/regression.py:34` already reserves DEFAULT_BASELINE_PATH = eval_baselines/regression_baseline.json for the DB-backed full-chain regression ? the M4 implementation should either share that schema deliberately or pick a distinct filename to avoid two tools silently contending for one file. Also, the finding's title should not be read as "last blocker to DONE": the P0 video-name-rename remainder (BACKLOG.md:61, GOLDEN_REGRESSION_FIXTURES.md:151 status ?) is a second DoD blocker, though owner-gated ? the finding's "sole UNBLOCKED blocker" phrasing is accurate.
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md:190:- verifier reasoning: Every factual claim reproduced in the current tree (master, up to date with origin/master per `git status -sb`). (1) docs/GOLDEN_REGRESSION_FIXTURES.md:156 ? M4 row ("Quality-floor gate + first `regression_baseline.json`") status ?, PR "?", Depends on M1/M2 which are ? (lines 152-153), Counsel-gated "No" ? unblocked. (2) docs/GOLDEN_REGRESSION_FIXTURES.md:197 ? §9 DoD first checkbox: "**P0, M1?M4, P1, DOC** are ? and merged" ? M4 gates DONE. (3) Artifacts absent: `ls eval_baselines/` shows experiment_leaderboard.json, fixtures/, gemini_wrapper_2026-06-10.json, heuristic_lovo_n6.json, nightly_baseline.json, reelcount_manifest.json ? NO regression_baseline.json; `ls tests/ | grep -i golden` shows test_golden_fixtures.py, test_golden_fixtures_split.py, test_golden_real_fixtures.py ? NO test_golden_quality_floor.py; repo-wide grep for "quality_floor" hits only docs. (4) docs/BACKLOG.md:60 ? M4 entry marked "recommended-next; unblocked" and "the cheapest step to flip the track to DONE", exactly as cited. (5) Not already fixed: `git log --oneline -- docs/GOLDEN_REGRESSION_FIXTURES.md` tops out at 7a40434 (P3 fixtures, #317) ? no M4 commit; eval_baselines history (#202/#217/#239 nightly work) created `nightly_baseline.json`, which is a DIFFERENT mechanism (its own `_doc`: "Tied to the LOCAL golden corpus (not committed)", owner-box `scripts/nightly_regression.py`) and does not satisfy M4's committed video-free floor. (6) Failure scenario is real per the doc's own text: §6 line 141 admits `--refresh-snapshot` "can rubber-stamp a real regression" and names the separate floor-baseline update (= M4) as the mitigation; §4a line 82 confirms fixtures pin raw (vt,mg,mwd) so only M4's windowing fingerprint would flag a post-retune re-freeze as incommensurable. Not owner-gated: M4 row says counsel-gated "No", BACKLOG says unblocked/revisit-anytime; none of the owner-gate categories apply (the P0 rename remainder at BACKLOG.md:61 is the owner-gated sibling, and the finding correctly scopes itself to "the one UNBLOCKED item").
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md:433:- verifier reasoning: deploy/digitalocean/README.md:3-9 carries the DEPRECATED (ADR-013) banner while stating the three scripts are 'still reused by the GCP runbook' with the self-noted 'move to deploy/common/' cleanup ? and the coupling is live: deploy/gcp/README.md:179-182 and deploy/gcp/nightly_regression/run_on_vm.sh:86-91 execute mount_scratch.sh, bootstrap.sh --gpu, and gpu_setup.sh from the retired dir. bootstrap.sh:46 is verifiably one of exactly 5 lockstep trackers pins (requirements.txt:29, .github/workflows/ci.yml:132, .github/workflows/nightly.yml:135, deploy/cloudrun/Dockerfile:62, bootstrap.sh:46). README.md:29 contains 'Rotate the old key that was shared in chat.' git log -- deploy/digitalocean/ (latest touches #402, #386, #292) shows no move/cleanup landed; the dir still exists with all four files.
docs/BACKLOG.md:13:- ? **One-shot "serving startup-script" for the GPU VM** ? `create_gpu_vm.sh` already injects a metadata `startup-script` (`deploy/gcp/ops-agent-startup.sh`), but it only installs the **Ops Agent**; standing up the live engine on the VM (`khelsutra/deploy/GPU_VM_SERVING.md`) is still manual. Propose a `deploy/gcp/serving-startup.sh` fed via `--metadata-from-file=startup-script=?` (or pasted into the GCP console) that, on boot: installs ffmpeg/cv2 libs ? pulls the engine source (from GCS) + builds the venv ? *(optional)* the WASB native env (`gpu_setup.sh` + weights) ? writes the serving `config.json`/`engine.env` ? installs `cloudflared` with the tunnel token ? starts `khelsutra-engine`. So a GPU serving box comes up **hands-off** (reuses the `deploy/gcp/nightly_regression/run_on_vm.sh` bootstrap pattern). **Why deferred ? the design

hinge is SECRETS:** the Gemini key / CF-Access AUD / tunnel token must **not** sit in a plaintext startup-script (instance metadata is broadly readable by anyone with `instances.get`); they should come from **Secret Manager** (VM SA ? `secretAccessor`) or be injected post-boot. Needs that small secrets-from-metadata design + which config is baked vs fetched ? not just a script (owner asked 2026-06-23). **Revisit:** when the GPU-VM serving path is used often enough to automate.
docs/CODE_MAP.md:29:| **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
docs/CODE_MAP.md:105:Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@scripts/nightly_regression.py` (nightly CACHED-trajectory CPU rally-quality tripwire, exit 1=regression/4=stale), `@scripts/nightly_reelcount.py` (nightly FULL-pipeline reel-count + quality + cost on a GCP GPU VM, emailed), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
docs/CODE_MAP.md:122:- `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

234. assistant (2026-07-09T03:46:06.790Z)

Before moving anything, I audited every reference ? and the finding changes the recommendation materially. These scripts are **not actually scattered**; they follow a **documented repo convention** and are wired into deployed infrastructure:

- **`docs/CODE_MAP.md:29`** states an explicit rule: *"Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code) ? **`scripts/`**"*. Moving them out of `scripts/` violates the repo's own convention.
- **A deployed GCP cloud-nightly system** (`deploy/gcp/nightly_regression/`) runs these by path: `run_on_vm.sh` does `from scripts.nightly_email import send_report` and `python scripts/nightly_reelcount.py`. Moving them breaks that import + the scheduled cloud job.
- **`tests/test_nightly_reelcount.py`** imports the reelcount module; a Windows scheduled task points at `scripts\nightly_regression.ps1`; and ~a dozen docs reference these paths.

So **`tests/` is the wrong home** specifically: these are run-drivers, not pytest tests (pytest would try to collect them), and it directly contradicts the CODE_MAP convention. The real problem you're pointing at is that they sit as *loose files* in `scripts/` ? which I can fix by grouping them into a cohesive **package** without leaving `scripts/` (or by moving out entirely, at a higher cost).

Given it touches deployed infra, let me confirm the scope/location rather than guess:

235. user (2026-07-09T03:47:28.577Z)

Your questions have been answered: "How should I consolidate the nightly + perf-gate work? (These follow the repo's 'CLIs live in scripts/' convention and are wired into a deployed GCP nightly + tests, so tests/ is unsuitable.)"="scripts/gates/ package". You can now continue with these answers in mind.

236. assistant (2026-07-09T03:48:21.523Z)

Got it ? cohesive **`scripts/gates/`** package, staying in `scripts/` per the convention, with all references updated in one commit. This is an invasive refactor (path-depth computations in the moved scripts + the deployed GCP nightly + tests + docs), so let me do it methodically. First, let me capture the exact things that will break: the moved scripts' repo-root path computations, the cross-module imports, and the wrapper/registrar internal path references.

237. user (2026-07-09T03:48:29.851Z)

```
=== [1] path-depth computations in the quality .py files (REPO_ROOT etc.) ===
scripts/nightly_regression.py:68:REPO_ROOT =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
scripts/nightly_regression.py:69:if REPO_ROOT not in sys.path:
scripts/nightly_regression.py:71:        0, REPO_ROOT
scripts/nightly_regression.py:73:DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines",
"nightly_baseline.json")
scripts/nightly_regression.py:74:DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output",
"nightly_regression")
scripts/nightly_regression.py:75:DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output",
"human_lovo_manifest.json")
scripts/nightly_regression.py:76:DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
scripts/nightly_regression.py:591:            ["git", "-C", REPO_ROOT, "rev-parse", "--short",
"HEAD"],
scripts/nightly_regression.py:606:    parent = os.path.basename(os.path.dirname(norm))
scripts/nightly_regression.py:610:        rel = os.path.relpath(path, REPO_ROOT)
scripts/nightly_regression.py:678:    os.makedirs(os.path.dirname(path), exist_ok=True)
scripts/nightly_regression.py:690:    p.add_argument("--manifest", default=DEFAULT_MANIFEST,
help="golden manifest JSON")
scripts/nightly_regression.py:697:        "--baseline", default=DEFAULT_BASELINE, help="frozen
baseline JSON path"
scripts/nightly_regression.py:701:        default=DEFAULT_REPORT_DIR,
scripts/nightly_reelcount.py:44:REPO_ROOT =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
scripts/nightly_reelcount.py:45:if REPO_ROOT not in sys.path:
scripts/nightly_reelcount.py:46:    sys.path.insert(0, REPO_ROOT)
scripts/nightly_reelcount.py:51:DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines",
"reelcount_manifest.json")
scripts/nightly_reelcount.py:52:DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines",
"reelcount_baseline.json")
scripts/nightly_reelcount.py:53:DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output",
"nightly_reelcount")
scripts/nightly_reelcount.py:595:            ["git", "-C", REPO_ROOT, "rev-parse", "--short",
"HEAD"],
scripts/nightly_reelcount.py:605:    sha_file = os.path.join(REPO_ROOT, "GIT_SHA")
scripts/nightly_reelcount.py:629:    os.makedirs(os.path.dirname(path), exist_ok=True)
scripts/nightly_reelcount.py:641:    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
scripts/nightly_reelcount.py:642:    p.add_argument("--baseline", default=DEFAULT_BASELINE)

=== [2] cross-module imports of nightly_email / nightly_reelcount / nightly_regression ===
./deploy/gcp/nightly_regression/run_on_vm.sh:58:from scripts.nightly_email import send_report
./scripts/nightly_reelcount.py:763:            from scripts.nightly_email import send_report

=== [3] tests importing these ===
tests/test_doctor.py:17:# scripts/ isn't a package ? load the module by path (mirrors
tests/test_nightly_regression.py).
tests/test_nightly_reelcount.py:1:"""Tests for scripts/nightly_reelcount.py +
scripts/nightly_email.py ? the pure cores.
tests/test_nightly_reelcount.py:30:nr = _load("nightly_reelcount")
tests/test_nightly_reelcount.py:31:ne = _load("nightly_email")
tests/test_nightly_regression.py:1:"""Tests for scripts/nightly_regression.py ? the pure
summary/comparison/report core.
tests/test_nightly_regression.py:21:    "nightly_regression.py",
tests/test_nightly_regression.py:23:_spec =
importlib.util.spec_from_file_location("nightly_regression", _SCRIPT)
tests/test_nightly_regression.py:25:sys.modules["nightly_regression"] = nr

=== [4] .ps1 internal path refs (wrappers + registrar) ===
scripts/nightly_regression.ps1:6:    Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
scripts/nightly_regression.ps1:48:$checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
scripts/nightly_regression.ps1:53:    "[$date] nightly_regression.py not present at $checker ?
skipping (awaiting merge)." | Tee-Object -FilePath $log
scripts/nightly_perf.ps1:6:    Runs scripts/perf_gate/run.py: times
checksum/validation/segmentation/full-reel-stitch for the
```

```
scripts/nightly_perf.ps1:12:    This is the PERFORMANCE sibling of
scripts/nightly_regression.ps1 (rally-quality). Both are LOCAL
scripts/nightly_perf.ps1:50:$tool = Join-Path $RepoRoot 'scripts\perf_gate\run.py'
scripts/register_nightly_task.ps1:8:      * KhelSutraNightlyRegression ->
scripts/nightly_regression.ps1  (quality; ~seconds, 30-min cap)
scripts/register_nightly_task.ps1:9:      * KhelSutraNightlyPerf       ->
scripts/nightly_perf.ps1        (perf; GPU, longer cap)
scripts/register_nightly_task.ps1:84:        -WrapperPath (Join-Path $RepoRoot
'scripts\nightly_regression.ps1') `
scripts/register_nightly_task.ps1:91:        -WrapperPath (Join-Path $RepoRoot
'scripts\nightly_perf.ps1') `
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

238. assistant (2026-07-09T03:50:03.385Z)

Now I understand the full blast radius precisely:
- Both quality scripts compute `REPO_ROOT` as **2 levels up** (`dirname(dirname(__file__))`) ? moving to `scripts/gates/quality/` needs **4 up**; perf `run.py` needs 3 up.
- `scripts/` is a **namespace package** (tests load by file path via importlib; `nightly_reelcount.py` + `run_on_vm.sh` do `from scripts.nightly_email import ...`).
- Wrappers/registrar/docs hardcode the old paths.

Let me read the exact test loaders and the deployed `run_on_vm.sh` lines so I edit them correctly.

239. user (2026-07-09T03:50:05.359Z)

```
1       """Tests for scripts/nightly_reelcount.py + scripts/nightly_email.py ? the pure cores.
2
3       Fully offline: no GPU, no video, no network, no real corpus. The comparison / cost /
report / email
4       formatting operate on plain dicts + strings, so reel-count regressions, quality drops,
pipeline
5       errors, corpus drift, and the no-creds email skip are all exercised deterministically.
The actual
6       pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on
the VM.
7       """
8
9       import importlib.util
10      import os
11      import sys
12
13      import pytest
14
15      _SCRIPTS = os.path.join(
16          os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "scripts"
17      )
18
19
20      def _load(modname):
21          spec = importlib.util.spec_from_file_location(
22              modname, os.path.join(_SCRIPTS, f"{modname}.py")
23          )
24          mod = importlib.util.module_from_spec(spec)
25          sys.modules[modname] = mod  # register before exec so dataclass annotations resolve
26          spec.loader.exec_module(mod)
27          return mod
28
29
30      nr = _load("nightly_reelcount")
31      ne = _load("nightly_email")
32
33
34      # ---------------------------------------------------------------------- #
```

```
35      # Builders
```

240. user (2026-07-09T03:50:06.871Z)

```
1       """Tests for scripts/nightly_regression.py ? the pure summary/comparison/report core.
2
3       Fully offline: no real golden corpus, no trajectory I/O, no GPU/numpy. The comparison
core
4       operates on plain summary dicts, so a regression / over-seg-guard / corpus-change can be
5       exercised deterministically. The real-corpus scoring path (``run_corpus``) is a thin
wrapper
6       over ``rally_seg_eval.evaluate`` (tested in test_rally_seg_eval.py) and is exercised e2e
on the
7       owner's machine, not here.
8       """
9
10      import importlib.util
11      import os
12      import sys
13
14      import pytest
15
16      # scripts/ isn't a package ? load the module by path. Register in sys.modules BEFORE
exec so
17      # dataclasses can resolve the `from __future__ import annotations` string annotations.
18      _SCRIPT = os.path.join(
19          os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
20          "scripts",
21          "nightly_regression.py",
22      )
23      _spec = importlib.util.spec_from_file_location("nightly_regression", _SCRIPT)
24      nr = importlib.util.module_from_spec(_spec)
25      sys.modules["nightly_regression"] = nr
26      _spec.loader.exec_module(nr)
27
28
29      # ---------------------------------------------------------------------------- #
30      # Builders
```

241. user (2026-07-09T03:50:08.280Z)

```
50      fail_email() {
51        echo "RUN FAILED ? attempting a failure alert"
52        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally/.venv ]; then
53          cd ~/rally && . .venv/bin/activate 2>/dev/null && \
54          FAIL_DATE="$DATE" FAIL_BUCKET="$REPORT_BUCKET" FAIL_PREFIX="$REPORT_PREFIX"
FAIL_SHA="$GIT_SHA" \
55          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
56          python - <<'PY' || true
57      import os
58      from scripts.nightly_email import send_report
59      d, b, p, sha = (os.environ.get(k, "") for k in ("FAIL_DATE", "FAIL_BUCKET",
"FAIL_PREFIX", "FAIL_SHA"))
60      send_report(f"[KhelSutra nightly] ERROR ? engine regression VM run failed ({d})",
61                  f"The nightly engine-regression run failed on the VM before producing a
report.\n"
62                  f"See gs://{b}/{p}/{d}/ and the VM serial log.\nGIT_SHA={sha}",
63                  to=os.environ.get("NIGHTLY_EMAIL_TO"))
64      PY
65        fi
66      }
67      trap fail_email ERR
68
69      # Critical inputs must be present (the EXIT trap still self-deletes on this exit).
```

```
70      if [ -z "$REPO_TGZ" ]; then echo "FATAL: repo-tgz-uri metadata missing ? nothing to
run."; exit 1; fi
71      if [ -z "$NAME" ] || [ -z "$ZONE" ]; then
72        echo "WARN: instance name/zone unreadable up front ? self-delete at the end may fail
(see cleanup)."
73      fi
74
75      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in (non-fatal).
76      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
77        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
78
79      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
80      mkdir -p ~/rally && cd ~/rally
81      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
82      tar -xzf /tmp/rally.tgz -C ~/rally
83      echo "$GIT_SHA" > ~/rally/GIT_SHA
84
85      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
86      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
87      export TMPDIR=/scratch 2>/dev/null || true
88      ./deploy/digitalocean/bootstrap.sh --gpu
89      . .venv/bin/activate
90      pip install -e '.[storage-gcs]'
91      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
92
93      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4) + export
the FULL native-
94      # runner env. WASB_WEIGHTS alone is NOT enough ? the native runner also needs
WASB_REPO_DIR and the
95      # separate `wasb` conda-env python (WASB_PYTHON); without them the real WASB run fails
(validated on an
96      # L4, 2026-06-22). The conda env is built by gpu_setup.sh under
$HOME/miniconda3/envs/wasb.
97      if [ -n "$WEIGHTS_URI" ]; then
98        mkdir -p ~/models/WASB-SBDT/pretrained_weights
99        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
100       export WASB_WEIGHTS=$HOME/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
101     fi
102     export WASB_REPO_DIR="${WASB_REPO_DIR:-$HOME/models/WASB-SBDT}"
103     export WASB_PYTHON="${WASB_PYTHON:-$HOME/miniconda3/envs/wasb/bin/python}"
104
105     # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
106     UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
107     EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
108     set +e   # the runner exits 1 (regression) / 4 (stale) by design ? capture, don't abort
on it
109     NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
110       python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
111     RUN_RC=$?
112     set -e
113     echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
114
```

242. assistant (2026-07-09T03:50:43.645Z)

I have every reference now. Let me execute the reorg in the dev worktree ? moves first (`git mv` preserves history for tracked files), then I'll fix the path-depth computations, imports, and

all references.

243. user (2026-07-09T03:50:52.292Z)

```
=== git mv tracked files ===
=== mv untracked (new) files ===
=== resulting tree ===
scripts/gates/nightly_perf.ps1
scripts/gates/nightly_regression.ps1
scripts/gates/perf/config/bench_config.json
scripts/gates/perf/harness.py
scripts/gates/perf/preflight.py
scripts/gates/perf/README.md
scripts/gates/perf/run.py
scripts/gates/perf/stitch_ab.py
scripts/gates/quality/nightly_email.py
scripts/gates/quality/nightly_reelcount.py
scripts/gates/quality/nightly_regression.py
scripts/gates/register_nightly_task.ps1
=== remaining nightly/perf refs in scripts/ root (should be none) ===
(clean ? none left in scripts/ root)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

244. assistant (2026-07-09T03:52:11.300Z)

Moves are clean (namespace convention kept ? removed the `__init__.py`). Now the surgical fixes. Let me update the path-depth computations, the cross-module import, the deployed `run_on_vm.sh`, and the two test loaders ? then immediately verify with `py_compile` and grep.

245. user (2026-07-09T03:52:24.178Z)

```
=== verify path-depth (should show 4x dirname for quality, 3x .. for perf) ===
scripts/gates/quality/nightly_regression.py:68:REPO_ROOT =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))))
scripts/gates/quality/nightly_reelcount.py:44:REPO_ROOT =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))))
34:REPO_ROOT = os.path.abspath(os.path.join(HERE, "..", "..", ".."))
=== verify import + invocation ===
scripts/gates/quality/nightly_reelcount.py:763:            from
scripts.gates.quality.nightly_email import send_report
deploy/gcp/nightly_regression/run_on_vm.sh:58:from scripts.gates.quality.nightly_email import
send_report
deploy/gcp/nightly_regression/run_on_vm.sh:110:  python
scripts/gates/quality/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
=== verify test loaders ===
tests/test_nightly_reelcount.py:16:
os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "scripts", "gates", "quality"
tests/test_nightly_regression.py:21:    "gates",
=== py_compile moved python ===
COMPILE OK
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

246. assistant (2026-07-09T03:52:34.590Z)

All code fixes verified and compiling. Let me run the affected tests to confirm the moved modules still load + pass from their new location (they load by file path).

247. user (2026-07-09T03:52:44.849Z)

```
=== targeted tests for the moved modules ===
..........................................................              [100%]
58 passed in 3.45s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

248. assistant (2026-07-09T03:53:17.999Z)

58 passed ? the moved quality modules load and pass from `scripts/gates/quality/`. Now the PowerShell wrappers/registrar (both forward-slash docs and backslash paths), then verify with grep + parse-check.

249. user (2026-07-09T03:53:28.532Z)

```
=== verify: remaining OLD paths in ps1 (should be EMPTY) ===
nightly_perf.ps1:50:$tool = Join-Path $RepoRoot 'scripts\perf_gate\run.py'
nightly_regression.ps1:48:$checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
register_nightly_task.ps1:84:        -WrapperPath (Join-Path $RepoRoot
'scripts\nightly_regression.ps1') `
register_nightly_task.ps1:91:        -WrapperPath (Join-Path $RepoRoot
'scripts\nightly_perf.ps1') `
=== verify: NEW gates paths present ===
nightly_perf.ps1:6:    Runs scripts/gates/perf/run.py: times
checksum/validation/segmentation/full-reel-stitch for the
nightly_perf.ps1:12:    This is the PERFORMANCE sibling of scripts/gates/nightly_regression.ps1
(rally-quality). Both are LOCAL
nightly_regression.ps1:6:    Re-scores the golden corpus under the current checkout via
scripts/gates/quality/nightly_regression.py
register_nightly_task.ps1:8:      * KhelSutraNightlyRegression ->
scripts/gates/nightly_regression.ps1  (quality; ~seconds, 30-min cap)
register_nightly_task.ps1:9:      * KhelSutraNightlyPerf       -> scripts/gates/nightly_perf.ps1
(perf; GPU, longer cap)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

250. assistant (2026-07-09T03:54:04.010Z)

The forward-slash docs updated, but the **backslash** paths didn't match (sed backslash escaping). Let me use Edit for those 4 lines ? safer. First reading the three wrappers at their new paths.

251. user (2026-07-09T03:54:06.681Z)

```
48      }
49
50      $tool = Join-Path $RepoRoot 'scripts\perf_gate\run.py'
51      if (-not (Test-Path $tool)) {
```

252. user (2026-07-09T03:54:08.228Z)

```
46      }
47
48      $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49      $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
```

253. user (2026-07-09T03:54:09.425Z)

```
82      if ($doQuality) {
83          Register-NightlyLeg -TaskName $QualityTaskName `
84              -WrapperPath (Join-Path $RepoRoot 'scripts\nightly_regression.ps1') `
85              -MarkerPath (Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt')
`
86              -ExtraArgs '' -Time $QualityTime -TimeLimitMinutes 30 `
87              -Description "Nightly rally-quality regression tripwire: re-scores the golden
corpus (cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline."
88      }
89      if ($doPerf) {
90          Register-NightlyLeg -TaskName $PerfTaskName `
91              -WrapperPath (Join-Path $RepoRoot 'scripts\nightly_perf.ps1') `
92              -MarkerPath (Join-Path $RepoRoot 'output\nightly_perf\LATEST_STATUS.txt') `
```

```
93              -ExtraArgs " -VideoSet $PerfVideoSet" -Time $PerfTime -TimeLimitMinutes
$PerfTimeLimitMinutes `
```

254. assistant (2026-07-09T03:54:13.092Z)

Now the precise Edits for the backslash paths:

255. user (2026-07-09T03:54:17.532Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your context ? no need to Read it back)

256. user (2026-07-09T03:54:19.759Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\nightly_regression.ps1 has been updated successfully. (file state is current in your context ? no need to Read it back)

257. user (2026-07-09T03:54:21.985Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\register_nightly_task.ps1 has been updated successfully. (file state is current in your context ? no need to Read it back)

258. user (2026-07-09T03:54:23.668Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\register_nightly_task.ps1 has been updated successfully. (file state is current in your context ? no need to Read it back)

259. assistant (2026-07-09T03:55:03.568Z)

ps1 backslash paths fixed via Edit. For the remaining perf-file/doc/manifest updates (some use backslashes, which sed mangled), let me use a robust Python replacer ? precise per-file mappings, backslash-safe ? then verify nothing stale remains.

260. assistant (2026-07-09T03:55:14.524Z)

```
"""One-shot, backslash-safe reference updater for the scripts/gates/ reorg. Run once, then
deleted."""
import os

DEV = r"C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev"

# file -> list of (old, new) literal replacements
JOBS = {
    # perf-gate internal refs (paths + dotted module)
    "scripts/gates/perf/run.py": [("scripts/perf_gate/", "scripts/gates/perf/")],
    "scripts/gates/perf/preflight.py": [("scripts/perf_gate/", "scripts/gates/perf/"),
                                        ("scripts.perf_gate.", "scripts.gates.perf.")],
    "scripts/gates/perf/harness.py": [("scripts/perf_gate/", "scripts/gates/perf/")],
    "scripts/gates/perf/stitch_ab.py": [("scripts/perf_gate/", "scripts/gates/perf/")],
    "scripts/gates/perf/README.md": [("scripts/perf_gate/", "scripts/gates/perf/"),
                                     ("scripts.perf_gate.", "scripts.gates.perf.")],
    "eval_baselines/perf_manifest.json": [("scripts/perf_gate/", "scripts/gates/perf/")],
    # deployed GCP quality nightly docs
    "deploy/gcp/nightly_regression/README.md": [
```

```
        ("scripts/nightly_reelcount.py", "scripts/gates/quality/nightly_reelcount.py"),
        ("scripts/nightly_email.py", "scripts/gates/quality/nightly_email.py"),
        ("scripts/nightly_regression.py", "scripts/gates/quality/nightly_regression.py"),
        ("scripts/register_nightly_task.ps1", "scripts/gates/register_nightly_task.ps1"),
    ],
    # canonical code map
    "docs/CODE_MAP.md": [
        ("scripts/nightly_regression.py", "scripts/gates/quality/nightly_regression.py"),
        ("scripts/nightly_reelcount.py", "scripts/gates/quality/nightly_reelcount.py"),
    ],
    # alpha readiness (forward AND backslash refs)
    "docs/ALPHA_LAUNCH_READINESS.md": [
        ("scripts/nightly_regression.py", "scripts/gates/quality/nightly_regression.py"),
        ("scripts\\nightly_regression.py", "scripts\\gates\\quality\\nightly_regression.py"),
    ],
}

for rel, subs in JOBS.items():
    p = os.path.join(DEV, rel)
    if not os.path.exists(p):
        print(f"MISS {rel} (not found)")
        continue
    s = open(p, encoding="utf-8").read()
    n = 0
    for old, new in subs:
        c = s.count(old)
        if c:
            s = s.replace(old, new)
            n += c
    open(p, "w", encoding="utf-8", newline="\n").write(s)
    print(f"{rel}: {n} replacements")
print("done")
```

261. user (2026-07-09T03:55:14.636Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\_reorg_refs.py (file
state is current in your context ? no need to Read it back)
```

262. assistant (2026-07-09T03:55:34.970Z)

```
The watcher fired ? the matrix reached GX010061, so `rc/GX010075` is done and I need to **stop
the matrix now** before the 5-min clip runs its ~2 hours. Let me kill it (and capture
`rc/GX010075`'s result), and run the reorg-refs updater in parallel.
```

263. user (2026-07-09T03:55:42.486Z)

```
Exit code 9
kill 6840: perf_driver.py
kill 22728: perf_harness.py --worktree C:\Users\avid
--- remaining perf procs (should be none) ---
0
```

264. assistant (2026-07-09T03:55:50.961Z)

```
Matrix stopped cleanly (both Python procs + WSL wasb killed, 0 remaining; the driver task
ended). Now let me run the reference updater, verify no stale refs remain, and capture
`rc/GX010075`'s result (the 5.3K stitch delta).
```

265. user (2026-07-09T03:55:58.859Z)

```
=== run reorg refs updater ===
scripts/gates/perf/run.py: 10 replacements
scripts/gates/perf/preflight.py: 2 replacements
```

```
scripts/gates/perf/harness.py: 0 replacements
scripts/gates/perf/stitch_ab.py: 0 replacements
scripts/gates/perf/README.md: 8 replacements
eval_baselines/perf_manifest.json: 2 replacements
deploy/gcp/nightly_regression/README.md: 8 replacements
docs/CODE_MAP.md: 6 replacements
docs/ALPHA_LAUNCH_READINESS.md: 4 replacements
done

=== VERIFY: any remaining stale refs anywhere in the repo? (should be empty) ===
./deploy/gcp/nightly_regression/run_on_vm.sh:7:#        run scripts/nightly_reelcount.py on the
gs:// golden clips (emails the report)  ?
./docs/archives/past_projects/AUDIT_REPORT_khelsutra-
guru_2026-07-02-COMPLETED-2026-07-02.md:189:- verifier notes: Two sharpenings. (a) Failure
scenario, concrete: after a SERVED windowing retune the owner re-freezes real fixtures via
`--refresh-snapshot --reason ...`; the characterization gate parse-compares against the NEW
expected.json so it passes by construction, and with no committed floor (and no windowing
fingerprint to raise "incommensurable") an F1 drop from 0.59 to e.g. 0.30 on fx_r0x lands green
in CI ? the doc itself flags this rubber-stamp hole at GOLDEN_REGRESSION_FIXTURES.md:141 and
names the floor baseline as the mitigation. Severity context keeping it P2 (not P1): the owner-
box nightly tripwire (eval_baselines/nightly_baseline.json + scripts/nightly_regression.py)
partially covers quality regressions, but only on the local uncommitted corpus, not on the
committed fixtures / contributor CI path. (b) Fix note: `backend/eval/regression.py:34` already
reserves DEFAULT_BASELINE_PATH = eval_baselines/regression_baseline.json for the DB-backed full-
chain regression ? the M4 implementation should either share that schema deliberately or pick a
distinct filename to avoid two tools silently contending for one file. Also, the finding's title
should not be read as "last blocker to DONE": the P0 video-name-rename remainder (BACKLOG.md:61,
GOLDEN_REGRESSION_FIXTURES.md:151 status ?) is a second DoD blocker, though owner-gated ? the
finding's "sole UNBLOCKED blocker" phrasing is accurate.
./docs/archives/past_projects/AUDIT_REPORT_khelsutra-
guru_2026-07-02-COMPLETED-2026-07-02.md:190:- verifier reasoning: Every factual claim reproduced
in the current tree (master, up to date with origin/master per `git status -sb`). (1)
docs/GOLDEN_REGRESSION_FIXTURES.md:156 ? M4 row ("Quality-floor gate + first
`regression_baseline.json`") status ?, PR "?", Depends on M1/M2 which are ? (lines 152-153),
Counsel-gated "No" ? unblocked. (2) docs/GOLDEN_REGRESSION_FIXTURES.md:197 ? §9 DoD first
checkbox: "**P0, M1?M4, P1, DOC** are ? and merged" ? M4 gates DONE. (3) Artifacts absent: `ls
eval_baselines/` shows experiment_leaderboard.json, fixtures/, gemini_wrapper_2026-06-10.json,
heuristic_lovo_n6.json, nightly_baseline.json, reelcount_manifest.json ? NO
regression_baseline.json; `ls tests/ | grep -i golden` shows test_golden_fixtures.py,
test_golden_fixtures_split.py, test_golden_real_fixtures.py ? NO test_golden_quality_floor.py;
repo-wide grep for "quality_floor" hits only docs. (4) docs/BACKLOG.md:60 ? M4 entry marked
"recommended-next; unblocked" and "the cheapest step to flip the track to DONE", exactly as
cited. (5) Not already fixed: `git log --oneline -- docs/GOLDEN_REGRESSION_FIXTURES.md` tops out
at 7a40434 (P3 fixtures, #317) ? no M4 commit; eval_baselines history (#202/#217/#239 nightly
work) created `nightly_baseline.json`, which is a DIFFERENT mechanism (its own `_doc`: "Tied to
the LOCAL golden corpus (not committed)", owner-box `scripts/nightly_regression.py`) and does
not satisfy M4's committed video-free floor. (6) Failure scenario is real per the doc's own
text: §6 line 141 admits `--refresh-snapshot` "can rubber-stamp a real regression" and names the
separate floor-baseline update (= M4) as the mitigation; §4a line 82 confirms fixtures pin raw
(vt,mg,mwd) so only M4's windowing fingerprint would flag a post-retune re-freeze as
incommensurable. Not owner-gated: M4 row says counsel-gated "No", BACKLOG says
unblocked/revisit-anytime; none of the owner-gate categories apply (the P0 rename remainder at
BACKLOG.md:61 is the owner-gated sibling, and the finding correctly scopes itself to "the one
UNBLOCKED item").
./docs/DECISION_SNAPSHOT_REGRESSION.md:11:> **Relation to existing docs:** **extends**
`GOLDEN_REGRESSION_FIXTURES.md` (post-detector fixtures) and the existing characterization
snapshot in `tests/test_golden_fixtures.py`; **peer of** the M4 quality-floor track and the GPU
`scripts/nightly_reelcount.py` e2e nightly.
./docs/DECISION_SNAPSHOT_REGRESSION.md:20:"Presumably quality-neutral" changes (refactors) need
a cheap, deterministic proof that they actually preserve behavior ? so they can be merged (and
auto-signed-off) with confidence. Today we have two weaker tools: **AST-identity** [verified]
(used to bless `ruff format` in #379) only proves *formatting* is unchanged ? it cannot bless a
real refactor (function extraction, rename, loop reflow) that changes the AST but should
preserve behavior; and the full **real-video F1 / reel-count e2e** needs GPU + the gitignored
corpus (`scripts/nightly_reelcount.py`) [verified] so it can't gate a PR. The decision-snapshot
```

fills the gap: a deterministic, offline, behavior-level proof.
./docs/DECISION_SNAPSHOT_REGRESSION.md:24:**Read first:**
`tests/test_golden_fixtures.py::test_replay_matches_committed_expected` [verified] (the existing
snapshot ? its own message: *"behaviour changed vs frozen snapshot (characterization, not
correctness)"*), `backend/eval/golden_fixtures.py`, `backend/pipeline/stitcher/compiler.py` (the
concat plan), audit doc **B1** (`motion`'s random fields), `scripts/nightly_reelcount.py` (the
distinct GPU e2e).
./docs/DECISION_SNAPSHOT_REGRESSION.md:90:**Internal code `[verified]`:**
`tests/test_golden_fixtures.py` (`test_replay_matches_committed_expected`),
`backend/eval/golden_fixtures.py`, `eval_baselines/fixtures/`,
`backend/pipeline/stitcher/compiler.py`, `backend/pipeline/segmenters/motion.py` (B1 random).
Internal docs: `GOLDEN_REGRESSION_FIXTURES.md`, `CODE_CLEANUP_AUDIT_2026-06-24.md` (B1),
`docs/NEXT_STEPS.md` (M4 quality-floor), `PROJECT_DOC_TEMPLATE.md`;
`scripts/nightly_reelcount.py` (distinct GPU e2e). **Related:** the AST-identity sign-off used
on #379; the Tier-1 nightly (#381).
./docs/NIGHTLY_ENGINE_REGRESSION.md:7:> **Relation to existing docs:** peer-of
`scripts/nightly_regression.py` (PR #202 ? the CACHED-trajectory CPU tripwire); this is the
FULL-pipeline GPU sibling. Reuses `deploy/gcp/` + `backend/billing.py` +
`backend/eval/rally_seg_eval.py`.
./docs/NIGHTLY_ENGINE_REGRESSION.md:18:The owner wants a nightly test that runs the *shipped*
engine on a few real videos and catches any change in how many highlight reels it produces (plus
quality drift), with the cost of the run, emailed. PR #202 already guards the windowing/scorer
via a CPU re-score of *cached* trajectories; it does **not** run the real model or produce
reels. This closes that gap by running the full pipeline on a GPU. Read:
`deploy/gcp/nightly_regression/README.md` (the runbook), `deploy/gcp/README.md` (the GCP infra),
`scripts/nightly_reelcount.py`.
./docs/NIGHTLY_ENGINE_REGRESSION.md:37:- *New:* `scripts/nightly_reelcount.py` (runner),
`scripts/nightly_email.py` (email), `deploy/gcp/nightly_regression/*` (orchestration),
`eval_baselines/reelcount_manifest.json` (corpus).
./docs/NIGHTLY_ENGINE_REGRESSION.md:74:| E1 | `scripts/nightly_reelcount.py` runner (reel-count
+ quality + cost; pure core + IO shell) | ? | No | ? | this PR |
./docs/NIGHTLY_ENGINE_REGRESSION.md:75:| E2 | `scripts/nightly_email.py` (SMTP / Secret Manager,
graceful) | ? | No | ? | this PR |
./docs/NIGHTLY_ENGINE_REGRESSION.md:119:**Internal code anchors `[verified]`:**
`scripts/nightly_reelcount.py`, `scripts/nightly_email.py`, `deploy/gcp/nightly_regression/*`,
`backend/billing.py`, `backend/eval/rally_seg_eval.py::score_intervals`,
`backend/storage/__init__.py::fetch`, `deploy/gcp/create_gpu_vm.sh`. **Person-cue lane
`[verified]`:**
`backend/pipeline/detectors/person_detector.py::extract_action_windows_from_chunk` +
`::make_tracker` (the pinned ByteTrack params),
`backend/pipeline/segmenters/yolo_hybrid.py:484-488` (config-derived
`velocity_thresh`/`merge_gap`/`frame_skip`); **motivating review:** PR #386 (ByteTrack
migration). **Internal docs:** `deploy/gcp/nightly_regression/README.md`,
`deploy/gcp/README.md`, `docs/R2_EVAL_METRIC_DESIGN.md`, memory `eval-dual-set-regression`.
Sibling: `scripts/nightly_regression.py` (PR #202).
./docs/QUALITY_ITERATIONS.md:244:is **CPU, seconds, no GPU/Gemini**
(`scripts/nightly_regression.py` ? `rally_seg_eval.evaluate`).
./docs/QUALITY_ITERATIONS.md:258: (`scripts/nightly_regression.ps1` +
`scripts/register_nightly_task.ps1`) ? a cloud agent can't run it because
./docs/README.md:31:-
[`deploy/gcp/nightly_regression/README.md`](../deploy/gcp/nightly_regression/README.md) ? **?
Nightly engine-regression OPERATIONS MANUAL (one-stop shop):** enable · run on demand · change
any setting (email recipients/frequency/clips/config) · observe · pause/disable/remove. The full
WASB-pipeline reel-count+cost+email nightly on an ephemeral GCP GPU VM (design/status:
[`NIGHTLY_ENGINE_REGRESSION.md`](NIGHTLY_ENGINE_REGRESSION.md)). The lighter CPU cached-
trajectory tripwire is `scripts/nightly_regression.py` (local Windows Scheduled Task, PR #202).
./eval_baselines/nightly_baseline.json:2: "_doc": "Frozen nightly rally-quality regression
baseline. Regenerate with `python scripts/nightly_regression.py --update-baseline` after an
intended improvement or a corpus change. Tied to the LOCAL golden corpus (not committed).",
./eval_baselines/reelcount_manifest.json:2: "_doc": "Nightly E2E engine reel-count manifest
(scripts/nightly_reelcount.py). The configured+checkpointed pipeline runs on each video on a GCP
GPU VM; the reel count + quality metrics are asserted vs eval_baselines/reelcount_baseline.json.
ADD A VIDEO = append one row + re-freeze: `python scripts/nightly_reelcount.py --update-
baseline` (commit the updated baseline). URIs may be gs:// (recommended, co-located with the
VM), gdrive://, or local paths. labels_uri (optional) is a golden-features JSON (gts key) or a

```
start,end CSV ? present => quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the
example rows below to your actual short (~5-6 min) OWNED, minors-free clips before enabling.
skip_ai_handoff=true keeps this PURE-ENGINE (WASB candidates + windowing become the reels; NO
paid Gemini handoff) so the count is deterministic + free; the VM also needs WASB_REPO_DIR +
WASB_PYTHON (the wasb conda env) which run_on_vm.sh exports. Validated on an L4 2026-06-22 (180s
smoke clip -> 22 reels, deterministic).",
./tests/test_nightly_reelcount.py:1:"""Tests for scripts/nightly_reelcount.py +
scripts/nightly_email.py ? the pure cores.
./tests/test_nightly_regression.py:1:"""Tests for scripts/nightly_regression.py ? the pure
summary/comparison/report core.
--- (empty above = all refs updated) ---

=== rc/GX010075 result (the 5.3K #531 delta) ===
2026-07-09T03:37:26+00:00  START rc/GX010075
2026-07-09T03:54:45+00:00  DONE  rc/GX010075 rc=0 wall=1039.3s :: [rc/GX010075] checksum=2.068s
validation=49.575s segmentation=869.176s stitch=111.309s total=1032.128s segments=7 gpu_peak=66%
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

266. assistant (2026-07-09T03:57:12.403Z)

Two things surfaced:

1. The 5.3K stitch number is nuanced (I'll report it honestly). `rc/GX010075` finished: **stitch prod 106.4s ? rc 111.3s** ? RC *slightly slower* at 5.3K, opposite of the 1080p result (prod 14.85 ? rc 12.92, ?13%). This is single-run matrix timing (noisy), which is exactly why the controlled **stitch A/B** matters ? I'll run it now that the GPU is free. The honest read: #531's single-pass benefit is real at smaller reels but within noise at 5.3K, where the encode dominates (prod's real 5.3K speed comes from L4 NVENC, not reproducible locally with libx264).

2. My ref-updater missed several *active* docs + baseline `_doc` strings + test docstrings + a `run_on_vm.sh` comment (the archives correctly stay untouched). Let me extend it and re-run.

267. assistant (2026-07-09T03:57:24.089Z)

```
"""One-shot, backslash-safe reference updater for the scripts/gates/ reorg. Idempotent."""
import os

DEV = r"C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev"
```

The 5 nightly script moves (forward-slash). Applied to active docs/baselines/tests/dep

```
NIGHTLY = [
    ("scripts/nightly_regression.py", "scripts/gates/quality/nightly_regression.py"),
    ("scripts/nightly_reelcount.py", "scripts/gates/quality/nightly_reelcount.py"),
    ("scripts/nightly_email.py", "scripts/gates/quality/nightly_email.py"),
    ("scripts/nightly_regression.ps1", "scripts/gates/nightly_regression.ps1"),
    ("scripts/register_nightly_task.ps1", "scripts/gates/register_nightly_task.ps1"),
]
PERF = [("scripts/perf_gate/", "scripts/gates/perf/"), ("scripts.perf_gate.",
"scripts.gates.perf.")]

JOBS = {
    "scripts/gates/perf/run.py": PERF,
    "scripts/gates/perf/preflight.py": PERF,
    "scripts/gates/perf/README.md": PERF,
    "eval_baselines/perf_manifest.json": PERF + [("scripts/nightly_regression.py",
"scripts/gates/quality/nightly_regression.py"),
                                                 ("scripts/nightly_reelcount.py",
"scripts/gates/quality/nightly_reelcount.py")],
    "deploy/gcp/nightly_regression/README.md": NIGHTLY,
    "deploy/gcp/nightly_regression/run_on_vm.sh": NIGHTLY,
    "docs/CODE_MAP.md": NIGHTLY,
    "docs/ALPHA_LAUNCH_READINESS.md": NIGHTLY + [("scripts\\nightly_regression.py",
"scripts\\gates\\quality\\nightly_regression.py")],
```

```
    "docs/NIGHTLY_ENGINE_REGRESSION.md": NIGHTLY,
    "docs/DECISION_SNAPSHOT_REGRESSION.md": NIGHTLY,
    "docs/QUALITY_ITERATIONS.md": NIGHTLY,
    "docs/README.md": NIGHTLY,
    "eval_baselines/nightly_baseline.json": NIGHTLY,
    "eval_baselines/reelcount_manifest.json": NIGHTLY,
    "tests/test_nightly_reelcount.py": NIGHTLY,
    "tests/test_nightly_regression.py": NIGHTLY,
}

for rel, subs in JOBS.items():
    p = os.path.join(DEV, rel)
    if not os.path.exists(p):
        print(f"MISS {rel}")
        continue
    s = open(p, encoding="utf-8").read()
    n = 0
    for old, new in subs:
        c = s.count(old)
        if c:
            s = s.replace(old, new)
            n += c
    open(p, "w", encoding="utf-8", newline="\n").write(s)
    print(f"{rel}: {n}")
print("done")
```

268. user (2026-07-09T03:57:24.126Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\_reorg_refs.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

269. user (2026-07-09T03:57:42.101Z)

```
=== re-run extended ref updater ===
scripts/gates/perf/run.py: 2
scripts/gates/perf/preflight.py: 0
scripts/gates/perf/README.md: 0
eval_baselines/perf_manifest.json: 3
deploy/gcp/nightly_regression/README.md: 0
deploy/gcp/nightly_regression/run_on_vm.sh: 1
docs/CODE_MAP.md: 4
docs/ALPHA_LAUNCH_READINESS.md: 0
docs/NIGHTLY_ENGINE_REGRESSION.md: 9
docs/DECISION_SNAPSHOT_REGRESSION.md: 4
docs/QUALITY_ITERATIONS.md: 3
docs/README.md: 1
eval_baselines/nightly_baseline.json: 1
eval_baselines/reelcount_manifest.json: 2
tests/test_nightly_reelcount.py: 2
tests/test_nightly_regression.py: 1
done

=== VERIFY: stale refs OUTSIDE archives (should be empty) ===
--- (empty above = all ACTIVE refs updated; archives intentionally left) ---
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

270. assistant (2026-07-09T03:58:05.627Z)

```
All active references updated (16 files); only archives remain (correct). Let me now run the
full verification sweep on the reorganized tree ? check for any malformed/doubled paths, lint,
compile, preflight from the new path, and the moved tests.
```

271. user (2026-07-09T03:58:24.952Z)

```
=== [a] malformed/doubled paths? (should be empty) ===
scripts/gates/perf/run.py:252:    ap.add_argument("--worktree-root",
default=os.path.join(REPO_ROOT, ".perf_gate_worktrees"))
--- (empty = clean) ---
=== [b] ruff on perf gate ===
All checks passed!
=== [c] py_compile moved py ===
COMPILE OK
=== [d] preflight from NEW path (script + module form) ===
  [OK  ] gcloud (prod auto-resolve) (optional): C:\Users\avidu\AppData\Local\Google\Cloud
SDK\google-cloud-sdk\bin\gcloud.CMD
  => 9/9 ok
  => 9/9 ok
=== [e] moved tests still pass ===
................................................                     [100%]
51 passed in 2.23s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

272. assistant (2026-07-09T03:59:12.475Z)

Reorg verified end-to-end: **no malformed paths** (the one `perf_gate` hit is just the internal `.perf_gate_worktrees` temp-dir name), **ruff clean, compiles, preflight 9/9 from both the script and `python -m scripts.gates.perf.run` forms, and 51 tests pass** from the new location.

Now let me kick off the tool's end-to-end self-test (`--stitch-only`, GPU is free) in the background ? this validates `run.py` and gives the clean controlled #531 numbers ? while I write the gate-suite index README.

273. user (2026-07-09T03:59:15.525Z)

Command running in background with ID: b62n2wgof. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\b62n2wgof.output. You will be notified when it completes. To check interim output, use Read on that file path.

274. assistant (2026-07-09T03:59:36.204Z)

KhelSutra gate suite (`scripts/gates/`)

Cohesive home for the release/nightly **gates** ? the harnesses that catch a regression before it ships. Two legs, mirroring each other, both **local** (the GPU, WSL WASB detector, and golden/vault corpus live on this machine ? a cloud runner can't run them):

Leg	What it guards	Lives in
Quality	Rally-detection accuracy (precision/recall/F1) + reel-count + cost	`scripts/gates/quality/`
Performance	Per-stage latency of the shipped pipeline (prod vs HEAD)	`scripts/gates/perf/`

> These are standalone run-driver CLIs (`__main__`, not imported by app code), so per the
> `docs/CODE_MAP.md` convention they live under `scripts/` ? now grouped here instead of
> scattered loose. The offline CI/nightly guards (lint, types, suite, fixtures) stay in
> `.github/workflows/`; these GPU/real-video gates are the owner-box Tier-2 complement.

Layout

```
scripts/gates/
  register_nightly_task.ps1     # registers BOTH scheduled tasks (quality + perf)
  nightly_regression.ps1        # quality wrapper (Windows Scheduled Task)
  nightly_perf.ps1              # performance wrapper (Windows Scheduled Task)
  quality/
```

```
    nightly_regression.py       # CPU re-score of CACHED trajectories vs frozen baseline (fast
tripwire)
    nightly_reelcount.py        # FULL-pipeline reel-count + quality + cost on a GCP GPU VM
(emailed)
    nightly_email.py            # SMTP report sender (graceful skip without creds)
  perf/
    run.py harness.py stitch_ab.py preflight.py  config/bench_config.json  README.md
```

Exit-code family (shared)

`0` PASS · `1` REGRESSION · `2` nothing-ran / preflight failed · `3` no baseline · `4` STALE
baseline.

Quality leg

- **`quality/nightly_regression.py`** ? re-scores the cached WASB trajectories on CPU (seconds)
  vs `eval_baselines/nightly_baseline.json`. Wrapper: `nightly_regression.ps1` (local Scheduled
  Task). Design: `docs/QUALITY_ITERATIONS.md`.
- **`quality/nightly_reelcount.py`** + **`nightly_email.py`** ? the full WASB pipeline on real
  clips on an ephemeral GCP L4 VM (reel-count + quality + USD/INR cost, emailed). Runbook:
  [`deploy/gcp/nightly_regression/README.md`](../../deploy/gcp/nightly_regression/README.md);
  design: `docs/NIGHTLY_ENGINE_REGRESSION.md`. Baselines: `eval_baselines/reelcount_*.json`.

```bash
python scripts/gates/quality/nightly_regression.py            # check (exit 1 on regression)
python scripts/gates/quality/nightly_regression.py --update-baseline
python scripts/gates/quality/nightly_reelcount.py            # full-pipeline reel-count + cost
```

Performance leg

- **`perf/run.py`** ? times checksum ? validation ? segmentation ? full-reel stitch for the
  commit deployed on `api.khelsutra.guru` **vs repo HEAD**, on the local GPU, and gates the
  release. Full docs, methodology, self-help: [`perf/README.md`](perf/README.md).

```bash
python scripts/gates/perf/run.py --preflight     # self-help env check
python scripts/gates/perf/run.py                 # gate HEAD vs production (quick profile)
python scripts/gates/perf/run.py --stitch-only   # just the #531 stitch A/B (fast)
python scripts/gates/perf/run.py --update-baseline
```

Nightly (both legs, one command)

```powershell
# Registers KhelSutraNightlyRegression (03:00, 30-min cap) + KhelSutraNightlyPerf (03:
pwsh -File scripts/gates/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'
pwsh -File scripts/gates/register_nightly_task.ps1 -Unregister     # remove both
```

Each wrapper writes `output/nightly_<leg>/<date>.{md,log}` + a `LATEST_STATUS.txt` marker and
exits with the gate's code, so Task Scheduler records PASS / REGRESSION / STALE. Both are
graceful no-ops until their prerequisites land, so registering before merge is safe.

275. user (2026-07-09T03:59:36.236Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\README.md (file state is current in your context ? no need to Read it back)

276. assistant (2026-07-09T03:59:43.282Z)

The self-test finished quickly (exit 0) ? possibly too quickly for a 5.3K stitch, so let me check what actually happened.

277. user (2026-07-09T03:59:44.214Z)

```
1       Preflight:
2         [OK  ] git: git version 2.45.2.windows.1
3         [OK  ] ffmpeg: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Micr
osoft.Winget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffmpeg.EXE
4         [OK  ] ffprobe: C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Mic
rosoft.Winget.Source_8wekyb3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffprobe.EXE
5         [OK  ] torch+cuda: torch 2.6.0+cu124 · NVIDIA GeForce RTX 2070 Super with Max-Q Design
6         [OK  ] wsl:Ubuntu: ok
7         [OK  ] wsl conda env 'wasb': ENV_OK
8         [OK  ] wsl WASB-SBDT repo: ~/models/WASB-SBDT
9         [OK  ] wsl WASB weights: ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
10        [OK  ] gcloud (prod auto-resolve) (optional):
C:\Users\avidu\AppData\Local\Google\Cloud SDK\google-cloud-sdk\bin\gcloud.CMD
11        => 9/9 ok
12      2026-07-09T03:59:26+00:00  Cloud Run resolve failed ([WinError 2] The system cannot find
the file specified); falling back to pinned prod_commit
13      2026-07-09T03:59:26+00:00  prod=304af7b (pinned prod_commit (Cloud Run resolve
unavailable))  ·  HEAD(HEAD)=3249451
14      Traceback (most recent call last):
15        File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py", line 388, in <module>
16          raise SystemExit(main())
17                           ~~~~^^
18        File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py", line 322, in main
19          worktree(REPO_ROOT, wt_prod, prod_commit, log)
20          ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
21        File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py", line 97, in worktree
22          raise RuntimeError(f"worktree add {path}@{commit} failed: {err}")
23      RuntimeError: worktree add C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-
Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\.perf_gate_worktrees\prod@304af7b failed: Preparing worktree (detached HEAD 304af7b)
24      error: unable to create file backend/ui/assets/noto-sans-devanagari-devanagari-wght-
normal-DWI_L5Cu.woff2: Filename too long
25      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_2026-07-04.md: Filename too long
26      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_a1-cuj_2026-07-04.md: Filename too long
27      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_a1-e2e_2026-07-04.md: Filename too long
28      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_alpha-all1v1_2026-07-06.md: Filename too long
29      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_cp-rebuild_2026-07-05.md: Filename too long
30      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md: Filename too long
31      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_validation-latency_2026-07-07.md: Filename too long
32      error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-
serving_exec-status-fastfail_2026-07-07.md: Filename too long
33      error: unable to create file docs/archives/past_projects/AUDIT_REPORT_khelsutra-
guru_2026-07-02-COMPLETED-2026-07-02.md: Filename too long
34      error: unable to create file
docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md: Filename too
```

```
long
35      error: unable to create file docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-
MAP-AND-CONTRACT.md: Filename too long
36      error: unable to create file docs/archives/past_projects/DECOUPLED_COMPUTE/06-RISKS-AND-
OPEN-QUESTIONS.md: Filename too long
37      error: unable to create file
docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md: Filename too long
38      error: unable to create file
docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md:
Filename too long
39      error: unable to create file docs/archives/quality-iterations/2026-06-18-lovo-
resweep-n13/lovo_resweep.py: Filename too long
40      fatal: Could not reset index file to revision 'HEAD'.
41
```

278. assistant (2026-07-09T04:00:44.026Z)

The self-test caught **two real Windows bugs** (exactly why we self-test):
1. **gcloud resolve fails** ? `shutil.which` returns `gcloud.CMD`, which `subprocess` can't run directly on Windows (it fell back to the pinned commit, so it still worked ? but the auto-resolve is broken).
2. **`worktree add` ? "Filename too long"** ? the deep scratchpad path + long repo filenames blow past Windows MAX_PATH (the real repo's shorter path wouldn't, but the tool must be robust).

Both are worth fixing in the committed tool. Let me read the exact code and patch.

279. user (2026-07-09T04:00:46.164Z)

```
65      # ------------------------------------------------------------------------ resolve
prod
66      def resolve_prod(manifest, args, log):
67          if args.prod_commit:
68              return git_short(REPO_ROOT, args.prod_commit) or args.prod_commit, "explicit
--prod-commit"
69          if args.prod_ref:
70              return git_short(REPO_ROOT, args.prod_ref), f"--prod-ref {args.prod_ref}"
71          prod = manifest.get("prod", {})
72          cr = prod.get("cloud_run", {})
73          mp = prod.get("image_digest_to_commit", {})
74          if shutil.which("gcloud") and cr:
75              try:
76                  rc, out, _ = sh(["gcloud", "run", "services", "describe", cr["service"],
77                                   "--region", cr["region"], "--project", cr["project"],
78                                   "--format=value(spec.template.spec.containers[0].image)"],
timeout=90)
79                  if rc == 0 and "@" in out:
80                      digest = out.split("@")[-1].strip()
81                      if digest in mp:
82                          log(f"prod resolved from live Cloud Run image {digest[:19]}? ->
{mp[digest]}")
83                          return git_short(REPO_ROOT, mp[digest]) or mp[digest], f"live Cloud
Run {digest[:19]}?"
84                      log(f"live Cloud Run digest {digest[:19]}? not in image_digest_to_commit
map (update it / #532)")
85              except Exception as e:
86                  log(f"Cloud Run resolve failed ({e}); falling back to pinned prod_commit")
87          pin = prod.get("pinned_commit")
88          return (git_short(REPO_ROOT, pin) or pin), "pinned prod_commit (Cloud Run resolve
unavailable)"
89
90
91      # ------------------------------------------------------------------------ worktrees
92      def worktree(repo, path, commit, log):
93          if os.path.exists(path):
94              sh(["git", "-C", repo, "worktree", "remove", "--force", path], timeout=120)
```

```
95          rc, _, err = sh(["git", "-C", repo, "worktree", "add", "--detach", path, commit],
timeout=180)
96          if rc != 0:
97              raise RuntimeError(f"worktree add {path}@{commit} failed: {err}")
98          log(f"worktree {os.path.basename(path)} -> {git_short(path, 'HEAD')}")
```

280. user (2026-07-09T04:00:54.225Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

281. user (2026-07-09T04:01:01.315Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

282. assistant (2026-07-09T04:01:24.767Z)

Both fixes in. Let me verify they compile/lint, clean up the partial worktree from the failed attempt, and re-run the self-test with a short worktree root (to sidestep the pathological scratchpad depth ? the committed default now also has the `core.longpaths` fix for robustness).

283. user (2026-07-09T04:01:34.150Z)

```
COMPILE OK
All checks passed!
=== cleanup partial worktrees from failed attempt ===
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/bhi-nvenc-probe-wt': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/fix-worktree': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/kg-wt521': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/kg-wt525fix': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/master-compare': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465-merge': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr467': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr469': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-465fix': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
```

```
indexer/.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-docs': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-input-jail': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-review-492-mergecheck': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-signed-download': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-source-cache': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-upload-staging': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt524-969': Permission denied
(no perf-gate worktrees registered)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

284. assistant (2026-07-09T04:01:59.812Z)

Good ? no perf-gate worktree lingered (the Permission-denied entries are **other sessions' worktrees** I must not touch; prune left them alone). Let me re-run the self-test with a short worktree root (background, ~7 min), and in parallel render the matrix's per-stage report from the incremental results, re-run leak-scan on the reorganized tree, and parse-check the wrappers.

285. user (2026-07-09T04:02:03.414Z)

Command running in background with ID: bzb2f69ld. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bzb2f69ld.output. You will be notified when it completes. To check interim output, use Read on that file path.

286. user (2026-07-09T04:02:09.855Z)

=== render matrix per-stage report (3 clips) from raw_results.json ===
rendered C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/report\REPORT.md and baseline.json
--- REPORT.md (headline section) ---

Headline

=== leak_scan on reorganized dev tree ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

287. user (2026-07-09T04:02:11.920Z)

OK register_nightly_task.ps1
OK nightly_regression.ps1
OK nightly_perf.ps1

288. assistant (2026-07-09T04:03:17.425Z)

Reorg fully verified: **leak-scan clean, all 3 wrappers parse-clean, matrix report rendered.** The self-test A/B is running (background, ~7 min).

One more robustness fix for the committed tool: the default worktree root is *inside* the repo (`REPO_ROOT/.perf_gate_worktrees`), which pollutes git status and worsens the MAX_PATH issue. Let me move it to a short temp path. Let me read the import + arg regions and patch.

289. user (2026-07-09T04:03:19.444Z)

24   import argparse
25   import json
26   import os
27   import shutil
28   import subprocess
29   import sys
30   import time
31   from datetime import datetime, timezone
32

290. user (2026-07-09T04:03:20.655Z)

253   ap.add_argument("--vault-dir", default=os.environ.get("RALLY_VAULT_DIR") or os.path.expanduser("~/Videos"))
254   ap.add_argument("--prod-commit", default=None)
255   ap.add_argument("--prod-ref", default=None)
256   ap.add_argument("--rc-ref", default="HEAD")

291. user (2026-07-09T04:03:29.907Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

292. user (2026-07-09T04:03:32.671Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-

```
dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

293. assistant (2026-07-09T04:03:38.195Z)

```
Let me verify the edits compile/lint and check the self-test's progress.
```

294. user (2026-07-09T04:03:43.240Z)

```
COMPILE OK
All checks passed!
=== self-test2 progress ===
  [OK  ] wsl:Ubuntu: ok
  [OK  ] wsl conda env 'wasb': ENV_OK
  [OK  ] wsl WASB-SBDT repo: ~/models/WASB-SBDT
  [OK  ] wsl WASB weights: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  [OK  ] gcloud (prod auto-resolve) (optional): C:\Users\avidu\AppData\Local\Google\Cloud
SDK\google-cloud-sdk\bin\gcloud.CMD
  => 9/9 ok
2026-07-09T04:02:15+00:00  prod resolved from live Cloud Run image sha256:7b62a8df2dc6? ->
304af7b
2026-07-09T04:02:15+00:00  prod=304af7b (live Cloud Run sha256:7b62a8df2dc6?)  ·
HEAD(HEAD)=3249451
2026-07-09T04:02:15+00:00  worktree prod -> 304af7b
2026-07-09T04:02:16+00:00  worktree rc -> 3249451
2026-07-09T04:02:16+00:00  STITCH prod/1080p (reps=3)
2026-07-09T04:03:12+00:00  STITCH rc/1080p (reps=3)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

295. assistant (2026-07-09T04:04:11.611Z)

```
The self-test is validating beautifully ? **the gcloud fix works** (`prod resolved from live
Cloud Run image sha256:7b62a8df? -> 304af7b`), worktrees created, stitch A/B running. It'll
finish in ~5 min.

Meanwhile, an important commit-readiness check: the moved quality scripts do `sys.path`
manipulation before imports, so `ruff.toml` likely has **per-file E402 ignores keyed by the old
paths** ? which the move would break. Let me check and fix.
```

296. user (2026-07-09T04:04:17.023Z)

```
=== ruff.toml per-file-ignores referencing moved scripts ===
19:"scripts/run_phase3_eval.py" = ["E402"]
20:"scripts/run_process_and_stitch.py" = ["E402"]

=== ruff check the WHOLE reorganized gates dir (incl quality) ===
All checks passed!
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

297. assistant (2026-07-09T04:04:30.714Z)

```
No `ruff.toml` changes needed ? the moved quality scripts **pass ruff at the new path** (only
`run_phase3_eval.py`/`run_process_and_stitch.py`, which stay put, have E402 ignores). The whole
reorganized `scripts/gates/` is lint-clean.

Let me confirm the exact deliverable file set on the branch (and that no build artifacts/reports
would sneak in), while the A/B finishes.
```

298. user (2026-07-09T04:04:37.214Z)

```
=== output/ gitignored? ===
2:__pycache__/
47:output/
48:output_dir/
```

```
=== git status of the feat/perf-release-gate branch (deliverable set) ===
 M deploy/gcp/nightly_regression/README.md
 M deploy/gcp/nightly_regression/run_on_vm.sh
 M docs/ALPHA_LAUNCH_READINESS.md
 M docs/CODE_MAP.md
 M docs/DECISION_SNAPSHOT_REGRESSION.md
 M docs/NIGHTLY_ENGINE_REGRESSION.md
 M docs/QUALITY_ITERATIONS.md
 M docs/README.md
 M eval_baselines/nightly_baseline.json
 M eval_baselines/reelcount_manifest.json
RM scripts/nightly_regression.ps1 -> scripts/gates/nightly_regression.ps1
R  scripts/nightly_email.py -> scripts/gates/quality/nightly_email.py
RM scripts/nightly_reelcount.py -> scripts/gates/quality/nightly_reelcount.py
RM scripts/nightly_regression.py -> scripts/gates/quality/nightly_regression.py
RM scripts/register_nightly_task.ps1 -> scripts/gates/register_nightly_task.ps1
 M tests/test_nightly_reelcount.py
 M tests/test_nightly_regression.py
?? eval_baselines/perf_manifest.json
?? scripts/gates/README.md
?? scripts/gates/nightly_perf.ps1
?? scripts/gates/perf/

=== any stray untracked junk (pyc, out, report) that must NOT commit? ===
?? scripts/gates/nightly_perf.ps1
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

299. assistant (2026-07-09T04:04:53.896Z)

Deliverable set is clean and coherent: **5 files moved as git-detected renames** (history preserved), docs/tests/baselines modified for the ref updates, and the new perf package + manifest + wrappers untracked. `output/` and `__pycache__/` are gitignored, and worktrees now go to system temp ? so no build artifacts or reports can sneak into the commit.

Let me check the A/B self-test result.

300. user (2026-07-09T04:04:59.414Z)

```
=== self-test2 tail ===
  [OK  ] wsl WASB weights: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  [OK  ] gcloud (prod auto-resolve) (optional): C:\Users\avidu\AppData\Local\Google\Cloud
SDK\google-cloud-sdk\bin\gcloud.CMD
  => 9/9 ok
2026-07-09T04:02:15+00:00  prod resolved from live Cloud Run image sha256:7b62a8df2dc6? ->
304af7b
2026-07-09T04:02:15+00:00  prod=304af7b (live Cloud Run sha256:7b62a8df2dc6?)  ·
HEAD(HEAD)=3249451
2026-07-09T04:02:15+00:00  worktree prod -> 304af7b
2026-07-09T04:02:16+00:00  worktree rc -> 3249451
2026-07-09T04:02:16+00:00  STITCH prod/1080p (reps=3)
2026-07-09T04:03:12+00:00  STITCH rc/1080p (reps=3)
2026-07-09T04:04:12+00:00  STITCH prod/5p3k (reps=1)

=== if done, the generated report (output/nightly_perf) ===
```

301. assistant (2026-07-09T04:05:22.076Z)

The reorg is essentially done and the self-test is on its final stage (`STITCH prod/5p3k`, ~4 min left). Here's the milestone:

? Consolidated into `scripts/gates/` ? verified

Structure (cohesive package, honoring the `scripts/` CLI convention):
```
```

```
scripts/gates/
  register_nightly_task.ps1        # registers BOTH quality + perf tasks
  nightly_regression.ps1  nightly_perf.ps1
  README.md                        # gate-suite index
  quality/  nightly_regression.py  nightly_reelcount.py  nightly_email.py
  perf/     run.py  harness.py  stitch_ab.py  preflight.py  config/  README.md
```

Moves preserved git history (5 files as `R`/`RM` renames), and I updated **every** reference in one change set ? the deployed GCP nightly (`run_on_vm.sh` import + invocation), `tests/`, `eval_baselines/` docstrings, `docs/CODE_MAP.md`, and 5 other active docs. Archives intentionally left as historical.

Verified: ruff clean across the whole package · **51 tests pass** from the new location · leak-scan clean · all 3 wrappers parse-clean · preflight **9/9** from both `python scripts/gates/perf/run.py` and `python -m scripts.gates.perf.run` · `output/`+`__pycache__` gitignored, worktrees now in system temp (no repo pollution).

The self-test earned its keep ? it caught and I fixed two real Windows bugs in the committed tool:
1. **gcloud `.CMD` resolution** ? now correctly auto-resolves prod from the live Cloud Run image (`sha256:7b62a8df? ? 304af7b`, confirmed in the run above).
2. **worktree MAX_PATH** ? added `core.longpaths=true` + a short temp worktree root.

I'll report the clean stitch-A/B numbers, seed the committed baseline, and lay out the commit/PR the moment the self-test finishes.

302. user (2026-07-09T04:05:23.378Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

303. assistant (2026-07-09T04:05:36.999Z)

The self-test is still on the 5.3K stitch stage. I'll pick up automatically when it completes (~3?4 min) with the clean A/B numbers, seed the committed baseline, and give you the final summary plus the ready-to-commit branch.

304. user (2026-07-09T04:08:03.716Z)

<task-notification>
<task-id>bzb2f69ld</task-id>
<tool-use-id>toolu_0113EMSvAE3NYhznp2QUUWFw</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bzb2f69ld.output</output-file>
<status>completed</status>
<summary>Background command "Re-run stitch-only self-test with short worktree root" completed (exit code 0)</summary>
</task-notification>

305. assistant (2026-07-09T04:08:08.006Z)

Self-test complete. Let me read the A/B result and the generated gate report.

306. user (2026-07-09T04:08:11.639Z)

```
=== self-test2 full output ===
  => 9/9 ok
2026-07-09T04:02:15+00:00  prod resolved from live Cloud Run image sha256:7b62a8df2dc6? ->
304af7b
2026-07-09T04:02:15+00:00  prod=304af7b (live Cloud Run sha256:7b62a8df2dc6?)  ·
HEAD(HEAD)=3249451
2026-07-09T04:02:15+00:00  worktree prod -> 304af7b
2026-07-09T04:02:16+00:00  worktree rc -> 3249451
```

```
2026-07-09T04:02:16+00:00  STITCH prod/1080p (reps=3)
2026-07-09T04:03:12+00:00  STITCH rc/1080p (reps=3)
2026-07-09T04:04:12+00:00  STITCH prod/5p3k (reps=1)
2026-07-09T04:06:13+00:00  STITCH rc/5p3k (reps=1)
2026-07-09T04:08:03+00:00  report: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-
Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\output\nightly_perf\2026-07-09.md  ·  exit 0

=== generated gate report (output/nightly_perf) ===
| segmentation | ? | ? | ? |
| stitch* | ? | ? | ? |
| total* | ? | ? | ? |
```

SmokeTestBadmintonClip · segments prod=None rc=None

```
| stage | prod | HEAD | ?% |
|---|---|---|---|
| checksum | ? | ? | ? |
| validation | ? | ? | ? |
| segmentation | ? | ? | ? |
| stitch* | ? | ? | ? |
| total* | ? | ? | ? |
```

Stitch A/B (#531, segmentation excluded) ? gated*

```
| reel | prod median | HEAD median | ?% |
|---|---|---|---|
| 1080p* | 18.11s | 19.28s | +6.5% |
| 5p3k* | 118.71s | 107.03s | -9.8% |

_* gated stage. `--update-baseline` freezes these numbers into
eval_baselines/perf_baseline.json._
```

307. assistant (2026-07-09T04:09:34.898Z)

The tool ran **fully end-to-end** ? preflight, **prod auto-resolve from live Cloud Run** (`sha256:7b62a8df? ? 304af7b`), worktrees, stitch A/B, gating, report, **exit 0 (PASS)**.

The clean controlled #531 result (median of reps, identical reels both sides):

reel	prod (two-pass)	HEAD (single-pass)	?
1080p	18.11s	19.28s	**+6.5%**
5.3K	118.71s	107.03s	**?9.8%**

The two directions **disagree** with each other *and* with the matrix single-runs ? which is the honest finding: **#531 is performance-neutral within noise on this local hardware** (libx264/RTX 2070). Its real production win is the L4 **NVENC** path, which a local libx264 encode can't reproduce. Both deltas are well within the 20% gate tolerance ? **PASS**, correctly.

I spotted one cosmetic bug in the report (a `--stitch-only` run prints empty per-video tables) ? let me fix that, then wrap up.

308. user (2026-07-09T04:09:37.982Z)

```
356-            "rc_commit": rc_commit, "rc_ref": args.rc_ref, "gpu": gpu, "profile":
args.profile,
357-            "gate": gate, "segmenter": manifest.get("segmenter", "wasb_hybrid")}
358:    summary, findings = build(results, stitch_results, [n for n, _ in videos], meta, gate)
359-
360-    if args.update_baseline:
```

309. user (2026-07-09T04:09:43.158Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

310. assistant (2026-07-09T04:09:53.639Z)

Fix applied. Let me do the final verification pass and confirm the self-test's worktrees were cleaned up.

311. user (2026-07-09T04:10:00.560Z)

```
=== compile + lint after fix ===
COMPILE OK
All checks passed!
=== self-test worktrees cleaned up? (should be no kswt) ===
(clean ? tool removed its worktrees)
C:/kswt
(removed leftover C:/kswt dir)
=== FINAL branch status (deliverable) ===
changed files: 21
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

312. assistant (2026-07-09T04:10:40.552Z)

Clean: **compile OK, ruff clean, self-test worktrees removed, 21-file deliverable with zero build artifacts.** Let me kick off the full CI gate (suite + coverage + ruff + mypy) in the background to confirm the reorg broke nothing repo-wide, then I'll lay out the complete result and the commit offer.

313. user (2026-07-09T04:10:44.617Z)

Command running in background with ID: bgkhy1aqm. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bgkhy1aqm.output. You will be notified when it completes. To check interim output, use Read on that file path.

314. assistant (2026-07-09T04:11:20.053Z)

The full CI gate is running in the background. Here's the complete picture on both fronts.

---

1) The performance comparison ? HEAD vs the deployed prod binary (your original ask)

Setup (verified): prod = `304af7b` (live on `api.khelsutra.guru`), HEAD = `origin/master` `3249451`, production **`wasb_hybrid` GPU detector via WSL**, run on your RTX 2070 over 3 vault clips (you opted to skip the 2-hr 5-min clip).

Per-stage ? (HEAD vs prod; negative = HEAD faster):

stage	GX010016 (1080p, 1 rally)	Smoke (1080p60, 6 rallies)	GX010075 (5.3K, 7 rallies)
checksum	+1.3% (0.08s)	?2.9% (0.2s)	?6.3% (2.1s)
validation	+2.7% (5.5s)	+0.9% (5.5s)	?2.7% (50s)
segmentation	+0.7% (94s)	+1.2% (314s)	?1.6% (876s)
stitch	**?10.0%** (1.0s)	**?13.0%** (14?13s)	**+4.6%** (106?111s)
total	+0.7%	+0.5%	?1.0%

Controlled stitch A/B (#531, identical reels, median of reps): 1080p **+6.5%**, 5.3K **?9.8%**.

Verdict: the release is performance-neutral locally ? no regression (gate PASSES). checksum/validation/segmentation are unchanged (same detector/inputs ? they didn't change

between the commits). The one code-level delta, **#531 single-pass stitch**, lands **within run-to-run noise** here (matrix and A/B disagree on direction) ? its real win is the L4 **NVENC** path, which a local libx264 encode can't reproduce. The other 8 release commits (GCSFuse mount, CP gating, cancel propagation, telemetry) are cloud-only and not locally exercisable.

2) The consolidated gate tool (your consolidation ask)

`scripts/gates/` ? a cohesive package (perf + quality legs), history-preserving renames, **every** reference updated (deployed GCP nightly, tests, baselines, 6 docs). The self-test ran the whole tool end-to-end and caught + fixed **two real Windows bugs** (gcloud `.CMD` prod-resolution; worktree MAX_PATH). Verified: ruff clean, 51 targeted tests pass, leak-scan clean, wrappers parse-clean, preflight 9/9.

21-file branch `feat/perf-release-gate` (cut from `origin/master`, your working tree untouched), no build artifacts. The full CI gate (suite + coverage + mypy + `ruff check .`) is running now to confirm the reorg broke nothing repo-wide.

How would you like me to land it?

315. user (2026-07-09T04:11:36.754Z)

Your questions have been answered: "The tool + reorg are on branch feat/perf-release-gate (cut from origin/master), pending the full CI gate now running. How should I land it?"="Commit + open PR". You can now continue with these answers in mind.

316. assistant (2026-07-09T04:12:07.758Z)

On it ? I'll commit and open the PR **once the CI gate is green**. Let me check the gate's progress and confirm `gh` is ready + re-verify the branch (git-safety) while it finishes.

317. user (2026-07-09T04:12:11.693Z)

```
=== CI gate progress ===
=== FULL CI GATE (reorg must not break the repo) ===
--- ruff check . ---
   |

Found 68 errors.
--- mypy backend training ---
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
--- full suite + coverage ---

=== git-safety pre-commit checks ===
branch: feat/perf-release-gate  ·  HEAD: 3249451 feat(#534): live detector progress heartbeat +
L4 worker resource telemetry (#546)
commits ahead of origin/master (should be 0 before my commit):
=== gh CLI ready? ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

318. assistant (2026-07-09T04:12:47.646Z)

`ruff check .` flagged **68 errors** ? but `ruff check scripts/gates/` passed earlier, so these are almost certainly **not** in my files (likely a local ruff-version drift vs CI's pinned 0.15.16). mypy is clean; the suite is still running. Let me confirm none of the 68 are in my changed files.

319. user (2026-07-09T04:12:52.195Z)

```
=== local ruff version (CI pins 0.15.16) ===
ruff 0.6.9
=== which FILES have the 68 errors? (are ANY mine?) ===
      9 tests\test_upload_gcs_streaming.py
      8 tests\test_submit_md5_cache.py
      8 tests\test_api_cloud_dispatch.py
      7 tests\test_restart_recovery.py
      6 tests\test_upload_download.py
      6 tests\test_api_assets.py
      5 tests\test_job_worker_loop.py
      5 tests\test_async_jobs.py
      5 tests\test_api_endpoints.py
      4 tests\test_api_annotations.py
      3 tests\test_bind_and_host.py
      2 tests\test_ui_bundle.py
=== any errors in MY changed .py files? (should be empty) ===
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

320. assistant (2026-07-09T04:13:10.318Z)

Confirmed harmless: local ruff is **0.6.9** but CI pins **0.15.16**, and all 68 errors are in **pre-existing test files** ? **zero in my changed files**. To match CI exactly (per your pre-commit convention), let me install the pinned ruff and re-check.

321. user (2026-07-09T04:13:20.704Z)

```
=== install CI-pinned ruff 0.15.16 ===
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
ruff 0.15.16
=== ruff check . with pinned version (matches CI ? should be clean) ===
All checks passed!
=== suite progress ===
Found 68 errors.
--- mypy backend training ---
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
--- full suite + coverage ---
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

322. assistant (2026-07-09T04:13:57.621Z)

Under the CI-pinned ruff 0.15.16, `ruff check .` ? All checks passed (the 68 were pure 0.6.9-version noise), and **mypy is clean** (no issues in 150 files). The full test suite + coverage is the last step still running.

Everything else is green and `gh` is authenticated (account `avidullu`). I'll verify the suite passes with coverage ?70, then stage explicit paths, commit, push, and open the PR. Waiting on the suite to finish.

323. user (2026-07-09T04:18:23.571Z)

```
=== suite + coverage tail ===
--- mypy backend training ---
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
--- full suite + coverage ---
Required test coverage of 70% reached. Total coverage: 84.28%
```

```
========================== short test summary info ==========================
FAILED tests/test_storage.py::test_gcs_fuse_helpers - AssertionError: assert ...
FAILED tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available
2 failed, 1933 passed, 8 skipped, 57 warnings in 378.62s (0:06:18)
=============================================================================
 RESULT: FAIL - pytest exit 1
=============================================================================
CIGATE_DONE rc=0
```

324. assistant (2026-07-09T04:18:53.631Z)

The suite result: **coverage 84.28% (?70 ?), 1933 passed**, but **2 failed** in `tests/test_storage.py` (`test_gcs_fuse_helpers`, `test_acquire_local_input_uses_gcs_fuse_when_available`). My reorg touches **zero** `backend/` or storage code, so these can't be my doing ? but I need to confirm they're pre-existing on `origin/master` (likely env-dependent: the GCSFuse `#535` code failing on Windows where CI's Linux passes). Let me verify.

325. user (2026-07-09T04:19:00.539Z)

```
=== [1] Does MY diff touch storage at all? (origin/master..worktree, backend/storage +
test_storage) ===
(empty above = my changes don't touch storage ? failures are pre-existing on master)

=== [2] the 2 failures verbose (env-dependent? gcsfuse/Windows) ===
platform win32 -- Python 3.13.13, pytest-8.4.2, pluggy-1.6.0
        assert storage._gcs_mount_root(sc) == str(mount)
>       assert fused == str(mount / "videos/clip.mp4")
E       AssertionError: assert 'C:\\Users\\a...deos/clip.mp4' == 'C:\\Users\\a...eos\\clip.mp4'
tests\test_storage.py:579: AssertionError
FAILED tests/test_storage.py::test_gcs_fuse_helpers - AssertionError: assert ...
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

326. assistant (2026-07-09T04:19:47.427Z)

Confirmed pre-existing and not mine: my diff touches **zero** storage code, and the 2 failures are a **Windows path-separator bug in the `#535` GCSFuse helper** (`videos/clip.mp4` vs `videos\clip.mp4`) ? it passes on CI's Linux (why master is green) and fails only on local Windows. So for this change the gate is green: **ruff (pinned) ?, mypy ?, coverage 84.28% ?, 1933 passed, all my changed-file tests pass.**

Proceeding with commit + PR. Let me stage explicit paths (git-safety ? no `git add -A`) and re-verify the branch.

327. user (2026-07-09T04:19:53.765Z)

```
branch: feat/perf-release-gate  ·  HEAD: 3249451
warning: in the working copy of 'deploy/gcp/nightly_regression/README.md', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'docs/ALPHA_LAUNCH_READINESS.md', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'docs/CODE_MAP.md', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'docs/DECISION_SNAPSHOT_REGRESSION.md', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'docs/NIGHTLY_ENGINE_REGRESSION.md', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'docs/QUALITY_ITERATIONS.md', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'docs/README.md', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'eval_baselines/nightly_baseline.json', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'eval_baselines/reelcount_manifest.json', LF will be replaced by
```

```
CRLF the next time Git touches it
warning: in the working copy of 'scripts/gates/nightly_regression.ps1', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'scripts/gates/quality/nightly_reelcount.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'scripts/gates/quality/nightly_regression.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'scripts/gates/register_nightly_task.ps1', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_nightly_reelcount.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_nightly_regression.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'eval_baselines/perf_manifest.json', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'scripts/gates/README.md', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'scripts/gates/nightly_perf.ps1', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'scripts/gates/perf/README.md', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'scripts/gates/perf/config/bench_config.json', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'scripts/gates/perf/harness.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'scripts/gates/perf/preflight.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'scripts/gates/perf/run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'scripts/gates/perf/stitch_ab.py', LF will be replaced by CRLF
the next time Git touches it
=== staged set (should be my 21 files; no __pycache__/artifacts) ===
M  deploy/gcp/nightly_regression/README.md
M  deploy/gcp/nightly_regression/run_on_vm.sh
M  docs/ALPHA_LAUNCH_READINESS.md
M  docs/CODE_MAP.md
M  docs/DECISION_SNAPSHOT_REGRESSION.md
M  docs/NIGHTLY_ENGINE_REGRESSION.md
M  docs/QUALITY_ITERATIONS.md
M  docs/README.md
M  eval_baselines/nightly_baseline.json
A  eval_baselines/perf_manifest.json
M  eval_baselines/reelcount_manifest.json
A  scripts/gates/README.md
A  scripts/gates/nightly_perf.ps1
R  scripts/nightly_regression.ps1 -> scripts/gates/nightly_regression.ps1
A  scripts/gates/perf/README.md
A  scripts/gates/perf/config/bench_config.json
A  scripts/gates/perf/harness.py
A  scripts/gates/perf/preflight.py
A  scripts/gates/perf/run.py
A  scripts/gates/perf/stitch_ab.py
R  scripts/nightly_email.py -> scripts/gates/quality/nightly_email.py
R  scripts/nightly_reelcount.py -> scripts/gates/quality/nightly_reelcount.py
R  scripts/nightly_regression.py -> scripts/gates/quality/nightly_regression.py
A  scripts/gates/register_nightly_task.ps1
D  scripts/register_nightly_task.ps1
M  tests/test_nightly_reelcount.py
M  tests/test_nightly_regression.py
=== unstaged/untracked leftovers in my scope? ===
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

328. assistant (2026-07-09T04:20:17.596Z)

Staging is exactly right ? the 5 moves (renames detected), all ref-update modifications, the new

perf package, and the registrar (delete+add since it was substantially rewritten). No artifacts, no leftovers; the LF?CRLF warnings are normal Windows autocrlf. Let me commit.

329. user (2026-07-09T04:20:27.406Z)

```
[feat/perf-release-gate 0aa6d4b] feat(gates): local release perf gate + consolidate nightly into scripts/gates/
 27 files changed, 1487 insertions(+), 113 deletions(-)
 create mode 100644 eval_baselines/perf_manifest.json
 create mode 100644 scripts/gates/README.md
 create mode 100644 scripts/gates/nightly_perf.ps1
 rename scripts/{ => gates}/nightly_regression.ps1 (96%)
 create mode 100644 scripts/gates/perf/README.md
 create mode 100644 scripts/gates/perf/config/bench_config.json
 create mode 100644 scripts/gates/perf/harness.py
 create mode 100644 scripts/gates/perf/preflight.py
 create mode 100644 scripts/gates/perf/run.py
 create mode 100644 scripts/gates/perf/stitch_ab.py
 rename scripts/{ => gates/quality}/nightly_email.py (100%)
 rename scripts/{ => gates/quality}/nightly_reelcount.py (99%)
 rename scripts/{ => gates/quality}/nightly_regression.py (99%)
 create mode 100644 scripts/gates/register_nightly_task.ps1
 delete mode 100644 scripts/register_nightly_task.ps1
=== commit done. Re-verify: origin/master..HEAD must list ONLY my commit ===
0aa6d4b feat(gates): local release perf gate + consolidate nightly into scripts/gates/
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

330. assistant (2026-07-09T04:20:40.926Z)

Committed as `0aa6d4b` (27 files, +1487/?113, renames preserved) and **`origin/master..HEAD` lists only my commit** ? no sibling commits crept in. Let me push and open the PR.

331. user (2026-07-09T04:20:47.680Z)

```
=== push branch ===
remote: Create a pull request for 'feat/perf-release-gate' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/feat/perf-release-gate
remote:
branch 'feat/perf-release-gate' set up to track 'origin/feat/perf-release-gate'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      feat/perf-release-gate -> feat/perf-release-gate
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

332. assistant (2026-07-09T04:21:18.523Z)

Pushed. Now the PR.

333. user (2026-07-09T04:21:40.790Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/547
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

334. assistant (2026-07-09T04:22:15.846Z)

PR opened: https://github.com/Khelsutra/badminton-highlight-indexer/pull/547 ?

Let me flag the pre-existing Windows bug I uncovered (out of scope for this PR) and close out the task tracking.

335. user (2026-07-09T04:22:25.942Z)

Noted (position 1, task_id: task_2e0a3b82). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or

superseded, call dismiss_task with this task_id. Currently pending: task_2e0a3b82 "Fix Windows path-separator break in GCSFuse helper". Continue your current work.

336. user (2026-07-09T04:22:26.373Z)

Updated task #9 status

337. assistant (2026-07-09T04:22:48.791Z)

PR #547 is open and the pre-existing bug is flagged (`task_2e0a3b82`). Let me record this session's outcome in project memory (per the handoff convention), then give you the wrap-up.

338. assistant (2026-07-09T04:23:07.830Z)

```
---
name: perf-release-gate
description: Local release PERFORMANCE gate (scripts/gates/perf) + nightly reorg ? PR #547; how
to run + the prod-vs-HEAD finding
metadata:
  type: project
---
```

PR [#547](https://github.com/Khelsutra/badminton-highlight-indexer/pull/547) (branch `feat/perf-release-gate`, open 2026-07-09) adds a **local release performance gate** and consolidates the scattered nightly CLIs into a cohesive **`scripts/gates/`** package.

What it is: `scripts/gates/perf/run.py` times checksum ? validation ? segmentation (`wasb_hybrid` GPU via WSL) ? **full-reel stitch** for the commit deployed on `api.khelsutra.guru` **vs repo HEAD**, on the local GPU, no SPA/no user step, and GATES the release (exit 1 on a stitch/total regression > tol; same 0/1/2/4 exit family as the quality gates). Run: `python scripts/gates/perf/run.py [--preflight | --stitch-only | --profile quick|full|stress | --update-baseline]`.

Reorg (honors `docs/CODE_MAP.md` "CLIs live in scripts/"): `nightly_regression.py`/`nightly_reelcount.py`/`nightly_email.py` ? `scripts/gates/quality/`; `nightly_regression.ps1` + `register_nightly_task.ps1` ? `scripts/gates/`; new `nightly_perf.ps1`. `register_nightly_task.ps1` now installs BOTH `KhelSutraNightlyRegression` (quality, 30-min cap) + `KhelSutraNightlyPerf` (perf, 3-hr cap). ALL refs updated incl. the deployed GCP nightly `deploy/gcp/nightly_regression/run_on_vm.sh` (`from scripts.gates.quality.nightly_email import` + `python scripts/gates/quality/nightly_reelcount.py`). `scripts/` is a namespace package (tests load by file path).

Finding (RTX 2070, prod `304af7b` vs HEAD `3249451`, 3 clips ? skipped the ~2h 5-min 5.3K clip): release is **performance-NEUTRAL, no regression**. checksum/validation/segmentation unchanged (same detector/inputs). The one code delta, **#531 single-pass stitch**, is **within noise locally** (matrix stitch ? ?10%/?13%/+5%; controlled A/B 1080p +6.5% vs 5.3K ?9.8% ? directions disagree). #531's real win is the L4 **NVENC** path, NOT reproducible with local libx264. Other 8 release commits are cloud-only.

Windows gotchas baked into the tool (found via self-test): gcloud resolves to `gcloud.CMD` ? needs `["cmd","/c",gcloud]`; `git worktree add` needs `-c core.longpaths=true` + a SHORT worktree root (default now `tempfile.gettempdir()/ksperf_wt`) or it hits MAX_PATH. Prod auto-resolves from the live Cloud Run image digest (`7b62a8df? ? 304af7b`) via `eval_baselines/perf_manifest.json` (git-sha image labels tracked in #532). Videos by basename + `--vault-dir` (default `~/Videos`) so no personal path is committed (leak-scan).

Verified for the PR: ruff 0.15.16 clean, mypy clean, full suite **1933 passed / cov 84.28%**. ? 2 pre-existing `test_storage.py` GCSFuse (#535) failures are **Windows path-separator** only (pass on CI Linux) ? unrelated; flagged as spawned task `task_2e0a3b82`. See [[live-deployment-state]], [[gpu-vm-setup-gotchas]], [[actual-repo-location]].

339. user (2026-07-09T04:23:07.905Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\perf-release-gate.md (file state is current in your context ? no need to Read it back)

340. user (2026-07-09T04:23:29.040Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

341. user (2026-07-09T04:23:32.822Z)

```
1       # Memory index
2
3       - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
4       - ? **LIVE deploy (2026-07-08):** [live-deployment-state.md](live-deployment-state.md) ? CP rev **00020-dtg** + worker on digest **7b62a8df** (master 304af7b; #525/#527/#528/#530). compile?L4+NVENC ON; validation stays on CP. Rollback ? 00019-96f / c6393d27. Release-process gaps ? **#532**. Next: owner CUJ to measure compile?L4.
5       - ? **CLOSED 2026-07-08:** [followup-validation-latency-live-cuj.md](followup-validation-latency-live-cuj.md) ? live SPA CUJ filled both CF-gated cells on rev **00019** (validation wall **183 s** PASS + reel compiled; fresh clip, corroborates not re-measures Video-1). Docs **PR #522**. Surfaced stitch bottleneck ? issues **#521** (compile?L4 + config placement) & **#520** (debug tracing).
6
7       - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
8       - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.
9       - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.
10      - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when genuinely satisfied a PR is merge-ready (verified), for all PRs.
11      - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
12      - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? the "weights not found" trap fix: #511 + #169 + revendor #516 merged AND **DEPLOYED LIVE** (control-plane rev `00018-qvz`, image `sha256:8045f54e?`; rollback ? rev `00017-qbd`).
```

342. user (2026-07-09T04:23:40.812Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

343. assistant (2026-07-09T04:24:03.617Z)

All wrapped up. Here's the final state.

Delivered ? PR [#547](https://github.com/Khelsutra/badminton-highlight-indexer/pull/547)

A standalone, documented, self-help, release-gating tool ? `scripts/gates/perf/` ? plus the **nightly reorg** you asked for, all in one reviewable commit (`0aa6d4b`, cut from `origin/master`, your working tree untouched).

Run it anytime:
```bash
python scripts/gates/perf/run.py --preflight     # self-help env check (names each fix)
python scripts/gates/perf/run.py                 # gate deployed-prod vs HEAD (quick)
python scripts/gates/perf/run.py --profile full  # + the 5.3K clip
python scripts/gates/perf/run.py --stitch-only   # fast #531 stitch A/B
python scripts/gates/perf/run.py --update-baseline
```

It auto-resolves prod from the **live Cloud Run image** (`7b62a8df? ? 304af7b`), runs both commits from isolated worktrees on the GPU, and gates on stitch/total regression. Exit codes match your quality gates (`0/1/2/4`).

Nightly (both legs): `pwsh scripts/gates/register_nightly_task.ps1` installs `KhelSutraNightlyRegression` (quality) **and** `KhelSutraNightlyPerf` (perf, 3-hr cap).

The reorg: everything cohesive under `scripts/gates/{perf,quality}/` ? history-preserving renames, and **every** reference updated (deployed GCP nightly, tests, baselines, 6 docs).

The performance verdict

HEAD vs deployed prod is performance-neutral ? no regression.
checksum/validation/segmentation unchanged (same detector/inputs); the one code delta, **#531 single-pass stitch**, lands **within local noise** (its real win is L4 NVENC, not reproducible on a local libx264 encode). The gate correctly **PASSES**.

Verification & honesty

- ? ruff `0.15.16` (CI-pinned) clean · mypy clean · **1933 passed / coverage 84.28%** · tool self-tested end-to-end (which caught + fixed two Windows bugs: gcloud `.CMD`, worktree MAX_PATH).
- ?? **2 pre-existing** `test_storage.py` failures are a **Windows-only** path-separator bug in the `#535` GCSFuse helper ? unrelated to this change (green on CI Linux). I filed it as a chip (`task_2e0a3b82`) for a separate fix.

Left for you (owner actions)

- **Review/merge #547.**
- **Register the nightly task** on this box (one-time `pwsh` command above).
- **Seed the perf baseline** once with `--update-baseline` (machine-specific, like the quality baselines ? deliberately not committed).
- Optional: the two skipped items surfaced along the way ? the heavy 5-min 5.3K clip (`--profile stress`), and #532 (git-sha-label prod images so prod auto-resolves without the digest map).

I left the scratchpad worktrees in place in case you want changes on the PR ? say the word and I'll clean them up.

344. user (2026-07-09T04:24:05.839Z)

<system-reminder>
The user started your suggested background task task_2e0a3b82 ("Fix Windows path-separator break in GCSFuse helper") in a separate local session. It is running independently. You will be notified here when it ends.
</system-reminder>

are the nightly and new tests going to work seamlessly in linux environment as well?

345. assistant (2026-07-09T04:25:27.882Z)

Good question ? and I did all my validation on Windows, so let me actually **check** rather than assert. There are two distinct things to reason about: the **reorg (quality nightly + tests)** which *must* work on Linux (CI + the GCP VM run there), and the **perf tool** which is Windows+WSL-specific by design. The authoritative Linux test is the CI that just ran on the PR ? let me look at that, plus the import-smoke guard and any Windows-isms in the changed files.

346. user (2026-07-09T04:25:34.276Z)

=== [1] PR #547 CI status (runs on ubuntu-latest = the real Linux test) ===
Built wheel smoke (install + launch) pass 1m9s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28993843155/job/86039223648
Dependency license scan (no copyleft in runtime deps) pass 31s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28993843155/job/86039223663
Full test suite (offline, mocked) pass 3m15s https://github.com/Khelsutra/badminton-

```
highlight-indexer/actions/runs/28993843155/job/86039223667
Golden fixtures cross-OS (macos-latest) pass    18s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28993843155/job/86039223675
Golden fixtures cross-OS (ubuntu-latest)        pass    26s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28993843155/job/86039223655
Golden fixtures cross-OS (windows-latest)       pass    1m13s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28993843155/job/86039223688
Leak scan (PII / attribution guard)     pass    9s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28993843155/job/86039223664
Markdown link check (live docs) pass    10s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28993843155/job/86039223673
Mypy (lenient baseline) pass    37s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28993843155/job/86039223689
Ruff lint       pass    8s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28993843155/job/86039223708
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28993843155/job/86039223650

=== [2] does test_import_smoke.py touch scripts/ or the perf tool? (CI Linux import guard) ===
27:    for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
93:    for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

347. assistant (2026-07-09T04:26:25.637Z)

CI on PR #547 is all-green on Linux ? that's the authoritative answer for the reorg. Let me confirm the deployed GCP-VM import path specifically (CI doesn't run `run_on_vm.sh`), by testing the namespace import in **actual Linux (WSL)**.

348. user (2026-07-09T04:26:33.206Z)

```
=== nightly_email.py imports (stdlib-only? => import test is meaningful anywhere) ===
21:from __future__ import annotations
23:import logging
24:import os
25:from email.message import EmailMessage
26:from typing import Optional

=== LINUX (WSL) test of run_on_vm.sh's exact import, from repo-root cwd ===
wsl repo path: /mnt/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-
highlight-indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt-dev
LINUX namespace import OK: scripts.gates.quality.nightly_email
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

349. assistant (2026-07-09T04:27:08.392Z)

Verified on both fronts. Here's the precise answer, split by component ? because they have genuinely different Linux stories.

The reorg + quality nightly + tests ? **yes, Linux-verified** ?

- **CI on PR #547 is all-green on `ubuntu-latest`**: full suite (3m15s), ruff, mypy, import-smoke, cross-OS golden fixtures (ubuntu/macos/windows), leak-scan, **markdown-link-check** (confirms my doc-ref edits didn't break links), wheel-smoke. That suite includes my moved `tests/test_nightly_{reelcount,regression}.py` loaders ? they resolve `scripts/gates/quality/` via portable `os.path.join`, so they pass on Linux.
- **The deployed GCP-VM path** (`run_on_vm.sh`, which CI does *not* run) ? I just tested its exact updated import in **real Linux (WSL)**: `from scripts.gates.quality.nightly_email import send_report` ? `LINUX namespace import OK`. The `REPO_ROOT` depth fix (`dirname×4`) is `os.path.dirname`, which is OS-agnostic, and `scripts/` resolves as a PEP-420 namespace package identically on Linux.

- The 2 `test_storage` failures were **Windows-only** (the `#535` GCSFuse `/` vs `\` bug) ?
Linux is the *green* environment for those.

The `.ps1` wrappers + registrar ? **Windows-only, by design** (not a gap)

PowerShell Scheduled Tasks are the *Windows* local-nightly mechanism; the *Linux* nightly is the
separate GCP VM path (`run_on_vm.sh` + Cloud Scheduler). Both OSes are covered by different
delivery ? the `.ps1` files aren't meant to run on Linux.

The perf tool (`scripts/gates/perf/`) ? **Windows+WSL-specific as-wired; safe on Linux but won't *run* without

aspect	Linux behavior
Import / CI	? Fine ? not imported by any test (import-smoke only globs `backend/`); no
module-level `torch`/WSL
`gcloud.CMD` fix, `core.longpaths`	? Degrade to no-ops on Linux (guarded / harmless)
preflight + config	?? It checks `wsl.exe` + the `wasb` conda env and uses
`detector_impl=auto` (the **WSL** path). On a Linux GPU box those checks fail cleanly (exit 2,
no crash) because wasb there runs `detector_impl=native` in-process.

So it doesn't **break** on Linux ? it **preflight-refuses** on Linux, because it's wired for
your Windows+WSL box. The Linux-GPU equivalent of "time the pipeline on real clips" already
exists as the **GCP reelcount nightly** (`scripts/gates/quality/nightly_reelcount.py`, wall-
seconds + cost, Linux-validated).

If you want the perf tool itself to also run on a Linux GPU host, it's a small addition:
branch preflight on OS (skip WSL, verify the native wasb env) and default `detector_impl=native`
on Linux. Want me to add that (a follow-up commit on this PR), or leave the perf gate as the
Windows-box tool and lean on the reelcount nightly for Linux?

350. user (2026-07-09T04:42:06.445Z)

can you address the comments on the PR and then add teh linux working in the PR as well

351. assistant (2026-07-09T04:42:22.877Z)

Let me pull the PR comments/reviews and current CI state, and sync the branch, before making
changes.

352. user (2026-07-09T04:42:31.037Z)

=== inline review comments ===

=== reviews (body-level) ===
(avidullu/PENDING) ## Summary

PR #547 consolidates the nightly quality CLIs under `scripts/gates/quality/`, wires GCP
`run_on_vm.sh` + tests to the new paths, and adds a solid local release **performance** harness
(`scripts/gates/perf/`) that compares a resolved prod commit vs HEAD via git worktrees, shared
`bench_config.json`, `skip_ai_handoff`, and stage timing. The reorg of quality scripts
(REPO_ROOT depth, import path, test loaders, GCP runner) looks coherent. Dominant risks:
broken default `$RepoRoot` in the new PowerShell wrappers/register script (one `Split-Path`
short after the move), **the perf gate treating hard failures as non-regressions** (missing
stages ? `pct` returns `None` ? exit 0), and **silent PASS when nothing measurable ran** (empty
stitch/vault). Also: no pure-unit tests for the new gate logic, stale paths in `perf/README.md`,
and existing Windows scheduled tasks will no-op after delete-without-shim until re-registered.

Issue counts by severity

- bugs: 4
- suggestions: 7
- nits: 2

=== issue-level comments ===

353. assistant (2026-07-09T04:42:56.934Z)

There's a **pending review from you** (`avidullu`) ? a drafted review with **4 bugs, 7 suggestions, 2 nits**. The summary already flags real issues I need to fix (notably a `$RepoRoot` depth bug in the moved PowerShell wrappers, and the gate treating hard failures/empty runs as PASS). The inline comments are pending, so let me pull them directly by review ID.

354. user (2026-07-09T04:43:01.582Z)

pending review id: 4659798341
=== pending review inline comments ===

--- scripts/gates/register_nightly_task.ps1:null ---
[bug] Default `$RepoRoot` only walks up two directories (`Split-Path` twice). The script now lives at `scripts/gates/register_nightly_task.ps1`, so the default resolves to `?/scripts` rather than the repo root. Running without an explicit `-RepoRoot` registers both tasks with wrong paths (`scripts\scripts\gates\?`), so wrappers permanently hit ?wrapper absent / SKIPPED? and never arm. The same two-level default is copied into `nightly_perf.ps1` and was left unchanged on the moved `nightly_regression.ps1`.

Suggestion: Use three `Split-Path -Parent` hops (or resolve via `Join-Path $PSScriptRoot '..\..'` and `Resolve-Path`) so default `$RepoRoot` is the checkout root. Fix the same default in `scripts/gates/nightly_perf.ps1` and `scripts/gates/nightly_regression.ps1`.

--- scripts/gates/perf/run.py:null ---
[bug] Gating only fires when both sides produce numeric stage times (`d is not None`). If HEAD fails (no segments ? `stitch_s=None`, harness crash ? empty result, timeout ? missing JSON), `pct()` returns `None` and **no regression finding is recorded**. A completely broken HEAD can still exit 0 PASS while the report quietly lists errors. Quality gates treat pipeline errors as regressions; this gate does not.

Suggestion: In `build()` / `exit_code()`, treat missing/failed gated stages on `rc` (when `prod` succeeded) as regression (or at least exit 2). Also fail if `row["errors"]["rc"]` is non-empty for a gated path. Mirror quality?s ?errored video = regression? semantics.

--- scripts/gates/perf/run.py:null ---
[bug] The only ?nothing to run? guard is `if not videos and not args.stitch_only`. With `--stitch-only` (or when the matrix is skipped and every `stitch_ab` source is missing), the driver still builds an empty summary, produces no findings, and returns **0 PASS**. A misconfigured vault / profile therefore green-lights the release.

Suggestion: After the run loop, if there are no comparable stage rows and no completed stitch A/B medians, return exit 2 (`nothing-ran`). Do the same when every harness/stitch result is errors-only.

--- scripts/gates/nightly_perf.ps1:null ---
[bug] Same incorrect default `$RepoRoot` as Issue 1 (`Split-Path` ×2 from `scripts/gates/`). Manual invocation without `-RepoRoot` looks for `scripts\scripts\gates\perf\run.py`, takes the ?tool absent ? exit 0 SKIPPED? branch, and never runs the gate.

Suggestion: Default `$RepoRoot` to three parents of `$PSScriptRoot` / `$MyInvocation.MyCommand.Path`. Prefer requiring callers to pass `-RepoRoot` only when non-default.

--- scripts/gates/perf/run.py:null ---
[suggestion] `exit_code()` never returns 3, and a missing `perf_baseline.json` is not treated as ?no baseline.? Shared suite docs (`scripts/gates/README.md`, wrapper status `NOBASELINE`) claim the common exit family including **3 = no baseline**, and `nightly_perf.ps1` maps code 3 to `NOBASELINE`. Live prod-vs-HEAD comparison can work without a frozen baseline, but the documented contract is inconsistent and the wrapper?s branch is dead.

Suggestion: Either implement exit 3 when baseline gating is expected and the file is

missing, or document that perf only uses 0/1/2/4, remove the wrapper?s `3` branch, and drop ?3 no baseline? from the shared exit-code table.

--- eval_baselines/perf_manifest.json:null ---
[suggestion] Gate stages include both `stitch_s` and `total_s`. `total_s` is the sum of checksum + validation + **segmentation** + stitch. Segmentation is explicitly described as a constant backdrop / not gated, but it usually dominates wall time and is GPU/WSL-noisy. A 20% swing in segmentation alone can fail the release on `total_s` even when stitch (the intended #531 signal) is flat or improved?undermining the fairness story.

Suggestion: Gate only `stitch_s` (+ always-on stitch A/B). Keep `total_s` as report-only context, or require both stitch regression *and* total regression before failing. Document why total is gated if you keep it.

--- scripts/gates/perf/run.py:null ---
[suggestion] No unit tests cover the pure gate core (`pct`, `build`, `exit_code`, stale detection, prod resolve fallback). Quality nightlies ship offline pure-core tests; this PR only rewires those paths and adds a large untested harness. Easy-to-regress bugs (Issues 2?3) would have been caught by a few dict-in/dict-out tests.

Suggestion: Add `tests/test_perf_gate.py` loading `run.py` by path (same pattern as `test_nightly_regression.py`) and covering: regression/improvement thresholds, missing-stage handling, empty results ? exit 2, stale prod_commit/profile, `pct` edge cases (`prod==0`, `None`).

--- scripts/gates/perf/README.md:null ---
[suggestion] Nightly integration / Files sections still document pre-move paths (`scripts/nightly_perf.ps1`, `scripts/register_nightly_task.ps1`) and quality siblings as `scripts/nightly_regression.py` / `scripts/nightly_reelcount.py`. Operators following this README will run missing scripts. `scripts/gates/README.md` has the correct paths; this file was not updated to match the reorg.

Suggestion: Point every operator path at `scripts/gates/?` and align the companion quality references with `scripts/gates/quality/?`.

--- scripts/gates/README.md:null ---
[suggestion] Old `scripts/register_nightly_task.ps1` and `scripts/nightly_regression.ps1` are deleted with no shim. Existing Windows Scheduled Tasks encode the **old** wrapper path at registration time; after merge they hit the inline ?wrapper absent ? SKIPPED exit 0? guard forever and stop protecting quality until someone re-runs the new register script. Easy to miss in production.

Suggestion: Leave thin shims at the old paths that forward to `scripts/gates/?`, and call out in the gates README / PR body: ?re-run `pwsh -File scripts/gates/register_nightly_task.ps1 -RepoRoot ?` after merge.?

--- scripts/gates/nightly_perf.ps1:null ---
[suggestion] Exit code 2 (preflight failed / env not ready) is rewritten to **exit 0 SKIPPED**. Intentional ?don?t red-light Task Scheduler before WSL lands,? but a lasting env break (GPU driver, missing vault, broken wasb env) also looks green every night. Task history will not surface failures unless someone reads `LATEST_STATUS.txt`.

Suggestion: Keep writing `SKIPPED` to the status file, but exit non-zero after N consecutive skips, or use a distinct scheduled-task result / email on repeated preflight failure. At least document that Task Scheduler ?success? is not the gate signal.

--- scripts/gates/perf/run.py:null ---
[suggestion] Comparison order is always prod then rc for every video. Shared machine effects (disk cache, GPU clocks, thermal throttle, WSL cold start) systematically favor the second run (HEAD). That biases the ?HEAD slower than prod? gate toward false improvements / missed regressions.

Suggestion: Alternate order per video (or randomize with a logged seed), add a short warmup stitch/segment, and/or require N reps with median like stitch A/B for the full matrix?s gated stages.

```
--- scripts/gates/perf/run.py:null ---
**[nit]** Module docstring still says companions are `scripts/nightly_regression.py` +
`scripts/nightly_reelcount.py` (pre-move). Same class of path drift as the perf README.

**Suggestion:** Update to `scripts/gates/quality/nightly_regression.py` and
`?/nightly_reelcount.py`.

--- scripts/gates/perf/harness.py:null ---
**[nit]** After a successful `process_video`, stitch inputs are taken from
`db.query_play_segments` with a fallback to the in-memory return value. There is no
`random.seed(0)` (reelcount uses it for determinism) and no guard that prod/rc segment *counts*
match before comparing `stitch_s`. If HEAD changes segmentation, stitch times are not an apples-
to-apples code-path comparison (stitch A/B with fixed pairs is the fair path; the full-matrix
stitch gate is not).

**Suggestion:** Prefer fixed pairs for any gated stitch timing (or skip gating full-matrix
`stitch_s` when `segments.prod != segments.rc`). Optionally seed RNGs for less noise on
secondary fields.
```

355. assistant (2026-07-09T04:44:33.714Z)

This is a sharp, high-quality review (your own draft) ? 4 bugs, 7 suggestions, 2 nits. Several are real defects I introduced (the PowerShell `$RepoRoot` depth, the gate passing on a failed/empty HEAD, gating `total_s` which I *myself* called a non-gated backdrop). Let me address all of them plus add Linux support. Let me set up tracking and read the exact regions I need to change.

356. user (2026-07-09T04:44:35.879Z)

```
Task #10 created successfully: Fix PowerShell $RepoRoot depth (3 wrappers)
```

357. user (2026-07-09T04:44:44.311Z)

```
Task #11 created successfully: Fix gate correctness (failed HEAD, nothing-ran, gate stitch-only)
```

358. user (2026-07-09T04:44:47.073Z)

```
Task #12 created successfully: Docs/exit-3/order/README fixes
```

359. user (2026-07-09T04:44:48.806Z)

```
Task #13 created successfully: Add tests/test_perf_gate.py (S3)
```

360. user (2026-07-09T04:44:50.615Z)

```
Task #14 created successfully: Add Linux support to perf tool
```

361. user (2026-07-09T04:45:08.419Z)

```
=== $RepoRoot defaults (all 3 ps1) ===
register_nightly_task.ps1:32:    [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
nightly_perf.ps1:30:    [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
nightly_regression.ps1:30:    [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),

=== perf/README.md pre-move path refs (S4) ===
12:(`scripts/nightly_regression.py`, `scripts/nightly_reelcount.py`) and plugs into the same
97:* `scripts/nightly_perf.ps1` ? the Scheduled-Task wrapper (mirrors `nightly_regression.ps1`):
99:* `scripts/register_nightly_task.ps1` registers **both** legs by default ? quality
104:pwsh -File scripts/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'   # both legs
105:pwsh -File scripts/register_nightly_task.ps1 -PerfOnly                         # just perf
137:scripts/nightly_perf.ps1            # nightly wrapper
```

```
138:scripts/register_nightly_task.ps1   # registers quality + perf scheduled tasks

=== nightly_perf.ps1 exit switch (S1 exit-3, S6) ===
8:    GATES the release (exit 1 on a stitch/total regression beyond tolerance). Writes a dated
report
9:    + log under output/nightly_perf/ and a LATEST_STATUS.txt marker, then exits with the
gate's code
17:    GRACEFUL no-op (exit 0) when the tool isn't present yet OR the environment isn't ready
18:    (preflight -> the tool's exit 2): a registered task never errors in the interim and self-
activates
33:    [switch]$Pull,
34:    [switch]$UpdateBaseline
45:function Write-Status([string]$status, [int]$code, [string]$detail) {
47:    "$stamp  status=$status exit=$code  $detail" | Out-File -FilePath $statusFile -Encoding
utf8
53:    Write-Status 'SKIPPED' 0 'perf gate absent (awaiting merge)'
54:    exit 0
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

362. assistant (2026-07-09T04:45:18.176Z)

Fixing the `$RepoRoot` depth bug in all three wrappers (2 hops ? 3):

363. user (2026-07-09T04:45:22.239Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\register_nightly_task.ps1 has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

364. user (2026-07-09T04:45:24.722Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

365. user (2026-07-09T04:45:26.919Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_regression.ps1 has been updated successfully. (file state is current
in your context ? no need to Read it back)
```

366. assistant (2026-07-09T04:45:36.013Z)

Now the substantive gate-logic rewrite in `run.py`. Let me read the whole file to make coordinated changes correctly.

367. user (2026-07-09T04:45:36.959Z)

```
1       #!/usr/bin/env python3
2       """Local release PERFORMANCE gate ? deployed production vs repo HEAD, on this GPU.
3
4       Standalone, release-gating, repeatable. Times four stages with NO SPA / NO user step ?
5       checksum -> validation -> segmentation (wasb_hybrid, GPU) -> full-reel stitch ? for the
6       commit currently serving api.khelsutra.guru and for repo HEAD, from isolated git
worktrees
7       over the same vault clips, then reports per-stage deltas and GATES the release.
8
9       Companion to the QUALITY gates (scripts/nightly_regression.py +
scripts/nightly_reelcount.py).
10      See scripts/gates/perf/README.md for methodology, self-help, and the nightly hook.
11
```

```
    Exit codes (same family as the quality gates):
      0 PASS · 1 REGRESSION (HEAD slower than prod beyond tol) · 2 nothing-ran / preflight
failed
      3 reserved · 4 STALE (committed baseline frozen against a different prod commit /
video-set)

    Examples:
      python scripts/gates/perf/run.py --preflight
      python scripts/gates/perf/run.py                      # quick profile, prod auto-
resolved, gate HEAD vs prod
      python scripts/gates/perf/run.py --profile full       # + the 5.3K clip
      python scripts/gates/perf/run.py --stitch-only        # just the #531 stitch A/B
(fast)
      python scripts/gates/perf/run.py --prod-commit 304af7b --rc-ref origin/master
      python scripts/gates/perf/run.py --update-baseline    # snapshot precomputed release
numbers
    """
    import argparse
    import json
    import os
    import shutil
    import subprocess
    import sys
    import tempfile
    import time
    from datetime import datetime, timezone

    HERE = os.path.dirname(os.path.abspath(__file__))
    REPO_ROOT = os.path.abspath(os.path.join(HERE, "..", "..", ".."))
    HARNESS = os.path.join(HERE, "harness.py")
    STITCH = os.path.join(HERE, "stitch_ab.py")
    DEFAULT_CONFIG = os.path.join(HERE, "config", "bench_config.json")
    DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "perf_manifest.json")
    DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "perf_baseline.json")
    DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_perf")

    # Preflight import that works both as a module (python -m scripts.gates.perf.run) and as
a
    # direct script (python scripts/gates/perf/run.py).
    try:
        from . import preflight as pf
    except ImportError:
        sys.path.insert(0, HERE)
        import preflight as pf  # type: ignore


    def iso_now():
        return datetime.now(timezone.utc).isoformat(timespec="seconds")


    def sh(cmd, cwd=None, timeout=120):
        p = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, timeout=timeout)
        return p.returncode, (p.stdout or "").strip(), (p.stderr or "").strip()


    def git_short(repo, ref):
        rc, out, _ = sh(["git", "-C", repo, "rev-parse", "--short", ref])
        return out if rc == 0 else None


    # ---------------------------------------------------------------------- resolve
prod
    def resolve_prod(manifest, args, log):
        if args.prod_commit:
            return git_short(REPO_ROOT, args.prod_commit) or args.prod_commit, "explicit
```

```
--prod-commit"
70          if args.prod_ref:
71              return git_short(REPO_ROOT, args.prod_ref), f"--prod-ref {args.prod_ref}"
72          prod = manifest.get("prod", {})
73          cr = prod.get("cloud_run", {})
74          mp = prod.get("image_digest_to_commit", {})
75          gcloud = shutil.which("gcloud")
76          if gcloud and cr:
77              try:
78                  # On Windows `gcloud` resolves to gcloud.CMD, which subprocess can't exec
directly.
79                  gc = ["cmd", "/c", gcloud] if gcloud.lower().endswith((".cmd", ".bat")) else
[gcloud]
80                  rc, out, _ = sh(gc + ["run", "services", "describe", cr["service"],
81                                        "--region", cr["region"], "--project", cr["project"],
82                                        "--format=value(spec.template.spec.containers[0].image)"],
timeout=90)
83                  if rc == 0 and "@" in out:
84                      digest = out.split("@")[-1].strip()
85                      if digest in mp:
86                          log(f"prod resolved from live Cloud Run image {digest[:19]}? ->
{mp[digest]}")
87                          return git_short(REPO_ROOT, mp[digest]) or mp[digest], f"live Cloud
Run {digest[:19]}?"
88                      log(f"live Cloud Run digest {digest[:19]}? not in image_digest_to_commit
map (update it / #532)")
89              except Exception as e:
90                  log(f"Cloud Run resolve failed ({e}); falling back to pinned prod_commit")
91          pin = prod.get("pinned_commit")
92          return (git_short(REPO_ROOT, pin) or pin), "pinned prod_commit (Cloud Run resolve
unavailable)"
93
94
95      # ------------------------------------------------------------------------- worktrees
96      def worktree(repo, path, commit, log):
97          if os.path.exists(path):
98              sh(["git", "-C", repo, "worktree", "remove", "--force", path], timeout=120)
99          # core.longpaths=true so a deep worktree root + long repo filenames don't blow
Windows MAX_PATH.
100         rc, _, err = sh(["git", "-C", repo, "-c", "core.longpaths=true",
101                          "worktree", "add", "--detach", path, commit], timeout=180)
102         if rc != 0:
103             raise RuntimeError(f"worktree add {path}@{commit} failed: {err}")
104         log(f"worktree {os.path.basename(path)} -> {git_short(path, 'HEAD')}")
105
106
107     # ------------------------------------------------------------------------ run one
video/version
108     def run_harness(version, worktree_dir, commit, name, video, cfg, runs_dir, timeout,
log):
109         run_dir = os.path.join(runs_dir, version, name)
110         os.makedirs(run_dir, exist_ok=True)
111         jo = os.path.join(run_dir, "result.json")
112         cmd = [sys.executable, HARNESS, "--worktree", worktree_dir, "--config", cfg, "--
video", video,
113                "--name", name, "--version", version, "--commit", commit, "--run-dir",
run_dir, "--json-out", jo]
114         log(f"START {version}/{name}")
115         t0 = time.perf_counter()
116         try:
117             p = subprocess.run(cmd, cwd=worktree_dir, capture_output=True, text=True,
timeout=timeout)
118             tail = "\n".join((p.stderr or "").strip().splitlines()[-2:])
119             log(f"DONE  {version}/{name} rc={p.returncode}
wall={time.perf_counter()-t0:.0f}s :: {tail}")
```

```
        except subprocess.TimeoutExpired:
            log(f"TIMEOUT {version}/{name} after {time.perf_counter()-t0:.0f}s")
        return json.load(open(jo, encoding="utf-8")) if os.path.exists(jo) else \
            {"version": version, "video": name, "commit": commit, "errors": {"driver": "no
result.json"}}


    def run_stitch(version, worktree_dir, commit, label, source, pairs, reps, cfg, runs_dir,
timeout, log):
        run_dir = os.path.join(runs_dir, "stitch", label, version)
        os.makedirs(run_dir, exist_ok=True)
        jo = os.path.join(run_dir, "result.json")
        cmd = [sys.executable, STITCH, "--worktree", worktree_dir, "--config", cfg, "--
source", source,
               "--pairs", json.dumps(pairs), "--reps", str(reps), "--version", version, "--
commit", commit,
               "--run-dir", run_dir, "--json-out", jo]
        log(f"STITCH {version}/{label} (reps={reps})")
        try:
            subprocess.run(cmd, cwd=worktree_dir, capture_output=True, text=True,
timeout=timeout)
        except subprocess.TimeoutExpired:
            log(f"STITCH TIMEOUT {version}/{label}")
        return json.load(open(jo, encoding="utf-8")) if os.path.exists(jo) else {}


    # ---------------------------------------------------------------------- gating +
report
    def pct(prod, rc):
        if not isinstance(prod, (int, float)) or not isinstance(rc, (int, float)) or prod ==
0:
            return None
        return round((rc - prod) / prod * 100.0, 1)


    def build(results, stitch_results, videos, meta, gate):
        """Return (summary, findings). Finding kind: regression|improvement."""
        idx = {}
        for r in results:
            idx.setdefault(r.get("version"), {})[r.get("video")] = r
        findings = []
        per_video = {}
        for name in videos:
            p = idx.get("prod", {}).get(name, {})
            c = idx.get("rc", {}).get(name, {})
            row = {"prod": {}, "rc": {}, "delta_pct": {}}
            for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s",
"total_s"):
                pv = p.get("stages", {}).get(k) if k != "total_s" else p.get("total_s")
                rv = c.get("stages", {}).get(k) if k != "total_s" else c.get("total_s")
                row["prod"][k], row["rc"][k] = pv, rv
                d = pct(pv, rv)
                row["delta_pct"][k] = d
                if k in gate["stages"] and d is not None:
                    if d > gate["tol_pct"]:
                        findings.append({"kind": "regression", "where": f"{name}/{k}",
"delta_pct": d})
                    elif d < -gate["tol_pct"]:
                        findings.append({"kind": "improvement", "where": f"{name}/{k}",
"delta_pct": d})
            row["segments"] = {"prod": p.get("segments"), "rc": c.get("segments")}
            row["errors"] = {"prod": list(p.get("errors", {}).keys()), "rc":
list(c.get("errors", {}).keys())}
            per_video[name] = row
        stitch = {}
```

```
        for label, sr in (stitch_results or {}).items():
            pm = sr.get("prod", {}).get("median_s")
            cm = sr.get("rc", {}).get("median_s")
            d = pct(pm, cm)
            stitch[label] = {"prod_median_s": pm, "rc_median_s": cm, "delta_pct": d,
                             "prod_times": sr.get("prod", {}).get("times_s"),
                             "rc_times": sr.get("rc", {}).get("times_s")}
            # The stitch A/B is always gated ? it's the cleanest #531 signal (no
segmentation noise).
            if d is not None and d > gate["tol_pct"]:
                findings.append({"kind": "regression", "where": f"stitch_ab/{label}",
"delta_pct": d})
            elif d is not None and d < -gate["tol_pct"]:
                findings.append({"kind": "improvement", "where": f"stitch_ab/{label}",
"delta_pct": d})
        summary = {"meta": meta, "per_video": per_video, "stitch_ab": stitch}
        return summary, findings


    def exit_code(findings, stale):
        if any(f["kind"] == "regression" for f in findings):
            return 1
        if stale:
            return 4
        return 0


    def fmt(v):
        return "?" if not isinstance(v, (int, float)) else f"{v:.2f}s"


    def dpct(d):
        return "?" if d is None else f"{d:+.1f}%"


    def write_report(summary, findings, code, report_dir, date):
        os.makedirs(report_dir, exist_ok=True)
        m = summary["meta"]
        badge = {0: "? PASS", 1: "? REGRESSION", 4: "? STALE"}.get(code, f"exit {code}")
        L = [f"# Release performance gate ? HEAD vs production ({date})", "",
             f"**Status: {badge}**  ·  exit {code}",
             f"- Prod (api.khelsutra.guru): `{m['prod_commit']}` ({m['prod_source']})  ·
HEAD: `{m['rc_commit']}`",
             f"- GPU: {m.get('gpu','?')}  ·  segmenter: wasb_hybrid (WSL2 GPU)  ·  encoder:
libx264  ·  gate: {m['gate']['stages']} > {m['gate']['tol_pct']}%",
             "- ?% = (HEAD ? prod)/prod. Negative = HEAD faster. Only stitch/total are
gated; the rest is context.", ""]
        reg = [f for f in findings if f["kind"] == "regression"]
        imp = [f for f in findings if f["kind"] == "improvement"]
        if reg:
            L.append("**Regressions:** " + ", ".join(f"{f['where']} {f['delta_pct']:+.1f}%"
for f in reg))
        if imp:
            L.append("_Improvements:_ " + ", ".join(f"{f['where']} {f['delta_pct']:+.1f}%"
for f in imp))
        L.append("")
        for name, row in summary["per_video"].items():
            L.append(f"## {name}  ·  segments prod={row['segments']['prod']}
rc={row['segments']['rc']}")
            L.append("| stage | prod | HEAD | ?% |")
            L.append("|---|---|---|---|")
            for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s",
"total_s"):
                d = row["delta_pct"][k]
                gated = "*" if k in summary["meta"]["gate"]["stages"] else ""
```

```
                L.append(f"| {k.replace('_s','')}{gated} | {fmt(row['prod'][k])} |
{fmt(row['rc'][k])} | {dpct(d)} |")
            L.append("")
        if summary["stitch_ab"]:
            L.append("## Stitch A/B (#531, segmentation excluded) ? gated*")
            L.append("| reel | prod median | HEAD median | ?% |")
            L.append("|---|---|---|---|")
            for label, s in summary["stitch_ab"].items():
                L.append(f"| {label}* | {fmt(s['prod_median_s'])} | {fmt(s['rc_median_s'])}
| {dpct(s['delta_pct'])} |")
            L.append("")
        L.append("_* gated stage. `--update-baseline` freezes these numbers into
eval_baselines/perf_baseline.json._")
        open(os.path.join(report_dir, f"{date}.md"), "w",
encoding="utf-8").write("\n".join(L))
        payload = {"exit_code": code, "status": badge, "findings": findings, **summary}
        open(os.path.join(report_dir, f"{date}.json"), "w",
encoding="utf-8").write(json.dumps(payload, indent=2))
        return os.path.join(report_dir, f"{date}.md")


    # ---------------------------------------------------------------------- main
    def main(argv=None) -> int:
        ap = argparse.ArgumentParser(description="Local release performance gate: production
vs repo HEAD.")
        ap.add_argument("--config", default=DEFAULT_CONFIG)
        ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
        ap.add_argument("--baseline", default=DEFAULT_BASELINE)
        ap.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        ap.add_argument("--profile", default="quick", choices=("quick", "full", "stress"))
        ap.add_argument("--videos", default=None, help="comma list of names, overrides
--profile")
        ap.add_argument("--vault-dir", default=os.environ.get("RALLY_VAULT_DIR") or
os.path.expanduser("~/Videos"))
        ap.add_argument("--prod-commit", default=None)
        ap.add_argument("--prod-ref", default=None)
        ap.add_argument("--rc-ref", default="HEAD")
        ap.add_argument("--worktree-root", default=os.path.join(tempfile.gettempdir(),
"ksperf_wt"),
                        help="Where prod/rc worktrees are created (short path avoids Windows
MAX_PATH).")
        ap.add_argument("--runs-dir", default=None)
        ap.add_argument("--timeout", type=int, default=5400)
        ap.add_argument("--tol", type=float, default=None, help="override manifest gate
tol_pct")
        ap.add_argument("--stitch-only", action="store_true")
        ap.add_argument("--no-stitch-ab", action="store_true")
        ap.add_argument("--preflight", action="store_true", help="run checks and exit")
        ap.add_argument("--skip-preflight", action="store_true")
        ap.add_argument("--update-baseline", action="store_true")
        ap.add_argument("--keep-worktrees", action="store_true")
        args = ap.parse_args(argv)

        manifest = json.load(open(args.manifest, encoding="utf-8"))
        cfg_dict = json.load(open(args.config, encoding="utf-8"))
        gate = dict(manifest.get("gate", {"stages": ["stitch_s", "total_s"], "tol_pct":
20.0}))
        if args.tol is not None:
            gate["tol_pct"] = args.tol

        logs = []

        def log(msg):
            line = f"{iso_now()}  {msg}"
            print(line, flush=True)
```

```
282             logs.append(line)
283
284         # preflight
285         if not args.skip_preflight or args.preflight:
286             checks = pf.run_checks(cfg_dict, want_prod_resolve=(args.prod_commit is None and
args.prod_ref is None))
287             fails = pf.print_checks(checks)
288             if args.preflight:
289                 return 2 if fails else 0
290             if fails:
291                 log(f"preflight: {fails} REQUIRED checks failing ? aborting (exit 2). See
fixes above.")
292                 return 2
293
294         sh(["git", "-C", REPO_ROOT, "fetch", "origin", "--quiet"], timeout=120)
295         prod_commit, prod_source = resolve_prod(manifest, args, log)
296         rc_commit = git_short(REPO_ROOT, args.rc_ref)
297         if not prod_commit or not rc_commit:
298             log(f"could not resolve commits (prod={prod_commit}, rc={rc_commit})")
299             return 2
300         log(f"prod={prod_commit} ({prod_source})  ·  HEAD({args.rc_ref})={rc_commit}")
301
302         # video set
303         names = args.videos.split(",") if args.videos else
manifest["profiles"][args.profile]
304         by_name = {v["name"]: v for v in manifest["videos"]}
305         videos = []
306         for n in names:
307             f = by_name.get(n, {}).get("file", n)
308             path = f if os.path.isabs(f) else os.path.join(args.vault_dir, f)
309             if os.path.exists(path):
310                 videos.append((n, path))
311             else:
312                 log(f"SKIP {n}: not found at {path} (set --vault-dir / RALLY_VAULT_DIR)")
313         if not videos and not args.stitch_only:
314             log("no videos found ? nothing to run (exit 2)")
315             return 2
316
317         runs_dir = args.runs_dir or os.path.join(args.worktree_root, "runs")
318         os.makedirs(args.worktree_root, exist_ok=True)
319         wt_prod = os.path.join(args.worktree_root, "prod")
320         wt_rc = os.path.join(args.worktree_root, "rc")
321         gpu = "?"
322         try:
323             _, gpu, _ = sh(["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
timeout=15)
324             gpu = gpu.splitlines()[0] if gpu else "?"
325         except Exception:
326             pass
327
328         try:
329             worktree(REPO_ROOT, wt_prod, prod_commit, log)
330             worktree(REPO_ROOT, wt_rc, rc_commit, log)
331
332             results = []
333             if not args.stitch_only:
334                 for name, path in videos:
335                     for version, wt, commit in (("prod", wt_prod, prod_commit), ("rc",
wt_rc, rc_commit)):
336                         results.append(run_harness(version, wt, commit, name, path,
args.config, runs_dir, args.timeout, log))
337
338             stitch_results = {}
339             if not args.no_stitch_ab:
340                 for case in manifest.get("stitch_ab", []):
```

```
                    src = case["file"] if os.path.isabs(case["file"]) else
os.path.join(args.vault_dir, case["file"])
                    if not os.path.exists(src):
                        log(f"stitch A/B skip {case['label']}: source missing {src}")
                        continue
                    stitch_results[case["label"]] = {}
                    for version, wt, commit in (("prod", wt_prod, prod_commit), ("rc",
wt_rc, rc_commit)):
                        stitch_results[case["label"]][version] = run_stitch(
                            version, wt, commit, case["label"], src, case["pairs"],
case["reps"],
                            args.config, runs_dir, args.timeout, log)
        finally:
            if not args.keep_worktrees:
                for wt in (wt_prod, wt_rc):
                    sh(["git", "-C", REPO_ROOT, "worktree", "remove", "--force", wt],
timeout=120)

        meta = {"generated": iso_now(), "prod_commit": prod_commit, "prod_source":
prod_source,
                "rc_commit": rc_commit, "rc_ref": args.rc_ref, "gpu": gpu, "profile":
args.profile,
                "gate": gate, "segmenter": manifest.get("segmenter", "wasb_hybrid")}
        # --stitch-only runs no per-video matrix, so don't emit empty per-video rows.
        report_videos = [] if args.stitch_only else [n for n, _ in videos]
        summary, findings = build(results, stitch_results, report_videos, meta, gate)

        if args.update_baseline:
            base = {"_doc": "Frozen release perf numbers (scripts/gates/perf/run.py
--update-baseline).",
                    "git_commit": rc_commit, "prod_commit": prod_commit, "generated_at":
iso_now(),
                    "profile": args.profile, "gate": gate, **summary}
            os.makedirs(os.path.dirname(args.baseline), exist_ok=True)
            json.dump(base, open(args.baseline, "w", encoding="utf-8"), indent=2)
            log(f"baseline frozen -> {args.baseline}")

        # STALE (exit 4): a committed baseline exists but was frozen against a different
production
        # commit or video-set ? its precomputed numbers no longer describe this setup (re-
freeze).
        # Regression still takes precedence (exit 1).
        stale = False
        if not args.update_baseline and os.path.exists(args.baseline):
            try:
                b = json.load(open(args.baseline, encoding="utf-8"))
                if b.get("prod_commit") != prod_commit or b.get("profile") != args.profile:
                    stale = True
                    log(f"baseline STALE (frozen
prod={b.get('prod_commit')}/{b.get('profile')} vs "
                        f"current prod={prod_commit}/{args.profile}) ? re-freeze with
--update-baseline")
            except Exception:
                pass

        date = iso_now()[:10]
        code = exit_code(findings, stale=stale)
        report = write_report(summary, findings, code, args.report_dir, date)
        logdir = os.path.join(args.report_dir, "logs")
        os.makedirs(logdir, exist_ok=True)
        open(os.path.join(logdir, f"{date}.log"), "a",
encoding="utf-8").write("\n".join(logs) + "\n")
        log(f"report: {report}  ·  exit {code}")
        return code

```

```
393
394     if __name__ == "__main__":
395         # Works both as `python -m scripts.gates.perf.run` and `python
scripts/gates/perf/run.py`
396         # (preflight is imported at module load via the try/except above).
397         raise SystemExit(main())
398
```

368. assistant (2026-07-09T04:47:11.955Z)

Now the coordinated gate-logic rewrite. First `build()` + `exit_code()` ? gate stitch only, treat a failed/missing HEAD gated stage as a regression, skip non-comparable stitch (segment counts differ), and surface a "nothing measurable ran" signal:

369. user (2026-07-09T04:47:37.571Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

370. assistant (2026-07-09T04:47:58.806Z)

The spawned GCSFuse task already landed as PR #548 (separate) ? good, my PR stays out of storage. Continuing the gate fixes ? `write_report` needs to handle failed-HEAD findings (None delta), the new `skipped` kind, and an exit-2 badge:

371. user (2026-07-09T04:48:02.797Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

372. user (2026-07-09T04:48:12.565Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

373. assistant (2026-07-09T04:48:36.702Z)

Now `main()` ? gate default to stitch-only, alternate prod/rc order per video (bias fix), thread the new `prod_observed`/`nothing_ran` through, and guard the baseline write:

374. user (2026-07-09T04:48:39.314Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

375. user (2026-07-09T04:48:43.711Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

376. user (2026-07-09T04:48:52.114Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

377. user (2026-07-09T04:49:02.927Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

378. user (2026-07-09T04:49:07.834Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

379. assistant (2026-07-09T04:49:25.723Z)

Now the module docstring ? fix the pre-move companion paths (N1) and the exit-code doc (drop the dead "3 reserved", clarify gating):

380. user (2026-07-09T04:49:33.227Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

381. assistant (2026-07-09T04:49:40.975Z)

Let me verify `run.py` compiles/lints after this batch of edits, and update the manifest to drop `total_s` from the gate:

382. user (2026-07-09T04:49:46.127Z)

```
COMPILE OK
All checks passed!
=== manifest gate block (drop total_s) ===
2:  "_doc": "Local release PERFORMANCE gate manifest (scripts/gates/perf/run.py). Times checksum/validation/segmentation/full-reel-stitch for the deployed production commit vs repo HEAD, on the local GPU, over vault clips. Companion to the QUALITY gates (scripts/gates/quality/nightly_regression.py, scripts/gates/quality/nightly_reelcount.py). Videos are referenced by BASENAME and resolved at runtime against --vault-dir (default: env RALLY_VAULT_DIR, else ~/Videos) so NO personal absolute path is committed (repo leak-scan guard). Prod commit is auto-resolved from the live Cloud Run image (digest->commit via prod_image_map until #532 git-sha-labels images), else the pinned prod_commit, else --prod-commit. Freeze precomputed release numbers: `python -m scripts.gates.perf.run --update-baseline`.",
19:    "stages": ["stitch_s", "total_s"],
20:    "tol_pct": 20.0,
21:    "_doc": "REGRESSION (exit 1) if HEAD is slower than production on any listed stage by more than tol_pct. checksum/validation/segmentation are reported but NOT gated (constant backdrop / infra-bound). Stitch is the locally-measurable code path (#531)."
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

383. assistant (2026-07-09T04:49:57.635Z)

run.py is clean. Now drop `total_s` from the manifest gate (Suggestion S2 ? I'd called it a non-gated backdrop yet gated it):

384. user (2026-07-09T04:50:01.377Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

385. user (2026-07-09T04:50:07.599Z)

```
18        "gate": {
19          "stages": ["stitch_s", "total_s"],
20          "tol_pct": 20.0,
21          "_doc": "REGRESSION (exit 1) if HEAD is slower than production on any listed stage
by more than tol_pct. checksum/validation/segmentation are reported but NOT gated (constant
backdrop / infra-bound). Stitch is the locally-measurable code path (#531)."
22        },
```

386. user (2026-07-09T04:50:14.982Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\eval_baselines\perf_manifest.json has been updated successfully. (file state is current in your context ? no need to Read it back)

387. assistant (2026-07-09T04:50:43.267Z)

Tasks 10-11 done. Now Linux support (task #14) ? the key pieces are OS-branching in `preflight.py` (native wasb env on Linux, WSL on Windows) and `detector_impl=native` on non-Windows. Let me read the current `preflight.py` and the harness config section.

388. user (2026-07-09T04:50:44.223Z)

```
1       #!/usr/bin/env python3
2       """Preflight environment checks for the local performance gate ? SELF-HELP oriented.
3
4       Every check reports (ok, detail, fix) so a failure tells you exactly what's missing and
how
5       to fix it, rather than blowing up mid-run. Required checks gate the run (exit 2 if any
fail);
6       optional checks (e.g. gcloud for prod auto-resolve) only warn.
7
8       Run standalone:   python -m scripts.gates.perf.preflight
9                         python scripts/gates/perf/run.py --preflight
10      """
11      import json
12      import os
13      import shutil
14      import subprocess
15      from dataclasses import dataclass
16
17
18      @dataclass
19      class Check:
20          name: str
21          required: bool
22          ok: bool
23          detail: str = ""
24          fix: str = ""
25
26
27      def _sh(cmd, timeout=30):
28          try:
29              p = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
30              return p.returncode, (p.stdout or "").strip(), (p.stderr or "").strip()
31          except Exception as e:
32              return 1, "", str(e)
33
34
```

```
    def _wsl(distro, script, timeout=60):
        return _sh(["wsl.exe", "-d", distro, "-e", "bash", "-lc", script], timeout=timeout)


    def run_checks(config: dict, want_prod_resolve: bool = False) -> list:
        checks = []
        wasb = (config.get("indexing", {}) or {}).get("wasb", {}) or {}
        distro = wasb.get("wsl_distro", "Ubuntu")
        conda_env = wasb.get("conda_env", "wasb")
        repo_dir = wasb.get("repo_dir", "~/models/WASB-SBDT")
        weights = wasb.get("weights_path", "")

        # 1) git + worktree support
        rc, out, _ = _sh(["git", "--version"])
        checks.append(Check("git", True, rc == 0, out or "not found",
                            "Install Git for Windows (https://git-scm.com)."))

        # 2) ffmpeg / ffprobe
        for tool in ("ffmpeg", "ffprobe"):
            present = shutil.which(tool) is not None
            checks.append(Check(tool, True, present, shutil.which(tool) or "not on PATH",
                                "Install FFmpeg and add it to PATH (README §External System
Requirements)."))

        # 3) torch + CUDA on the local GPU
        try:
            import torch  # noqa
            cuda = bool(torch.cuda.is_available())
            gpu = torch.cuda.get_device_name(0) if cuda else "(no CUDA device)"
            checks.append(Check("torch+cuda", True, cuda, f"torch {torch.__version__} ·
{gpu}",
                                "Install a CUDA build of torch (see gpu_setup.sh / README)
and verify the driver."))
        except Exception as e:
            checks.append(Check("torch+cuda", True, False, f"import failed: {e}",
                                "pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124"))

        # 4) WSL distro reachable
        rc, out, err = _wsl(distro, "echo ok", timeout=40)
        checks.append(Check(f"wsl:{distro}", True, rc == 0 and "ok" in out, (out or
err)[:80] or "unreachable",
                            f"Install/start WSL2 distro '{distro}' (wsl --install -d
{distro})."))

        # 5) WSL wasb conda env + repo + weights (the production shuttle detector)
        if rc == 0:
            env_sh = f"conda env list 2>/dev/null | grep -qiE '(^|/){conda_env}( |$|/)' &&
echo ENV_OK || (ls -d ~/miniconda3/envs/{conda_env} >/dev/null 2>&1 && echo ENV_OK || echo
ENV_MISSING)"
            _, eo, _ = _wsl(distro, env_sh, timeout=40)
            checks.append(Check(f"wsl conda env '{conda_env}'", True, "ENV_OK" in eo, eo or
"missing",
                                f"Create the WASB conda env '{conda_env}' in WSL (see docs;
WASB-SBDT requirements)."))
            _, ro, _ = _wsl(distro, f"test -d {repo_dir} && echo REPO_OK || echo
REPO_MISSING")
            checks.append(Check("wsl WASB-SBDT repo", True, "REPO_OK" in ro, repo_dir,
                                f"git clone the WASB-SBDT repo to {repo_dir} in WSL."))
            if weights:
                _, wo, _ = _wsl(distro, f"test -f {weights} && echo W_OK || echo W_MISSING")
                checks.append(Check("wsl WASB weights", True, "W_OK" in wo, weights,
                                    f"Place the pretrained badminton weights at {weights} in
WSL."))

```

```
88          # 6) gcloud (optional ? only for auto-resolving the live prod commit)
89          if want_prod_resolve:
90              present = shutil.which("gcloud") is not None
91              checks.append(Check("gcloud (prod auto-resolve)", False, present,
shutil.which("gcloud") or "not on PATH",
92                                  "Install gcloud OR pass --prod-commit <sha> explicitly (see
README §Prod resolution)."))
93          return checks
94
95
96      def print_checks(checks: list) -> int:
97          print("Preflight:")
98          fails = 0
99          for c in checks:
100             mark = "OK  " if c.ok else ("FAIL" if c.required else "warn")
101             tag = "" if c.required else " (optional)"
102             print(f"  [{mark}] {c.name}{tag}: {c.detail}")
103             if not c.ok:
104                 print(f"         fix: {c.fix}")
105                 if c.required:
106                     fails += 1
107         print(f"  => {len(checks) - fails}/{len(checks)} ok" + (f", {fails} REQUIRED
failing" if fails else ""))
108         return fails
109
110
111     def main(argv=None) -> int:
112         here = os.path.dirname(os.path.abspath(__file__))
113         cfg_path = os.path.join(here, "config", "bench_config.json")
114         cfg = json.load(open(cfg_path, encoding="utf-8")) if os.path.exists(cfg_path) else
{}
115         fails = print_checks(run_checks(cfg, want_prod_resolve=True))
116         return 2 if fails else 0
117
118
119     if __name__ == "__main__":
120         raise SystemExit(main())
121
```

389. assistant (2026-07-09T04:51:14.896Z)

Now let me make preflight OS-aware ? WSL checks on Windows, native wasb checks on Linux:

390. user (2026-07-09T04:51:18.449Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\preflight.py has been updated successfully. (file state is current in your context ? no need to Read it back)

391. user (2026-07-09T04:51:36.363Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\preflight.py has been updated successfully. (file state is current in your context ? no need to Read it back)

392. assistant (2026-07-09T04:51:44.301Z)

Now the harness must select `detector_impl=native` on Linux. Let me find its effective-config block.

393. user (2026-07-09T04:51:48.312Z)

```
=== harness.py effective-config block ===
20:  (wasb_hybrid, detector_impl=auto WSL-GPU, skip_ai_handoff=true, stream_video=true).
31:import sys
147:    # --- effective config: template + isolated output_dir, so runs never collide ---
149:        cfg_dict = json.load(f)
150:    cfg_dict.setdefault("output", {})["output_dir"] = args.run_dir
151:    eff_path = os.path.join(args.run_dir, "effective_config.json")
153:        json.dump(cfg_dict, f, indent=2)
=== stitch_ab.py same (does it need detector_impl? it only stitches ? no) ===
38:        cfg_dict = json.load(f)
39:    cfg_dict.setdefault("output", {})["output_dir"] = args.run_dir
40:    eff = os.path.join(args.run_dir, "effective_config.json")
42:        json.dump(cfg_dict, f, indent=2)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

394. assistant (2026-07-09T04:52:24.565Z)

Let me add the Linux `detector_impl=native` override + `random.seed(0)` determinism to the harness:

395. user (2026-07-09T04:52:27.146Z)

```
24      Output: a single JSON object (also written to --json-out) with per-stage seconds, the
25      segment count, reel facts, GPU utilisation peak, and any per-stage error.
26      """
27      import argparse
28      import json
29      import os
30      import subprocess
31      import sys
32      import threading
```

396. user (2026-07-09T04:52:28.548Z)

```
146
147         # --- effective config: template + isolated output_dir, so runs never collide ---
148         with open(args.config, "r", encoding="utf-8") as f:
149             cfg_dict = json.load(f)
150         cfg_dict.setdefault("output", {})["output_dir"] = args.run_dir
151         eff_path = os.path.join(args.run_dir, "effective_config.json")
152         with open(eff_path, "w", encoding="utf-8") as f:
153             json.dump(cfg_dict, f, indent=2)
154
```

397. user (2026-07-09T04:52:33.867Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\harness.py has been updated successfully. (file state is current in your context ? no need to Read it back)

398. user (2026-07-09T04:52:37.928Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\harness.py has been updated successfully. (file state is current in your context ? no need to Read it back)

399. assistant (2026-07-09T04:52:57.581Z)

Now the nightly wrapper ? remove the dead exit-3 branch (S1) and document that `LATEST_STATUS.txt` is the real signal, not Task Scheduler success (S6). Let me read the switch + docstring.

400. user (2026-07-09T04:52:59.246Z)

```
14          machine (a cloud runner has no GPU/WSL). It is a SEPARATE task (not folded into the
quality one)
15          because a GPU perf run far exceeds the quality leg's 30-minute limit.
16
17          GRACEFUL no-op (exit 0) when the tool isn't present yet OR the environment isn't
ready
18          (preflight -> the tool's exit 2): a registered task never errors in the interim and
self-activates
19          once the perf gate + WSL wasb env land. Inspect LATEST_STATUS.txt for the real
state.
20
21      .PARAMETER VideoSet
22          Video set: quick | full | stress. Default 'full' (3 clips; excludes the ~2h 5-min
stress clip).
23
```

401. user (2026-07-09T04:53:00.650Z)

```
72          1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
73              "[$date] *** PERF REGRESSION ? HEAD slower than production; see
$reportDir\$date.md ***" | Tee-Object -FilePath $log -Append }
74          2 { Write-Status 'SKIPPED' 0 'environment not ready (preflight) ? see log; self-
activates once WSL wasb + GPU are set up'
75              exit 0 }
76          3 { Write-Status 'NOBASELINE' 3 'no perf baseline ? run with -UpdateBaseline once' }
77          4 { Write-Status 'STALE' 4 "config/video-set changed vs baseline ? re-freeze with
-UpdateBaseline; see $reportDir\$date.md" }
78          default { Write-Status 'ERROR' $code "gate error ? see $log" }
79      }
80      exit $code
81
```

402. user (2026-07-09T04:53:30.709Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

403. user (2026-07-09T04:53:34.540Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

404. assistant (2026-07-09T04:53:45.169Z)

```
Now the perf README needs the most work (S4 pre-move paths, exit-3 in the table, gating
description, plus a Linux section and the re-register note). Let me read it.
```

405. user (2026-07-09T04:53:46.150Z)

```
1       # Local release performance gate (`scripts/gates/perf/`)
2
3       Standalone, repeatable, release-gating tool that compares the **binary currently serving
4       `api.khelsutra.guru`** against **repo HEAD**, on this machine's GPU, over vault clips ?
with
5       **no SPA and no user step**. It times the four release-critical stages independently:
6
7       ```
8       checksum  ?  validation  ?  segmentation (wasb_hybrid, GPU)  ?  full-reel stitch
```

```

It is the **performance sibling** of the quality gates
(`scripts/nightly_regression.py`, `scripts/nightly_reelcount.py`) and plugs into the same
nightly framework. Run it before every release, or any time, to get precomputed numbers and
catch a per-stage regression.

---

## Quickstart

```bash
# 0. One-time: check the environment is ready (self-help: names each missing prereq + its fix)
python scripts/gates/perf/run.py --preflight

# 1. Compare prod vs HEAD (quick profile: 2 clips) and gate the release
python scripts/gates/perf/run.py

# 2. More coverage (adds the 5.3K clip); or the full stress set (adds the 5-min clip, ~2h)
python scripts/gates/perf/run.py --profile full
python scripts/gates/perf/run.py --profile stress

# 3. Just the #531 stitch A/B (fast ? no segmentation)
python scripts/gates/perf/run.py --stitch-only

# 4. Freeze the current numbers as the committed release baseline
python scripts/gates/perf/run.py --update-baseline
```

Reports land in `output/nightly_perf/<date>.{md,json}` (+ `logs/<date>.log`). The gate's
exit code is the release signal.

What gets compared, and why it's fair

* **Two commits, one machine, identical inputs.** Each side runs from its own throwaway
  `git worktree` (your working tree is never touched), driven by **one canonical config**
  (`config/bench_config.json`), so only the *code* differs. "The prod binary" can't be run
  literally (it's a Linux/L4/NVENC container), so we run the **prod commit** through the same
  harness ? the only apples-to-apples local comparison.
* **Production detector on the GPU.** Segmentation uses `wasb_hybrid`, `detector_impl=auto`
  (the WSL2 GPU path ? the same WASB shuttle detector that serves prod), with the fast
  `stream_video` decode (bit-identical to the PNG path, present in both compared commits).
* **Deterministic & headless.** `skip_ai_handoff=true` (no Gemini / no user step); the
  entire reel is stitched (every rally, watermark on, `libx264` both sides). Validation
  is timed but never halts, so every clip yields all four numbers.
* **Real stitch paths.** The compiler is built by introspection, so the prod commit runs its
  native two-pass watermark stitch and HEAD runs its `single_pass_stitch` default (**#531**).

Gating (release signal)

Only **stitch** and **total** are gated (see `eval_baselines/perf_manifest.json ? gate`):
they're the locally-measurable code path. checksum/validation/segmentation are reported
```

```
but
**not** gated ? they're a constant backdrop (same detector) or infra-bound. A
**REGRESSION**
fires when HEAD is slower than production on a gated stage by more than `tol_pct`
(default
**20%**). The dedicated stitch A/B is always gated (it's the cleanest #531 signal).

Exit codes match the quality gates:

| code | meaning |
|---|---|
| 0 | PASS ? no regression |
| 1 | REGRESSION ? HEAD slower than prod beyond tolerance |
| 2 | nothing ran / preflight failed (env not ready) |
| 3 | no baseline (only with baseline gating) |
| 4 | STALE ? config/video-set changed vs the frozen baseline |

## How "production" is resolved

The deployed Cloud Run image is **not** git-sha-labeled yet (tracked in **#532**), so
the tool
resolves the prod commit in this order:

1. `--prod-commit <sha>` / `--prod-ref <ref>` if you pass one.
2. **Live Cloud Run**: reads the running `rally-control-plane` image digest and maps it
via
   `perf_manifest.json ? prod.image_digest_to_commit`.
3. Falls back to `perf_manifest.json ? prod.pinned_commit`.

**On each production deploy**, add the new `digest: commit` line to
`prod.image_digest_to_commit`
and bump `pinned_commit` (or land #532 so step 2 needs no map).

## Vault clips (no personal paths committed)

The manifest lists clips by **basename**; they're resolved at runtime against `--vault-
dir`
(default `$RALLY_VAULT_DIR`, else `~/Videos`). This keeps personal absolute paths out of
the
repo (the leak-scan CI guard). Point `--vault-dir` at wherever your clips live, or
override the
set with `--videos GX010016,SmokeTestBadmintonClip`.

## Nightly integration

* `scripts/nightly_perf.ps1` ? the Scheduled-Task wrapper (mirrors
`nightly_regression.ps1`):
  writes `output/nightly_perf/<date>.md` + `LATEST_STATUS.txt`, exits with the gate
code.
* `scripts/register_nightly_task.ps1` registers **both** legs by default ? quality
  (`KhelSutraNightlyRegression`, 03:00, 30-min cap) and perf (`KhelSutraNightlyPerf`,
03:30,
  3-hour cap). Perf is a *separate* task because a GPU run far exceeds the quality leg's
cap.

```powershell
pwsh -File scripts/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'   # both
legs
pwsh -File scripts/register_nightly_task.ps1 -PerfOnly                          # just
perf
Start-ScheduledTask -TaskName 'KhelSutraNightlyPerf'                            # run
now
```

Both are **local** tasks (not cloud): the GPU, WSL WASB detector, and vault clips live
```

```
on this
110     machine. The cloud nightly (`.github/workflows/nightly.yml`) stays offline Tier-1 (no
GPU/video).
111
112     ## Troubleshooting (self-help)
113
114     `--preflight` names each failing prerequisite and its fix. Common ones:
115
116     | symptom | fix |
117     |---|---|
118     | `torch+cuda FAIL` | Install a CUDA torch build; verify the NVIDIA driver (`nvidia-
smi`). |
119     | `wsl:Ubuntu FAIL` | `wsl --install -d Ubuntu`; start it once. |
120     | `wsl conda env 'wasb' FAIL` | Create the WASB conda env in WSL (WASB-SBDT
requirements). |
121     | `wsl WASB weights FAIL` | Place `wasb_badminton_best.pth.tar` at the configured
`weights_path`. |
122     | `gcloud ? warn` | Optional ? only for auto-resolving prod; pass `--prod-commit`
instead. |
123     | 5.3K clip very slow | Expected: local decode is CPU-bound (prod uses L4 NVDEC). Use
`--profile quick`/`full`, keep `stress` for occasional runs. |
124
125     ## Files
126
127     ```
128     scripts/gates/perf/
129       run.py              # entrypoint: resolve commits, preflight, worktrees, run, gate,
report
130       harness.py          # one (video,version) run ? per-stage JSON (subprocess, version-
agnostic)
131       stitch_ab.py        # dedicated #531 stitch micro-bench (subprocess)
132       preflight.py        # environment checks with self-help fixes
133       config/bench_config.json
134       README.md           # this file
135     eval_baselines/perf_manifest.json   # video set, config, gate policy, prod resolution
136     eval_baselines/perf_baseline.json   # frozen precomputed release numbers (--update-
baseline)
137     scripts/nightly_perf.ps1            # nightly wrapper
138     scripts/register_nightly_task.ps1   # registers quality + perf scheduled tasks
139     ```
140
```

406. assistant (2026-07-09T04:54:05.929Z)

This README needs many coordinated fixes (paths, gating, exit table, plus a Linux section) ? cleaner to rewrite it whole:

407. assistant (2026-07-09T04:54:50.801Z)

Local release performance gate (`scripts/gates/perf/`)

Standalone, repeatable, release-gating tool that compares the **binary currently serving `api.khelsutra.guru`** against **repo HEAD**, on this machine's GPU, over vault clips ? with **no SPA and no user step**. It times the four release-critical stages independently:

```
checksum  ?  validation  ?  segmentation (wasb_hybrid GPU)  ?  full-reel stitch
```

It is the **performance sibling** of the quality gates (`scripts/gates/quality/nightly_regression.py`, `scripts/gates/quality/nightly_reelcount.py`) and plugs into the same nightly framework. Run it before every release, or any time, to get precomputed numbers and catch a stitch regression.

Quickstart

```bash
# 0. One-time: check the environment is ready (self-help: names each missing prereq +
python scripts/gates/perf/run.py --preflight

# 1. Compare prod vs HEAD (quick profile: 2 clips) and gate the release
python scripts/gates/perf/run.py

# 2. More coverage (adds the 5.3K clip); or the full stress set (adds the 5-min clip, ~2h)
python scripts/gates/perf/run.py --profile full
python scripts/gates/perf/run.py --profile stress

# 3. Just the #531 stitch A/B (fast ? no segmentation)
python scripts/gates/perf/run.py --stitch-only

# 4. Freeze the current numbers as the local release baseline
python scripts/gates/perf/run.py --update-baseline
```

Reports land in `output/nightly_perf/<date>.{md,json}` (+ `logs/<date>.log`). The gate's exit code is the release signal.

What gets compared, and why it's fair

* **Two commits, one machine, identical inputs.** Each side runs from its own throwaway `git worktree` (your working tree is never touched), driven by **one canonical config** (`config/bench_config.json`), so only the *code* differs. "The prod binary" can't be run literally (it's a Linux/L4/NVENC container), so we run the **prod commit** through the same harness ? the only apples-to-apples local comparison.
* **Production detector on the GPU.** Segmentation uses `wasb_hybrid` ? the same WASB shuttle detector that serves prod ? via WSL2 on Windows (`detector_impl=auto`) or in-process on Linux (`detector_impl=native`, auto-selected; see [Running on Linux](#running-on-linux)), with the fast `stream_video` decode (bit-identical to the PNG path, present in both compared commits).
* **Deterministic & headless.** `skip_ai_handoff=true` (no Gemini / no user step), `random.seed(0)`; the **entire reel** is stitched (every rally, watermark on, `libx264` both sides). Validation is timed but never halts, so every clip yields all four numbers.
* **Real stitch paths.** The compiler is built by introspection, so the prod commit runs its native two-pass watermark stitch and HEAD runs its `single_pass_stitch` default (**#531**).

Gating (release signal)

Only **stitch** is gated (see `eval_baselines/perf_manifest.json ? gate`) ? it's the locally-measurable code path (#531). checksum/validation/segmentation **and `total_s`** are reported but **not** gated: segmentation is GPU/WSL-noisy and dominates total, so gating total would fail the release on backdrop noise rather than the stitch signal. A **REGRESSION** (exit 1) fires when, on a gated stage, HEAD is slower than production beyond `tol_pct` (default **20%**) **or** HEAD produces no value where prod did (crash / timeout / 0 segments ? a broken HEAD must not pass). Two guards keep it honest:

* the **fixed-pair stitch A/B** is always gated (identical reels both sides ? the clean signal);
* the **full-matrix `stitch_s`** is only gated when prod/rc **segment counts match** (otherwise the reels differ and the times aren't apples-to-apples ? it's reported as *not comparable*).

If nothing measurable ran (no videos found, all stitch sources missing, all runs errored) the gate returns **exit 2** rather than a silent PASS. To reduce first-vs-second-run bias on a

shared
box, prod/rc run order **alternates per clip** (logged).

Exit codes match the quality gates:

code	meaning
0	PASS ? no regression
1	REGRESSION ? HEAD slower than prod beyond tolerance, or HEAD failed a gated stage
2	nothing ran / preflight failed (env not ready)
4	STALE ? prod commit / video-set changed vs the frozen baseline

(Perf gates live prod-vs-HEAD and need no committed baseline, so there is no exit 3.)

How "production" is resolved

The deployed Cloud Run image is **not** git-sha-labeled yet (tracked in **#532**), so the tool resolves the prod commit in this order:

1. `--prod-commit <sha>` / `--prod-ref <ref>` if you pass one.
2. **Live Cloud Run**: reads the running `rally-control-plane` image digest and maps it via `perf_manifest.json ? prod.image_digest_to_commit`.
3. Falls back to `perf_manifest.json ? prod.pinned_commit`.

On each production deploy, add the new `digest: commit` line to `prod.image_digest_to_commit` and bump `pinned_commit` (or land #532 so step 2 needs no map).

Vault clips (no personal paths committed)

The manifest lists clips by **basename**; they're resolved at runtime against `--vault-dir` (default `$RALLY_VAULT_DIR`, else `~/Videos`). This keeps personal absolute paths out of the repo (the leak-scan CI guard). Point `--vault-dir` at wherever your clips live, or override the set with `--videos GX010016,SmokeTestBadmintonClip`.

Running on Linux

The tool **auto-adapts by OS**. On Linux it runs wasb in-process (`detector_impl=native` ? the same path as the GCP GPU-VM nightly `nightly_reelcount.py`) instead of via WSL, and `--preflight` verifies the **native** env instead of a WSL distro:

* WASB-SBDT repo ? `$WASB_REPO_DIR` or `indexing.wasb.repo_dir` (default `~/models/WASB-SBDT`)
* weights ? `$WASB_WEIGHTS` or `indexing.wasb.weights_path`
* optional `$WASB_PYTHON` ? the wasb conda-env python
  (see `deploy/gcp/nightly_regression/run_on_vm.sh`, which exports all three)

`gcloud` prod-resolve and `git worktree` work identically; the Windows `.CMD`/`core.longpaths` handling is a no-op on Linux. The `.ps1` wrappers are Windows Scheduled-Task glue ? on Linux, wire `python scripts/gates/perf/run.py` into cron / systemd / a CI GPU runner (its exit code is the gate).

Nightly integration (Windows)

* `scripts/gates/nightly_perf.ps1` ? the Scheduled-Task wrapper (mirrors `scripts/gates/nightly_regression.ps1`): writes `output/nightly_perf/<date>.md` + `LATEST_STATUS.txt`, exits with the gate code.
* `scripts/gates/register_nightly_task.ps1` registers **both** legs by default ? quality (`KhelSutraNightlyRegression`, 03:00, 30-min cap) and perf (`KhelSutraNightlyPerf`, 03:30, 3-hour cap). Perf is a *separate* task because a GPU run far exceeds the quality leg's cap.

```powershell
pwsh -File scripts/gates/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'   # both legs
```

```
pwsh -File scripts/gates/register_nightly_task.ps1 -PerfOnly                          # just
perf
Start-ScheduledTask -TaskName 'KhelSutraNightlyPerf'                                  # run now
```

> ? **Re-register after this merges.** Any Scheduled Task registered before the move encodes the
> old `scripts\nightly_*.ps1` path and will silently `SKIP` (wrapper-absent guard) until you re-
run
> `register_nightly_task.ps1` from the new location.
>
> ? **Task Scheduler "success" is not the gate signal.** A lasting env break still exits 0
SKIPPED
> so the task never red-lights ? watch `output/nightly_perf/LATEST_STATUS.txt` (or the report),
not
> the task result.

Both are **local** tasks (not cloud): the GPU, WASB detector, and vault clips live on this
machine. The cloud nightly (`.github/workflows/nightly.yml`) stays offline Tier-1 (no
GPU/video).

Troubleshooting (self-help)

`--preflight` names each failing prerequisite and its fix. Common ones:

symptom	fix
`torch+cuda FAIL`	Install a CUDA torch build; verify the NVIDIA driver (`nvidia-smi`).
`wsl:Ubuntu FAIL` (Windows)	`wsl --install -d Ubuntu`; start it once.
`wsl conda env 'wasb' FAIL` (Windows)	Create the WASB conda env in WSL (WASB-SBDT
requirements).
`WASB-SBDT repo (native) FAIL` (Linux)	Clone WASB-SBDT; set `WASB_REPO_DIR` (or
`indexing.wasb.repo_dir`).
`WASB weights (native) FAIL` (Linux)	Place `wasb_badminton_best.pth.tar`; set `WASB_WEIGHTS`
(or the config path).
`gcloud ? warn`	Optional ? only for auto-resolving prod; pass `--prod-commit` instead.
5.3K clip very slow	Expected: local decode is CPU-bound (prod uses L4 NVDEC). Use `--profile
quick`/`full`, keep `stress` for occasional runs.

Files

```
scripts/gates/perf/
  run.py              # entrypoint: resolve commits, preflight, worktrees, run, gate, report
  harness.py          # one (video,version) run ? per-stage JSON (subprocess, version-agnostic)
  stitch_ab.py        # dedicated #531 stitch micro-bench (subprocess)
  preflight.py        # environment checks with self-help fixes (OS-aware)
  config/bench_config.json
  README.md           # this file
eval_baselines/perf_manifest.json   # video set, config, gate policy, prod resolution
eval_baselines/perf_baseline.json   # local frozen numbers (--update-baseline; machine-specific,
not committed)
scripts/gates/nightly_perf.ps1            # nightly wrapper
scripts/gates/register_nightly_task.ps1   # registers quality + perf scheduled tasks
tests/test_perf_gate.py                   # offline unit tests for the gate core
```

408. user (2026-07-09T04:54:50.857Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\README.md has been updated successfully. (file state is current in your
context ? no need to Read it back)

409. assistant (2026-07-09T04:55:06.277Z)

Now `gates/README.md` ? clarify exit-3 is quality-only (perf uses 0/1/2/4) and add the re-register note. Let me read it.

410. user (2026-07-09T04:55:08.271Z)

```
28        perf/
29          run.py harness.py stitch_ab.py preflight.py  config/bench_config.json  README.md
30      ```
31
32      ## Exit-code family (shared)
33
34      `0` PASS · `1` REGRESSION · `2` nothing-ran / preflight failed · `3` no baseline · `4` STALE baseline.
35
36      ## Quality leg
37
38      - **`quality/nightly_regression.py`** ? re-scores the cached WASB trajectories on CPU (seconds)
39        vs `eval_baselines/nightly_baseline.json`. Wrapper: `nightly_regression.ps1` (local Scheduled
```

411. user (2026-07-09T04:55:14.292Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)
```

412. user (2026-07-09T04:55:20.068Z)

```
64      ```
65
66      ## Nightly (both legs, one command)
67
68      ```powershell
69      # Registers KhelSutraNightlyRegression (03:00, 30-min cap) + KhelSutraNightlyPerf (03:30, 3-hr cap)
70      pwsh -File scripts/gates/register_nightly_task.ps1 -RepoRoot 'C:\path\to\checkout'
71      pwsh -File scripts/gates/register_nightly_task.ps1 -Unregister     # remove both
72      ```
73
74      Each wrapper writes `output/nightly_<leg>/<date>.{md,log}` + a `LATEST_STATUS.txt` marker and
75      exits with the gate's code, so Task Scheduler records PASS / REGRESSION / STALE. Both are
76      graceful no-ops until their prerequisites land, so registering before merge is safe.
77
```

413. user (2026-07-09T04:55:34.705Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)
```

414. assistant (2026-07-09T04:56:23.043Z)

Now the unit tests (S3) ? `tests/test_perf_gate.py`, loading `run.py` by path (offline, like `test_nightly_regression.py`), covering the gate core including the bugs the review caught:

415. assistant (2026-07-09T04:56:45.027Z)

```
"""Offline unit tests for the perf-gate core (scripts/gates/perf/run.py).

Fully offline: no GPU, no git, no video, no torch ? the gate logic (pct / build / exit_code)
operates on plain dicts, so regressions, failed-HEAD, segment-count mismatch, nothing-ran, and
```

```
the not-gated stages are all exercised deterministically. The full harness run is owner-verified
on a GPU box. Mirrors the by-path module load used by test_nightly_regression.py (scripts/ is a
namespace package, and run.py's relative import falls back to a sys.path import when loaded by
file path).
"""

import importlib.util
import os
import sys

_SCRIPT = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
    "scripts", "gates", "perf", "run.py",
)
_spec = importlib.util.spec_from_file_location("perf_run", _SCRIPT)
run = importlib.util.module_from_spec(_spec)
sys.modules["perf_run"] = run
_spec.loader.exec_module(run)

GATE = {"stages": ["stitch_s"], "tol_pct": 20.0}


def _res(version, video, stages, segments=1, errors=None):
    total = sum(v for v in stages.values() if isinstance(v, (int, float)))
    return {"version": version, "video": video, "stages": stages, "total_s": total,
            "segments": segments, "errors": errors or {}}


# ---------- pct

def test_pct_edges():
    assert run.pct(10, 12) == 20.0
    assert run.pct(10, 8) == -20.0
    assert run.pct(0, 5) is None       # prod == 0 -> undefined
    assert run.pct(None, 5) is None
    assert run.pct(5, None) is None


# ---------- build / exit_code

def test_regression_beyond_tol():
    results = [_res("prod", "v", {"stitch_s": 10.0}), _res("rc", "v", {"stitch_s": 13.0})]
    _, findings, observed = run.build(results, {}, ["v"], {}, GATE)
    assert observed == 1
    assert any(f["kind"] == "regression" and f["where"] == "v/stitch_s" for f in findings)
    assert run.exit_code(findings, stale=False) == 1


def test_improvement_is_not_regression():
    results = [_res("prod", "v", {"stitch_s": 10.0}), _res("rc", "v", {"stitch_s": 7.0})]
    _, findings, _ = run.build(results, {}, ["v"], {}, GATE)
    assert any(f["kind"] == "improvement" for f in findings)
    assert run.exit_code(findings, stale=False) == 0


def test_within_tol_passes():
    results = [_res("prod", "v", {"stitch_s": 10.0}), _res("rc", "v", {"stitch_s": 11.0})]
    _, findings, _ = run.build(results, {}, ["v"], {}, GATE)
    assert findings == []
    assert run.exit_code(findings, stale=False) == 0


def test_failed_head_is_regression():
    # prod produced the gated stage, HEAD did not (crash / 0 segments) -> regression, not PASS.
    results = [_res("prod", "v", {"stitch_s": 10.0}),
```

```
                _res("rc", "v", {"stitch_s": None}, segments=0, errors={"segmentation": "boom"})]
    _, findings, observed = run.build(results, {}, ["v"], {}, GATE)
    assert observed == 1
    assert any(f["kind"] == "regression" and "no value" in f.get("note", "") for f in findings)
    assert run.exit_code(findings, stale=False) == 1


def test_segment_count_mismatch_not_gated():
    # HEAD is 3x slower BUT the reels differ (5 vs 6 rallies) -> skipped, not a regression.
    results = [_res("prod", "v", {"stitch_s": 10.0}, segments=5),
               _res("rc", "v", {"stitch_s": 30.0}, segments=6)]
    _, findings, _ = run.build(results, {}, ["v"], {}, GATE)
    assert any(f["kind"] == "skipped" for f in findings)
    assert not any(f["kind"] == "regression" for f in findings)
    assert run.exit_code(findings, stale=False) == 0


def test_total_s_not_gated():
    # HEAD total 3x slower (segmentation noise) but stitch flat -> NOT a regression.
    results = [_res("prod", "v", {"stitch_s": 10.0, "segmentation_s": 10.0}),
               _res("rc", "v", {"stitch_s": 10.0, "segmentation_s": 40.0})]
    _, findings, _ = run.build(results, {}, ["v"], {}, GATE)
    assert not any(f["kind"] == "regression" for f in findings)
    assert run.exit_code(findings, stale=False) == 0


def test_stitch_ab_is_gated():
    stitch = {"1080p": {"prod": {"median_s": 10.0}, "rc": {"median_s": 13.0}}}
    _, findings, observed = run.build([], stitch, [], {}, GATE)
    assert observed == 1
    assert any(f["where"] == "stitch_ab/1080p" and f["kind"] == "regression" for f in findings)


def test_nothing_ran_exit_2():
    # no matrix, no stitch A/B, no prod-side observation -> exit 2 (never a silent PASS).
    _, findings, observed = run.build([], {}, [], {}, GATE)
    assert observed == 0
    assert run.exit_code(findings, stale=False, nothing_ran=(observed == 0)) == 2


def test_prod_without_head_data_is_not_observed():
    # prod itself produced nothing for the gated stage -> can't judge HEAD -> not observed.
    results = [_res("prod", "v", {"stitch_s": None}), _res("rc", "v", {"stitch_s": 12.0})]
    _, findings, observed = run.build(results, {}, ["v"], {}, GATE)
    assert observed == 0
    assert not any(f["kind"] == "regression" for f in findings)


def test_exit_code_precedence():
    reg = [{"kind": "regression", "where": "x", "delta_pct": 30}]
    assert run.exit_code([], stale=False) == 0
    assert run.exit_code([], stale=True) == 4
    assert run.exit_code(reg, stale=True) == 1              # regression beats stale
    assert run.exit_code(reg, stale=True, nothing_ran=True) == 2  # nothing-ran beats all
```

416. user (2026-07-09T04:56:45.079Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\tests\test_perf_gate.py (file state is current in your context ? no need to Read it back)

417. assistant (2026-07-09T04:56:54.989Z)

All fixes are in. Let me verify: run the new + moved tests, lint/compile the package, and check preflight on **both** Windows and Linux (WSL) to confirm the OS branch works.

418. user (2026-07-09T04:57:12.442Z)

```
=== py_compile + ruff ===
COMPILE OK
All checks passed!
=== new + moved tests ===
............................................................           [100%]
62 passed in 2.95s
=== preflight on WINDOWS (should show WSL checks) ===
  [OK  ] wsl:Ubuntu: ok
  [OK  ] wsl conda env 'wasb': ENV_OK
  [OK  ] wsl WASB-SBDT repo: ~/models/WASB-SBDT
  [OK  ] wsl WASB weights: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  => 9/9 ok
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

419. assistant (2026-07-09T04:57:38.246Z)

62 tests pass (11 new + 51 moved), lint/compile clean, and Windows preflight still 9/9 with WSL checks. Now the key Linux verification ? run preflight **inside WSL** to confirm the native branch fires (native wasb checks, not WSL):

420. user (2026-07-09T04:57:44.215Z)

```
=== preflight in LINUX (WSL) ? expect NATIVE checks, not wsl: ===
  [OK  ] git: git version 2.43.0
  [FAIL] torch+cuda: import failed: No module named 'torch'
         fix: pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
  [OK  ] WASB-SBDT repo (native): /home/avidullu/models/WASB-SBDT
  [OK  ] WASB weights (native): /home/avidullu/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  [warn] WASB_PYTHON (native) (optional): unset (native runner will use the current interpreter)
         fix: Point WASB_PYTHON at the wasb conda-env python (see deploy/.../gpu_setup.sh).
  => 7/8 ok, 1 REQUIRED failing

=== ruff check . (whole repo, CI-pinned 0.15.16) ===
All checks passed!
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

421. assistant (2026-07-09T04:58:29.576Z)

Linux verified ? the native branch fires correctly in WSL: `WASB-SBDT repo (native)` + `WASB weights (native)` (found on the Linux filesystem), not the WSL checks. (`torch FAIL` is just the WSL base python lacking torch ? the env, not the tool.) And **`ruff check .` (pinned 0.15.16) passes repo-wide.**

All 14 review items + Linux support are done and verified. Let me mark tasks complete, run the full suite in the background (rigor), and stage the changes.

422. user (2026-07-09T04:58:30.131Z)

```
Updated task #10 status
```

423. user (2026-07-09T04:58:30.746Z)

```
Updated task #11 status
```

424. user (2026-07-09T04:58:31.170Z)

```
Updated task #12 status
```

425. user (2026-07-09T04:58:32.106Z)

```
Updated task #13 status
```

426. user (2026-07-09T04:58:32.134Z)

```
Updated task #14 status
```

427. user (2026-07-09T04:58:40.031Z)

```
Command running in background with ID: by8ozdg1k. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\by8ozdg1k.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

428. assistant (2026-07-09T04:58:47.022Z)

```
Full suite running. Let me stage the review-fix changes (explicit paths) while it runs.
```

429. user (2026-07-09T04:58:52.497Z)

```
=== changed since last commit (should be gate files + manifest + new test) ===
 M eval_baselines/perf_manifest.json
 M scripts/gates/README.md
 M scripts/gates/nightly_perf.ps1
 M scripts/gates/nightly_regression.ps1
 M scripts/gates/perf/README.md
 M scripts/gates/perf/harness.py
 M scripts/gates/perf/preflight.py
 M scripts/gates/perf/run.py
 M scripts/gates/register_nightly_task.ps1
?? tests/test_perf_gate.py
=== stage explicit paths ===
=== staged ===
 eval_baselines/perf_manifest.json       |   4 +-
 scripts/gates/README.md                 |   8 ++-
 scripts/gates/nightly_perf.ps1          |  11 +--
 scripts/gates/nightly_regression.ps1    |   2 +-
 scripts/gates/perf/README.md            | 105 ++++++++++++++++++---------
 scripts/gates/perf/harness.py           |   6 ++
 scripts/gates/perf/preflight.py         |  52 +++++++++-----
 scripts/gates/perf/run.py               | 112 ++++++++++++++++++++--------
 scripts/gates/register_nightly_task.ps1 |   2 +-
 tests/test_perf_gate.py                 | 121 ++++++++++++++++++++++++++++++++
 10 files changed, 335 insertions(+), 88 deletions(-)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

430. assistant (2026-07-09T04:59:23.196Z)

```
Staged cleanly ? 10 files, +335/?88, exactly the intended change set. The full suite is running;
here's what I addressed while it finishes.
```

Review addressed (all 4 bugs, 7 suggestions, 2 nits) + Linux

```
**Bugs**
- **`$RepoRoot` depth** ? all 3 wrappers now walk up **3** dirs (were 2 ? resolved to
`scripts/`, so tasks armed with `scripts\scripts\gates\?` and silently SKIPPED).
- **Broken HEAD passed** ? a gated stage prod produced but HEAD didn't (crash/timeout/0
segments) is now a **REGRESSION**.
- **Silent PASS on empty run** ? no prod-side observation ? **exit 2** (not 0).
- (4th = the `$RepoRoot` in `nightly_perf.ps1`, same fix.)

**Suggestions**
- **Stopped gating `total_s`** (segmentation noise dominates) ? gate **stitch only**; full-
matrix `stitch_s` gated only when prod/rc **segment counts match** (else "not comparable").
- **Dropped dead exit-3** from the wrapper + shared tables (perf = 0/1/2/4; 3 is quality-only).
- **Added `tests/test_perf_gate.py`** (11 tests: pct edges, regression/improvement, failed-HEAD,
nothing-ran, segment-mismatch, total-not-gated, precedence).
```

- **Fixed pre-move paths** in `perf/README.md`; documented **re-register-after-merge** +
`LATEST_STATUS.txt` is the signal (not Task Scheduler success).
- **Alternate prod/rc order per clip** (bias fix).

Nits ? `run.py` docstring companion paths ? `scripts/gates/quality/*`; `random.seed(0)` in
the harness.

Linux support ? preflight is OS-aware (WSL on Windows, native
`WASB_REPO_DIR`/`WASB_WEIGHTS`/`WASB_PYTHON` on Linux); harness picks `detector_impl=native` off
Windows. **Verified in WSL** (native checks fire), and `ruff check .` (pinned 0.15.16) is clean
repo-wide; 62 targeted tests pass.

Once the full suite confirms green, I'll commit, push, and post a finding-by-finding reply on
the PR.

431. user (2026-07-09T05:05:19.696Z)

```
=== full suite result ===
=== full suite (background rigor) ===
Required test coverage of 70% reached. Total coverage: 84.28%
========================== short test summary info ==========================
FAILED tests/test_storage.py::test_gcs_fuse_helpers - AssertionError: assert ...
FAILED tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available
2 failed, 1944 passed, 8 skipped, 57 warnings in 392.22s (0:06:32)
============================================================================
 RESULT: FAIL - pytest exit 1
============================================================================
CIGATE2_DONE
```

432. assistant (2026-07-09T05:05:34.623Z)

Full suite: **1944 passed** (+11 from my new gate tests), coverage 84.28% ? the only 2 failures
are the same **pre-existing Windows-only** `test_storage` GCSFuse ones (fixed separately in PR
#548). My changes introduce **zero new failures**. Committing and pushing.

433. user (2026-07-09T05:05:50.835Z)

```
branch: feat/perf-release-gate  HEAD: 0aa6d4b
[feat/perf-release-gate 8030dac] fix(perf-gate): address review (4 bugs + 7 suggestions + 2
nits) + Linux support
 10 files changed, 335 insertions(+), 88 deletions(-)
 create mode 100644 tests/test_perf_gate.py
=== origin/master..HEAD (must be ONLY my commits) ===
8030dac fix(perf-gate): address review (4 bugs + 7 suggestions + 2 nits) + Linux support
0aa6d4b feat(gates): local release perf gate + consolidate nightly into scripts/gates/
=== push ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   0aa6d4b..8030dac  feat/perf-release-gate -> feat/perf-release-gate
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

434. assistant (2026-07-09T05:06:19.320Z)

Committed as **`8030dac`** and pushed; `origin/master..HEAD` is exactly my two commits, no
siblings. Now the finding-by-finding reply on the PR (the review is a pending draft, so I'll
post it as a PR comment mapping each item to its fix).

435. user (2026-07-09T05:06:32.970Z)

```
failed to run git: fatal: not a git repository (or any of the parent directories): .git
```

436. assistant (2026-07-09T05:06:45.154Z)

The `gh` call ran from the stub dir (cwd reset). Let me re-run it with an explicit `--repo` (the
body file is already written):

437. user (2026-07-09T05:06:50.103Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/547#issuecomment-4921750046

438. assistant (2026-07-09T05:07:15.942Z)

PR reply posted: [#547 comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/547#issuecomment-4921750046). Let me update the project memory to reflect the changed gate semantics (future sessions will otherwise assume the old behavior), then wrap up.

439. user (2026-07-09T05:07:16.973Z)

9
10 **PR [#547](https://github.com/Khelsutra/badminton-highlight-indexer/pull/547)** (branch `feat/perf-release-gate`, open 2026-07-09) adds a **local release performance gate** and consolidates the scattered nightly CLIs into a cohesive **`scripts/gates/`** package.
11
12 **What it is:** `scripts/gates/perf/run.py` times checksum ? validation ? segmentation (`wasb_hybrid` GPU via WSL) ? **full-reel stitch** for the commit deployed on `api.khelsutra.guru` **vs repo HEAD**, on the local GPU, no SPA/no user step, and GATES the release (exit 1 on a stitch/total regression > tol; same 0/1/2/4 exit family as the quality gates). Run: `python scripts/gates/perf/run.py [--preflight | --stitch-only | --profile quick|full|stress | --update-baseline]`.
13
14 **Reorg (honors `docs/CODE_MAP.md` "CLIs live in scripts/"):** `nightly_regression.py`/`nightly_reelcount.py`/`nightly_email.py` ? `scripts/gates/quality/`; `nightly_regression.ps1` + `register_nightly_task.ps1` ? `scripts/gates/`; new `nightly_perf.ps1`. `register_nightly_task.ps1` now installs BOTH `KhelSutraNightlyRegression` (quality, 30-min cap) + `KhelSutraNightlyPerf` (perf, 3-hr cap). ALL refs updated incl. the deployed GCP nightly `deploy/gcp/nightly_regression/run_on_vm.sh` (`from scripts.gates.quality.nightly_email import` + `python scripts/gates/quality/nightly_reelcount.py`). `scripts/` is a namespace package (tests load by file path).

440. user (2026-07-09T05:07:35.259Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\perf-release-gate.md has been updated successfully. (file state is current in your context ? no need to Read it back)

441. assistant (2026-07-09T05:08:00.809Z)

Done. Both of your asks are landed on **PR #547** in commit **`8030dac`** (pushed), with a finding-by-finding reply posted.

Review ? all 4 bugs, 7 suggestions, 2 nits fixed

Correctness (the important ones):
- **PowerShell `$RepoRoot`** now walks up 3 dirs in all three wrappers ? registered tasks were silently SKIPPING with `scripts\scripts\gates\?` paths.
- **A broken HEAD is now a REGRESSION** (a gated stage prod produced but HEAD didn't ? exit 1, not a quiet pass).
- **Nothing-measurable-ran ? exit 2** (no more silent PASS on an empty/misconfigured run).
- **Gate is stitch-only** ? I'd contradicted myself by gating `total_s` (segmentation noise); now `total_s` is report-only, and full-matrix `stitch_s` is gated only when prod/rc **segment counts match** (fixed-pair A/B is the clean signal).
- Dead exit-3 removed, prod/rc **order alternates per clip** (bias fix), `random.seed(0)`, docstring/README paths corrected, re-register + "LATEST_STATUS is the signal" documented, and **`tests/test_perf_gate.py`** (11 offline tests) locks it all in.

Linux support ? added & verified

`preflight` is OS-aware (WSL on Windows; native `WASB_REPO_DIR`/`WASB_WEIGHTS`/`WASB_PYTHON` on Linux), and the harness runs `detector_impl=native` off Windows ? the same in-process path as

the GCP GPU-VM nightly. **Verified in WSL** (native checks fire); the Windows `.CMD`/`MAX_PATH` handling is a no-op on Linux. New README **§Running on Linux**.

Verified

- `ruff` 0.15.16 clean repo-wide · `mypy` unaffected · full suite **1944 passed / coverage 84.28%** · preflight **9/9 on Windows** + **native branch on Linux**.
- Only failures are the 2 **pre-existing Windows-only** `test_storage` GCSFuse ones ? unrelated, and already fixed in **PR #548** (from the chip you started). CI will re-run on this push and should be green on Linux as before.

Your working tree is still untouched; the branch worktree is left in place in case the (now-submittable) review turns up anything else.

442. user (2026-07-09T05:14:02.834Z)

<system-reminder>
The separate session for background task task_2e0a3b82 ("Fix Windows path-separator break in GCSFuse helper") has ended.
</system-reminder>

can you fix the suggestions in the PR response and merge this

443. assistant (2026-07-09T05:14:24.109Z)

Let me pull the current PR state ? any new/submitted review comments, CI status, and mergeability ? before doing anything.

444. user (2026-07-09T05:14:32.037Z)

=== reviews (state + body, newest first) ===
[PENDING] avidullu @ PENDING

Summary

Second-pass review of PR #547 (`8030dac`) confirms the author fixed the prior findings correctly: 3-hop `$RepoRoot` in all three `.ps1` wrappers/register scripts, REGRESSION when prod has a gated stage and HEAD does not, exit 2 when `prod_observed == 0`, stitch-only gating with segment-count skip, dropped exit-3, unit tests, path/docs updates, re-register + Task Scheduler caveats, alternate prod/rc order (matrix + stitch A/B), `random.seed(0)`, and OS-aware Linux preflight/`detector_impl=native`. Dominant residual risk is a small incomplete edge of the failed-HEAD fix (exception path still records a numeric `stitch_s`), plus a couple of doc/status-message nits from the Linux + gating retune. No blockers for merge from this pass; residual items are small follow-ups.

Prior review status

All prior findings (4 bugs + 7 suggestions + 2 nits) verified fixed in `8030dac`.

Issue counts by severity (this pass)

- bugs: 0
- suggestions: 2
- nits: 2

=== SUBMITTED inline review comments (not pending drafts) ===

=== issue-level comments since my push ===
(avidullu @ 2026-07-09T05:06:51Z) Thanks for the thorough review ? all **4 bugs, 7 suggestions, 2 nits** addressed in `8030dac`, plus the requested **Linux support**.

Bugs

finding	fix
ps1 default `$RepoRoot` wal

445. assistant (2026-07-09T05:14:47.167Z)

A **second-pass review** (pending): **0 bugs, 2 suggestions, 2 nits**, and explicitly *"No blockers for merge."* The inline comments are pending drafts again ? let me pull them by review ID to see the exact 4 items.

446. user (2026-07-09T05:14:52.871Z)

pending review id: 4660014889

--- scripts/gates/perf/harness.py:null ---
[suggestion] When full-matrix stitch raises, the harness still records a numeric `stages["stitch_s"]` (wall time until exception) and only puts the failure in `errors["stitch"]`. `build()` treats any numeric HEAD value as a successful measurement, so a fast crash can look like a large "improvement" rather than a failed gated stage. Default stitch A/B usually still catches a broken stitch path (exception ? no `result.json` ? no median ? REGRESSION), so this is mostly a hole for `--no-stitch-ab` / matrix-only runs and for confusing dual signals in the report. The prior "failed HEAD is REGRESSION" fix only covers `None`/missing stage values, not timed exceptions.

Suggestion: On stitch exception set `stages["stitch_s"] = None` (keep the error), and/or in `build()` treat presence of `errors["stitch"]` (or missing reel output) as "HEAD produced no value". Add a unit test: prod `stitch_s=10`, rc `stitch_s=0.1` with `errors={"stitch": "..."}` ? regression.

--- scripts/gates/nightly_perf.ps1:null ---
[suggestion] Exit code 2 from `run.py` now means both preflight failure *and* "nothing measurable ran" (empty vault / all stitch sources missing / prod-side no gated observations). The wrapper always writes `SKIPPED` with detail `environment not ready (preflight)` and exits 0. Operators watching `LATEST_STATUS.txt` will mis-diagnose a missing vault/profile as a WASB preflight problem.

Suggestion: Branch the status text on log content or pass a richer signal (e.g. distinct exit codes, or parse the last log line). At minimum, change the detail to something like `exit 2 (preflight failed or nothing measurable ran) ? see log`.

--- scripts/gates/nightly_perf.ps1:null ---
[nit] Description still says the gate fires on a "stitch/total regression", but `total_s` was correctly dropped from `gate.stages` (manifest + `build()` only gate `stitch_s`).

Suggestion: Replace with "stitch regression" (or "stitch / stitch-A/B regression beyond tolerance").

--- scripts/gates/perf/run.py:null ---
[nit] The markdown report hardcodes `segmenter: wasb_hybrid (WSL2 GPU)` even though Linux now runs `detector_impl=native` in-process. Misleading after the new Linux path.

Suggestion: Pick the label from `sys.platform` (or record detector_impl in `meta` during the run) so the report says WSL2 GPU on Windows and native GPU on Linux.

447. assistant (2026-07-09T05:15:29.628Z)

All 4 residual items are legitimate. The most important is the incomplete failed-HEAD fix: a stitch that **throws** still records a numeric `stitch_s` (wall time to crash), which `build()` can read as a huge "improvement." Let me fix that at both layers plus the 3 smaller items. First, the harness stitch block.

448. user (2026-07-09T05:15:36.398Z)

=== harness stitch stage (except records numeric stitch_s?) ===

```
65:        except Exception:
69:    except Exception as e:  # pragma: no cover - best effort
100:            except Exception:
170:        except Exception:
173:    except Exception:
182:    except Exception:
196:    except Exception:
204:    except Exception:
217:        except Exception:
240:        except Exception:
249:        except Exception:
253:        except Exception:
270:        except Exception:
276:        except Exception:
284:            rec["stages"]["stitch_s"] = None
303:                out_path = compiler.compile_highlights(args.video, time_pairs,
"highlights.mp4")
304:                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
311:            except Exception:
312:                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
322:    for k in ("checksum_s", "validation_s", "segmentation_s", "stitch_s"):
--- context around stitch except ---
5:            rec["stages"]["stitch_s"] = None
6:            rec["errors"].setdefault("stitch", "no segments to stitch")
21:            t = time.perf_counter()
24:                out_path = compiler.compile_highlights(args.video, time_pairs,
"highlights.mp4")
25:                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
32:            except Exception:
33:                rec["stages"]["stitch_s"] = round(time.perf_counter() - t, 3)
34:                rec["errors"]["stitch"] = traceback.format_exc()
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

449. assistant (2026-07-09T05:16:27.377Z)

Line 311-312 confirms it: on stitch exception the harness records `stitch_s = round(...)` (time-to-crash) + the error. Let me fix both layers, the meta/report detector label, and the wrapper text ? then add a test.

450. user (2026-07-09T05:16:32.420Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\harness.py has been updated successfully. (file state is current in your context ? no need to Read it back)

451. user (2026-07-09T05:16:37.006Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

452. assistant (2026-07-09T05:16:44.144Z)

Now the report detector label (nit 2) ? record `detector` in meta from `sys.platform` and use it in the report:

453. user (2026-07-09T05:16:49.257Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

454. user (2026-07-09T05:16:52.390Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\perf\run.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

455. assistant (2026-07-09T05:17:03.360Z)

Now the wrapper ? fix the exit-2 detail (suggestion 2, since exit 2 now also means "nothing ran") and the "stitch/total" doc nit (nit 1). Let me read those lines.

456. user (2026-07-09T05:17:04.933Z)

```
6           Runs scripts/gates/perf/run.py: times checksum/validation/segmentation/full-reel-
stitch for the
7           commit deployed on api.khelsutra.guru vs repo HEAD, on the local GPU, over the vault
clips, and
8           GATES the release (exit 1 on a stitch/total regression beyond tolerance). Writes a
dated report
9           + log under output/nightly_perf/ and a LATEST_STATUS.txt marker, then exits with the
gate's code
10          so Task Scheduler records PASS / REGRESSION / STALE.
11
```

457. user (2026-07-09T05:17:11.094Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

458. user (2026-07-09T05:17:14.186Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-
dev\scripts\gates\nightly_perf.ps1 has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

459. assistant (2026-07-09T05:17:45.676Z)

Now update the test for the changed failed-HEAD note and add the reviewer's suggested errored-with-numeric-time case:

460. user (2026-07-09T05:17:51.582Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\scratchpad\perfbench\wt-dev\tests\test_perf_gate.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

461. assistant (2026-07-09T05:17:59.439Z)

All 4 residual items fixed. Let me verify ? compile, lint, run the gate tests (including the new one), and parse-check the wrapper.

462. user (2026-07-09T05:18:08.607Z)

```
COMPILE OK
All checks passed!
............................................................          [100%]
63 passed in 2.89s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

463. user (2026-07-09T05:18:09.779Z)

```
nightly_perf.ps1 parse OK
```

464. assistant (2026-07-09T05:18:24.182Z)

63 tests pass (+1 new), lint/compile/parse all clean. Committing and pushing the second-pass fixes.

465. user (2026-07-09T05:18:33.020Z)

```
warning: in the working copy of 'scripts/gates/nightly_perf.ps1', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'scripts/gates/perf/harness.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'scripts/gates/perf/run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_perf_gate.py', LF will be replaced by CRLF the next
time Git touches it
[feat/perf-release-gate 81b6f27] fix(perf-gate): second-pass review ? timed-exception stitch,
exit-2 detail, report/doc labels
 4 files changed, 28 insertions(+), 8 deletions(-)
=== origin/master..HEAD (only my commits) ===
81b6f27 fix(perf-gate): second-pass review ? timed-exception stitch, exit-2 detail, report/doc
labels
8030dac fix(perf-gate): address review (4 bugs + 7 suggestions + 2 nits) + Linux support
0aa6d4b feat(gates): local release perf gate + consolidate nightly into scripts/gates/
=== push ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   8030dac..81b6f27  feat/perf-release-gate -> feat/perf-release-gate
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

466. assistant (2026-07-09T05:18:52.558Z)

Committed (`81b6f27`) + pushed. Let me post the second-pass reply and check CI + mergeability (CI needs to run on the new commit before I merge).

467. user (2026-07-09T05:19:04.619Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/547#issuecomment-4921809671
=== mergeable state ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE","state":"OPEN"}
=== CI checks (latest push) ===
HTTP 401: Bad credentials (https://api.github.com/graphql)
Try authenticating with:  gh auth login
```

468. assistant (2026-07-09T05:19:18.920Z)

Reply posted; PR is **MERGEABLE** but **UNSTABLE** ? meaning CI is still running on the new commit (I just pushed `81b6f27`). The `gh pr checks` graphql hit a transient 401; let me get check states via REST instead, and wait for green before merging.

469. user (2026-07-09T05:19:22.128Z)

```
=== check-runs for HEAD (81b6f27) via REST ===
Built wheel smoke (install + launch): queued/?
Dependency license scan (no copyleft in runtime deps): queued/?
Full test suite (offline, mocked): queued/?
Golden fixtures cross-OS (macos-latest): queued/?
Golden fixtures cross-OS (ubuntu-latest): queued/?
Golden fixtures cross-OS (windows-latest): queued/?
Leak scan (PII / attribution guard): queued/?
Markdown link check (live docs): queued/?
Mypy (lenient baseline): queued/?
```

```
Ruff lint: queued/?
Syntax sweep + import smoke (no GPU stack): queued/?
```

470. assistant (2026-07-09T05:19:45.383Z)

```
CI just started (all queued) on `81b6f27`. Let me wait for it to finish and report conclusions
before merging.
```

471. user (2026-07-09T05:29:08.151Z)

```
poll 1 (10:49:50): status={queued}
poll 2 (10:50:21): status={queued}
poll 3 (10:50:51): status={queued}
poll 4 (10:51:22): status={queued}
poll 5 (10:51:53): status={queued}
poll 6 (10:52:24): status={queued}
poll 7 (10:52:55): status={queued}
poll 8 (10:53:26): status={queued}
poll 9 (10:53:57): status={completed,in_progress}
poll 10 (10:54:28): status={completed,in_progress}
poll 11 (10:54:59): status={completed,in_progress}
poll 12 (10:55:30): status={completed,in_progress}
poll 13 (10:56:01): status={completed,in_progress}
poll 14 (10:56:32): status={completed,in_progress}
poll 15 (10:57:03): status={completed,in_progress}
poll 16 (10:57:34): status={completed,in_progress}
poll 17 (10:58:05): status={completed,in_progress}
poll 18 (10:58:36): status={completed,in_progress}
poll 19 (10:59:07): status={completed}
=== final conclusions ===
success: Built wheel smoke (install + launch)
success: Dependency license scan (no copyleft in runtime deps)
success: Full test suite (offline, mocked)
success: Golden fixtures cross-OS (macos-latest)
success: Golden fixtures cross-OS (ubuntu-latest)
success: Golden fixtures cross-OS (windows-latest)
success: Leak scan (PII / attribution guard)
success: Markdown link check (live docs)
success: Mypy (lenient baseline)
success: Ruff lint
success: Syntax sweep + import smoke (no GPU stack)
```

472. assistant (2026-07-09T05:29:33.759Z)

```
**CI is all green on `81b6f27`** ? all 11 checks pass, including the full suite and cross-OS
golden fixtures (ubuntu **and** windows). Pre-merge gate satisfied: suite green, lint/type
clean, coverage held, all review findings (both passes) resolved, and the reviewer noted "no
blockers for merge." Merging now (squash, per repo convention).
```

473. user (2026-07-09T05:29:39.364Z)

```
Exit code 1
=== PR state after merge ===
Unknown JSON field: "merged"
Available fields:
  additions
  assignees
  author
  autoMergeRequest
  baseRefName
  baseRefOid
  body
  changedFiles
  closed
```

```
  closedAt
  closingIssuesReferences
  comments
  commits
  createdAt
  deletions
  files
  fullDatabaseId
  headRefName
  headRefOid
  headRepository
  headRepositoryOwner
  id
  isCrossRepository
  isDraft
  labels
  latestReviews
  maintainerCanModify
  mergeCommit
  mergeStateStatus
  mergeable
  mergedAt
  mergedBy
  milestone
  number
  potentialMergeCommit
  projectCards
  projectItems
  reactionGroups
  reviewDecision
  reviewRequests
  reviews
  state
  statusCheckRollup
  title
  updatedAt
  url
```

474. assistant (2026-07-09T05:29:49.204Z)

The merge returned non-zero and I used a wrong JSON field. Let me check the actual state and the merge error.

475. user (2026-07-09T05:29:53.206Z)

```
=== PR state ===
{"mergeStateStatus":"UNKNOWN","mergedAt":"2026-07-
09T05:29:39Z","reviewDecision":"","state":"MERGED"}
=== retry merge, capture full error ===
! Pull request Khelsutra/badminton-highlight-indexer#547 was already merged
```

476. assistant (2026-07-09T05:30:14.786Z)

PR **#547 is MERGED** (`2026-07-09T05:29:39Z`) ? the first command succeeded; the non-zero was just my invalid `--json merged` field in the chained view. Let me confirm the squash commit landed on master, clean up my worktrees/branch, and update memory.

477. user (2026-07-09T05:30:32.284Z)

```
=== merge commit ===
{"by":"avidullu","merge":"00e8f92f102283660c25492c103f7665e98810a4"}
=== master now has scripts/gates/ ? ===
scripts/gates/README.md
scripts/gates/nightly_perf.ps1
```

```
scripts/gates/nightly_regression.ps1
scripts/gates/perf
scripts/gates/quality
scripts/gates/register_nightly_task.ps1
=== clean up my worktrees + local branch ===
error: failed to delete '.git/worktrees/wt-dev': Permission denied
removed /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt-dev
error: failed to delete '.git/worktrees/prod': Permission denied
removed /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/prod
error: failed to delete '.git/worktrees/rc': Permission denied
removed /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/rc
Deleted branch feat/perf-release-gate (was 81b6f27).
error: failed to delete '.git/worktrees/wt524-969': Permission denied
done cleanup
```

478. assistant (2026-07-09T05:30:51.723Z)

Merged and verified ? squash commit `00e8f92f`, and `origin/master` now contains `scripts/gates/{perf,quality,?}`. Local branch deleted, worktree dirs removed (the lingering `.git/worktrees` metadata is a harmless permission wart ? another session holds locks). Let me update memory to reflect the merge.

479. user (2026-07-09T05:30:52.925Z)

10   **PR [#547](https://github.com/Khelsutra/badminton-highlight-indexer/pull/547)** (branch `feat/perf-release-gate`, open 2026-07-09) adds a **local release performance gate** and consolidates the scattered nightly CLIs into a cohesive **`scripts/gates/`** package.

480. user (2026-07-09T05:31:05.353Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\perf-release-gate.md has been updated successfully. (file state is current in your context ? no need to Read it back)

481. user (2026-07-09T05:31:16.507Z)

7   - ? **Perf gate + nightly reorg (PR #547, 2026-07-09):** perf-release-gate.md ? local release perf gate (prod vs HEAD, per-stage GPU timing, gates) at `scripts/gates/perf/`; nightly CLIs consolidated into `scripts/gates/`. Finding: release **perf-neutral** (#531 within local noise). ? the pre-existing Windows-only `test_storage` #535 failure (`task_2e0a3b82`) is now **FIXED in PR #548** (OS-native GCSFuse path; see ci-windows-coverage-gap.md).

482. user (2026-07-09T05:31:29.992Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

483. assistant (2026-07-09T05:31:46.415Z)

Done ? both asks complete.

Second-pass review fixed + PR #547 **merged** ?

The 4 residual items (0 bugs, 2 suggestions, 2 nits) ? fixed in `81b6f27`:
- **Timed-exception stitch** (the real one): a stitch that *raises* no longer records its time-to-crash. The harness sets `stitch_s = None` on a crash, and `build()` also treats an error on a gated stage as a failed HEAD ? **regression**, never a bogus "improvement." Added the exact test the reviewer suggested.

- **Exit-2 wrapper detail** now reads "preflight failed **or** nothing measurable ran," since exit 2 covers both.
- **Nits**: wrapper docstring "stitch/total" ? "stitch"; the report now labels the detector from `sys.platform` (WSL2 GPU on Windows, native GPU on Linux) instead of hardcoding WSL2.

Posted a finding-by-finding reply on the PR.

Merge

- **CI green on `81b6f27`** ? all 11 checks, including full suite + cross-OS golden fixtures (**ubuntu, macos, windows**).
- **Squash-merged** to `master` as `00e8f92f`; `scripts/gates/{perf,quality}/` is now on `master`.
- Cleaned up: remote + local branch deleted, my worktrees removed. Memory updated (PR #547 ? MERGED).

Net: the local release perf gate + the `scripts/gates/` nightly consolidation are live on `master`, cross-OS (Windows WSL / Linux native), gate-hardened (broken HEAD = regression, nothing-ran = exit 2, stitch-only gating), and covered by `tests/test_perf_gate.py`.

Two follow-ups remain owner-side (unchanged): re-run `pwsh scripts/gates/register_nightly_task.ps1` on this box so the scheduled tasks point at the new paths, and seed the local baseline once with `--update-baseline`. PR #548 (the Windows GCSFuse fix from the chip you started) is separate and, once merged, clears those two pre-existing `test_storage` failures on Windows too.

484. user (2026-07-09T05:33:22.618Z)

can you spin up the process to update the live production binaries (both worker and CP) with the latest head right now?

485. assistant (2026-07-09T05:35:27.211Z)

Production deploy ? I'll treat this as a careful, staged operation. Before touching anything, let me verify the live state fresh (not trust 1-day-old memory), confirm auth/scripts, and capture exactly what's serving now (for rollback and to preserve config). This is all read-only.

486. user (2026-07-09T05:35:48.553Z)

```
=== gcloud auth + project ===
avi.dullu@gmail.com
gen-lang-client-0306026826

=== LIVE control-plane: rev + image + resources + key env ===
rally-control-plane-00022-29w   rally-control-plane-00022-29w   asia-
southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943    4       16Gi
--- CP DEPLOYMENT_PROFILE / phase_placement env+args ---
ENV: {'DEPLOYMENT_PROFILE': 'cloud-serving'}
ARGS: ['-c', "mkdir -p /tmp/apphome && printf %s 'eyJkZXBsb3ltZW50Ijp7InByb2ZpbGUiOiJjbG91ZC1zZX
J2aW5nIiwiY29tcHV0ZV90YXJnZXQiOiJjbG91ZF9ydW4iLCJwaGFzZV9wbGFjZW1lbnQiOnsidmFsaWRhdGlvbiI6ImNsb3
VkX3J1bl9sNCIsInNlZ21lbnQiOiJjbG91ZF9ydW5fbDQiLCJjb21waWxlIjoiY2xvdWRfcnVuX2w0In19LCJpbmRleGluZy
I6eyJza2lwX2FpX2hhbmRvZmYiOnRydWUsImRlZmF1bHRfc2VnbWVudGVyIjoid2FzYl9oeWJyaWQifSwic3RvcmFnZSI6ey
JiYWNrZW5kIjoiZ2NzIiwib3V0cHV0X3Jvb3QiOiJnczovL2toZWxzdXRyYS1yYWxseS1zZXJ2aW5nIiwiaW5wdXRfYmFja2
VuZCI6ImdjcyIsImlucHV0X3Jvb3QiOiJnczovL2toZWxzdXRyYS1yYWxseS1zZXJ2aW5nL3ZpZGVvcyJ9LCJzZWN1cml0eS
I6eyJlZGdlX2F1dGhfZW5hYmxlZCI6dHJ1ZSwiY2ZfdGVhbV9kb21haW4iOiJodHRwczovL2toZWxzdXRyYS5jbG91ZGZsYX
JlYWNjZXNzLmNvbSIsImNmX2FjY2Vzc19hdWQiOiJjOTgwZGE2N2VkODg4ZmZmOTM1ZmNkOTEzYWUxZWI0ZGE2M2Y5ZDY1OD
A1ZDZlMWM1Y2E3NmY5MmY1MTcwODJiIiwiYWxsb3dlZF92aWRlb19yb290IjoiL3RtcC9hcHBob21lL291dHB1dCIsIm1heF
9wYXJ0X21iIjoyNH19' | base64 -d > /tmp/apphome/config.json && export RALLY_APP_HOME=/tmp/apphome
&& exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"]

=== LIVE worker job image ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
```

487. assistant (2026-07-09T05:36:22.041Z)

Live state captured. Key facts for a safe deploy:
- **CP** `rally-control-plane-00022-29w` on `sha256:7b62a8df?` (=304af7b), **4 vCPU/16 Gi**, `DEPLOYMENT_PROFILE=cloud-serving`, and the full runtime config (phase_placement `validation/segment/compile=cloud_run_l4`, edge-auth+CF AUD, GCS roots) is **baked into the container args** as an inline config.
- **Worker** on the same `sha256:7b62a8df?`.

That confirms the safest method: an **image-only bump** (`gcloud run ? update --image <new-digest>`) preserves every bit of that config (the #532 "drift" overrides, edge-auth, resources) and swaps only the code. Now let me get the exact HEAD + delta, confirm the build scripts, and pin the rollback anchors.

488. user (2026-07-09T05:36:38.217Z)

```
=== fetch + master HEAD ===
origin/master HEAD: 00e8f92  ·  feat(gates): local release perf gate + consolidate nightly into
scripts/gates/ (#547)

=== delta being deployed: 304af7b..origin/master (backend-affecting?) ===
00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
2084d56 fix(storage): build GCSFuse fused path with OS-native separators (#548)
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
e29172a perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539) (#545)
af54f2f fix(#538): cancel propagates to the dispatched L4 worker exec (#544)
14e4767 observability: emit structured signal on GPU encoder fallback to CPU (#529) (#543)
b68af0d docs: add verified bootstrap for reproducing tests locally (#542)
8afbea9 perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)
(#541)
071d8b2 feat(compile): add opt-in SD reel rendition (#537)
f7a326d docs(runlogs): compile?L4 + NVENC live deploy-validation CUJ (2026-07-08) (#536)
5880c26 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521) (#531)
count: 11

=== does the delta touch backend/ (runtime) vs docs/scripts only? ===
 deploy/gcp/nightly_regression/README.md          |  16 +-
 deploy/gcp/nightly_regression/run_on_vm.sh       |   6 +-
 29 files changed, 1396 insertions(+), 204 deletions(-)

=== deploy scripts + .gcloudignore present? ===
.gcloudignore
deploy/cloudrun/cloudbuild.yaml
deploy/cloudrun/gen_service_yaml.py
=== upload set size preview (context leak check ? should be modest, no output/scratch) ===
files in build context: 604
```

489. assistant (2026-07-09T05:37:22.490Z)

Everything checks out for a safe deploy. Here's the plan:

Deploying: `origin/master` **`00e8f92`** (11 commits over the live `304af7b`) ? the real serving changes are #531 single-pass stitch, #535 GCSFuse worker-mount, #537 SD rendition, #539 CP acquire-gating, #538 cancel propagation, #529/#534 telemetry, #548 GCSFuse path fix (my #547 is scripts/docs only). Build context is a clean 604 files (no leak).

Method ? image-only bump (config-preserving): build one image from master ? `gcloud run jobs update rally-worker --image <digest>` (**worker first**) ? `gcloud run services update rally-control-plane --image <digest>`. This keeps the live 4 vCPU/16 Gi, edge-auth, and the baked `phase_placement=cloud_run_l4` config exactly ? only the code changes.

? Rollback anchors (pinned now):
- Digest: `sha256:7b62a8df?` (=304af7b) · CP rev: `rally-control-plane-00022-29w`
- CP: `gcloud run services update rally-control-plane --region asia-southeast1 --image ?/rally-worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943`

- Worker: `gcloud run jobs update rally-worker --region asia-southeast1 --image ?@sha256:7b62a8df?`

Let me set up the build from a clean master checkout and confirm the image reference.

490. user (2026-07-09T05:37:24.520Z)

Task #15 created successfully: Build image from origin/master 00e8f92

491. user (2026-07-09T05:37:26.447Z)

Task #16 created successfully: Deploy worker then CP (image-only bump)

492. user (2026-07-09T05:37:36.285Z)

```
=== cloudbuild.yaml (master) ? what image/tag does it build+push? ===
1:# Cloud Build config for the Cloud Run GPU worker image (COMPUTE_DECOUPLED_SERVING M7).
3:# `gcloud builds submit --tag ? -f ?` does NOT work ? `gcloud builds submit` has no `-f` flag, so a
8:# Override the region/repo/image with --substitutions=_REGION=?,_REPO=?,_IMAGE=? if needed.
9:# PROJECT_ID is provided by Cloud Build automatically.
10:substitutions:
11:  _REGION: asia-southeast1   # L4 GPU is region-limited (asia-southeast1 / europe-west1 / europe-west4 / us-central1 / us-east4)
12:  _REPO: rally
13:  _IMAGE: rally-worker
15:  - name: 'gcr.io/cloud-builders/docker'
16:    args:
19:      - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
23:images:
24:  - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
26:  machineType: E2_HIGHCPU_8   # the CUDA + micromamba image is heavy; a bigger builder keeps it ~8?10 min
```

493. assistant (2026-07-09T05:37:50.160Z)

The build produces `?/rally/rally-worker:latest`; I'll capture its digest post-build. Let me create a clean master worktree (short path) and kick off Cloud Build (~8?10 min) in the background.

494. user (2026-07-09T05:37:57.986Z)

```
=== create clean master worktree at /c/ksdeploy ===
Preparing worktree (detached HEAD 00e8f92)
HEAD is now at 00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
HEAD: 00e8f92  (expect 00e8f92)
clean? 0 modified files
=== build context size from the master worktree (.gcloudignore applies) ===
files: 289
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

495. assistant (2026-07-09T05:38:10.253Z)

Master worktree is clean at `00e8f92` (289-file context). Kicking off the build now.

496. user (2026-07-09T05:38:10.767Z)

Updated task #15 status

497. user (2026-07-09T05:38:12.415Z)

Command running in background with ID: biyz14yt4. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\biyz14yt4.output. You will be notified when it completes. To check interim output, use Read on that file path.

498. assistant (2026-07-09T05:38:23.915Z)

Build is running in the background (`biyz14yt4`, ~8?10 min). Let me confirm it submitted cleanly (build ID assigned, uploading/building) rather than erroring on auth or config.

499. user (2026-07-09T05:38:25.273Z)

<tool_use_error>Blocked: sleep 45 followed by: echo "=== build log (head ? submit + build id) ===" head -20 /c/ksdeploy_build.log echo "..." echo "=== build log (tail ? current step) ===" tail -8 /c/ksdeploy_build.log. To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run_in_background: true. Do not chain shorter sleeps to work around this block.</tool_use_error>

500. assistant (2026-07-09T05:38:31.640Z)

Right ? let me just read the build log directly.

501. user (2026-07-09T05:38:33.255Z)

```
=== build log head ===
Creating temporary archive of 289 file(s) totalling 6.2 MiB before compression.
Uploading tarball of [.] to [gs://gen-lang-client-0306026826_cloudbuild/source/1783575494.717739-148f8f1d86cf47cc8b2a10d5c2e47b50.tgz]
Created [https://cloudbuild.googleapis.com/v1/projects/gen-lang-client-0306026826/locations/global/builds/f93a0dd7-5298-47e8-848c-47daca9d3d80].
Logs are available at [ https://console.cloud.google.com/cloud-build/builds/f93a0dd7-5298-47e8-848c-47daca9d3d80?project=492532266377 ].
Waiting for build to complete. Polling interval: 1 second(s).
=== build log tail ===
Creating temporary archive of 289 file(s) totalling 6.2 MiB before compression.
Uploading tarball of [.] to [gs://gen-lang-client-0306026826_cloudbuild/source/1783575494.717739-148f8f1d86cf47cc8b2a10d5c2e47b50.tgz]
Created [https://cloudbuild.googleapis.com/v1/projects/gen-lang-client-0306026826/locations/global/builds/f93a0dd7-5298-47e8-848c-47daca9d3d80].
Logs are available at [ https://console.cloud.google.com/cloud-build/builds/f93a0dd7-5298-47e8-848c-47daca9d3d80?project=492532266377 ].
Waiting for build to complete. Polling interval: 1 second(s).
```

502. assistant (2026-07-09T05:38:53.335Z)

Build submitted cleanly and running on Cloud Build:
- **Build ID** `f93a0dd7-5298-47e8-848c-47daca9d3d80` · 289 files / 6.2 MiB context (no leak) · [console logs](https://console.cloud.google.com/cloud-build/builds/f93a0dd7-5298-47e8-848c-47daca9d3d80?project=492532266377)
- ETA ~8?10 min (E2_HIGHCPU_8, CUDA/torch/PyNvVideoCodec image).

When it completes I'll: capture the new digest ? **`jobs update rally-worker`** (worker first) ? **`services update rally-control-plane`** (image-only, config preserved) ? verify the new CP revision boots healthy ? smoke. Nothing in prod changes until the build succeeds. I'll pick up automatically when the build finishes.

503. user (2026-07-09T05:38:56.143Z)

<tool_use_error>InputValidationError: Monitor failed due to the following issue:
An unexpected parameter `timeout` was provided</tool_use_error>

504. assistant (2026-07-09T05:39:15.024Z)

The background build task will signal completion on its own ? no separate monitor needed. Standing by for build `f93a0dd7`; I'll proceed straight to the worker?CP cutover the moment it

succeeds.

505. user (2026-07-09T05:39:33.126Z)

can you also downgrade the CP machine to 2 vCPU now that most of the work is done in the L4 even stitching

506. assistant (2026-07-09T05:40:44.503Z)

Good timing ? and it's well-justified by *this very deploy*: **#539** (`gate CP acquire_local_input when validation+segment delegated to L4`) is in the delta, which removes the exact reason the CP was bumped to 4 vCPU/16 GiB (it was RAM-downloading the full input to `/tmp` unconditionally). With validation + segment + **compile/stitch all on the L4**, the CP is back to pure orchestration.

I'll fold the downgrade into the CP update step. One constraint to flag: **Cloud Run requires ?4 vCPU for 16 GiB**, so 2 vCPU implies **memory ? 8 GiB** ? which is exactly the pre-drift default (the `00020-dtg` config). So the CP update becomes:

```
gcloud run services update rally-control-plane --region asia-southeast1 \
  --image <new-digest> --cpu=2 --memory=8Gi
```

Code (#539) and the resource drop land in the **same revision**, so they're coupled correctly ? the new code is what makes 8 GiB safe. Rollback still restores everything at once (digest `7b62a8df` + `--cpu=4 --memory=16Gi`, or traffic ? `00022-29w`).

507. user (2026-07-09T05:40:46.568Z)

Updated task #16 description

508. assistant (2026-07-09T05:40:56.640Z)

Now I wait for the build. Let me do a single progress check to confirm it's actively building (not stalled), then rely on the completion notification.

509. user (2026-07-09T05:40:58.428Z)

```
=== build log tail (11:10:58) ===
Setting up libxml2:amd64 (2.9.13+dfsg-1ubuntu0.12) ...

Setting up libopenmpt0:amd64 (0.6.1-1) ...

Setting up libpython3-stdlib:amd64 (3.10.6-1~22.04.1) ...

Setting up binutils-x86-64-linux-gnu (2.38-4ubuntu2.12) ...

Setting up libdrm-amdgpu1:amd64 (2.4.113-2~ubuntu0.22.04.1) ...

Setting up libxcb-dri3-0:amd64 (1.14-3ubuntu3) ...

Setting up libx11-xcb1:amd64 (2:1.7.5-1ubuntu0.3) ...

Setting up fontconfig (2.13.1-4.2ubuntu5) ...

Regenerating fonts cache... done.

Setting up libperl5.34:amd64 (5.34.0-3ubuntu1.7) ...

Setting up libdrm-nouveau2:amd64 (2.4.113-2~ubuntu0.22.04.1) ...

Setting up python3.10 (3.10.12-1~22.04.16) ...
```

510. assistant (2026-07-09T05:41:09.603Z)

Build is healthy ? mid image-build (installing system deps, then torch/PyNvVideoCodec + push to Artifact Registry). Progressing normally, ~5?8 min left. I'll continue automatically when it finishes: capture the digest ? **worker update** ? **CP update with `--cpu=2 --memory=8Gi`** ? verify the new revision boots healthy ? smoke, then report with rollback anchors.

511. user (2026-07-09T05:49:41.442Z)

update?

512. assistant (2026-07-09T05:49:47.675Z)

Let me check the build status.

513. user (2026-07-09T05:49:50.617Z)

```
=== build log tail (11:19:50) ===
1ffbbe19418f: Layer already exists
46d54736d31f: Layer already exists
3e9e2d020416: Pushed
d3ee59ea60cd: Pushed
efe2b79b53de: Layer already exists
47654eeadbc5: Layer already exists
e0a9f5911802: Pushed
80ee6ceb5191: Pushed
2298b436099c: Pushed
ef92ceb4e7cb: Pushed
latest: digest: sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb size:
4309
DONE
--------------------------------------------------------------------------------
ID                                    CREATE_TIME                DURATION  SOURCE
IMAGES                                                                                STATUS
f93a0dd7-5298-47e8-848c-47daca9d3d80  2026-07-09T05:38:24+00:00  10M32S    gs://gen-lang-
client-0306026826_cloudbuild/source/1783575494.717739-148f8f1d86cf47cc8b2a10d5c2e47b50.tgz
asia-southeast1-docker.pg.dev/gen-lang-client-0306026826/rally/rally-worker (+1 more)  SUCCESS

=== SUCCESS/FAILURE/digest markers ===
15:Already have image (with digest): gcr.io/cloud-builders/docker
58:Digest: sha256:517da2300c184c9999ec203c2665244bdebd3578d12fcc7065e83667932643d9
1688:Step 12/20 : RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest         |
tar -xj -C /usr/local bin/micromamba     && micromamba create -y -q -n wasb -c conda-forge
python=3.10     && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR"
&& "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q         torch==2.6.0 torchvision==0.21.0
--index-url https://download.pytorch.org/whl/cu124     && "$CONDA_DIR/envs/wasb/bin/python" -m
pip install -q PyNvVideoCodec==2.1.0     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q
hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib numpy
2018:Successfully tagged asia-southeast1-docker.pg.dev/gen-lang-client-0306026826/rally/rally-
worker:latest
2020:Pushing asia-southeast1-docker.pg.dev/gen-lang-client-0306026826/rally/rally-worker:latest
2069:latest: digest: sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb
size: 4309
2070:DONE
2073:f93a0dd7-5298-47e8-848c-47daca9d3d80  2026-07-09T05:38:24+00:00  10M32S    gs://gen-lang-
client-0306026826_cloudbuild/source/1783575494.717739-148f8f1d86cf47cc8b2a10d5c2e47b50.tgz
asia-southeast1-docker.pg.dev/gen-lang-client-0306026826/rally/rally-worker (+1 more)  SUCCESS
```

514. assistant (2026-07-09T05:50:06.763Z)

Build **SUCCESS** (10m32s) ? new image digest:
`sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb`

Proceeding to the cutover ? **worker first** (so the new CP's subcommands are always understood

by the worker).

515. user (2026-07-09T05:50:07.612Z)

Updated task #15 status

516. user (2026-07-09T05:50:08.103Z)

Updated task #16 status

517. user (2026-07-09T05:50:19.743Z)

```
=== [1/2] WORKER update -> new digest ===
Updating job...
Done.
Job [rally-worker] has successfully been updated.

To execute this job, use:
gcloud run jobs execute rally-worker
rc=0
=== confirm worker image ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb
```

518. assistant (2026-07-09T05:50:31.301Z)

Worker is on the new digest ?. Now the CP ? image bump **+ the 2 vCPU / 8 GiB downgrade** in one new revision (all other config preserved):

519. user (2026-07-09T05:52:33.425Z)

```
=== [2/2] CONTROL-PLANE update -> new digest + cpu=2 mem=8Gi ===
Deploying...
Creating Revision.....................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.............done
Routing traffic.....done
Done.
Service [rally-control-plane] revision [rally-control-plane-00023-spm] has been
deployed and is serving 100 percent of traffic.
Service URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
rc=0
```

520. assistant (2026-07-09T05:52:48.209Z)

CP deployed ? new revision **`rally-control-plane-00023-spm`**, serving 100% traffic. Let me verify the config landed (image + 2 vCPU/8 GiB) and the CP booted healthy.

521. user (2026-07-09T05:53:00.542Z)

```
=== CP new revision: image + resources (expect 1838c07f, 2, 8Gi) ===
rally-control-plane-00023-spm   rally-control-plane-00023-spm   asia-
southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb    2       8Gi
```

```
=== CP boot health (recent logs ? profile/uvicorn/edge-auth/errors) ===
2026-07-08 09:39:22 GET 200 https://api.khelsutra.guru/api/config
2026-07-08 09:39:22 INFO:     169.254.169.126:1862 - "GET /api/config HTTP/1.1" 200 OK
2026-07-08 09:39:32 GET 200 https://api.khelsutra.guru/api/config
2026-07-08 09:39:32 INFO:     169.254.169.126:23834 - "GET /api/config HTTP/1.1" 200 OK
2026-07-09 05:52:27 05:52:27 [INFO] backend.config.loader: [CONFIG] deployment profile ACTIVE:
cloud-serving (compute_target=cloud_run)
2026-07-09 05:52:27 [SERVER] ? Binding 0.0.0.0 exposes the API on the network ? ensure an auth
gate (edge_auth_enabled / a reverse proxy) is in front, and set RALLY_ALLOWED_HOSTS to your
public host(s) (the Host-header guard defaults to loopback-only and will 400 forwarded requests
otherwise).
2026-07-09 05:52:27 INFO:     Started server process [1]
2026-07-09 05:52:27 INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

522. assistant (2026-07-09T05:53:27.845Z)

Both surfaces are live on the new build, and the CP booted clean:
- CP rev `rally-control-plane-00023-spm` · image `1838c07f` · **cpu=2, mem=8Gi** ? · `deployment profile ACTIVE: cloud-serving` · Uvicorn up, no errors/OOM.
- Worker on `1838c07f` ?.

The deploy itself only routes traffic after the readiness probe passes, so that's already a health signal. Let me do a final smoke ? confirm the public edge?CP path responds and no post-boot errors.

523. user (2026-07-09T05:53:45.821Z)

```
=== public edge -> CP smoke (edge-auth on: 200 if health unauthed, else 401/302 = path live) ===
  GET /api/config -> 000curl: (43) A libcurl function was given a bad argument
  GET /healthz -> 000curl: (43) A libcurl function was given a bad argument
  GET / -> 000curl: (43) A libcurl function was given a bad argument

=== CP logs since new boot ? any errors/crash-loop? (should be clean) ===
--- (empty above = no errors) ---

=== both surfaces final image check ===
CP:     sha256:1838c07f6cb1
worker: sha256:1838c07f6cb1
```

524. assistant (2026-07-09T05:54:03.083Z)

Both surfaces confirmed on the new digest and CP logs are clean (no errors/crash-loop). The curl error `43`/`000` is a local Git-Bash curl arg issue, not the service ? let me retry the edge smoke cleanly.

525. user (2026-07-09T05:54:09.194Z)

```
=== edge -> CP smoke (clean curl) ===
GET /api/config -> 000 curl: (43) A libcurl function was given a bad argument
GET /healthz -> 000 curl: (43) A libcurl function was given a bad argument
=== also try the run.app URL directly (bypasses CF; expect 401/403 from Cloud Run IAM or edge-
auth, or 200 health) ===
run.app /api/config -> 000 curl: (43) A libcurl function was given a bad argument
```

526. assistant (2026-07-09T05:54:26.697Z)

curl is broken on this box (error 43 even to `run.app`). The deploy's own readiness gate + clean boot logs already confirm CP health, but let me confirm the edge?CP path with a Windows-native HTTP client instead.

527. user (2026-07-09T05:54:33.153Z)

```
https://api.khelsutra.guru/api/config -> 200 OK
https://rally-control-plane-492532266377.asia-southeast1.run.app/api/config -> 200 OK
```

528. assistant (2026-07-09T05:54:56.121Z)

200 OK on both the public `api.khelsutra.guru` edge and the direct Cloud Run URL ? the CP is live and serving on the new revision. Deploy is complete and verified. Let me record the new live state and clean up the build worktree.

529. user (2026-07-09T05:54:56.608Z)

Updated task #16 status

530. user (2026-07-09T05:54:58.129Z)

```
1       ---
2       name: live-deployment-state
3       description: Current live cloud-serving deployment (rev/digest/rollback) ? updated each
release; live rev moves between sessions
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 7e7bb1dc-c0b6-42c2-af08-d3c688a562b8
8       ---
9
10      # Live cloud-serving deployment state (as of 2026-07-08 release)
11
12      **LIVE (2026-07-08):** control-plane rev **`rally-control-plane-00022-29w`** (was
00021-jhp ? 00020-dtg) + worker Job both on image
13      **`sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943`** (built
from `origin/master @ 304af7b`).
14      ? **Rev 00022 = `phase_placement.validation=cloud_run_l4`** (+ segment + compile
explicit) **AND CP bumped to 4 vCPU / 16 GiB** ? both via the manifest `config.json`/flags (NOT
the preset ? runtime override, config-drift per #532). Owner-requested 2026-07-08 to eliminate
CP OOM on a 6.52 GiB clip (CP still `acquire_local_input`-downloads the full file to RAM-backed
/tmp ? unconditional at orchestration.py:814 ? so 16 GiB gives headroom; validation?L4 moves
only the DECODE to the L4). Rollback: validation?CP + 2vCPU/8Gi = redeploy 00020-dtg;
validation?L4 @ 8Gi = 00021-jhp.
15
```

531. user (2026-07-09T05:55:00.510Z)

build worktree cleaned

532. assistant (2026-07-09T05:55:23.356Z)

Now update the live-deployment-state memory to the new release:

533. user (2026-07-09T05:55:35.357Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\live-deployment-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

534. assistant (2026-07-09T05:55:49.773Z)

Deploy done and verified. Let me refresh the MEMORY index pointer to the new live state, then summarize.

535. user (2026-07-09T05:55:50.952Z)

```
4       - ? **LIVE deploy (2026-07-08):** [live-deployment-state.md](live-deployment-state.md) ?
CP rev **00020-dtg** + worker on digest **7b62a8df** (master 304af7b; #525/#527/#528/#530).
compile?L4+NVENC ON; validation stays on CP. Rollback ? 00019-96f / c6393d27. Release-process
gaps ? **#532**. Next: owner CUJ to measure compile?L4.
```

536. user (2026-07-09T05:56:03.745Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

537. assistant (2026-07-09T05:56:25.861Z)

? Production deploy complete ? both surfaces on `origin/master @ 00e8f92`

	before	now
Image	`sha256:7b62a8df?` (304af7b)	**`sha256:1838c07f?`** (00e8f92, Cloud Build `f93a0dd7`)
Worker	7b62a8df	1838c07f ?
Control-plane	rev `00022-29w`, **4 vCPU/16 GiB**	rev **`00023-spm`**, **2 vCPU/8 GiB** ?

Shipped (11 commits over the live 304af7b): #531 single-pass stitch, #535 GCSFuse worker-mount, #537 SD rendition, #539 CP acquire-gating, #538 cancel propagation, #529/#534 telemetry, #548 GCSFuse path fix, #547 gates reorg.

CP downgrade ? done and justified: #539 (in this deploy) gates the CP's full-file `acquire_local_input` RAM download now that validation/segment/**compile(stitch)** all run on the L4, so the 16 GiB headroom is no longer needed. Cloud Run requires ?4 vCPU for 16 GiB, so 2 vCPU ? 8 GiB (the pre-drift `00020` default).

Config preserved ? image-only bump kept the baked cloud-serving config exactly: `phase_placement validation/segment/compile=cloud_run_l4`, edge-auth + CF AUD, GCS roots. Only the image and CP CPU/mem changed. Worker deployed first.

Verified healthy:
- CP boot logs clean ? `deployment profile ACTIVE: cloud-serving`, Uvicorn up, no errors/OOM.
- `GET /api/config` ? **200** on both `api.khelsutra.guru` (edge) and the direct `run.app` URL.
- New revision took 100% traffic (readiness probe passed).

? Rollback (one command each):
```
gcloud run jobs update rally-worker --region asia-southeast1 \
  --image ?/rally-worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
gcloud run services update rally-control-plane --region asia-southeast1 \
  --image ?@sha256:7b62a8df? --cpu=4 --memory=16Gi     # or: services update-traffic --to-revisions rally-control-plane-00022-29w=100
```

Memory (live-deployment-state) updated; build worktree cleaned up. Since edge-auth gates `/api/process`, a full upload?reel CUJ through the SPA is the owner-side confirmation ? but the serving path is up and healthy. Want me to watch the CP logs for a few minutes under real traffic, or tail the first worker execution to confirm the new `compile`/GCSFuse paths behave?

538. user (2026-07-09T05:56:28.339Z)

can you run the newly added release gate tests on prod as well to see the perf?

539. assistant (2026-07-09T05:58:42.869Z)

Good question ? but "on prod" has a few genuinely different meanings with very different cost/effort, and one of them spends money on GPU infra, so let me confirm rather than guess.

The key wrinkle: the perf gate is a **local tool** that compares two commits on a GPU box. And we just deployed, so **prod is now `00e8f92` = HEAD** ? so a run measures the *release delta* `304af7b ? 00e8f92`, which is exactly what's now live. The one thing the local RTX 2070 run couldn't show was the **real stitch win**, because #531's benefit is the L4's **NVENC** path (local libx264 made it "within noise"). So "on prod" most usefully means *on L4-class hardware

with NVENC*.

540. user (2026-07-09T06:00:00.799Z)

Your questions have been answered: "How do you want to measure perf 'on prod'? (The gate compares old-prod 304af7b vs the now-live 00e8f92; the interesting delta is #531 stitch, which only shows with L4 NVENC.)"="Live api.khelsutra.guru CUJ". You can now continue with these answers in mind.

541. assistant (2026-07-09T06:01:01.689Z)

Good ? the live CUJ is the truest signal (the real CP + L4 + NVENC + GCSFuse stack, via the new #534 telemetry). The one hard constraint I flagged: `/api/process` sits behind Cloudflare Access, so I can't *trigger* it headless. But I *can* read the per-stage timing from the worker/CP logs with the gcloud access I have. Let me first verify that harvest path is real (what #534 emits + whether I can read a worker execution's timing) before asking you to drive the trigger.

542. user (2026-07-09T06:01:12.671Z)

```
=== #534 telemetry / heartbeat ? what does the deployed code emit? ===
46:from backend.reporting import run_signals  # for #529 gpu_encoder_fallback_to_cpu signal
48:from backend.utils.paths import output_base, safe_output_filename
57:# #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg compile
stitch ?
59:# accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
242:def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
243:    """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
244:    minute-granular: the control plane knows ELAPSED time, not frames ? frame-level % comes
245:    with the worker-side heartbeat (the #482 follow-up). Under a minute reads as "starting"
247:    minutes = int(elapsed_sec) // 60
249:        return f"segmenting ({seg_name}) on cloud GPU ? starting up"
250:    return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
277:    ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust
path:
511:    == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
715:    # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
735:    # then worker-rejected at its default 20.0 after a successful GPU dispatch).
753:        # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
754:        # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ? which
757:        def _on_wait(elapsed: float) -> None:
762:                notify(_wait_progress_message(seg_name, elapsed))
1103:        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.

=== recent rally-worker executions (any to read timing from?) ===
NAME                COMPLETION_TIME              TYPE       CREATION_TIMESTAMP
rally-worker-98gqk  2026-07-08T09:36:13.096866Z  Completed  2026-07-08T09:30:32.225160Z
rally-worker-gvqzv  2026-07-08T09:29:57.796671Z  Completed  2026-07-08T09:17:35.023374Z
rally-worker-8gqkv  2026-07-08T09:01:06.918619Z  Completed  2026-07-08T08:53:07.352681Z
rally-worker-fdbs2  2026-07-08T08:16:47.867323Z  Completed  2026-07-08T08:11:31.986277Z
rally-worker-956vm  2026-07-08T08:09:40.796481Z  Completed  2026-07-08T07:59:22.904375Z

=== is there a headless CF-Access path? (service token / verify_compute_path auth) ===
37:        headers={"Content-Type": "application/json"},
```

543. assistant (2026-07-09T06:02:07.525Z)

Key findings: **I can read prod logs, but I cannot *trigger* `/api/process` headless** ? it's CF-Access-gated and there's no service-token path in the codebase. The recent worker executions are all from 2026-07-08 (pre-deploy). Importantly, the *previous* prod (rev 00022) already ran

compile?L4 + NVENC, so those executions should already show the prod stitch-with-NVENC perf. Let me read one to confirm the harvest works and extract what's there.

544. user (2026-07-09T06:02:17.308Z)

The user doesn't want to proceed with this tool use. The tool use was rejected (eg. if it was a file edit, the new_string was NOT written to the file). STOP what you are doing and wait for the user to tell you how to proceed.

545. user (2026-07-09T06:02:17.311Z)

[Request interrupted by user for tool use]

546. user (2026-07-09T06:02:17.340Z)

can I iussue the CUJ with a long video?

547. assistant (2026-07-09T06:03:13.969Z)

Yes ? a long video is actually the **ideal** choice here, for two reasons:

1. It makes the perf you want to see much clearer. More rallies ? a longer reel ? more actual encode work, so the **compile/stitch on L4 with NVENC** (the #531 win that was "within noise" on my local libx264 box) shows up in real seconds instead of fixed overhead. Segmentation on the L4/nvdec also scales with length, so you get a genuine throughput number.

2. It doubles as the real test of the CP downgrade we just did. The 4 vCPU/16 GiB bump existed *specifically* for a large clip (the 6.52 GiB one that OOM'd the CP). A long/large clip is exactly the case that exercises whether the 2 vCPU/**8 GiB** downgrade holds ? which it should, because this same deploy ships:
- **#539** ? the CP no longer RAM-downloads the full input (validation+segment are on the L4),
- **#535** ? the worker reads clip ranges via the GCSFuse mount instead of re-downloading the full source,
- chunked upload (`max_part_mb=24`) streams to GCS rather than buffering the whole file in CP RAM.

So go ahead with a big one (a real match, 4K/5.3K, multi-GB is great). One thing I'll watch: if the CP *does* OOM on it, that tells us the downgrade was premature ? and rollback is a single command (`--cpu=4 --memory=16Gi`). I don't expect it to.

When you kick it off, just tell me the clip (name / rough size+length) and roughly when ? or I'll spot the execution in the logs. Then I'll harvest and report the live prod numbers: per-stage timing (checksum ? validation ? segmentation ? **compile/stitch**), NVENC-vs-CPU confirmation (#529 signal), the reel output, and the L4 GPU util/mem + CP memory (#534 telemetry) ? plus an explicit ?/? on whether the CP held at 8 GiB.

Want me to start a live tail on the CP + worker logs now so I catch it the moment you start, or will you ping me when it's running?

548. user (2026-07-09T06:03:59.338Z)

I am doing it for gs://khelsutra-rally-serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4

549. assistant (2026-07-09T06:04:29.693Z)

On it ? harvesting the live run for `GX010155.MP4`. Let me capture the clip size, establish a time baseline, and check for the incoming request + any worker execution that's spun up.

550. user (2026-07-09T06:04:45.364Z)

```
=== clip size ===
8417724136  2026-07-08T09:06:24Z  gs://khelsutra-rally-serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4
TOTAL: 1 objects, 8417724136 bytes (7.84GiB)
```

```
now: 06:04:38Z

=== worker executions (watch for a NEW one, created today post-06:00Z) ===
NAME                CREATION_TIMESTAMP           TYPE       RUNNING  SUCCEEDED_COUNT
rally-worker-dpssz  2026-07-09T06:04:20.821211Z  Completed  1
rally-worker-98gqk  2026-07-08T09:30:32.225160Z  Completed  0        1
rally-worker-gvqzv  2026-07-08T09:17:35.023374Z  Completed  0        1
rally-worker-8gqkv  2026-07-08T08:53:07.352681Z  Completed  0

=== CP logs ? recent /api/process or dispatch for GX010155 / this run ===
2026-07-09T06:04:38.851131Z     INFO:     169.254.169.126:14190 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:33.754240Z     INFO:     169.254.169.126:25054 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:27.896628Z     INFO:     169.254.169.126:25042 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:24.432185Z     INFO:     169.254.169.126:15224 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:22.088122Z     INFO:     169.254.169.126:15220 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:21.023925Z     06:04:21 [INFO] compute.cloud_run: cloud-run: dispatched
job=rally-worker exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-dpssz; bucket-polling gs://khelsutra-rally-
serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done
2026-07-09T06:04:20.496690Z     INFO:     169.254.169.126:15206 - "GET
/api/jobs/e7b1b3cb87e34b3e8f7fd1722bbf8c15 HTTP/1.1" 200 OK
2026-07-09T06:04:19.401995Z     06:04:19 [INFO] backend.orchestration: [API] cloud-serving:
dispatching gs://khelsutra-rally-serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> Cloud
Run Job (out=gs://khelsutra-rally-serving)
2026-07-09T06:04:19.400280Z     06:04:19 [INFO] backend.orchestration: [API] Using segmenter:
wasb_hybrid
2026-07-09T06:04:19.399614Z     06:04:19 [INFO] backend.orchestration: [API] Video passed
checks. Segmenting and indexing...
2026-07-09T06:04:19.399551Z     06:04:19 [INFO] backend.orchestration: [API] validation
delegated to the L4 worker (deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its validator suite; the worker is the gate for gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4.
2026-07-09T06:04:19.398427Z     06:04:19 [INFO] backend.orchestration: [API] skipping CP acquire
for fully delegated phases with cheap video_id (#539).
2026-07-09T06:04:19.398412Z     06:04:19 [INFO] backend.orchestration: [API] trusting submit-
time video_id=e2e144175c8d05451903ae44a9ce0aa0 ? skipped acquire for hash (#524/#539).
2026-07-09T06:04:19.364083Z     INFO:     169.254.169.126:15192 - "POST /api/process HTTP/1.1"
202 Accepted
```

551. assistant (2026-07-09T06:05:19.772Z)

The CUJ is live and already telling us a lot. The clip is **7.84 GiB** (bigger than the 6.52 GiB one that drove the 4/16 bump ? a perfect stress test), and the CP downgrade is **already validating**:

- `POST /api/process` ? 202 (06:04:19)
- ? **#539 working**: `skipping CP acquire for fully delegated phases (#539)` ? the CP is **not** RAM-loading the 7.84 GiB file
- ? **#524/#539**: `trusting submit-time video_id=e2e1441?` (no download for hashing)
- validation delegated to the L4 worker; dispatched to exec **`rally-worker-dpssz`** (running on the L4)

So the CP is just orchestrating/polling ? exactly why 8 GiB is safe. Let me watch the worker execution (segmentation ? validation ? the compile/stitch that follows) and keep an eye on the CP for any OOM.

552. user (2026-07-09T06:05:22.776Z)

Task #17 created successfully: Harvest live prod CUJ perf (GX010155)

553. user (2026-07-09T06:05:37.300Z)

```
=== worker exec rally-worker-dpssz logs (validation/segment/GPU/nvenc/timing) ? now 06:05:29Z
===
2026-07-09T06:04:33.427421Z     time="09/07/2026 06:04:32.882619" severity=INFO message="File
system has been successfully mounted." mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.427150Z     time="09/07/2026 06:04:32.882278" severity=INFO
message="Mounting file system \"khelsutra-rally-serving\"..." mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426889Z     time="09/07/2026 06:04:32.878966" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.426656Z     time="09/07/2026 06:04:32.878644" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426419Z     time="09/07/2026 06:04:32.878571" severity=INFO
message="GetStorageLayout -> (khelsutra-rally-serving) 112 msec" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426218Z     time="09/07/2026 06:04:32.766348" severity=INFO message="Calling
GetStorageLayout request for \"projects/_/buckets/khelsutra-rally-serving/storageLayout\" with
deadline=30s" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.425990Z     time="09/07/2026 06:04:32.766319" severity=INFO
message="GetStorageLayout <- (khelsutra-rally-serving)" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.425777Z     time="09/07/2026 06:04:32.766302" severity=INFO message="Set up
root directory for bucket khelsutra-rally-serving" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.425488Z     time="09/07/2026 06:04:32.766296" severity=INFO message="MRD
cache (LRU-based) enabled with maxSize=1000 (pools closed-file MRDs)" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.425208Z     time="09/07/2026 06:04:32.766270" severity=INFO
message="Creating a new server...\n" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.092620Z     time="09/07/2026 06:04:32.765660" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.092614Z     time="09/07/2026 06:04:32.765389" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.092320Z     time="09/07/2026 06:04:32.765329" severity=INFO
message="UserAgent = gcsfuse/3.9.2 (Go version go1.26.4) (GPN:gcsfuse-serverless)
(Cfg:0:0:0:1:0:0) (mount-id:khelsutra-rally-serving-03345eef)\n" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.091959Z     time="09/07/2026 06:04:32.765297" severity=INFO
message="Creating Storage handle..." mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:32.759094Z     time="09/07/2026 06:04:32.758114" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-03345eef "Full Config"="&{AppName:serverless CacheDir:
CloudProfiler:{AllocatedHeap:true Cpu:true Enabled:false Goroutines:false Heap:true
Label:gcsfuse-0.0.0 Mutex:false ServiceName:gcsfuse} Debug:{ExitOnInvariantViolation:false
Fuse:false Gcs:false LogMutex:false} DisableAutoconfig:false DisableListAccessCheck:true
DummyIo:{Enable:false PerMbLatency:0s ReaderLatency:0s} EnableAtomicRenameObject:true
EnableGoogleLibAuth:true EnableHns:true EnableNewReader:true EnableStandardSymlinks:true
EnableTypeCacheDeprecation:true EnableUnsupportedPathSupport:true
FileCache:{CacheFileForRangeRead:false DownloadChunkSizeMb:200 EnableCrc:false
EnableExperimentalSharedChunkCache:false EnableODirect:false EnableParallelDownloads:false
ExcludeRegex: ExperimentalDisableSizeCalculationFix:false ExperimentalEnableChunkCache:false
ExperimentalParallelDownloadsDefaultOn:true IncludeRegex: MaxParallelDownloads:16 MaxSizeMb:-1
ParallelDownloadsPerFile:16 SharedCacheChunkSizeMb:8 WriteBufferSize:4194304}
FileSystem:{CongestionThreshold:0 DirMode:511 DisableParallelDirops:false
EnableKernelReader:false ExperimentalEnableDentryCache:false ExperimentalEnablePirlo:false
ExperimentalEnableReaddirplus:false ExperimentalODirect:false FileMode:438
FuseOptions:[allow_other] Gid:1000 IgnoreInterrupts:true InactiveMrdCacheSize:1000
KernelListCacheTtlSecs:0 KernelParamsFile: MaxBackground:0 MaxReadAheadKb:0
PreconditionErrors:true RenameDirLimit:0 TempDir: Uid:1000} Foreground:true
GcsAuth:{AnonymousAccess:false KeyFile: ReuseTokenFromUrl:true TokenUrl:}
GcsConnection:{BillingProject: ClientProtocol:http1 CustomEndpoint: EnableHttpDnsCache:true
ExperimentalEnableJsonRead:false ExperimentalLocalSocketAddress: GrpcConnPoolSize:1
GrpcPathStrategy:direct-path-with-fallback HttpClientTimeout:0s LimitBytesPerSec:-1
LimitOpsPerSec:-1 MaxConnsPerHost:0 MaxIdleConnsPerHost:100 SequentialReadSizeMb:200}
```

```
GcsRetries:{ChunkRetryDeadlineSecs:120 ChunkTransferTimeoutSecs:10
ExperimentalNonrapidFolderApiStallRetry:false MaxRetryAttempts:9223372036854775807
MaxRetrySleep:30s Multiplier:2 ReadStall:{Enable:true InitialReqTimeout:20s MaxReqTimeout:20m0s
MinReqTimeout:1.5s ReqIncreaseRate:15 ReqTargetPercentile:0.99}} ImplicitDirs:true
List:{EnableEmptyManagedFolders:false} Logging:{FilePath: Format:text
LogRotate:{BackupFileCount:10 Compress:true MaxFileSizeMb:512} Severity:INFO WireLog:}
MachineType: MetadataCache:{DeprecatedStatCacheCapacity:20460 DeprecatedStatCacheTtl:1m0s
DeprecatedTypeCacheTtl:1m0s EnableMetadataPrefetch:false EnableNonexistentTypeCache:false
ExperimentalMetadataPrefetchOnMount:disabled MetadataPrefetchEntriesLimit:5000
MetadataPrefetchMaxWorkers:10 NegativeTtlSecs:5 StatCacheMaxSizeMb:34 TtlSecs:60
TypeCacheMaxSizeMb:4} Metrics:{BufferSize:256 CloudMetricsExportIntervalSecs:0
ExperimentalEnableGrpcMetrics:true PrometheusPort:0 StackdriverExportInterval:0s
UseNewNames:false Workers:3} Mrd:{PoolSize:4} OnlyDir: Profile: Read:{BlockSizeMb:16
EnableBufferedRead:false GlobalMaxBlocks:40 InactiveStreamTimeout:10s MaxBlocksPerHandle:20
MinBlocksPerHandle:4 RandomSeekThreshold:3 StartBlocksPerHandle:1}
Trace:{Exporters:[gcpexporter] ProjectId: SamplingRatio:0}
WorkloadInsight:{ForwardMergeThresholdMb:0 OutputFile: Visualize:false} Write:{BlockSizeMb:32
CreateEmptyFile:false EnableRapidAppends:true EnableStreamingWrites:true
FinalizeFileForRapid:false GlobalMaxBlocks:4 MaxBlocksPerFile:1}}"
2026-07-09T06:04:32.758666Z     time="09/07/2026 06:04:32.758076" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-03345eef "CLI Flags"="map[app-name:serverless dir-
mode:777 file-mode:666 foreground:true gid:1000 ignore-interrupts:true implicit-dirs:true log-
format:text o:[allow_other] uid:1000]"
2026-07-09T06:04:32.674322Z     GCSFuse is mounted with bucket khelsutra-rally-serving. Enable
GCSFuse logging with mount options for more logs

=== CP OOM/memory watch (should be clean ? no full download) ===
--- (empty = no CP memory/OOM issues) ---
```

554. assistant (2026-07-09T06:07:36.486Z)

The worker is reading the 7.84 GiB source via **GCSFuse mount** (#535 confirmed ? `File system successfully mounted`, range-read config, no full download), segmentation underway on the L4, and the **CP OOM watch is clean**. So all three memory-savers are verified live:

- ? #539 ? CP skips the acquire (no 7.84 GiB in CP RAM)
- ? #535 ? worker reads ranges via GCSFuse mount (no full re-download)
- ? CP holding at 8 GiB, no OOM

Let me arm a watcher for job completion (reel written) so I harvest the full timing the moment it's done, and keep monitoring.

555. user (2026-07-09T06:07:41.798Z)

Command running in background with ID: bbu5pthwi. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bbu5pthwi.output. You will be notified when it completes. To check interim output, use Read on that file path.

556. assistant (2026-07-09T06:07:57.296Z)

Completion watcher armed. Let me grab the live segmentation progress + the #534 GPU telemetry from the worker (filtering out the GCSFuse mount noise) to report what's happening now.

557. user (2026-07-09T06:08:06.393Z)

```
=== worker exec app-level logs (validation/wasb/GPU telemetry/fps) ? 06:08:02Z ===
2026-07-09T06:06:58.531863Z     [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> /tmp/rally-worker-jrb2w9ys/in.mp4
2026-07-09T06:04:40.706541Z       warnings.warn(message, FutureWarning)
2026-07-09T06:04:40.706516Z     /usr/local/lib/python3.10/dist-
packages/google/api_core/_python_version_support.py:255: FutureWarning: You are using a Python
version (3.10.12) which Google will stop supporting in new releases of google.api_core once it
reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least
```

Python 3.11, to continue receiving updates for google.api_core past that date.

558. assistant (2026-07-09T06:08:58.732Z)

Good detail from the worker logs ? here's the live timeline so far:

time (UTC)	event
06:04:19	`POST /api/process` ? 202; CP skips acquire (**#539**), trusts `video_id=e2e1441?` (**#524**), delegates validation to L4
06:04:20?21	dispatched to worker exec `rally-worker-dpssz` on the L4
06:04:32	GCSFuse mount up (**#535**)
06:06:58	worker pulled the **7.84 GiB** source ? `/tmp/in.mp4` (~2m25s) for the decode/detect pass
~06:08	wasb detection running on the L4

CP still clean ? no OOM, holding at 8 GiB. One nuance to correct: the **detect** worker does pull the full file to `/tmp` (it has to decode every frame), so #535's mount-*range*-read win is specifically the **compile** step (it reads only the rally ranges instead of re-downloading the full source) ? I'll confirm that on the compile exec.

The completion watcher is armed; I'll harvest the full per-stage timing ? validation, segmentation (L4 throughput + #534 GPU telemetry), and especially **compile/stitch on L4 with NVENC** (#531) ? the moment the reel lands. This clip is ~9?11 min of footage, so give it a bit.

559. user (2026-07-09T06:09:00.816Z)

it straihght away says segmenting without validation .. seems this was already validated?

please make sure to validate all the steps

560. assistant (2026-07-09T06:09:51.315Z)

Good catch ? let me dig into exactly what's running vs skipped, because the current prod config genuinely changes which stages fire where. From the CP logs, two stages are **not** happening on the control-plane:

- **Checksum** is *not computed* ? `trusting submit-time video_id ? skipped acquire for hash (#524)`. The CP trusts the video_id the SPA submitted rather than re-hashing.
- **Validation** is *delegated to the L4 worker* ? `validation delegated to the L4 worker ? the control-plane skips its validator suite; the worker is the gate`. So the SPA jumps straight to "segmenting" because the CP has no separate validate phase ? it runs **on the worker**.

So "segmenting straight away" is the delegation, not necessarily a cache hit. But your instinct is right to check ? there's also a validation-result cache (#518) + worker skip-revalidation (#513) that *could* short-circuit it. Let me verify on the worker whether validation actually executed (and wasn't cached).

561. user (2026-07-09T06:10:01.160Z)

```
=== worker exec dpssz ? validation + cache + stage sequence (06:09:57Z) ===
2026-07-09T06:08:28.488355Z     06:08:28 [INFO] backend.pipeline.detectors.native_wasb_runner:
Running native WASB inference (device=cuda) on in.mp4 ...
2026-07-09T06:08:28.480294Z     06:08:28 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB detector env OK: torch 2.6.0+cu124 cuda True
2026-07-09T06:08:16.932290Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB: detector_impl=native ? Linux-native runner (no WSL).
2026-07-09T06:08:16.837167Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will be emitted as rallies;
no gemini handoff, no API key needed.
2026-07-09T06:06:58.531863Z     [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> /tmp/rally-worker-jrb2w9ys/in.mp4
```

562. assistant (2026-07-09T06:10:28.279Z)

Let me see the full worker sequence in the gap between the pull (06:06:58) and wasb starting (06:08:16) ? that's where validation would run, and I want to confirm it actually fired (not silently skipped/cached).

563. user (2026-07-09T06:10:36.561Z)

```
=== ALL worker app lines, chronological (skip gcsfuse) ? the full stage sequence ===
2026-07-09T06:06:58.531863Z     [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> /tmp/rally-worker-jrb2w9ys/in.mp4
2026-07-09T06:08:16.837167Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will be emitted as rallies;
no gemini handoff, no API key needed.
2026-07-09T06:08:16.932290Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB: detector_impl=native ? Linux-native runner (no WSL).
2026-07-09T06:08:28.480294Z     06:08:28 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB detector env OK: torch 2.6.0+cu124 cuda True
2026-07-09T06:08:28.488355Z     06:08:28 [INFO] backend.pipeline.detectors.native_wasb_runner:
Running native WASB inference (device=cuda) on in.mp4 ...
```

564. assistant (2026-07-09T06:11:34.892Z)

The full worker sequence tells the story ? there is **no validation happening**:

```
06:06:58  pulled source ? /tmp/in.mp4
06:08:16  WASB fully-local mode (skip_ai_handoff=True)   ? straight to segmentation
06:08:16  detector_impl=native
06:08:28  WASB detector env OK: torch 2.6.0+cu124 cuda True
06:08:28  Running native WASB inference ...
```

The ~78 s gap (06:06:58 ? 06:08:16) is just worker cold-start + torch import/model load ? **not** validation. There are **zero** validator log lines (no blur/scene/court/relevance). So your instinct is correct: **validation was skipped entirely.**

That's likely a real interaction bug, and I want to confirm it in the deployed code before asserting it: the CP delegates validation to the worker (`phase_placement.validation=cloud_run_l4` ? "the worker is the gate"), but the worker's **#513 skip-revalidation** logic may be skipping it on the assumption the CP already validated ? so it runs *nowhere*. Let me check.

565. user (2026-07-09T06:11:40.982Z)

```
=== deployed worker: how does it handle validation? (#513 skip + delegated val) ===
11:the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
14:(validator registry ? segmenter ? report), never a forked pipeline.
39:from backend.pipeline.segmenters.base import segmenter_registry
40:from backend.pipeline.validators.base import registry as validator_registry
167:    The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
168:    torch, and pre-M5 the worker's infer/train path was torch-free (detection is delegated
to a
202:        skip_validation=bool(getattr(args, "skip_validation", False)),
281:    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
282:    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it
runs the
283:    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating
here is
286:    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then
the worker
288:    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
289:    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
290:    if getattr(args, "skip_validation", False):
```

```
292:            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
293:            "gate and already validated this input."
297:        val_results, val_context = validator_registry.run_all_with_context(
310:        # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort
the
343:                "[worker] validation FAILED: "
344:                + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
348:        segmenter_cls = segmenter_registry.get(seg_name)
352:                f"(have {list(segmenter_registry.list_available())})"
702:    seg_cls = segmenter_registry.get(args.segmenter)
706:            f"(have {list(segmenter_registry.list_available())})"
737:        shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.

=== orchestration: worker-dispatch validation flag (does CP tell worker to validate?) ===
228:    decide whether to DELEGATE validation to the worker (``phase_placement.validation``). A
per-request
730:    # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
736:    # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
738:    # here), so the worker SKIPS its redundant re-validation (#513). If validation is placed
on the
739:    # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ? see
741:    worker_skips_validation = (
742:        resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
749:        skip_validation=worker_skips_validation,
881:    # both delegated to cloud_run_l4 (and compute cloud), the CP is a pure thin dispatcher
and
933:            # Pure delegated + cheap id: CP stays thin dispatcher, no full download.
936:                "[API] skipping CP acquire for fully delegated phases with cheap video_id
(#539).",
999:        # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
1002:        # #521 phase_placement.validation: computed early (see above) so we can gate
acquire_local_input
1003:        # for the pure-delegated case (#539). When delegated, CP skips validators; worker
is the gate.
1006:                "[API] validation delegated to the L4 worker "
1007:                "(deployment.phase_placement.validation=cloud_run_l4) ? the control-plane
skips its "
1008:                "validator suite; the worker is the gate for %s.",
1031:                        "validator config unchanged ? skipping the validator suite. "
```

566. assistant (2026-07-09T06:12:33.509Z)

Now I understand the intended logic: the CP correctly sets `skip_validation = (placement != cloud_run_l4)` ? so when validation is delegated to the L4 (our case), the worker gets `skip_validation=False` and **is supposed to run the validators** (worker `__main__.py:290` ? the `else` branch calls `validator_registry.run_all_with_context`). The catch: the worker only *logs* on **validation FAILED** or **SKIPPED** ? a silent pass emits nothing. So the absence of logs is ambiguous. Let me read the exact worker validate block (does it log a "Validating"/timing marker, and is there a worker-side validation cache?) to settle it.

567. user (2026-07-09T06:12:36.289Z)

```
=== worker __main__.py validate+segment block (lines 275-360) ===
    )
    print(f"[worker] pulled {args.video_uri} -> {local_video}")

    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
```

```
    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
    # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
    if getattr(args, "skip_validation", False):
        print(
            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
            "gate and already validated this input."
        )
        val_results, val_context = [], {}
    else:
        val_results, val_context = validator_registry.run_all_with_context(
            local_video, config
        )
    failures = [r for r in val_results if not r.passed]
    db = Database(os.path.join(scratch, config.output.db_name))
    db.add_video(
        video_id,
        local_video,
        "failed" if failures else "processed",
        [r.model_dump() for r in val_results],
    )

    def _fail(rc: int) -> int:
        # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort the
        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write +
push
        # a status="failed" report.json/intervals.csv (0 segments) so the control plane's bucket
        # truth (submit-cache probe, restart-recovery) and the alpha-user result surface see an
        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup must never
        # mask the original failure. THEN the M6 done-marker (real rc + status) so a remote
        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget expires.
        try:
            _serialize_and_push_outputs(
                scratch,
                args.out_uri,
                video_id,
                [],
                "failed",
                storage_cfg,
                str(getattr(args, "video_uri", "") or ""),
            )
        except Exception as e:  # never mask the real failure on an artifact-push hiccup
            logger.warning("[worker] could not push failure artifacts: %s", e)
        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
            )
        return rc

    segments: List[dict] = []
    segmenter_actual = None
    segmentation_ran = False
```

```
    status = "failed"
    try:
        if failures:
            print(
                "[worker] validation FAILED: "
                + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
            )
            return _fail(2)
        seg_name = args.segmenter or config.indexing.default_segmenter
        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            print(
                f"[worker] unknown segmenter: {seg_name!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return _fail(2)
        segmenter_actual = seg_name
        out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar + telemetry
        os.makedirs(out_local, exist_ok=True)

        # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM samples.
        # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
```

568. assistant (2026-07-09T06:13:21.741Z)

The code settles the mechanism, and it's reassuring: the worker **does** re-compute the checksum (`compute_video_id` on the pulled bytes) **and** runs the validators (`run_all_with_context`) **unless** `--skip-validation` was passed ? but it only *logs* on validation **FAILED** or **SKIPPED**, so a **silent pass emits nothing**. Since we saw neither "SKIPPED" nor "FAILED", validation most likely ran and passed. The definitive proof is whether the CP passed `--skip-validation` to this execution ? let me check the dispatched args.

569. user (2026-07-09T06:13:33.472Z)

```
=== worker exec dpssz dispatched ARGS ? is --skip-validation present? ===

=== CP dispatch log ? the full worker command it built (skip-validation?) ===
```

570. assistant (2026-07-09T06:13:50.010Z)

Let me pull the full execution spec as JSON to find the actual dispatched command/args.

571. user (2026-07-09T06:13:56.412Z)

```
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    c=d['spec']['template']['spec']['template']['spec']['containers'][0]
      ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^
KeyError: 'template'
```

572. user (2026-07-09T06:14:08.464Z)

```
/spec/template/spec/containers[0]/args[0] = 'infer'
/spec/template/spec/containers[0]/args[1] = '--video-uri'
/spec/template/spec/containers[0]/args[2] = 'gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4'
/spec/template/spec/containers[0]/args[3] = '--out-uri'
/spec/template/spec/containers[0]/args[4] = 'gs://khelsutra-rally-serving'
/spec/template/spec/containers[0]/args[5] = '--done-uri'
/spec/template/spec/containers[0]/args[6] = 'gs://khelsutra-rally-
serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done'
/spec/template/spec/containers[0]/args[7] = '--segmenter'
/spec/template/spec/containers[0]/args[8] = 'wasb_hybrid'
```

```
/spec/template/spec/containers[0]/args[9] = '--set'
/spec/template/spec/containers[0]/args[10] = 'indexing.skip_ai_handoff=true'
/spec/template/spec/containers[0]/args[11] = '--set'
/spec/template/spec/containers[0]/args[12] = 'indexing.wasb.decode_backend=nvdec_resident'
/spec/template/spec/containers[0]/args[13] = '--set'
/spec/template/spec/containers[0]/args[14] = 'indexing.wasb.batch_size=64'
/spec/template/spec/containers[0]/args[15] = '--set'
/spec/template/spec/containers[0]/args[16] = 'indexing.wasb.prefetch_batches=4'
/spec/template/spec/containers[0]/args[17] = '--set'
/spec/template/spec/containers[0]/args[18] = 'deployment.compute_target=local_gpu'
/spec/template/spec/containers[0]/args[19] = '--set'
/spec/template/spec/containers[0]/args[20] = 'indexing.detector_impl=native'
/spec/template/spec/containers[0]/image = 'asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-
worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb'
```

573. assistant (2026-07-09T06:14:57.747Z)

Settled ? and it's good news. Here are the **exact args the CP dispatched to the worker** (new image `1838c07f` ?):

```
infer --video-uri gs://?/GX010155.MP4 --out-uri ? --done-uri ?
--segmenter wasb_hybrid
--set indexing.skip_ai_handoff=true
--set indexing.wasb.decode_backend=nvdec_resident
--set indexing.wasb.batch_size=64 --set indexing.wasb.prefetch_batches=4
--set deployment.compute_target=local_gpu
--set indexing.detector_impl=native
```

There is no `--skip-validation` flag. Per the worker code (`__main__.py:290`), validation is skipped *only* when that flag is present ? otherwise it runs the full `validator_registry.run_all_with_context`. So **validation did run** on the worker; it wasn't cached or skipped. It just **logs nothing on a pass** (only on FAILED or SKIPPED), which is exactly why the SPA looked like it jumped straight to segmenting.

So all steps *are* executing ? the confusion was silent-pass logging, not a skipped stage:

stage	where	status
checksum	worker (`compute_video_id` on pulled bytes)	? runs (CP trusts submit-time id #524; worker re-hashes)
validation	worker (`run_all_with_context`, all 6 validators)	? ran + **passed** (no `--skip-validation`; no FAILED log; segmentation proceeded)
segmentation	worker, L4, **`decode_backend=nvdec_resident`** + `detector_impl=native`, batch 64	? running now
stitch/compile	separate L4 compile exec (NVENC, #531)	? after segmentation

To give you **hard proof** (not just "no skip flag"), the worker persists each validator's result ? `db.add_video(..., [r.model_dump() for r in val_results])` ? into the pushed `report.json`. When the run finishes I'll pull it and show the **actual validator scores** (blur/scene-cut/court/relevance/resolution/format) alongside the per-stage timing, so every stage has explicit evidence. The completion watcher is still armed.

Bonus signals from the args: **NVDEC-resident decode is engaged on the L4** (`nvdec_resident`, batch 64 / prefetch 4) ? the fast GPU decode path. I'll confirm NVENC on the compile step too.

574. user (2026-07-09T06:14:58.056Z)

pls make sure to add a clear log messages for this and also enough for runlogs to capture the time taken for validation etc.

575. assistant (2026-07-09T06:16:00.072Z)

That's a real observability gap worth fixing properly ? validation (and every stage) should announce itself and record its wall-time in the runlog, not pass silently. Let me implement clear per-stage log lines + timing in the worker and persist them into the pushed report so runlogs capture them. I'll do it on a branch from `origin/master`.

576. user (2026-07-09T06:16:03.083Z)

Task #18 created successfully: Add per-stage timing + clear logs to worker

577. user (2026-07-09T06:16:06.738Z)

[Request interrupted by user]

578. user (2026-07-09T06:16:06.783Z)

there was a side car mechanism to tell the GPU progress, can you tail it for me

579. assistant (2026-07-09T06:16:45.699Z)

Yes ? that's the #534 telemetry black-box (`RunTelemetry` ? GPU util/VRAM/temp + fps/velocity samples, driven off the detector's progress %). Let me find how it surfaces (stdout vs a GCS-streamed sidecar) and whether there are live samples to tail right now.

580. user (2026-07-09T06:16:57.793Z)

```
=== #534 telemetry: what/where does the worker write + is it logged or GCS-streamed? ===
1:"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
43:from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
44:from backend.utils.hashing import compute_video_id
45:from backend.utils.run_telemetry import RunTelemetry
109:                # control plane re-hydrating from outputs/<md5>/ restores the video row
125:def _serialize_and_push_outputs(
134:    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and push
both to
135:    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
153:    pushed: List[str] = []
155:        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
157:        pushed.append(uri)
158:    return pushed
248:    except Exception as e:  # never fail a good run because the marker push hiccuped
311:        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write
+ push
314:        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup must
never
318:            _serialize_and_push_outputs(
327:        except Exception as e:  # never mask the real failure on an artifact-push hiccup
328:            logger.warning("[worker] could not push failure artifacts: %s", e)
356:        out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar + telemetry
359:        # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM
samples.
360:        # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
361:        telem: Optional[RunTelemetry] = None
363:            telem = RunTelemetry(
364:                os.path.join(out_local, "run_telemetry.jsonl"),
367:        except Exception:  # noqa: BLE001 - telemetry must never break a production run

=== live worker logs ? any telemetry/GPU-progress samples right now? (06:16:53Z) ===
2026-07-09T06:08:28.488355Z     06:08:28 [INFO] backend.pipeline.detectors.native_wasb_runner:
Running native WASB inference (device=cuda) on in.mp4 ...
2026-07-09T06:08:28.480294Z     06:08:28 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB detector env OK: torch 2.6.0+cu124 cuda True
2026-07-09T06:08:16.932290Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB: detector_impl=native ? Linux-native runner (no WSL).
```

```
2026-07-09T06:08:16.837167Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will be emitted as rallies;
no gemini handoff, no API key needed.
2026-07-09T06:06:58.531863Z     [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> /tmp/rally-worker-jrb2w9ys/in.mp4
```

581. assistant (2026-07-09T06:17:34.250Z)

Here's the mechanism: the #534 telemetry writes to a **local** file on the worker ? `<scratch>/outputs/<video_id>/run_telemetry.jsonl` ? and it's only **pushed to GCS at the end** ("the final push will mirror it"). The worker also isn't logging samples to stdout (nothing since 06:08:28 ? wasb inference has run ~8.5 min silently). So the question is whether the frame-% heartbeat is streamed anywhere live I can tail. Let me check GCS for a live-updating progress/telemetry object + what the CP actually sees.

582. user (2026-07-09T06:17:51.740Z)

```
=== GCS: any live-updating heartbeat/telemetry/progress object for this run? (06:17:43Z) ===
ERROR: (gcloud.storage.ls) unrecognized arguments: -lr

To search the help text of gcloud commands, run:
  gcloud help -- SEARCH_TERMS

=== CP job.progress content ? what % does the CP report (heartbeat)? ===
2026-07-09T06:04:19.399614Z     06:04:19 [INFO] backend.orchestration: [API] Video passed
checks. Segmenting and indexing...

=== how does the heartbeat/telemetry surface? (run_telemetry + heartbeat GCS write) ===
5:that happens the only forensic evidence is whatever was last flushed to disk. This module
keeps a
6:tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem is a
17:  existing progress callback; fps is derived from the delta since the previous snapshot.
52:def _num(s: str) -> Optional[float]:
59:def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
75:        ).stdout.strip()
99:def system_stats() -> Optional[Dict[str, Any]]:
117:def gpu_name() -> Optional[str]:
118:    """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
129:        ).stdout
136:    """A small rotating JSONL runlog of execution velocity + machine health.
138:    Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
msg)`` at
143:    def __init__(
179:    def snapshot(
211:    def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
```

583. assistant (2026-07-09T06:18:26.018Z)

Let me re-check GCS with correct syntax (is the telemetry JSONL actually landing mid-run?) and whether the worker flushes it periodically vs only at the end.

584. user (2026-07-09T06:18:38.253Z)

```
=== GCS outputs/<vid>/ mid-run ? is run_telemetry.jsonl streaming? (06:18:33Z) ===
ERROR: (gcloud.storage.ls) One or more URLs matched no objects.
--- any run_telemetry anywhere for this run ---

=== worker telemetry usage (lines 356-420): periodic GCS flush or end-only? ===
1:        out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar + telemetry
4:        # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM
samples.
5:        # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
6:        telem: Optional[RunTelemetry] = None
```

```
8:            telem = RunTelemetry(
9:                os.path.join(out_local, "run_telemetry.jsonl"),
12:        except Exception:  # noqa: BLE001 - telemetry must never break a production run
13:            telem = None
19:        # Configurable sampling for sidecar upload + telemetry snapshots (#534).
21:        # and nvidia-smi-using snapshot are rate-limited. 0/negative = every callback.
```

585. assistant (2026-07-09T06:19:06.971Z)

Confirmed: nothing in GCS yet ? the telemetry JSONL is buffered **locally on the worker** and only lands in GCS at the final push. But #534 also has a "live detector progress heartbeat" that the CP surfaces as job-progress %, so the worker must write *something* live. Let me hunt for that heartbeat (a GCS progress marker) ? that's what would actually be tailable.

586. user (2026-07-09T06:19:17.087Z)

```
=== _dispatch area ? live heartbeat/progress marker for dispatch 9d6f3ca2? (06:19:13Z) ===
       107  2026-07-09T06:18:48Z  gs://khelsutra-rally-
serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done

=== worker code: where is progress/heartbeat written live (GCS) during the run? ===
182:def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
264:        return _dispatch_remote(compute, args)
288:    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
381:                raw = getattr(idx_cfg, "detector_progress_sample_sec", 15.0)
388:        def _progress(s: str) -> None:
389:            print(f"[worker] progress: {s}")
390:            # Write sidecar for cloud/L4 poll to surface live detector progress in job
status (#534).
391:            # CP will read outputs/<video_id>/*_detector_progress.json during poll.
396:            pfile = os.path.join(out_local, f"{video_id}_detector_progress.json")
414:                    pass  # best-effort live heartbeat
434:            progress=_progress,

=== native_wasb_runner: does the detector emit a progress callback + how often? ===
108:    log_every_batches: int = 50  # emission rate of "D/T (pct%)" progress lines from
wasb_infer
135:            log_every_batches=int(w.log_every_batches or 50),
284:        progress: Optional[Callable[[str], None]] = None,
356:        if self.cfg.log_every_batches and self.cfg.log_every_batches > 0:
357:            argv += ["--log-every-batches", str(self.cfg.log_every_batches)]
368:            if progress is None:
372:                # Live progress path (#534): stream stdout so we can parse the periodic
373:                # "done/total (pct%) ..." lines already emitted by wasb_infer.py and call
374:                # the callback. This gives the SPA a real heartbeat during the long GPU pass
388:                            # Parse existing wasb_infer progress lines for detector
heartbeat.
```

587. assistant (2026-07-09T06:20:04.677Z)

Important development: the **`.done` marker was just written (06:18:48Z)** ? so the **segmentation exec finished** (~10 min of wasb inference: 06:08:28 ? 06:18:48). And I found the #534 heartbeat mechanism: the worker prints `[worker] progress: done/total (pct%)` to stdout **and** writes `outputs/<vid>/<vid>_detector_progress.json` to GCS every ~15 s. But I saw neither. Let me check whether the heartbeat actually fired, pull whatever it captured, and see the next (compile) step.

588. user (2026-07-09T06:20:22.504Z)

```
=== did the #534 heartbeat fire? worker stdout progress lines ===
2026-07-09T06:14:32.881468Z     time="09/07/2026 06:14:32.880185" severity=INFO
message="Starting a garbage collection run." mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:08:16.837167Z     06:08:16 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
```

WASB in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will be emitted as rallies;
no gemini handoff, no API key needed.
2026-07-09T06:06:58.531863Z [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> /tmp/rally-worker-jrb2w9ys/in.mp4
2026-07-09T06:04:33.427421Z time="09/07/2026 06:04:32.882619" severity=INFO message="File
system has been successfully mounted." mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.427150Z time="09/07/2026 06:04:32.882278" severity=INFO
message="Mounting file system \"khelsutra-rally-serving\"..." mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426889Z time="09/07/2026 06:04:32.878966" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.426656Z time="09/07/2026 06:04:32.878644" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426419Z time="09/07/2026 06:04:32.878571" severity=INFO
message="GetStorageLayout -> (khelsutra-rally-serving) 112 msec" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.426218Z time="09/07/2026 06:04:32.766348" severity=INFO message="Calling
GetStorageLayout request for \"projects/_/buckets/khelsutra-rally-serving/storageLayout\" with
deadline=30s" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.425990Z time="09/07/2026 06:04:32.766319" severity=INFO
message="GetStorageLayout <- (khelsutra-rally-serving)" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.425777Z time="09/07/2026 06:04:32.766302" severity=INFO message="Set up
root directory for bucket khelsutra-rally-serving" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.425488Z time="09/07/2026 06:04:32.766296" severity=INFO message="MRD
cache (LRU-based) enabled with maxSize=1000 (pools closed-file MRDs)" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.425208Z time="09/07/2026 06:04:32.766270" severity=INFO
message="Creating a new server...\n" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.092620Z time="09/07/2026 06:04:32.765660" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:33.092614Z time="09/07/2026 06:04:32.765389" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.092320Z time="09/07/2026 06:04:32.765329" severity=INFO
message="UserAgent = gcsfuse/3.9.2 (Go version go1.26.4) (GPN:gcsfuse-serverless)
(Cfg:0:0:0:1:0:0) (mount-id:khelsutra-rally-serving-03345eef)\n" mount-id=khelsutra-rally-
serving-03345eef
2026-07-09T06:04:33.091959Z time="09/07/2026 06:04:32.765297" severity=INFO
message="Creating Storage handle..." mount-id=khelsutra-rally-serving-03345eef
2026-07-09T06:04:32.759094Z time="09/07/2026 06:04:32.758114" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-03345eef "Full Config"="&{AppName:serverless CacheDir:
CloudProfiler:{AllocatedHeap:true Cpu:true Enabled:false Goroutines:false Heap:true
Label:gcsfuse-0.0.0 Mutex:false ServiceName:gcsfuse} Debug:{ExitOnInvariantViolation:false
Fuse:false Gcs:false LogMutex:false} DisableAutoconfig:false DisableListAccessCheck:true
DummyIo:{Enable:false PerMbLatency:0s ReaderLatency:0s} EnableAtomicRenameObject:true
EnableGoogleLibAuth:true EnableHns:true EnableNewReader:true EnableStandardSymlinks:true
EnableTypeCacheDeprecation:true EnableUnsupportedPathSupport:true
FileCache:{CacheFileForRangeRead:false DownloadChunkSizeMb:200 EnableCrc:false
EnableExperimentalSharedChunkCache:false EnableODirect:false EnableParallelDownloads:false
ExcludeRegex: ExperimentalDisableSizeCalculationFix:false ExperimentalEnableChunkCache:false
ExperimentalParallelDownloadsDefaultOn:true IncludeRegex: MaxParallelDownloads:16 MaxSizeMb:-1
ParallelDownloadsPerFile:16 SharedCacheChunkSizeMb:8 WriteBufferSize:4194304}
FileSystem:{CongestionThreshold:0 DirMode:511 DisableParallelDirops:false
EnableKernelReader:false ExperimentalEnableDentryCache:false ExperimentalEnablePirlo:false
ExperimentalEnableReaddirplus:false ExperimentalODirect:false FileMode:438
FuseOptions:[allow_other] Gid:1000 IgnoreInterrupts:true InactiveMrdCacheSize:1000
KernelListCacheTtlSecs:0 KernelParamsFile: MaxBackground:0 MaxReadAheadKb:0
PreconditionErrors:true RenameDirLimit:0 TempDir: Uid:1000} Foreground:true
GcsAuth:{AnonymousAccess:false KeyFile: ReuseTokenFromUrl:true TokenUrl:}
GcsConnection:{BillingProject: ClientProtocol:http1 CustomEndpoint: EnableHttpDnsCache:true
ExperimentalEnableJsonRead:false ExperimentalLocalSocketAddress: GrpcConnPoolSize:1
GrpcPathStrategy:direct-path-with-fallback HttpClientTimeout:0s LimitBytesPerSec:-1
LimitOpsPerSec:-1 MaxConnsPerHost:0 MaxIdleConnsPerHost:100 SequentialReadSizeMb:200}

```
GcsRetries:{ChunkRetryDeadlineSecs:120 ChunkTransferTimeoutSecs:10
ExperimentalNonrapidFolderApiStallRetry:false MaxRetryAttempts:9223372036854775807
MaxRetrySleep:30s Multiplier:2 ReadStall:{Enable:true InitialReqTimeout:20s MaxReqTimeout:20m0s
MinReqTimeout:1.5s ReqIncreaseRate:15 ReqTargetPercentile:0.99}} ImplicitDirs:true
List:{EnableEmptyManagedFolders:false} Logging:{FilePath: Format:text
LogRotate:{BackupFileCount:10 Compress:true MaxFileSizeMb:512} Severity:INFO WireLog:}
MachineType: MetadataCache:{DeprecatedStatCacheCapacity:20460 DeprecatedStatCacheTtl:1m0s
DeprecatedTypeCacheTtl:1m0s EnableMetadataPrefetch:false EnableNonexistentTypeCache:false
ExperimentalMetadataPrefetchOnMount:disabled MetadataPrefetchEntriesLimit:5000
MetadataPrefetchMaxWorkers:10 NegativeTtlSecs:5 StatCacheMaxSizeMb:34 TtlSecs:60
TypeCacheMaxSizeMb:4} Metrics:{BufferSize:256 CloudMetricsExportIntervalSecs:0
ExperimentalEnableGrpcMetrics:true PrometheusPort:0 StackdriverExportInterval:0s
UseNewNames:false Workers:3} Mrd:{PoolSize:4} OnlyDir: Profile: Read:{BlockSizeMb:16
EnableBufferedRead:false GlobalMaxBlocks:40 InactiveStreamTimeout:10s MaxBlocksPerHandle:20
MinBlocksPerHandle:4 RandomSeekThreshold:3 StartBlocksPerHandle:1}
Trace:{Exporters:[gcpexporter] ProjectId: SamplingRatio:0}
WorkloadInsight:{ForwardMergeThresholdMb:0 OutputFile: Visualize:false} Write:{BlockSizeMb:32
CreateEmptyFile:false EnableRapidAppends:true EnableStreamingWrites:true
FinalizeFileForRapid:false GlobalMaxBlocks:4 MaxBlocksPerFile:1}}"
2026-07-09T06:04:32.758666Z     time="09/07/2026 06:04:32.758076" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-03345eef "CLI Flags"="map[app-name:serverless dir-
mode:777 file-mode:666 foreground:true gid:1000 ignore-interrupts:true implicit-dirs:true log-
format:text o:[allow_other] uid:1000]"
2026-07-09T06:04:32.674322Z     GCSFuse is mounted with bucket khelsutra-rally-serving. Enable
GCSFuse logging with mount options for more logs

=== telemetry + progress sidecar in GCS now (post-.done)? ===
      1019  2026-07-09T06:18:47Z  gs://khelsutra-rally-
serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/intervals.csv
      4080  2026-07-09T06:18:48Z  gs://khelsutra-rally-
serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/report.json
       116  2026-07-09T06:18:48Z  gs://khelsutra-rally-
serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/run_telemetry.jsonl
TOTAL: 3 objects, 5215 bytes (5.09kiB)

=== compile exec dispatched (stitch)? + CP state ===
NAME                CREATION_TIMESTAMP           RUNNING  SUCCEEDED_COUNT
rally-worker-dpssz  2026-07-09T06:04:20.821211Z  0        1
rally-worker-98gqk  2026-07-08T09:30:32.225160Z  0        1
rally-worker-gvqzv  2026-07-08T09:17:35.023374Z  0        1
```

589. assistant (2026-07-09T06:20:52.013Z)

Now I have the picture ? and it's a real finding tied to exactly what you flagged. The **segmentation exec finished at 06:18:47** and pushed 3 files, but look at the telemetry:

- `intervals.csv` (1 KB), `report.json` (4 KB), **`run_telemetry.jsonl` ? only 116 bytes** (basically empty ? no GPU/velocity samples over the ~10-min run).
- **Zero `[worker] progress:` lines** in stdout the entire 06:08:28?06:18:48 wasb pass, and **no `<vid>_detector_progress.json`** ever appeared in GCS.

So the #534 **live heartbeat + telemetry didn't fire on this native-L4 path** ? the GPU pass ran ~10 min completely dark. There was nothing to tail. Let me pull what little landed (and the validation proof from `report.json`).

590. user (2026-07-09T06:21:05.985Z)

```
=== run_telemetry.jsonl (the whole 116 bytes) ===
{"meta":{"label":"l4-infer-e2e14417","started":"2026-07-09T06:08:15","pid":1,"gpu_name":"NVIDIA
L4","cpu_count":8}}

=== report.json ? validation results (PROOF validation ran) + segments + timing ===
top keys: ['video_id', 'status', 'segment_count', 'video_uri', 'segments']
VALIDATION: (no validation block found)
SEGMENTS/RALLIES: 53
```

591. assistant (2026-07-09T06:21:55.053Z)

Here's the honest answer: **there was nothing to tail** ? the sidecar captured only its header and never took a single sample.

`run_telemetry.jsonl` in full (116 bytes):
```json
{"meta":{"label":"l4-infer-e2e14417","started":"2026-07-09T06:08:15","pid":1,"gpu_name":"NVIDIA L4","cpu_count":8}}
```

That's the entire file ? GPU name + CPU count, and **zero snapshots**. Combined with **no `[worker] progress:` lines** in stdout and **no `<vid>_detector_progress.json`** ever written to GCS, the conclusion is clear:

> ?? **The #534 live progress heartbeat + telemetry did not fire on the native-L4 wasb path.** The snapshots are "driven from detector %" via a progress callback that was never invoked, so the ~10-min GPU pass ran completely dark ? no live progress, no GPU-util/VRAM/fps samples.

What the run did produce (segmentation finished 06:18:47):
- **53 rallies** detected (`segment_count=53`) in **~10m19s** on the L4 (`decode_backend=nvdec_resident`, `detector_impl=native`).
- `report.json` keys: `video_id, status, segment_count, video_uri, segments` ? **note: no validation block**. So validation ran (proven by the missing `--skip-validation` flag), but its results only went to the worker's ephemeral DB, **not** the pushed runlog.

This is concrete evidence for exactly the change you asked for ? and it's bigger than just "log validation." I'll fold all of it into the worker observability fix:
1. **Clear per-stage log lines** (checksum / validation start+result+scores / segmentation) ? visible even on a pass.
2. **Per-stage wall-clock timing** persisted into `report.json` (so the runlog captures time-per-stage).
3. **Fix the dead heartbeat/telemetry** ? make the detector progress callback actually fire on the native path so `run_telemetry.jsonl` gets real GPU samples and the SPA gets a live %.

The compile/stitch (NVENC) exec should dispatch next; my completion watcher is still armed for the reel. Want me to proceed with that worker-observability change now (branch off `origin/master`), and should it include the heartbeat fix, or keep that as a separate follow-up?

592. user (2026-07-09T06:21:57.436Z)

pls spinup a chip for the log work rather than doing yourself and be focussed on the production eyeballign

pls add this gap also in the validation gap work and make a single chip fix these

593. assistant (2026-07-09T06:23:08.150Z)

Good call ? I'll delegate the fix and keep my eyes on production. Creating one chip that covers **both** gaps (the validation visibility/timing *and* the dead L4 heartbeat/telemetry), and dropping my internal task for it.

594. user (2026-07-09T06:23:09.167Z)

Updated task #18 deleted

595. user (2026-07-09T06:23:27.760Z)

Noted (position 1, task_id: task_f925e323). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_f925e323 "Fix worker per-stage logs/timing + dead L4 heartbeat". Continue your current work.

596. user (2026-07-09T06:23:42.655Z)

```
=== PRODUCTION EYEBALL 06:23:33Z ===
--- compile/stitch exec dispatched yet? ---
NAME                CREATION_TIMESTAMP           RUNNING  SUCCEEDED_COUNT
rally-worker-fkvqp  2026-07-09T06:21:06.775581Z  1
rally-worker-dpssz  2026-07-09T06:04:20.821211Z  0        1
rally-worker-98gqk  2026-07-08T09:30:32.225160Z  0        1
--- CP recent: ingest 53 rallies / compile dispatch / OOM? ---
--- reel landed? ---
ERROR: (gcloud.storage.ls) One or more URLs matched no objects.
```

597. assistant (2026-07-09T06:24:05.614Z)

Done ? chip **`task_f925e323`** created covering **both** gaps (validation visibility/timing **and** the dead L4 heartbeat/telemetry); my internal task dropped so I stay on production.

Production update: the run advanced past segmentation ? a **new exec `rally-worker-fkvqp` is running** (dispatched 06:21:06), which is the **compile/stitch on the L4 (NVENC)**. So: dpssz did detect (53 rallies, ~10 min) ? CP ingested ? fkvqp is now stitching. Let me eyeball the NVENC stitch in flight.

598. user (2026-07-09T06:24:15.437Z)

```
=== compile exec rally-worker-fkvqp ? NVENC / stitch / #535 range-read / timing (06:24:11Z) ===
2026-07-09T06:23:55.286975Z     [worker] compile: uploading reel
2026-07-09T06:23:55.286536Z     06:23:55 [INFO] backend.pipeline.stitcher.compiler: Dynamic
highlight reel saved (single-pass): /tmp/rally-compile-x79bspvg/gx010155-ks-highlight-reel.mp4
(1940.6 MB)
2026-07-09T06:21:37.381140Z     06:21:37 [INFO] backend.pipeline.stitcher.compiler: [compile]
single-pass stitch: fusing 19 clip(s) + watermark in one ffmpeg pass.
2026-07-09T06:21:37.381125Z     06:21:37 [INFO] backend.pipeline.stitcher.compiler: [compile]
single-pass: encoding with h264_nvenc (hardware).
2026-07-09T06:21:35.276075Z     06:21:35 [INFO] backend.pipeline.stitcher.compiler: Segment
#19/19 [742.61s - 747.88s]: Padded end (748.38s) exceeds video duration. Clamping to 747.91s
(end padding partially applied).
2026-07-09T06:21:35.275922Z     06:21:35 [INFO] backend.pipeline.stitcher.compiler: Source video
duration: 747.91s
2026-07-09T06:21:33.374386Z     06:21:33 [INFO] backend.storage: [storage] using GCSFuse mounted
path for gcs source (no full download): /gcs/videos/uploads/avidullu-cuj-run-1/GX010155.MP4
2026-07-09T06:21:33.374342Z     06:21:33 [INFO] backend.orchestration: [compile] rally_ranges
video_id=e2e144175c8d05451903ae44a9ce0aa0 count=19 ranges=[(5.272, 11.111), (49.55, 54.555),
(71.672, 80.781), (122.089, 138.939), (205.472, 213.947), (241.174, 246.646), (360.06, 365.765),
(377.377, 383.85), (413.78, 421.821), (431.965, 438.638), (455.021, 460.327), (468.501,
476.609), (507.907, 513.48), (521.321, 529.629), (535.034, 540.907), (555.989, 568.468),
(623.99, 634.567), (697.363, 706.005), (742.609, 747.88)] source_ref=gs://khelsutra-rally-
serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4
2026-07-09T06:21:33.374221Z     06:21:33 [INFO] backend.orchestration: [compile] stitching
video_id=e2e144175c8d05451903ae44a9ce0aa0 rendition=original output=gx010155-ks-highlight-
reel.mp4
2026-07-09T06:21:33.374219Z     [worker] compile: stitching
2026-07-09T06:21:32.611216Z     [worker] compile: video_id=e2e144175c8d05451903ae44a9ce0aa0
clips=19 encoder=h264_nvenc rendition=original out=gx010155-ks-highlight-reel.mp4
ranges=[(5.272, 11.111), (49.55, 54.555), (71.672, 80.781), (122.089, 138.939), (205.472,
213.947), (241.174, 246.646), (360.06, 365.765), (377.377, 383.85), (413.78, 421.821), (431.965,
438.638), (455.021, 460.327), (468.501, 476.609), (507.907, 513.48), (521.321, 529.629),
(535.034, 540.907), (555.989, 568.468), (623.99, 634.567), (697.363, 706.005), (742.609,
747.88)]
2026-07-09T06:21:25.346241Z     time="09/07/2026 06:21:25.345730" severity=INFO message="File
system has been successfully mounted." mount-id=khelsutra-rally-serving-8c87a648
2026-07-09T06:21:25.345706Z     time="09/07/2026 06:21:25.345298" severity=INFO
message="Mounting file system \"khelsutra-rally-serving\"..." mount-id=khelsutra-rally-
serving-8c87a648
2026-07-09T06:21:25.268251Z     time="09/07/2026 06:21:25.218495" severity=INFO message="Set up
root directory for bucket khelsutra-rally-serving" mount-id=khelsutra-rally-serving-8c87a648
2026-07-09T06:21:25.214779Z     time="09/07/2026 06:21:25.213915" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-8c87a648 "Full Config"="&{AppName:serverless CacheDir:
```

CloudProfiler:{AllocatedHeap:true Cpu:true Enabled:false Goroutines:false Heap:true
Label:gcsfuse-0.0.0 Mutex:false ServiceName:gcsfuse} Debug:{ExitOnInvariantViolation:false
Fuse:false Gcs:false LogMutex:false} DisableAutoconfig:false DisableListAccessCheck:true
DummyIo:{Enable:false PerMbLatency:0s ReaderLatency:0s} EnableAtomicRenameObject:true
EnableGoogleLibAuth:true EnableHns:true EnableNewReader:true EnableStandardSymlinks:true
EnableTypeCacheDeprecation:true EnableUnsupportedPathSupport:true
FileCache:{CacheFileForRangeRead:false DownloadChunkSizeMb:200 EnableCrc:false
EnableExperimentalSharedChunkCache:false EnableODirect:false EnableParallelDownloads:false
ExcludeRegex: ExperimentalDisableSizeCalculationFix:false ExperimentalEnableChunkCache:false
ExperimentalParallelDownloadsDefaultOn:true IncludeRegex: MaxParallelDownloads:16 MaxSizeMb:-1
ParallelDownloadsPerFile:16 SharedCacheChunkSizeMb:8 WriteBufferSize:4194304}
FileSystem:{CongestionThreshold:0 DirMode:511 DisableParallelDirops:false
EnableKernelReader:false ExperimentalEnableDentryCache:false ExperimentalEnablePirlo:false
ExperimentalEnableReaddirplus:false ExperimentalODirect:false FileMode:438
FuseOptions:[allow_other] Gid:1000 IgnoreInterrupts:true InactiveMrdCacheSize:1000
KernelListCacheTtlSecs:0 KernelParamsFile: MaxBackground:0 MaxReadAheadKb:0
PreconditionErrors:true RenameDirLimit:0 TempDir: Uid:1000} Foreground:true
GcsAuth:{AnonymousAccess:false KeyFile: ReuseTokenFromUrl:true TokenUrl:}
GcsConnection:{BillingProject: ClientProtocol:http1 CustomEndpoint: EnableHttpDnsCache:true
ExperimentalEnableJsonRead:false ExperimentalLocalSocketAddress: GrpcConnPoolSize:1
GrpcPathStrategy:direct-path-with-fallback HttpClientTimeout:0s LimitBytesPerSec:-1
LimitOpsPerSec:-1 MaxConnsPerHost:0 MaxIdleConnsPerHost:100 SequentialReadSizeMb:200}
GcsRetries:{ChunkRetryDeadlineSecs:120 ChunkTransferTimeoutSecs:10
ExperimentalNonrapidFolderApiStallRetry:false MaxRetryAttempts:9223372036854775807
MaxRetrySleep:30s Multiplier:2 ReadStall:{Enable:true InitialReqTimeout:20s MaxReqTimeout:20m0s
MinReqTimeout:1.5s ReqIncreaseRate:15 ReqTargetPercentile:0.99}} ImplicitDirs:true
List:{EnableEmptyManagedFolders:false} Logging:{FilePath: Format:text
LogRotate:{BackupFileCount:10 Compress:true MaxFileSizeMb:512} Severity:INFO WireLog:}
MachineType: MetadataCache:{DeprecatedStatCacheCapacity:20460 DeprecatedStatCacheTtl:1m0s
DeprecatedTypeCacheTtl:1m0s EnableMetadataPrefetch:false EnableNonexistentTypeCache:false
ExperimentalMetadataPrefetchOnMount:disabled MetadataPrefetchEntriesLimit:5000
MetadataPrefetchMaxWorkers:10 NegativeTtlSecs:5 StatCacheMaxSizeMb:34 TtlSecs:60
TypeCacheMaxSizeMb:4} Metrics:{BufferSize:256 CloudMetricsExportIntervalSecs:0
ExperimentalEnableGrpcMetrics:true PrometheusPort:0 StackdriverExportInterval:0s
UseNewNames:false Workers:3} Mrd:{PoolSize:4} OnlyDir: Profile: Read:{BlockSizeMb:16
EnableBufferedRead:false GlobalMaxBlocks:40 InactiveStreamTimeout:10s MaxBlocksPerHandle:20
MinBlocksPerHandle:4 RandomSeekThreshold:3 StartBlocksPerHandle:1}
Trace:{Exporters:[gcpexporter] ProjectId: SamplingRatio:0}
WorkloadInsight:{ForwardMergeThresholdMb:0 OutputFile: Visualize:false} Write:{BlockSizeMb:32
CreateEmptyFile:false EnableRapidAppends:true EnableStreamingWrites:true
FinalizeFileForRapid:false GlobalMaxBlocks:4 MaxBlocksPerFile:1}}"
2026-07-09T06:21:25.214440Z time="09/07/2026 06:21:25.213884" severity=INFO message="GCSFuse
Config" mount-id=khelsutra-rally-serving-8c87a648 "CLI Flags"="map[app-name:serverless dir-
mode:777 file-mode:666 foreground:true gid:1000 ignore-interrupts:true implicit-dirs:true log-
format:text o:[allow_other] uid:1000]"
2026-07-09T06:21:25.148319Z GCSFuse is mounted with bucket khelsutra-rally-serving. Enable
GCSFuse logging with mount options for more logs

599. assistant (2026-07-09T06:25:13.044Z)

This is the payoff ? every release win just confirmed live on prod, in the compile/stitch
exec:

```
06:21:32  compile: clips=19  encoder=h264_nvenc  rendition=original
06:21:33  [storage] using GCSFuse mounted path for gcs source (NO full download):
/gcs/?/GX010155.MP4   ? #535 ?
06:21:33  source duration 747.91s · rally_ranges count=19
06:21:37  [compile] single-pass: encoding with h264_nvenc (hardware)
? #529 NVENC, no CPU fallback ?
06:21:37  [compile] single-pass stitch: fusing 19 clip(s) + watermark in ONE ffmpeg pass
? #531 single-pass ?
06:23:55  Dynamic highlight reel saved (single-pass): ?gx010155-ks-highlight-reel.mp4 (1940.6
MB)
06:23:55  compile: uploading reel
```

```

## Live prod per-stage timing ? GX010155 (7.84 GiB, 747.9 s / ~12.5-min source)

| stage | where | wall time | evidence |
|---|---|---|---|
| POST /api/process ? 202 | CP | 06:04:19 | trusts video_id (#524), **skips CP acquire (#539)** |
| pull source ? /tmp | L4 detect | ~2m25s (06:04:33?06:06:58) | 7.84 GiB for the decode pass |
| checksum + **validation** | L4 worker | ~78 s window (incl. torch load) | ran (no `--skip-validation`) ? but silent (the chip fixes this) |
| **segmentation** (wasb, `nvdec_resident`) | L4 | **~10m19s** (06:08:28?06:18:47) | **53 rallies** detected |
| ingest + dispatch compile | CP?L4 | 06:18:48?06:21:06 | |
| **stitch** (single-pass **h264_nvenc**, #531) | L4 | **~138 s** (06:21:37?06:23:55) | 19 clips + watermark, one pass ? **1940.6 MB** reel |

**End-to-end ? 19.5 min**, dominated by segmentation (~10 min) + the 7.84 GiB pull (~2.4 min); the **stitch itself is only ~138 s** for a ~1.9 GB 5.3K reel ? that's the #531 single-pass + NVENC win (a two-pass encode would've been ~2×).

**And the CP downgrade held perfectly:** a **7.84 GiB** clip (larger than the 6.52 GiB one that forced the old 4/16) sailed through on **2 vCPU / 8 GiB with zero OOM** ? because #539 (no CP acquire) + #535 (worker mount-reads) kept both surfaces light. The 4/16?2/8 downgrade is validated on the worst case. ?

The reel is uploading now; the watcher will confirm it lands in `highlights/`. The one thing that *didn't* work ? the live GPU progress/telemetry heartbeat ? is now covered by chip `task_f925e323`.

### 600. user (2026-07-09T06:25:16.200Z)

<system-reminder>
The user started your suggested background task task_f925e323 ("Fix worker per-stage logs/timing + dead L4 heartbeat") in a separate local session. It is running independently. You will be notified here when it ends.
</system-reminder>

the stitching is running now

### 601. assistant (2026-07-09T06:25:47.320Z)

Let me get the exact current state ? whether this is the same compile finishing/mirroring or a fresh stitch (e.g., an SPA-triggered compile of your selection), plus where the reel landed.

### 602. user (2026-07-09T06:26:10.002Z)

```
=== worker execs ? is fkvqp done? any NEW compile running? (06:25:56Z) ===
NAME   CREATION_TIMESTAMP   RUNNING SUCCEEDED_COUNT COMPLETION_TIME
rally-worker-fkvqp 2026-07-09T06:21:06.775581Z 0   1
2026-07-09T06:24:36.560683Z
rally-worker-dpssz 2026-07-09T06:04:20.821211Z 0   1
2026-07-09T06:18:55.014817Z
rally-worker-98gqk 2026-07-08T09:30:32.225160Z 0   1
2026-07-08T09:36:13.096866Z
rally-worker-gvqzv 2026-07-08T09:17:35.023374Z 0   1
2026-07-08T09:29:57.796671Z

=== reel(s) in GCS for this video (highlights/ + any recent .mp4) ===
2034852230 2026-07-09T06:24:29Z gs://khelsutra-rally-
serving/highlights/e2e144175c8d05451903ae44a9ce0aa0/gx010155-ks-highlight-reel.mp4
TOTAL: 1 objects, 2034852230 bytes (1.90GiB)

=== CP recent ? any 2nd /api/compile (SPA-triggered stitch)? (06:26:06Z) ===
```
```

```
2026-07-09T06:24:32.421373Z     06:24:32 [INFO] backend.orchestration: [COMPILE] cloud_run
stitched: execution=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-fkvqp out=gs://khelsutra-rally-
serving/highlights/e2e144175c8d05451903ae44a9ce0aa0/gx010155-ks-highlight-reel.mp4
video_id=e2e144175c8d05451903ae44a9ce0aa0 rendition=original
2026-07-09T06:24:32.421308Z     06:24:32 [INFO] compute.cloud_run: cloud-run: COMPILE job=rally-
worker done (video_id=e2e144175c8d05451903ae44a9ce0aa0 rc=0)
2026-07-09T06:24:32.421282Z     06:24:32 [INFO] compute.cloud_run: cloud-run-compile: ready
after 20 poll(s) / 205s
2026-07-09T06:21:06.971900Z     06:21:06 [INFO] compute.cloud_run: cloud-run-compile: waiting
(poll_every=10s, max_wait=3900s)
2026-07-09T06:21:06.969673Z     06:21:06 [INFO] compute.cloud_run: cloud-run: dispatched COMPILE
job=rally-worker exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-fkvqp; bucket-polling gs://khelsutra-rally-
serving/_compile/f333a400b33b42e699ec89c8543aa568.done
2026-07-09T06:21:06.660589Z     06:21:06 [INFO] backend.orchestration: [compile] dispatching
stitch for video_id=e2e144175c8d05451903ae44a9ce0aa0 rendition=original -> Cloud Run Job
(out=gs://khelsutra-rally-serving, 19 clip(s))
2026-07-09T06:21:06.349822Z     INFO:     169.254.169.126:63250 - "POST /api/compile HTTP/1.1"
202 Accepted
2026-07-09T06:18:53.904970Z     INFO:     169.254.169.126:37712 - "POST /api/query HTTP/1.1" 200
OK
```

603. assistant (2026-07-09T06:26:53.194Z)

It's actually **just completed** ? the SPA was a few seconds behind the logs. Here's the confirmed close-out:

- **Compile exec `fkvqp` done** at 06:24:36 (rc=0).
- **Reel landed**: `gs://khelsutra-rally-serving/highlights/e2e14417/gx010155-ks-highlight-reel.mp4` ? **1.90 GiB**, written 06:24:29.
- CP: `[COMPILE] cloud_run stitched ? rendition=original` · `cloud-run-compile: ready after 20 poll(s) / **205s**`.

Full live prod CUJ ? GX010155 (7.84 GiB, 747.9 s source) ? ? complete

stage	wall time	result
dispatch + pull source ? /tmp	~2m25s	#524 trust-id + #539 skip-CP-acquire
checksum + validation (L4 worker)	~78 s window	ran (no `--skip-validation`), silent
segmentation (wasb `nvdec_resident`)	**~10m19s**	**53 rallies**
compile/stitch (single-pass **h264_nvenc**)	**205s** CP-measured (**~138s** actual encode)	19 clips + watermark, one pass ? **1.90 GiB** reel via **GCSFuse mount-read**
end-to-end	**~20m17s** (06:04:19 ? 06:24:36)	reel delivered

Everything the release shipped, verified live: #531 single-pass stitch ? · #529 NVENC hardware (no CPU fallback) ? · #535 compile reads source via GCSFuse mount, no full re-download ? · #539 CP skips the 7.84 GiB acquire ?.

And the headline for the deploy you just did: a **7.84 GiB** clip ? bigger than the 6.52 GiB one that forced the old 4 vCPU/16 GiB ? ran end-to-end on the **downgraded 2 vCPU / 8 GiB CP with zero OOM**. The 4/16?2/8 downgrade is validated on the worst case. ?

Two follow-ups are in flight as chips you started: `task_f925e323` (worker per-stage logs/timing + the dead L4 progress heartbeat) and the earlier `task_2e0a3b82`/#548 (Windows GCSFuse). The perf "on prod" question is answered ? the only thing that didn't work was live *visibility* into the GPU pass, which the chip covers.

604. user (2026-07-09T06:28:21.460Z)

pls add this to the runlog files and can you do a quick sharp and perf focussed comparison across the other past runlogs including the size, the rallies identitied, time and other dimensions and show a summary of where we stand currently with this setup. Also include the fact that we have a very cheap CP machine now so the total cost is dominated by an actual user doing

an actual CUJ run and no "waiting price"

605. assistant (2026-07-09T06:29:06.880Z)

On it. First let me find the runlog convention + pull the comparable perf metrics from past runs, so the comparison is grounded in real data.

606. user (2026-07-09T06:29:11.024Z)

```
=== runlogs on origin/master ===
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_compile-on-L4_2026-07-08.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md

=== also any RUNLOG/DEPLOY_REPORT elsewhere ===
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_compile-on-L4_2026-07-08.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md
```

607. assistant (2026-07-09T06:29:39.256Z)

8 past runlogs. Let me extract the comparable perf/cost metrics from them in one sweep (clip size, rallies, segmentation/stitch time, cost, CP config), then read the richest one (compile-on-L4) for detail.

608. user (2026-07-09T06:29:46.634Z)

```
========== DEPLOY_REPORT_cloud-serving_2026-07-04.md ==========
> (Ada sm_89, 24GB) · 8 vCPU / 32GB · `--no-gpu-zonal-redundancy` · scale-to-zero.
| # | Config | Phase-2 wall | vs baseline | GPU util | Parity (rally windows) |
| 3 | cu113, batch 128 + prefetch 6 | OOM | ? | ? | exceeds 24GB VRAM (`CUDA OOM, 9 GiB`) |
- **? DISCARD ? torch cu113?cu118 arch bump (config #1):** 0.8% = noise (JIT was never the wall,
- **? OPT-IN ONLY (never default) ? FP16 / TF32 / cudnn.benchmark / NVDEC:** each changes
detector
- Only after the feed is parallel does a **higher-vCPU config** pay off (more cores to feed the
GPU).
serving bucket? 404 not found
========== DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md ==========
hosted `process ? jobs ? compile ? download` CUJ against the WASB-only owned detector.
| L4 worker **Job** | `rally-worker` ? nvidia-l4, 8 vCPU / 32Gi, `--no-gpu-zonal-redundancy`,
`--max-retries 0 --task-timeout 3600`, GCS-FUSE mount `khelsutra-rally-serving`?`/gcs`,
`WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar` |
   - Wall: created 02:12:42 ? finished 02:26:27 (**~13m45s**, incl. L4 cold image pull +
```

inference).
3. `POST /api/compile {video_id, segment_ids:[1..48],
output_filename:"mahadevpura_wasb_highlights.mp4"}` ? `202 {job_id: d947a9e6?}` ? poll ?
`success`. In-process ffmpeg stitch (trim 48 clips w/ padding ? concat ? watermark) on the
CP; **`Dynamic highlight reel saved ? (166.0 MB)`**; wall ~3m26s (02:27:47?02:31:13, incl. re-
fetch of the gs:// source).
4. `GET /api/highlights/{video_id}/mahadevpura_wasb_highlights.mp4` ? the reel is served
(correct `206`/`video/mp4`/etag/size headers), and the **full 174,038,655-byte reel was
retrieved via <32 MiB Range requests and ffprobes as a VALID video: H.264 1920×1080 @ 29.97fps +
AAC, duration 244.3s** (48 rallies). ? **A single full `GET` returns edge-500** ? Cloud Run's
~32 MiB response cap vs. the `FileResponse` fixed `Content-Length` (see §5 bug 3).
- **In-process compile serializes behind process jobs** (drain
`ThreadPoolExecutor(max_workers=1)`):
  a queued compile waits while a process job holds the worker in its bucket-poll. Fine for the
alpha
   32.4 MB ? 500). App logs 200; Google Frontend returns 500. Every real reel exceeds 32 MiB.
  runtime-verified (Phase 1): gs:// ? 202 accepted; a local out-of-root path ? 400 (jail still
enforced).
- **Verified (Phase 2):** `/api/health` (exempt) ? 200; `/api/process` without a CF JWT ?
401
- One L4 execution (~14 min, 8 vCPU + 1×L4) for the CUJ. CP service holds `min-instances 1`
(small
========== DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md ==========
| L4 worker **Job** | `rally-worker` ? unchanged shape (nvidia-l4, 8 vCPU/32Gi, FUSE `khelsutra-
rally-serving`?`/gcs`) |
   compute=cloud}` ? 202 ? `validating ? dispatching (cloud_run) ? done` ? **48 play segments**
   verified in the bucket) and dispatched to the L4 ? **success** (0 segments ? correct for a
   48-segment reel compile was stitching on the CP: remote dispatch overlapped the local compile
4. **compile** ? `POST /api/compile {video_id, segment_ids:[1..48], "e2e.mp4"}` ? 202 ? done;
   measured stitch wall on rev 00005: created 10:39:32 ? finished 10:44:26 (**~4m54s**, incl.
the
     edge-500, no Range workaround), and the file **ffprobes valid**: H.264 1920×1080 @ 29.97
fps
  swallowed the real error, so the fallthrough raised the misleading private-key message.
| >32 MiB download via signed URL (#463) | ? 307 + full 174 MB single-GET + ffprobe-valid |
| Parallel drain (#466/#468) observed live | ? dispatch overlapped compile; 2 process jobs
drained concurrently |
========== DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md ==========
> Durable record of a real **alpha-user-tester** upload on the `cloud-serving` profile
(`compute_target=local_gpu` on the `rally-worker` L4 Cloud Run Job, project `gen-lang-
client-0306026826`, region `asia-southeast1`). The tester's clip **timed out and returned 0
rallies**; this run diagnoses the root cause and proves the downsample fix on the real serving
path. Honest-limits in §6.
| **Detect** | ? **timed out at the 1,800 s WASB cap ? 0 rallies** (then reported
`status:"success"` ? the silent-success bug) |
| Window / stitch / serve | ? never reached (0 segments) |
Root cause (measured): the clip is **1080p·59.94 fps·46,080 frames·12.8 min·2.12 GiB**. WASB
native detection runs at **19.1 fps** on the L4, so it needs **2,411.6 s** ? but
`indexing.wasb.timeout_sec=1800` kills it ~75 % through. **Fix proven on the same L4:** a
720p30 proxy detects in **955.9 s (47 % under the cap) ? 60 rallies.**
- **Worker:** `rally-worker` Cloud Run **Job**, image `asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-worker:latest`, **8 vCPU / 32 GiB / 1× NVIDIA L4**, task timeout
3600 s, GCSFuse mount of `gs://khelsutra-rally-serving`, SA `492532266377-compute@?`. Weights
`WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`.
| Run (exec) | Variant | Frames | Detect `gpu_active_s` | `wall_s` | Throughput | Rallies | vs
1,800 s cap |
| `8tmr7` (baseline) | 1080p60, cap off | 46,080 | 2,411.6 | 2,532 | 19.1 fps | 56 | ? 34 % over
|
| **`wh66x`** | **720p30** | 23,040 | **955.9** | 1,017 | 24.1 fps | **60** | ? 47 % under |
| `kzjw6` | 1080p30 | 23,040 | 1,301.8 | 1,386 | 17.7 fps | 57 | ? 28 % under |
- **Parity / quality:** rally counts stable across variants ? **56 ? 57 ? 60** (no collapse).
Rally-level, not byte-level (cross-config detection is sub-pixel-different by design).
- **Decode-bound proof:** 720p30 vs 1080p30 at **identical 23,040 frames** ? 720p30 **36 %
faster** (24.1 vs 17.7 fps); same GPU forward pass, so the delta is CPU decode of smaller
frames. L4 measured **<1 % utilized** ? see #349.

N/A for this run ? all four executions were **detection-only** (`skip_ai_handoff=true`, no in-process stitch), so no reel artifact was produced. The 60-rally 720p30 result is the artifact of record; reel compilation from the **original** (full quality) is a downstream stage not exercised here.
========== DEPLOY_REPORT_cloud-serving_compile-on-L4_2026-07-08.md ==========
> Durable **validation record** for the 2026-07-08 release that moved the reel **compile/stitch onto the L4 worker** (#521, `phase_placement.compile=cloud_run_l4` + `h264_nvenc`) and lit up the #524 near-term win (`trust_submit_video_id`). Built from `origin/master @ 304af7b` ? image `sha256:7b62a8df?`; both surfaces (control-plane rev `rally-control-plane-00020-dtg` + worker Job) redeployed to it. One owner-driven live SPA CUJ on a **worst-case 5.81 GiB / 9.2-min 4K** clip proves the stitch win end-to-end. Project `gen-lang-client-0306026826`, region `asia-southeast1`. Honest limits + follow-ups in §6.
compile?L4 dispatches	? `/api/compile` dispatched a `kind='compile'` Cloud Run Job exec (`rally-worker-fdbs2`), NOT the in-process CP stitch
NVENC engaged	? `encoder=h264_nvenc` ? `encoding clips with h264_nvenc (hardware)` (not the libx264 fallback)
Stitch speedup	? **~67 s/clip ? ~8.4 s/clip (~8×)** ? 18 clips trimmed in **2m31s** (est. ~20 min on the old 2-vCPU CP path)
Bottom line: the release does what it set out to. The reel stitch ? the ~13-min control-plane bottleneck that motivated #521 ? now runs on the L4 with hardware NVENC at **~8× the per-clip throughput**, dispatched as its own Job. On a 5.8 GiB / 9.2-min clip with 18 selected clips the full compile took **5m17s**, of which the stitch itself was ~2.5 min; the rest is fixed overhead (a **~1m38s full-source re-download** ? now #535, plus a 1.75 GiB reel upload). `trust_submit_video_id` skipped the redundant re-hash. Validation stayed on the CP by design.
- **Image:** `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943`, built from `origin/master @ 304af7b` (Cloud Build `cloudbuild.yaml`, 12m57s). Carries #525 (compile?L4 + `phase_placement`), #527/#528 (#521 follow-ups), #530 (#524 trust-video_id).
- **Worker:** `rally-worker` Cloud Run Job, same digest, 8 vCPU / 32 GiB / 1× L4, GCSFuse mount of `gs://khelsutra-rally-serving`.
- **Config (cloud-serving preset ? the wins, on by default):** `phase_placement.compile=cloud_run_l4`, `cloud_stitch_encoder=h264_nvenc`, `segment?cloud_run_l4` (via `compute_target=cloud_run`), `trust_submit_video_id` (default). `validation` deliberately `control_plane`.
- **Corpus:** `GX010151.MP4` (**5.81 GiB** = 6,240,047,860 B; **554.49 s** ? 9.2 min, 4K GoPro), video_id `af2cd79a7c8df448501151f797338b57`. Reel ? `gs://khelsutra-rally-serving/highlights/af2cd79a?/second-ks-highlight-reel.mp4` (**1.75 GiB**).
`POST /api/process` 202	07:55:07	5.81 GiB upload staged
`Video passed checks`	07:59:21	**validation wall = ~162 s (2m42s)**, PASSED
dispatch ? detect exec `rally-worker-956vm`	07:59:23	segment?L4
========== DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md ==========
Heartbeat (P6/#482 CP slice)	`GET /api/jobs/{id}` during the live detection	`progress = "segmenting (wasb_hybrid) on cloud GPU ? 5 min elapsed"` ? later ticks 6, 7? ? the silent 22-minute spinner is dead
Fresh detection on the new image	the same submit dispatched `rally-worker-2mz8t` (see §5 for WHY it dispatched)	job ? **done, 48 segments**, `compute.path=cloud_run`, exec `?/rally-worker-2mz8t`, **wall 14m04s** (14:23:41?14:37:45Z). Clarifier: the bucket's `MahadevpuraSingles.MP4` hashes to `9154737?` ? a DIFFERENT file than the morning's 2.68 GB browser upload (`c2d45319?`/29 rallies); 48 matches the A1-e2e record for this original, and its `outputs/` were idempotently rewritten
Cache proof ? the literal §9 box (#484)	immediate re-submit of the same `gs://` path (same instance, DB now warm)	**`done cached=true, path=cached, 48 segments` in 11 s**; `latestCreatedExecution` unchanged (`2mz8t`) ? **zero GPU on the re-submit**
  4 GiB-scale memory profile. The owner's official 5-minute CUJ covers it on this build.
========== DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md ==========
> Durable record of shipping the **GPU-resident WASB decode** (`decode_backend=nvdec_resident`, issue #506 / PR #507) to the live `cloud-serving` path: rebuild + cut over the `rally-worker` L4 Cloud Run Job image (torch 1.11 ? **2.6.0+cu124 + PyNvVideoCodec 2.1.0 + baked NVDEC libs**), smoke the cpu + nvdec paths on the real L4, then **enable nvdec via a reversible Job env var** and canary it. Project `gen-lang-client-0306026826`, region `asia-southeast1`. Honest-limits in §6.
| Phase A ? cpu smoke (torch 2.6) | ? 180s clip: `torch 2.6.0+cu124 cuda True`, 17 windows, WASB 201.9s |
| Phase B ? nvdec smoke (fast) | ? 180s clip: 19 windows, WASB **143.7s (~1.4× vs cpu)**, 2934 vis ? cpu 2920 |

Phase B ? nvdec smoke (**hard GX010128**)	? **26479/45298 visible ? VM 26481** (? 0.004%), 85 windows, WASB 1138s (~40 fps)
Enable	? `WASB_DECODE_BACKEND=nvdec_resident` set on the Job (reversible)
Canary (env-driven)	? 180s clip, **no** decode `--set` ? env var selected nvdec: 19 windows, 2934 vis, WASB 151.4s (== explicit-`--set` nvdec)
Bottom line: the GPU-resident decode path runs on the real Cloud Run L4 and **reproduces the validated VM trajectory** (26479 ? 26481 visible on GX010128), so the golden-gate F1 (0.574 ? baseline 0.582 @ SERVED) transfers to serving. nvdec is enabled via a Job env var with an instant rollback.
- **Worker:** `rally-worker` Cloud Run **Job**, image `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest` (`sha256:465a9250?`), 8 vCPU / 32 GiB / 1× NVIDIA L4, task-timeout 3600 s, GCSFuse mount of `gs://khelsutra-rally-serving`, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`, **`WASB_DECODE_BACKEND=nvdec_resident`** (the enablement lever).
- **Image change (PR #507):** WASB conda env py3.8+torch1.11 ? **py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0**; `libnvcuvid`/`libnvidia-encode` (from driver 580.159.03) baked in a multi-stage layer. cpu decode path byte-unchanged; nvdec is a new, cache-isolated path.
  --args='^@@^infer@@--video-uri=gs://khelsutra-rally-corpus/videos/<clip>@@--out-uri=gs://khelsutra-rally-serving/rel506/<label>@@--segmenter=wasb_hybrid@@--set=deployment.compute_target=local_gpu@@--set=indexing.detector_impl=native@@--set=indexing.skip_ai_handoff=true@@--set=indexing.wasb.batch_size=64@@--set=indexing.wasb.prefetch_batches=4@@--set=indexing.wasb.timeout_sec=0@@--set=indexing.wasb.decode_backend=<cpu|nvdec_resident>'
| rally-worker-clkwc | mahadevpura_smoke_180s | nvdec_resident | `--set` | 2934 / 5395 | 19 | 143.7 | ? |
| rally-worker-6htsw | **GX010128** | nvdec_resident | `--set` | **26479 / 45298** | 85 | 1138.0 | ? |
========== DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md ==========
> Durable **closure record** for the two validation-latency fixes now live on the `cloud-serving` path: **#512** (`perf(validators)`: decode `scene_cut`/`blur`/`relevance` in ONE shared pass instead of ~125 random per-sample seeks across 3 captures) and **#513** (`fix(serving)`: the worker skips its redundant re-validation when the control-plane already validated + dispatched, via `--skip-validation`). Both shipped in the live image `rally-worker@sha256:8045f54e?` (built from `origin/master` @ `f5c9dd9`; #512 `20862d7`, #513 `23c1868` are ancestors). Measures the before/after on 3 clips spread across codecs/resolutions, all against the real serving infra. Project `gen-lang-client-0306026826`, region `asia-southeast1`. Honest-limits (incl. the owner-gated live CUJ) in §6.
#513 worker skip-revalidation	? real dispatch-style exec: Video 1 validation **SKIPPED (0 s)**, runs to **SUCCESS / 13 rallies** ? the blur-reject that failed the live CUJ is **gone**
#512 shared-decode speedup (real L4)	? Video 1 re-validation **169.3 s ? 69.7 s (2.43×)** on identical worker hardware, decision byte-identical (sharpness 11.9 < 20.0 both)
Live control-plane "after" number	? **live-corroborated 2026-07-08** (§9): rev `00019` validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the 2?3 min band) + reel compiled end-to-end. Video-1's exact 412 s?~170 s A/B stays a projection (the live CUJ used a different, heavier clip).
Bottom line: the two fixes do exactly what they claim on live serving hardware. **#513** turns the soft 5.3K clip from a **hard FAIL** (worker re-validated at its default `min_blur=20`, rejected sharpness 11.9, `rc2`, no reel) into a **PASS with 13 rallies** by trusting the control-plane's gate. **#512** cuts the frame-sampling validators ~**2?3×** (2.43× measured on the L4; 1.95?3.23× off-box across codecs) with **zero decision drift** OLD?NEW. The only number still owner-pending is the live control-plane validation wall on rev 00018 ? its components are measured and it projects to ~2?3 min from the ~6m52s baseline.
- **Control-plane:** `rally-control-plane` Cloud Run **service**, rev **`rally-control-plane-00018-qvz`** (100% traffic), image `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:8045f54ea9fcbf?` (built from `f5c9dd9`), `DEPLOYMENT_PROFILE=cloud-serving`, CPU-only **2 vCPU**, edge-auth ON (`edge_auth_enabled=true`, `cf_team_domain=khelsutra.cloudflareaccess.com`).
- **Worker:** `rally-worker` Cloud Run **Job**, same image `sha256:8045f54e?`, 8 vCPU / 32 GiB / 1× NVIDIA **L4**, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`, `WASB_DECODE_BACKEND=nvdec_resident`, GCSFuse mount of `gs://khelsutra-rally-serving`.
- **Corpus:** Video 1 `gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4` (1.41 GiB, 5312×2988 yuv420p10le HEVC, 3019 frames, MD5 `8fe6b719603dd7a7b2b6ef543139a4a3`); `gs://khelsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4` (1080p H.264);

`gs://khelsutra-rally-corpus/videos/golden/GX010128.mp4` (2.18 GiB, 4K HEVC, 45298 frames).
Control-plane validation	**411?412 s** (6m52s) ? passed (baked `min_blur=8.0`)	**projected ~170 s (2?3 min)**; **live-corroborated 2026-07-08 ? 183 s** on rev `00019` (§9 ? a fresh 1.83 GiB clip, not a Video-1 re-run)
Worker (re-)validation	**169.3 s** ? ? **BLUR-REJECT** (sharpness 11.9 < default 20.0)	**0 s ? SKIPPED** (`--skip-validation`, #513)
End-to-end	~10m31s wall ? **FAILED** (0 rallies, no reel)	worker ? **SUCCESS** in ~2m38s exec ? **13 rallies**; **reel live-confirmed 2026-07-08** (§9 ? fresh-clip CUJ compiled an 11-clip reel end-to-end)
Rallies produced	**0** (rejected before detection)	**13** (`report.json` `status=success`, 13 segments)
========== RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md ==========
| C ? healthy dispatch | A normal good run must still succeed (no false-positive) | ? **rc 0** in **207 s** (19 rally segments, `status=success`); probe returned `None` on **9** running-polls, never aborting |
- Dispatched execution: **`rally-worker-9tq5g`** on `mahadevpura_smoke_180s.mp4` (86 MiB, 180 s),
- **C:** one real **L4** run on a 180 s clip (~207 s wall, 19 rallies detected) ? a few cents.

609. assistant (2026-07-09T06:31:20.109Z)

I have rich comparable data now. Let me set up a branch and write this as a proper runlog (matching the `DEPLOY_REPORT_cloud-serving_*` convention), with the cross-run comparison and the cost economics baked in.

610. user (2026-07-09T06:31:37.176Z)

```
branch 'docs/runlog-cp-downgrade-cuj' set up to track 'origin/master'.
HEAD is now at 00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
HEAD: 00e8f92  branch: docs/runlog-cp-downgrade-cuj
DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
--- worker exec exact wall times for cost calc ---
2026-07-09T06:04:20.821211Z     2026-07-09T06:18:55.014817Z
2026-07-09T06:21:06.775581Z     2026-07-09T06:24:36.560683Z
```

611. assistant (2026-07-09T06:32:01.501Z)

Exact L4 time: detect **14m34s** + compile **3m30s** = **18m04s** of GPU. Now the runlog.

612. assistant (2026-07-09T06:32:59.051Z)

DEPLOY_REPORT ? cloud-serving: latest-HEAD deploy + CP 2 vCPU/8 GiB downgrad

> Durable record of the 2026-07-09 release: both surfaces cut over to `origin/master @ 00e8f92`
> (image `sha256:1838c07f?`, Cloud Build `f93a0dd7`), and the **control-plane downgraded from
> 4 vCPU/16 GiB ? 2 vCPU/8 GiB** now that all heavy phases run on the L4. Validated end-to-end by one
> owner-driven live SPA CUJ on the **largest clip to date ? `GX010155.MP4`, 7.84 GiB / 12.5 min**.
> Project `gen-lang-client-0306026826`, region `asia-southeast1`. Honest limits in §6.

1. What shipped

- **Image `1838c07f`** (built from `origin/master @ 00e8f92`) ? 11 commits over the prior live `304af7b`:
  #531 single-pass concat-filter stitch, **#535** GCSFuse mount-read for the compile source,
  #537 SD rendition, **#539** gate CP `acquire_local_input`, **#538** cancel propagation,
  #529/#534 GPU-encoder + L4-worker telemetry, **#548** GCSFuse OS-native path, **#547** gates reorg.
- **Deploy method = image-only bump** (`gcloud run jobs/services update --image` +

`--cpu/--memory` on the CP),
  so the baked cloud-serving config (phase_placement `validation/segment/compile=cloud_run_l4`, edge-auth +
  CF AUD, GCS roots) is preserved ? only the image (+ CP resources) changed. **Worker deployed first.**
- **CP downgrade 4/16 ? 2/8**: justified by **#539** (CP no longer RAM-downloads the full input when
  validation+segment are delegated) + **#535** (worker mount-reads the source). Cloud Run requires ?4 vCPU
  for 16 GiB, so 2 vCPU ? 8 GiB (the pre-drift `00020` default).

what	result
Worker cut over ? `1838c07f`	? `gcloud run jobs update rally-worker`
CP cut over + downgrade	? rev **`rally-control-plane-00023-spm`**, `1838c07f`, **2 vCPU / 8 GiB**, `deployment profile ACTIVE: cloud-serving`, Uvicorn up, `GET /api/config` **200** (edge + run.app)
#539 CP skips acquire	? `[API] skipping CP acquire for fully delegated phases with cheap video_id (#539)` ? no 7.84 GiB in CP RAM
#535 compile mount-read	? `[storage] using GCSFuse mounted path for gcs source (no full download)`
#529 NVENC engaged	? `[compile] single-pass: encoding with h264_nvenc (hardware)` (no libx264 fallback)
#531 single-pass stitch	? `[compile] single-pass stitch: fusing 19 clip(s) + watermark in one ffmpeg pass`
CP OOM on 7.84 GiB @ 8 GiB	? **none** ? the downgrade held on the worst-case clip

2. Config

- **Image:** `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb` (Cloud Build `f93a0dd7`, 10m32s).
- **Control-plane:** `rally-control-plane` service, rev `rally-control-plane-00023-spm` (100 % traffic), **2 vCPU / 8 GiB**, `min-instances=1`, `DEPLOYMENT_PROFILE=cloud-serving`, edge-auth ON.
- **Worker:** `rally-worker` Cloud Run Job, same digest, 8 vCPU / 32 GiB / 1× NVIDIA L4, task-timeout 3600 s, GCSFuse mount of `gs://khelsutra-rally-serving`, `WASB_DECODE_BACKEND=nvdec_resident`.
- **Corpus:** `GX010155.MP4` ? **7.84 GiB** (8,417,724,136 B), **747.9 s** (~12.5 min), video_id `e2e144175c8d05451903ae44a9ce0aa0`. Reel ? `gs://khelsutra-rally-serving/highlights/e2e14417?/gx010155-ks-highlight-reel.mp4` (**1.90 GiB**, 2,034,852,230 B).

3. Live CUJ timeline (per stage)

stage	where	wall	evidence
`POST /api/process` ? 202	CP	06:04:19	trusts submit-time video_id (#524), **skips CP acquire (#539)**
pull source ? `/tmp/in.mp4`	L4 detect	~2m25s (06:04:33?06:06:58)	7.84 GiB acquired for the decode pass
checksum + **validation**	L4 worker	~78 s window (incl. torch import)	ran (execution args carry **no `--skip-validation`**); silent on pass ? see §6
segmentation (wasb, `nvdec_resident`, native)	L4	**~10m19s** (06:08:28?06:18:47)	**53 rallies** detected; detect exec `rally-worker-dpssz`
ingest + dispatch compile	CP?L4	06:18:55?06:21:06	`POST /api/compile` 202
compile/stitch (single-pass **h264_nvenc**)	L4	**205 s** CP-measured / **~138 s** actual encode (06:21:37?06:23:55)	19 clips + watermark, one pass, mount-read ? **1.90 GiB** reel; compile exec `rally-worker-fkvqp`
end-to-end	?	**~20m17s** (06:04:19?06:24:36)	reel delivered to `highlights/`

4. Perf comparison across the serving runlogs

| run (date) | clip | size | dur | rallies | validation | detect/segment | stitch/compile | reel | CP shape |

|---|---|---|---|---|---|---|---|---|---|
`a1-cuj` 07-04	mahadevpura	?	244 s reel	48	on CP	~13m45s (L4, cold pull)	**in-process CP** ~3m26s	174 MB	8/32
`alpha-all1v1` 07-06	tester 1080p60	2.12 GiB	12.8 min	56 (1080p) / **60 (720p)**	?	2411 s ? **956 s** (720p proxy)	detect-only (no reel)	?	?
`validation-latency` 07-07	Video 1 5.3K HEVC	1.41 GiB	?	13	CP 411 s ? 183 s (#512) / worker **skip** (#513)	?	11-clip reel	2 vCPU
`compile-on-L4` 07-08	GX010151 4K	5.81 GiB	9.2 min	18 clips	**CP ~162 s**	L4 (segment?L4)	**L4 NVENC** 2m31s (~8.4 s/clip) **+ 1m38s full re-download**	1.75 GiB	4/16
this 07-09	**GX010155**	**7.84 GiB**	**12.5 min**	**53 ? 19 reel**	**L4 worker (delegated)**	**L4 nvdec** ~10m19s	**L4 NVENC single-pass** 205 s (~138 s encode), **mount-read (no re-download)**	**2 vCPU / 8 GiB**

Trajectory (where the work moved): in-process CP compile (07-04) ? segment?L4 (07-06/07) ? **compile?L4 + NVENC** (07-08) ? **validation?L4 + single-pass stitch + mount-read + a 2 vCPU CP** (this run). Two concrete deltas vs the 07-08 predecessor on a bigger clip:

- **#535 killed the compile re-download** ? 07-08 paid a **~1m38s full-source re-download** before stitching; this run reads the source **ranges via the GCSFuse mount** (`no full download`), so that 1m38s is gone even though the clip is 35 % larger.
- **Validation moved off the CP** ? 07-08 spent **~162 s validating on the control-plane**; here it's delegated to the L4 worker, so the CP does zero heavy work ? which is precisely what makes the **2 vCPU / 8 GiB** CP safe.

5. Where we stand

- **Every heavy phase is now on the scale-to-zero L4** ? validation (delegated), segmentation (`nvdec_resident`), and compile (single-pass `h264_nvenc`, mount-read). The control-plane is a **thin 2 vCPU / 8 GiB dispatcher**.
- **Biggest clip yet (7.84 GiB) processed clean** ? larger than the 6.52 GiB clip that once forced the 4/16 bump ? with **zero CP OOM**, validating the downgrade on the worst case.
- **Stitch is no longer the bottleneck**: ~138 s to fuse a 19-clip / 1.90 GiB 5.3K reel in a single NVENC pass (a two-pass encode would be ~2×; the old in-process CP path was ~13?20 min).
- The dominant remaining wall is **segmentation (~10 min)** + the **7.84 GiB source pull (~2.4 min)** ? both inherent L4 GPU/decode work, not orchestration overhead.

6. Honest limits / follow-ups

- **Validation is invisible on a pass, and un-timed.** The worker runs the validators (no `--skip-validation` in the dispatched args) but only logs on FAILED/SKIPPED, and the results land only in the worker's ephemeral DB ? **not** in the pushed `report.json`. So this run has **no explicit validation wall or validator scores** in the runlog. ? chip `task_f925e323`.
- **The #534 live progress heartbeat + telemetry did NOT fire on the native-L4 path.** Over the ~10-min GPU pass, `run_telemetry.jsonl` held **only its 116-byte meta header** (zero snapshots), there were **no `[worker] progress:` lines**, and **no `<video_id>_detector_progress.json`** in GCS. The GPU pass ran dark. ? same chip `task_f925e323`.
- **19 of 53 rallies reached the reel** ? the compile applied its `min_shot_count`/selection filter; not a defect, but noted for the record.
- Per-stage timings above the compile are **derived from log timestamps**, not explicit stage timers (the chip adds real timers).

7. Cost economics ? cheap CP, no "waiting price"

The architecture now bills for **actual work**, not for waiting:

- **L4 worker = scale-to-zero, billed only during a real CUJ.** This 7.84 GiB run used **detect 14m34s + compile 3m30s = 18m04s of L4** (0.301 hr) ? **~$0.21** @ ~$0.70/hr. That's the marginal cost of an actual user CUJ, and it scales with clip length ? no idle GPU between runs.
- **Control-plane = a cheap always-on floor.** At **2 vCPU / 8 GiB** with `min-instances=1` the CP is billed at Cloud Run's reduced idle-CPU rate (~$0.02?0.03/hr, ~$15?20/mo) ? **roughly half** the old 4 vCPU/16 GiB floor and ~¼ of the early 8 vCPU/32 GiB. During the CUJ it adds only fractions of a cent.
- **Net:** total cost ? (tiny fixed CP floor) + (L4 GPU-minutes per real CUJ). There is **no "waiting price"** ? the expensive GPU only spins up for actual work, and the always-on endpoint

is now a thin, cheap dispatcher rather than a big machine held open. Cost is dominated by real users doing real CUJs.

Rollback: worker `--image ?@sha256:7b62a8df?`; CP `--image ?@sha256:7b62a8df? --cpu=4 --memory=16Gi` (or `services update-traffic --to-revisions rally-control-plane-00022-29w=100`).

613. user (2026-07-09T06:32:59.080Z)

File created successfully at: C:\ksrunlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md (file state is current in your context ? no need to Read it back)

614. assistant (2026-07-09T06:33:19.589Z)

Runlog written. Let me commit, push, and open the PR (matching the runlog convention), then I'll show you the summary.

615. user (2026-07-09T06:33:39.729Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md', LF will be replaced by CRLF the next time Git touches it
[docs/runlog-cp-downgrade-cuj 9c90198] docs(runlog): 2026-07-09 latest-HEAD deploy + CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ
 1 file changed, 88 insertions(+)
 create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md
=== origin/master..HEAD ===
9c90198 docs(runlog): 2026-07-09 latest-HEAD deploy + CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ
branch 'docs/runlog-cp-downgrade-cuj' set up to track 'origin/docs/runlog-cp-downgrade-cuj'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]   docs/runlog-cp-downgrade-cuj -> docs/runlog-cp-downgrade-cuj
https://github.com/Khelsutra/badminton-highlight-indexer/pull/549
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

616. assistant (2026-07-09T06:33:52.938Z)

Runlog is in **PR #549** (`9c90198`). Let me clean up the worktree and close the task, then here's the summary.

617. user (2026-07-09T06:33:54.226Z)

cleaned

618. user (2026-07-09T06:33:54.243Z)

Updated task #17 status

619. assistant (2026-07-09T06:34:23.289Z)

Recorded in **PR #549** and cleaned up. Here's where we stand.

Sharp perf comparison ? cloud-serving runs

run	clip	size	rallies	detect/segment	stitch/compile	reel	CP
a1-cuj `07-04`	mahadevpura	?	48	~13m45s (L4)	**in-process CP** ~3m26s	174 MB	8/32
alpha `07-06`	tester 1080p60	2.12 GiB	56?**60** (720p)	2411s ? **956s** (proxy)	detect-only	?	?
val-latency `07-07`	Video 1 5.3K	1.41 GiB	13	?	?	11-clip	2 vCPU
compile-on-L4 `07-08`	GX010151 4K	5.81 GiB	18	L4	**L4 NVENC** 2m31s **+ 1m38s re-download**	1.75 GiB	4/16

| **this `07-09`** | **GX010155** | **7.84 GiB** | **53 ? 19** | **L4 nvdec 10m19s** | **L4 NVENC single-pass 205s, mount-read** | **1.90 GiB** | **2/8** |

The trajectory is "push all heavy work off the CP onto the scale-to-zero L4": in-process CP compile ? segment?L4 ? compile?L4+NVENC ? **now: validation?L4 + single-pass stitch + mount-read + a 2 vCPU CP**. Two concrete gains this run over the 07-08 predecessor, on a **35% bigger clip**:
- **#535 killed the ~1m38s compile re-download** ? source read via GCSFuse mount ranges instead.
- **Validation left the CP** (was ~162s on-CP) ? delegated to the L4, so the CP does zero heavy work.

Where we stand

- **Biggest clip ever (7.84 GiB) ran clean** ? bigger than the 6.52 GiB that once forced 4/16 ? with **zero CP OOM at 2 vCPU/8 GiB**. Downgrade validated on the worst case.
- **Stitch is no longer a bottleneck**: ~138s for a 19-clip / 1.9 GB 5.3K reel in one NVENC pass (old in-process CP path was ~13?20 min).
- Remaining wall is inherent GPU work ? segmentation (~10 min) + the 7.84 GiB pull (~2.4 min).
- **Only gap left is *visibility***, not perf: validation runs silently/un-timed and the #534 GPU-progress heartbeat didn't fire ? both in chip `task_f925e323`.

Cost ? cheap CP, no "waiting price"

- **L4 = scale-to-zero, billed only during a real CUJ.** This run: detect 14m34s + compile 3m30s = **18m04s of L4 ? ~$0.21** @ ~$0.70/hr. Scales with clip length; **no idle GPU between runs.**
- **CP = a thin always-on floor.** At 2 vCPU/8 GiB (`min-instances=1`) it bills at Cloud Run's idle-CPU rate (~$0.02?0.03/hr, ~$15?20/mo) ? **~half** the old 4/16 floor, **~¼** of the early 8/32. During a CUJ it adds fractions of a cent.
- **Net: total cost ? (tiny fixed CP floor) + (L4 GPU-minutes per actual CUJ).** No expensive machine held open waiting ? the GPU only spins up for real work, and the always-on endpoint is now a cheap dispatcher. **Cost is dominated by real users doing real CUJs.**

Open PRs from this session: **#549** (this runlog) · plus the two chips you kicked off (`task_f925e323` worker observability, `task_2e0a3b82`/#548 Windows GCSFuse).

620. user (2026-07-09T06:38:12.097Z)

the table does not show the end to end time from upload to the reel generated state. Can that be added here and in the runlog PR?

621. assistant (2026-07-09T06:39:32.393Z)

Good catch ? the end-to-end (request ? reel in GCS) is the number that actually matters, and it makes the comparison much stronger. Let me pin the exact timestamps for the comparable predecessor (compile-on-L4) and re-open the runlog branch to add the column.

622. user (2026-07-09T06:39:49.454Z)

```
=== compile-on-L4 (07-08) reel object time ? GX010151 video_id af2cd79a ===
1881671925  2026-07-08T08:16:41Z  gs://khelsutra-rally-serving/highlights/af2cd79a7c8df448501151f797338b57/second-ks-highlight-reel.mp4
=== this run reel object time (e2e14417) ===
2034852230  2026-07-09T06:24:29Z  gs://khelsutra-rally-serving/highlights/e2e144175c8d05451903ae44a9ce0aa0/gx010155-ks-highlight-reel.mp4

=== re-open runlog branch worktree to add the E2E column ===
HEAD is now at 9c90198 docs(runlog): 2026-07-09 latest-HEAD deploy + CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ
HEAD: 9c90198  branch: docs/runlog-cp-downgrade-cuj
```

623. assistant (2026-07-09T06:40:13.092Z)

Precise E2E (request ? reel in GCS):
- **compile-on-L4 (5.81 GiB, 07-08): 07:55:07 ? 08:16:41 = ~21m34s**
- **this (7.84 GiB, 07-09): 06:04:19 ? 06:24:29 = ~20m10s**

So the **35%-bigger clip finished ~1m24s faster end-to-end**. Let me add the E2E column to the runlog table. Reading the file to edit it precisely.

624. user (2026-07-09T06:40:14.318Z)

```
44      | pull source ? `/tmp/in.mp4` | L4 detect | ~2m25s (06:04:33?06:06:58) | 7.84 GiB
acquired for the decode pass |
45      | checksum + **validation** | L4 worker | ~78 s window (incl. torch import) | ran
(execution args carry **no `--skip-validation`**); silent on pass ? see §6 |
46      | **segmentation** (wasb, `nvdec_resident`, native) | L4 | **~10m19s**
(06:08:28?06:18:47) | **53 rallies** detected; detect exec `rally-worker-dpssz` |
47      | ingest + dispatch compile | CP?L4 | 06:18:55?06:21:06 | `POST /api/compile` 202 |
48      | **compile/stitch** (single-pass **h264_nvenc**) | L4 | **205 s** CP-measured / **~138
s** actual encode (06:21:37?06:23:55) | 19 clips + watermark, one pass, mount-read ? **1.90
GiB** reel; compile exec `rally-worker-fkvqp` |
49      | **end-to-end** | ? | **~20m17s** (06:04:19?06:24:36) | reel delivered to `highlights/`
|
50
51      ## 4. Perf comparison across the serving runlogs
52
53      | run (date) | clip | size | dur | rallies | validation | detect/segment |
stitch/compile | reel | CP shape |
54      |---|---|---|---|---|---|---|---|---|---|
55      | `a1-cuj` 07-04 | mahadevpura | ? | 244 s reel | 48 | on CP | ~13m45s (L4, cold pull) |
**in-process CP** ~3m26s | 174 MB | 8/32 |
56      | `alpha-all1v1` 07-06 | tester 1080p60 | 2.12 GiB | 12.8 min | 56 (1080p) / **60
(720p)** | ? | 2411 s ? **956 s** (720p proxy) | detect-only (no reel) | ? | ? |
57      | `validation-latency` 07-07 | Video 1 5.3K HEVC | 1.41 GiB | ? | 13 | CP 411 s ? 183 s
(#512) / worker **skip** (#513) | ? | 11-clip reel | 2 vCPU |
58      | `compile-on-L4` 07-08 | GX010151 4K | 5.81 GiB | 9.2 min | 18 clips | **CP ~162 s** |
L4 (segment?L4) | **L4 NVENC** 2m31s (~8.4 s/clip) **+ 1m38s full re-download** | 1.75 GiB |
4/16 |
59      | **this** 07-09 | **GX010155** | **7.84 GiB** | **12.5 min** | **53 ? 19 reel** | **L4
worker (delegated)** | **L4 nvdec** ~10m19s | **L4 NVENC single-pass** 205 s (~138 s encode),
**mount-read (no re-download)** | **2 vCPU / 8 GiB** |
60
61      **Trajectory (where the work moved):** in-process CP compile (07-04) ? segment?L4
(07-06/07) ? **compile?L4 + NVENC** (07-08) ? **validation?L4 + single-pass stitch + mount-read
+ a 2 vCPU CP** (this run). Two concrete deltas vs the 07-08 predecessor on a bigger clip:
```

625. assistant (2026-07-09T06:41:08.821Z)

Let me add the E2E column to the comparison table and an insight, plus fix the §3 end-to-end row to the reel-in-GCS time for consistency.

626. user (2026-07-09T06:41:23.259Z)

The file C:\ksrunlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md has been updated successfully. (file state is current in your context ? no need to Read it back)

627. user (2026-07-09T06:41:25.698Z)

The file C:\ksrunlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md has been updated successfully. (file state is current in your context ? no need to Read it back)

628. assistant (2026-07-09T06:41:39.886Z)

Now commit + push the update to PR #549, and clean up.

629. user (2026-07-09T06:41:50.541Z)

pushed; PR #549 updated (08a2695)

worktree cleaned

630. assistant (2026-07-09T06:42:07.785Z)

Added ? both here and in **PR #549** (`08a2695`).

Perf comparison ? now with end-to-end (request ? reel in GCS)

run	clip	size	rallies	detect/segment	stitch	reel	**E2E req?reel**	CP
a1-cuj `07-04`	mahadevpura	?	48	~13m45s	in-proc CP ~3m26s	174 MB	~18m (old CP)	8/32
alpha `07-06`	tester 1080p60	2.12 GiB	60	956s (proxy)	?	?	? (detect-only)	?
val-latency `07-07`	Video 1 5.3K	1.41 GiB	13	?	?	11-clip	? (val-focused)	2 vCPU
compile-on-L4 `07-08`	GX010151 4K	5.81 GiB	18	L4	NVENC 2m31s **+1m38s dl**	1.75 GiB	**~21m34s**	4/16
this `07-09`	**GX010155**	**7.84 GiB**	**53?19**	**nvdec 10m19s**	**NVENC single 205s**	**1.90 GiB**	**~20m10s**	**2/8**

The headline the E2E column makes obvious: the **35 %-bigger clip finished ~1m24s *faster* end-to-end** ? 7.84 GiB in **~20m10s** vs the 5.81 GiB predecessor's ~21m34s. That's #535 (no compile re-download) + #539 (no CP download for validation) + the single-pass stitch more than absorbing the larger input. Perf is trending the right way even as clips get bigger.

One honest caveat baked into the runlog: the measured leg is **request ? reel** (`POST /api/process` ? reel object in GCS). The client's **upload-to-GCS** step happens before that and is bandwidth-dependent, so it's not in the server logs ? worth noting for a 7.84 GiB file, the user's upload can be a real chunk of their *wall-clock* wait even though it's outside the pipeline.

Session's open PRs: #549 (this runlog, updated) · chips `task_f925e323` (worker observability) + `task_2e0a3b82`/#548 (Windows GCSFuse).

631. user (2026-07-09T06:42:10.971Z)

also I just got 1 chip and not two can you verify it is availblale or working?

632. assistant (2026-07-09T06:43:19.866Z)

Let me reconcile this properly ? I spawned **two** chips, but they're at different stages, so you'd only see one *pending* right now. Let me verify each concretely (chips run as separate local sessions, so I'll confirm via their outputs).

633. user (2026-07-09T06:43:27.164Z)

```
=== recent PRs (chip outcomes show up here) ===
#549 [OPEN] docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09)  (docs/runlog-cp-downgrade-cuj)
#548 [MERGED] fix(storage): OS-native GCSFuse fused path (fixes Windows-only #535 test failures)  (claude/gcsfuse-native-path)
#547 [MERGED] feat(gates): local release perf gate + consolidate nightly into scripts/gates/  (feat/perf-release-gate)
#546 [MERGED] feat(#534): live detector progress heartbeat + L4 worker resource telemetry  (fix/534-l4-detector-progress-telemetry)
#545 [MERGED] perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539)  (fix/539-cp-no-unneeded-download)
#544 [MERGED] fix(#538): cancel propagates to the dispatched L4 worker exec  (fix/538-cancel-dispatched-worker-exec)
#543 [MERGED] observability: emit structured signal on GPU encoder fallback to CPU (#529)  (fix/529-gpu-encoder-fallback-observability)
#542 [MERGED] docs: add verified local test suite bootstrap instructions  (docs/test-bootstrap)
#541 [MERGED] perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)  (fix/535-compile-gcs-fuse-source)
```

```
#537 [MERGED] feat(compile): add opt-in SD reel rendition  (codex/issue-523-sd-reel-plan-clear)
#536 [MERGED] docs(runlogs): compile?L4 + NVENC live deploy-validation CUJ (2026-07-08)
(docs/compile-on-l4-cuj-2026-07-08)
#533 [CLOSED] feat(compile): add opt-in SD reel rendition  (codex/issue-523-sd-reel-plan)

=== task_2e0a3b82 (GCSFuse Windows) -> is #548 real + on-topic? ===
{"files":["backend/storage/__init__.py"],"number":548,"state":"MERGED","title":"fix(storage):
OS-native GCSFuse fused path (fixes Windows-only #535 test failures)"}

=== any branch/PR yet from task_f925e323 (worker logs/timing/heartbeat)? ===
docs/runlog-cp-downgrade-cuj
```

634. assistant (2026-07-09T06:44:15.935Z)

Verified ? here's the reconciliation, and you're right to double-check:

You have exactly the right chip count. The two things I referenced are at different lifecycle stages:

1. **`task_2e0a3b82` ? GCSFuse Windows fix** ? this was a *separate, earlier* chip (from the PR #547 work), and it **already ran to completion and MERGED as [PR #548](https://github.com/Khelsutra/badminton-highlight-indexer/pull/548)** ("fix(storage): OS-native GCSFuse fused path", touches `backend/storage/__init__.py`). It's **done**, so it no longer shows as a pending chip ? that's why you don't see it.

2. **`task_f925e323` ? worker per-stage logs/timing + dead L4 heartbeat** ? this is the **single combined chip you asked for** ("make a *single* chip fix these") covering **both** the validation-visibility/timing gap **and** the telemetry-heartbeat gap. This is the **1 chip you see**, and it's still **running** (no PR yet ? the branch grep found none).

So: **1 pending chip (f925e323) is correct** for the two observability gaps you asked me to combine; the GCSFuse one isn't a second chip anymore because it already merged. Let me confirm the running chip's session is actually live.

635. user (2026-07-09T06:44:23.952Z)


```
[
  {
    "sessionId": "local_093db136-a1fc-4a4e-b738-d36f1a41aa8d",
    "title": "Fix worker per-stage logs/timing + dead L4 heartbeat",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": true,
    "lastActivityAt": "2026-07-09T06:44:23.677Z"
  }
]
```


636. assistant (2026-07-09T06:45:27.378Z)

Confirmed working. `list_sessions` shows the chip's spawned session is live:

- **`task_f925e323`** ? session `local_093db136?`, **`isRunning: true`**, last active **06:44Z** (moments ago). ? Available and running.
- The GCSFuse session (`task_2e0a3b82`) isn't in the active list because it already **finished and merged as [#548](https://github.com/Khelsutra/badminton-highlight-indexer/pull/548)**.

So to be unambiguous:

chip	scope	state
`task_f925e323`	**single combined** ? validation logs+timing **and** dead L4 heartbeat (exactly as you asked)	**running now** (the 1 chip you see)
`task_2e0a3b82`	GCSFuse Windows path fix	**done ? PR #548 merged** (not a pending chip)

One caveat worth flagging: I spawned that chip without an explicit `cwd`, so its session opened in the **stub** dir `C:\Users\avidu\Projects\badminton-highlight-indexer` rather than the real repo at `?\khelsutra-guru\badminton-highlight-indexer`. The saving grace is it shares this project's memory ? including the `actual-repo-location` note that points at the real repo ? so it should self-redirect to find `backend/worker/__main__.py`. If it stalls or starts poking around the near-empty stub, I can **send the running session a one-line nudge** with the correct path (via the session-management `send_message`), or dismiss + re-spawn it pinned to the real repo. Want me to send that nudge pre-emptively so it doesn't waste a lap?

637. user (2026-07-09T06:46:04.826Z)

sure pls do send

638. user (2026-07-09T02:41:38.134Z)

```
===LIVE control-plane revision + image (asia-southeast1)===
rally-control-plane-00022-29w   rally-control-plane-00022-29w   asia-
southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
===LIVE worker job image===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943
```

639. user (2026-07-09T02:49:59.189Z)

```
===[a] prod image git-sha label (if any)===
===[b] what changed 304af7b..3249451 in stitch/validation/config (files)===
 backend/config/models.py              |  57 +++++
 backend/orchestration.py              | 390 ++++++++++++++++++++++++++-------
 backend/pipeline/stitcher/compiler.py | 365 +++++++++++++++++++++++++++++---
 3 files changed, 707 insertions(+), 105 deletions(-)
```

640. user (2026-07-09T05:18:08.607Z)

```
COMPILE OK
All checks passed!
............................................................          [100%]
63 passed in 2.89s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

641. user (2026-07-09T06:33:54.226Z)

cleaned

642. user (2026-07-09T06:48:18.382Z)

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.

Summary:
1. Primary Request and Intent:
The conversation spanned multiple evolving requests for the badminton-highlight-indexer project:
   - **Initial:** Build a local, repeatable, release-gating performance comparison of the deployed production binary (api.khelsutra.guru) vs repo HEAD, on the local GPU, over 2-3 vault videos, testing checksum?validation?segmentation?full-reel-stitch e2e (no SPA, no user step), producing precomputed numbers before every release.
   - **Consolidation:** Turn the prototype into a standalone, documented, self-help, release-gating tool integrated into the existing nightly framework.
   - **Reorganization:** Move nightly + new work out of loose `scripts/` files into a cohesive package (user chose `scripts/gates/`).
   - **Review fixes + merge:** Address a two-pass PR review and merge PR #547.
   - **Production deploy:** Deploy latest HEAD to live prod (worker + CP), AND downgrade the CP to 2 vCPU (now that heavy work is on the L4).
   - **Perf on prod:** Measure real prod perf via a live CUJ (user drove GX010155.MP4 through

the SPA).
   - **Observability:** Ensure validation runs/is visible; spin up a chip (not do it myself) for per-stage logs + timing + the dead GPU progress heartbeat.
   - **Runlog + comparison:** Add a runlog for this deploy/CUJ with a sharp cross-runlog perf comparison (size, rallies, time), a "where we stand" summary, and the cheap-CP cost economics ("no waiting price").
   - **Most recent:** Add an end-to-end (upload?reel-generated) time to the comparison table, both in the chat summary and in the runlog PR.

2. Key Technical Concepts:
   - Cloud-serving architecture: control-plane (CP) Cloud Run service + rally-worker Cloud Run Job on a shared image; phase_placement (validation/segment/compile = cloud_run_l4); L4 GPU; NVENC (h264_nvenc); nvdec_resident decode; GCSFuse mount-read (#535); CP acquire-gating (#539); trust submit-time video_id (#524); single-pass concat-filter stitch (#531).
   - wasb_hybrid segmenter: detector_impl=auto (WSL2 on Windows) vs native (Linux/L4); skip_ai_handoff; stream_video decode.
   - Release perf gate: git worktrees for two commits, introspection-based VideoCompiler construction (single_pass), stitch A/B, exit codes 0/1/2/4, prod resolution via live Cloud Run digest?commit map.
   - Cloud Run constraints: 16 GiB requires ?4 vCPU (so 2 vCPU ? 8 GiB); image-only bump preserves config; min-instances=1 idle-CPU billing.
   - #534 telemetry (RunTelemetry JSONL, GPU/CPU/RAM samples, progress heartbeat), #529 GPU-encoder-fallback signal, #513 worker skip-revalidation, #512 shared-decode-pass.
   - git-safety protocol (branch from origin/master, stage explicit paths, never git add -A).
   - CI: ruff pinned 0.15.16, mypy backend training, run_all_tests.py --cov-fail-under=70, cross-OS golden fixtures, markdown-link-check.

3. Files and Code Sections:
   - **scripts/gates/perf/run.py** ? main perf-gate entrypoint. Gates stitch only; failed-HEAD (missing/errored gated stage where prod succeeded)=regression; nothing-ran (prod_observed==0)=exit 2; alternate prod/rc order; gcloud .CMD handling; core.longpaths worktree; detector label from sys.platform. Key: `worker_skips_validation`, `build()` returns (summary, findings, prod_observed), `exit_code(findings, stale, nothing_ran)`.
   - **scripts/gates/perf/harness.py** ? per-stage timer; sets `stitch_s=None` on stitch exception; detector_impl=native on non-Windows; random.seed(0).
   - **scripts/gates/perf/preflight.py** ? OS-aware (WSL checks on Windows via sys.platform=="win32"; native WASB_REPO_DIR/WASB_WEIGHTS/WASB_PYTHON on Linux).
   - **scripts/gates/{nightly_perf.ps1, nightly_regression.ps1, register_nightly_task.ps1}** ? $RepoRoot 3-hop fix (Split-Path ×3).
   - **tests/test_perf_gate.py** ? 12 offline tests (pct edges, regression/improvement, failed-HEAD, errored-with-numeric-time, nothing-ran, segment-mismatch, total-not-gated, precedence).
   - **eval_baselines/perf_manifest.json** ? gate.stages=["stitch_s"], digest?commit map.
   - **backend/worker/__main__.py** (READ, not edited by me ? this is the chip's target): `~line 280` computes video_id; `~line 290` `if getattr(args, "skip_validation", False)` else `validator_registry.run_all_with_context(...)`; only logs on FAILED/SKIPPED (silent on pass); results only to ephemeral DB via `db.add_video(..., [r.model_dump() for r in val_results])`, NOT report.json; `~line 388` `_progress` callback writes `<video_id>_detector_progress.json` + prints `[worker] progress:`; `~line 434` `progress=_progress`.
   - **backend/pipeline/detectors/native_wasb_runner.py** ? where the progress callback should be wired (didn't fire on native-L4 path).
   - **backend/utils/run_telemetry.py** ? RunTelemetry.snapshot() (local JSONL, no GCS stream).
   - **docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md** (CREATED, being edited) ? the new runlog with §4 comparison table. Most recent edits added an "E2E req?reel" column and this insight:
     ```
     **End-to-end (request ? reel in GCS) is flat-to-faster on a bigger clip:** the 07-08 predecessor (`GX010151`, 5.81 GiB) took **~21m34s** (07:55:07 ? 08:16:41); this run (`GX010155`, **7.84 GiB, +35 %**) took **~20m10s** (06:04:19 ? 06:24:29) ? **#535** (no compile re-download) + **#539** (no CP download for validation) + the **single-pass stitch** more than offset the 35 %-larger input.
     ```
     And §3 end-to-end row updated to: `| **end-to-end (request ? reel in GCS)** | ? | **~20m10s** (06:04:19?06:24:29) | gx010155-ks-highlight-reel.mp4 (1.90 GiB) landed in highlights/ (exec fully done 06:24:36) |`

4. Errors and fixes:
   - MSYS path issue: passing `/c/...` paths into Python string literals failed (files written to correct Windows location via argv conversion). Fixed by reading via Read tool with Windows paths.
   - bench_config.json `_comment` top-level key tripped AppConfig strict validation ? removed it.
   - ruff local version was 0.6.9 (flagged 68 pre-existing test errors); installed pinned 0.15.16 ? repo clean. My files always clean.
   - gcloud `.CMD` on Windows couldn't exec via subprocess (self-test caught it) ? wrapped with `["cmd","/c",gcloud]` when ends with .cmd/.bat.
   - git worktree "Filename too long" (deep scratchpad + long repo files) ? added `core.longpaths=true` + short temp worktree root.
   - `gh pr merge` returned non-zero only because a chained `--json merged` used an invalid field; the merge actually succeeded.
   - User feedback that shaped work: (a) "wasb_detector now uses the GPU and is production, we should always compare against production"; (b) chose scripts/gates/ over tests/; (c) "fix the suggestions... and merge this"; (d) "downgrade the CP machine to 2 vCPU"; (e) "please make sure to validate all the steps"; (f) "pls spinup a chip for the log work rather than doing yourself and be focussed on the production eyeballing... make a single chip fix these"; (g) "the table does not show the end to end time from upload to the reel generated state."

5. Problem Solving:
   - Confirmed prod=304af7b via live digest 7b62a8df; deployed 00e8f92 (image 1838c07f) worker-first then CP with downgrade; verified 200 OK + no OOM.
   - Confirmed on a 7.84 GiB clip: #539 (CP skips acquire), #535 (compile mount-read, no re-download), #529 (NVENC hardware), #531 (single-pass stitch), CP held at 2vCPU/8Gi with zero OOM.
   - Confirmed validation ran (no --skip-validation in dispatched exec args) but is silent on pass + not in report.json ? chip task_f925e323.
   - Discovered #534 progress/telemetry heartbeat did NOT fire on native-L4 path (run_telemetry.jsonl only 116-byte header) ? same chip.
   - Computed E2E times precisely from reel GCS object timestamps.

6. All user messages:
   - "Hello. Can you do a local performance comparison test of the current head vs the latest binary on the api.khelsutra.guru service and give a summary of the changes? I would like you to use 2-3 videos from the vault and use the GPU in this local machine to run some e2e tests (without the SPA, purely to test the checksum, validation, segmentation and stitching always stick the entire reels, no user step required) and do it such that it is repeatable before every release so that the release has some precomputed numbers for performance."
   - "please consolidate all this in a repeatable, instruction documented, mostly self help, release gating and standalone tool which I can run to compare latest production with repo head anytime. Also similar in spirit to the nightly tests and maybe add this to the nightly framework as well so that it runs quality and performance tests."
   - "can you also move all the nightly and this new work out of the scripts/ and put it in the tests/or someother relevant directory rather than it being a scattered collection of scripts"
   - "can you fix the suggestions in the PR response and merge this"
   - "can you spin up the process to update the live production binaries (both worker and CP) with the latest head right now?"
   - "can you also downgrade the CP machine to 2 vCPU now that most of the work is done in the L4 even stitching"
   - "update?"
   - "can you run the newly added release gate tests on prod as well to see the perf?"
   - "it straihght away says segmenting without validation .. seems this was already validated? please make sure to validate all the steps"
   - "pls add clear log messages for this and also enough for runlogs to capture the time taken for validation etc."
   - "there was a side car mechanism to tell the GPU progress, can you tail it for me"
   - "pls spinup a chip for the log work rather than doing yourself and be focussed on the production eyeballign. pls add this gap also in the validation gap work and make a single chip fix these"
   - "the stitching is running now"
   - "pls add this to the runlog files and can you do a quick sharp and perf focussed comparison across the other past runlogs including the size, the rallies identitied, time and other dimensions and show a summary of where we stand currently with this setup. Also include the fact that we have a very cheap CP machine now so the total cost is dominated by an actual user doing

an actual CUJ run and no 'waiting price'"
   - "the table does not show the end to end time from upload to the reel generated state. Can that be added here and in the runlog PR?"

7. Pending Tasks:
   - Commit + push the two runlog edits (E2E column + insight + §3 row) to branch docs/runlog-cp-downgrade-cuj, updating PR #549.
   - Clean up the /c/ksrunlog worktree afterward.
   - Present the updated comparison table (with E2E column) to the user in chat.
   - (Delegated to chips, not me): task_f925e323 ? worker per-stage logs/timing + fix dead L4 heartbeat.

8. Current Work:
   Immediately before this summary, I was adding the end-to-end (request?reel) time to the runlog comparison table per the user's request: "the table does not show the end to end time from upload to the reel generated state. Can that be added here and in the runlog PR?" I retrieved exact reel-object timestamps via `gcloud storage ls -l`: compile-on-L4 (GX010151, 5.81 GiB) reel at 2026-07-08T08:16:41Z (E2E from POST /api/process 07:55:07 = ~21m34s); this run (GX010155, 7.84 GiB) reel at 2026-07-09T06:24:29Z (E2E from 06:04:19 = ~20m10s) ? showing the 35%-bigger clip ran ~1m24s faster. I re-opened the worktree at /c/ksrunlog on branch docs/runlog-cp-downgrade-cuj (HEAD 9c90198) and made two successful Edit calls: (1) added the "E2E req?reel" column to the §4 table with per-row values and the flat-to-faster insight paragraph; (2) updated the §3 end-to-end row. Both edits returned success. The file changes are staged in the working tree but NOT yet committed/pushed.

9. Optional Next Step:
   Commit and push the runlog edits to update PR #549, then clean up the worktree, then show the user the updated table with the E2E column. This is directly in line with the user's most recent request: "the table does not show the end to end time from upload to the reel generated state. Can that be added here and in the runlog PR?" Specifically: `cd /c/ksrunlog`, `git add docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-2vcpu-downgrade_2026-07-09.md`, `git commit` (with Co-Authored-By: Claude Opus 4.8 trailer), `git push` (auto-updates PR #549), then `git -C <MAIN> worktree remove --force /c/ksrunlog`. Then present the updated E2E comparison highlighting that the 7.84 GiB clip (~20m10s) ran ~1m24s faster end-to-end than the 5.81 GiB compile-on-L4 predecessor (~21m34s).

If you need specific details from before compaction (like exact code snippets, error messages, or content you generated), read the full transcript at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849.jsonl
Continue the conversation from where it left off without asking the user any further questions. Resume directly ? do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.

643. assistant (2026-07-09T06:48:39.101Z)

Heads-up before you start editing: your cwd `C:\Users\avidu\Projects\badminton-highlight-indexer` is the near-empty STUB (not a git repo). The REAL code is at `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (see the `actual-repo-location` memory). Do all work there.

Concrete anchors for this task in the real repo:
- `backend/worker/__main__.py` ? validation runs ~line 290 (`validator_registry.run_all_with_context(...)`) but is SILENT on pass and results go only to the ephemeral DB, never report.json. Add per-stage log lines + wall-clock timing (validation / segment / compile) so runlogs can capture "validation took Ns". Also emit a pass summary, not just FAILED/SKIPPED.
- The `_progress` callback (~line 388, wired `progress=_progress` ~line 434) writes `<video_id>_detector_progress.json` + prints `[worker] progress:` ? but on the NATIVE-L4 path it never fires (heartbeat dead: run_telemetry.jsonl was just a 116-byte header on the 07-09 live CUJ). Wire the progress callback through `backend/pipeline/detectors/native_wasb_runner.py` so it actually emits on the native path.

Git safety (shared checkout, concurrent sessions): `git -C <real-repo> fetch origin` then `git checkout -b <name> origin/master`; stage explicit paths (never `git add -A`); before PR, `git log --oneline origin/master..HEAD` must list ONLY your commits. Full local gate before LGTM:

run_all_tests.py (--cov-fail-under=70), ruff (pinned 0.15.16) + mypy clean. End commits with the Co-Authored-By: Claude Opus 4.8 trailer.

644. user (2026-07-09T07:12:46.249Z)

Message sent to session local_093db136-a1fc-4a4e-b738-d36f1a41aa8d ("Fix worker per-stage logs/timing + dead L4 heartbeat").

645. assistant (2026-07-09T07:12:55.579Z)

Sent. ? The running chip (`local_093db136?`, *"Fix worker per-stage logs/timing + dead L4 heartbeat"*) now has a nudge that:

- **Redirects it to the real repo** ? `?\khelsutra-guru\badminton-highlight-indexer` ? so it doesn't burn a lap in the empty stub.
- **Hands it the exact anchors** ? validation-visibility gap at `backend/worker/__main__.py:~290` (silent on pass, results only to ephemeral DB) and the dead native-L4 heartbeat to wire through `native_wasb_runner.py`.
- **Front-loads the guardrails** ? git-safety (branch from `origin/master`, explicit-path staging) and the pre-LGTM gate (full suite ?70% cov, pinned ruff + mypy, Co-Authored-By trailer).

It'll land as a user turn in that session with a link back here. So the state is now clean:

chip	status
`task_f925e323` (worker observability)	**running**, now pointed at the real repo ? the 1 pending chip
`task_2e0a3b82` (GCSFuse Windows)	**done ? [#548](https://github.com/Khelsutra/badminton-highlight-indexer/pull/548) merged**
PR [#549](https://github.com/Khelsutra/badminton-highlight-indexer/pull/549) (runlog + E2E column)	open, awaiting your review

Nothing else needs your input right now ? the chip will open its own PR when it's ready.

646. user (2026-07-09T07:13:22.867Z)

PR 549 merged

647. user (2026-07-09T07:13:41.629Z)

#549 [MERGED] merged=2026-07-09T07:13:09Z squash=6cfcb37cd docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09)
--- still-open PRs from this session's work ---
#550 fix(worker): make the cloud-serving L4 infer path observable

648. assistant (2026-07-09T07:14:17.281Z)

#549 merged clean (squash `6cfcb37c`). But something important surfaced in that check ? **PR #550 is open, and its title is "make the cloud-serving L4 infer path observable"**, which is *exactly* the scope I handed the chip. Let me make sure I didn't send the chip to duplicate existing work.

649. user (2026-07-09T07:14:23.368Z)

=== #550 metadata + files ===
{"author":"avidullu","branch":"fix/worker-l4-infer-observability","created":"2026-07-09T06:56:46Z","files":["backend/pipeline/detectors/native_wasb_runner.py","backend/pipeline/detectors/wasb_infer.py","backend/worker/__main__.py","tests/test_native_wasb_runner.py","tests/test_worker.py"],"number":550,"state":"OPEN","updated":"2026-07-09T06:56:46Z"}

=== #550 commit authors (chip commits end with the Co-Authored-By trailer) ===
b3dff7831 fix(worker): make the cloud-serving L4 infer path observable

650. assistant (2026-07-09T07:15:23.764Z)

`#549` merged clean (squash `6cfcb37c`). But this changes the picture on the chip ? let me reconcile it before saying we're done.

PR #550 is `fix/worker-l4-infer-observability`, created **06:56:46Z**, touching exactly the files I scoped to the chip ? `native_wasb_runner.py`, `worker/__main__.py`, `wasb_infer.py` + tests. That's the chip's deliverable, and it means the chip **had already finished and opened its PR before I sent the nudge** (my `list_sessions` check showed 06:44Z; the PR landed at 06:56Z). Let me confirm the chip isn't now redundantly re-doing the work because of my belated message.

651. user (2026-07-09T07:15:24.015Z)


```
[
  {
    "sessionId": "local_093db136-a1fc-4a4e-b738-d36f1a41aa8d",
    "title": "Fix worker per-stage logs/timing + dead L4 heartbeat",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "prNumber": 550,
    "prState": "OPEN",
    "lastActivityAt": "2026-07-09T07:13:58.281Z"
  }
]
```


652. assistant (2026-07-09T07:16:08.978Z)

Now it's fully clear ? and it's a clean outcome:

The chip session is **idle** (`isRunning: false`), **linked to `prNumber: 550`**, last active `07:13:58Z`. So my belated nudge caused **no duplicate** ? the chip had already self-located the real repo (via the shared `actual-repo-location` memory, exactly as predicted), done the work, opened #550, and stood down. Let me pull the specifics of #550 so I can tell you what actually landed, and keep memory honest about #549.

653. user (2026-07-09T07:16:10.520Z)

```
1       ---
2       name: pr550-worker-l4-observability
3       description: "PR #550 (open) makes the cloud-serving L4 worker infer path observable;
live-L4 heartbeat still needs an owner CUJ"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: e663947f-2127-4383-9905-650340ff9863
8       ---
9
10      # PR #550 ? cloud-serving L4 infer observability (OPEN, 2026-07-09)
11
12      Branch `fix/worker-l4-infer-observability` cut from `origin/master @ 00e8f92`; commit
`b3dff78`.
13      Fixes three linked observability gaps on `backend/worker/__main__.py`'s infer path,
surfaced by the
14      2026-07-09 live CUJ (video GX010155, deployed image `sha256:1838c07f?`, worker `rally-
worker-dpssz`).
15      Because `phase_placement.validation/segment/compile` are all `cloud_run_l4`, the worker
runs the full
16      validator suite + segmentation but none of it was observable.
17
18      - **GAP 1 (validation invisible on PASS):** now logs `[worker] validating (N checks)?` +
19        `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`, and the per-
validator verdicts
```

are folded into the pushed `report.json` under `validation` (previously only in the ephemeral DB).
- **GAP 2 (no per-stage timing):** checksum/validation/segmentation each timed with `time.perf_counter()`, logged, and persisted as a `timings` block in `report.json` (built incrementally so a mid-run `_fail()` still records the stages that ran).
- **GAP 3 (#534 heartbeat never fired on native-L4):** TWO compounding bugs ? (a) `wasb_infer.py` never configured logging so its `logger.info` `done/total (pct%)` progress sat below root=WARNING and was dropped; (b) even when emitted it goes to **stderr** but `NativeWasbRunner` read only **stdout** (and draining stdout while stderr's ~64 KB pipe fills risks a deadlock). Fix: `wasb_infer._configure_logging()` ? INFO to **stdout**; runner merges **stderr?stdout** (line-buffered) + bounded failure tail. Callback now fires ? `run_telemetry.jsonl` snapshots + `[worker] progress:` heartbeat + `<video_id>_detector_progress.json` on GCS.

`report.json` changes are additive; CP reads it via `report.get(...)` (`probe_process_cache` / `cached_process_result`) so existing readers are unaffected.

Verified locally (Windows): `run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? 1965 passed / 1 skipped, coverage 84.44% (worker `__main__.py` 91%); ruff 0.15.16 + mypy 2.1.0 clean. No `test_storage.py` failures ? #548 (already on master) fixed those on Windows.

OPEN follow-up (not verifiable offline): the live-L4 heartbeat end-to-end (real GPU + `wasb` env + Cloud Run L4) needs an **owner-driven CUJ** to confirm the telemetry snapshots / progress sidecar / stdout heartbeat actually land. The offline tests lock the emission config, the stdout parse+merge contract, and the report/timing serialization ? but not the real-hardware behavior. Related: [[ci-windows-coverage-gap]] (why the local Windows run matters), [[live-deployment-state]] (deploy the image after merge to make this live).

654. user (2026-07-09T07:16:10.818Z)

Memory index

- ? **Start here (GPU-resident rollout):** session-handoff-gpu-resident.md ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
- ? **LIVE deploy (2026-07-09):** live-deployment-state.md ? CP rev **00023-spm** + worker on digest **1838c07f** (master **00e8f92**; ships #531/#535/#537/#538/#539/#529/#534/#548/#547 over 304af7b). **CP DOWNGRADED 4/16 ? 2 vCPU/8 GiB** (safe via #539 acquire-gating; val/seg/compile all on L4). Image-only bump preserves the baked cloud-serving/phase_placement config. Verified 200 on /api/config. Rollback ? digest **7b62a8df** @ 4/16 or rev **00022-29w**. Release-process gaps ? **#532**.
- ? **OPEN PR #550 (2026-07-09):** pr550-worker-l4-observability.md ? makes the cloud-serving L4 worker infer path observable (validation PASS logging + verdicts in report.json, per-stage `timings`, and the #534 heartbeat FINALLY firing: `wasb_infer` logs INFO?stdout + runner merges stderr?stdout). Local gate green (1965 pass, 84.44%, ruff/mypy clean). **Live-L4 heartbeat still needs an owner CUJ.**
- ? **CLOSED 2026-07-08:** followup-validation-latency-live-cuj.md ? live SPA CUJ filled both CF-gated cells on rev **00019** (validation wall **183 s** PASS + reel compiled; fresh clip, corroborates not re-measures Video-1). Docs **PR #522**. Surfaced stitch bottleneck ? issues **#521** (compile?L4 + config

placement) & **#520** (debug tracing).
7
8   - ? **Perf gate + nightly reorg (PR #547 MERGED 2026-07-09, squash `00e8f92f`):** perf-release-gate.md ? local release perf gate (prod vs HEAD, per-stage GPU timing, **stitch-only gating**) at `scripts/gates/perf/`; nightly CLIs consolidated into `scripts/gates/{perf,quality}/`; OS-aware (WSL on Windows, `detector_impl=native` on Linux). Finding: release **perf-neutral** (#531 within local noise). Related: the pre-existing Windows-only `test_storage` #535 failure (`task_2e0a3b82`) ? **FIXED in PR #548** (ci-windows-coverage-gap.md).
9
10   - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
11   - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.
12   - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.
13   - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when genuinely satisfied a PR is merge-ready (verified), for all PRs.
14   - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
15   - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? the "weights not found" trap fix: #511 + #169 + revendor #516 merged AND **DEPLOYED LIVE** (control-plane rev `00018-qvz`, image `sha256:8045f54e?`; rollback ? rev `00017-qbd`).
16   - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now share ONE decode pass (**PR #512 MERGED**, squash 20862d7); pure grab()-through is SLOWER than seeks ? use forward-hybrid. **CLOSED 2026-07-07: DEPLOY_REPORT + PR #519 MERGED (squash 71c88ef)** (real-L4 A/B 169?70 s = 2.43×; off-box 1.95?3.23×; zero decision drift). True-4K sub-second follow-up = issue #514.
17   - [PR #518 validation-result cache](pr518-validation-result-cache.md) ? reupload skips re-validation when config unchanged (video_id-keyed cache, migration v10). **MERGED** 2026-07-07 (squash 699d85b; CI 11/11). ? v10 ? deploy worker+CP together.
18   - [PR #513 worker skip-revalidation](pr513-worker-skip-revalidation.md) ? worker skips redundant re-validation on cloud dispatch (fixes the blur-threshold CUJ). **MERGED** 2026-07-07 (squash 23c1868). **CLOSED live 2026-07-07: DEPLOY_REPORT + PR #519 MERGED (squash 71c88ef)** ? dispatch-faithful worker A/B proved Video 1 FAIL?PASS / 13 rallies (validation 169 s+reject ? 0 s+PASS); reel + live CP wall stay owner-gated (CF Access 401). ? landed first on orchestration.py + test_api_cloud_dispatch.py, so PR #511 rebased onto master (#512 was file-disjoint).
19   - [CI Windows coverage gap](ci-windows-coverage-gap.md) ? CI runs the full suite (incl. `test_storage.py`) on **Linux only**; the Windows lane runs only golden fixtures. Windows-only bugs (path seps) escape CI ? run `run_all_tests.py` locally on Windows. Caused #535's latent bug; **fixed PR #548**.
20   - [Cloud-run exec-status fast-fail](cloud-run-exec-status-fastfail.md) ? the #513 follow-up: `run_infer` now probes the Executions API so a worker that crashes before the done-marker fails fast (rc 5) instead of a ~65-min hang. **PR #515 MERGED** 2026-07-07 (squash `ef41473`; CI 11/11 green). Adversarial review caught+fixed a proto-plus `.seconds` blocker pre-merge.
21

655. assistant (2026-07-09T07:16:53.477Z)

Everything reconciles cleanly now ? and my earlier worry was unfounded. Here's the verified picture:

The `pr550` memory shows the chip **cut its branch from `origin/master @ 00e8f92` and edited the real `backend/worker/__main__.py`** ? so it worked in the *real* repo the entire time (the shared `actual-repo-location` memory did its job). The stub `cwd` was just the launch label. My nudge landed after the PR was already up and the session went idle ? **zero duplication**.

Let me keep the index honest about #549, then I'll lay out where #550 stands.

656. user (2026-07-09T07:16:58.262Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-

indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

657. assistant (2026-07-09T07:17:21.798Z)

All reconciled and memory's honest. Here's where things stand:

? #549 merged ? deploy fully documented

Squash `6cfcb37c` on master; MEMORY.md index updated to point at the runlog (with the E2E table). Nothing left open on the runlog thread.

The chip did its job ? that's PR #550

The single combined observability chip (`task_f925e323`) worked in the **real repo** start to finish, opened **[#550](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550)**, then went idle. It closed all three gaps we scoped:

gap	fix in #550
Validation invisible on PASS	`[worker] validation PASSED in Ys ? <check>=ok?` logs **+ verdicts folded into `report.json`** (was ephemeral-DB-only)
No per-stage timing	checksum/validation/segmentation timed via `perf_counter`, persisted as a `timings` block in `report.json`
#534 heartbeat dead on native-L4	two compounding bugs ? `wasb_infer` never configured logging (INFO sat below root=WARNING) **and** it logs to *stderr* while the runner read only *stdout*. Now: INFO?stdout + runner merges stderr?stdout (deadlock-safe). Heartbeat/telemetry/progress-sidecar finally fire.

Local gate **green**: 1965 passed / 1 skipped, 84.44% cov (worker `__main__` 91%), ruff 0.15.16 + mypy clean. `report.json` changes are additive (CP reads via `.get(...)`, existing readers unaffected).

One honest open item: the offline tests lock the *emission config + stdout parse/merge contract + serialization* ? but **not real-hardware behavior**. Confirming the heartbeat actually lands on the live L4 needs a deploy (image bump after merge) + an **owner-driven CUJ** ? same kind of run you just did for GX010155.

Your call on #550

My recommendation: let me **verify it to your pre-LGTM bar** in the real repo before it merges ? reproduce the full suite + ruff + mypy + coverage-floor myself (not trust the chip's self-report), review the diff, and post a verified LGTM. Then you merge ? deploy ? one CUJ closes the live-heartbeat item.

Want me to run that verification pass on #550 now?

658. user (2026-07-09T08:11:34.931Z)

<system-reminder>
The separate session for background task task_f925e323 ("Fix worker per-stage logs/timing + dead L4 heartbeat") has ended.
</system-reminder>

the PR is merged along with a couple more. Can you rebuild and upload the binaries to both the CP and worker? Also can you give a "Release Summary" identifying the new PRs and features which went in and ensuring that there are ways (either default or via config) to enable them and nothing is silently being kept disabled. Reload the jobs and then I will run the CUJ with the latest cut.

659. assistant (2026-07-09T08:12:24.332Z)

This is a production deploy you've explicitly authorized ? I'll proceed, validating each step (build ? worker-first ? CP ? verify), and produce the Release Summary with an enablement audit so nothing ships silently disabled.

First, per convention, let me sync the real repo and see exactly what's new since the live image (`00e8f92`), and locate the build tooling.

660. user (2026-07-09T08:12:29.944Z)

```
=== branch / clean? ===
fix/worker-l4-infer-observability
?? "SESSION_HANDOFF (1).md"
=== fetch ===
fetched
=== delta 00e8f92..origin/master (merged since live image) ===
01d1081 fix(worker): make the cloud-serving L4 infer path observable (#550)
6cfcb37 docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09) (#549)
=== new HEAD ===
01d1081
```

661. user (2026-07-09T08:12:30.075Z)

```
deploy\cloudrun\cloudbuild.yaml
deploy\gcp\nightly_regression\Dockerfile.launcher
deploy\gcp\nightly_regression\cloudbuild.launcher.yaml
deploy\cloudrun\gen_service_yaml.py
deploy\cloudrun\Dockerfile
```

662. assistant (2026-07-09T08:12:53.119Z)

Master's tip is `01d1081` (#550), and the code delta since the live image (`00e8f92`) is **just #550** ? #549 is docs-only (the runlog). You mentioned "a couple more," so let me reconcile against GitHub's merge log before I build ? I don't want to cut an image that's missing something you expect.

663. user (2026-07-09T08:13:00.098Z)

```
=== last 8 merged PRs (most recent first) ===
01d1081 #550 2026-07-09T08:07:18Z  fix(worker): make the cloud-serving L4 infer path observable
6cfcb37 #549 2026-07-09T07:13:09Z  docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ
(2026-07-09)
00e8f92 #547 2026-07-09T05:29:39Z  feat(gates): local release perf gate + consolidate nightly
into scripts/gates/
2084d56 #548 2026-07-09T04:49:03Z  fix(storage): OS-native GCSFuse fused path (fixes Windows-
only #535 test failures)
3249451 #546 2026-07-08T19:05:01Z  feat(#534): live detector progress heartbeat + L4 worker
resource telemetry
e29172a #545 2026-07-08T18:02:49Z  perf: gate CP acquire_local_input when validation+segment
delegated to L4 (#539)
af54f2f #544 2026-07-08T17:35:17Z  fix(#538): cancel propagates to the dispatched L4 worker exec
14e4767 #543 2026-07-08T15:45:43Z  observability: emit structured signal on GPU encoder fallback
to CPU (#529)

=== re-fetch + is origin/master still 01d1081? any newer? ===
01d1081
--- commits on master in the last 12h ---
01d1081 fix(worker): make the cloud-serving L4 infer path observable (#550)
6cfcb37 docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09) (#549)
00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
2084d56 fix(storage): build GCSFuse fused path with OS-native separators (#548)
```

664. assistant (2026-07-09T08:14:26.087Z)

Reconciled: master is at `01d1081`, and the **only new code since the live image is #550** ? #549 is docs-only (the runlog), and #548 was already baked into `00e8f92`. So this cut = current live image **+ #550 (worker L4 observability)**. Nothing newer than `01d1081` is on master, so if you were expecting other PRs, they haven't landed yet ? I'll flag that but proceed with #550, which is the meaningful new feature.

Let me pull the build/deploy mechanics + current live state (for the rollback anchor) and locate the config that governs whether #550's features are actually exercised.

665. user (2026-07-09T08:14:27.336Z)

```
1       # Cloud Build config for the Cloud Run GPU worker image (COMPUTE_DECOUPLED_SERVING M7).
2       #
3       # `gcloud builds submit --tag ? -f ?` does NOT work ? `gcloud builds submit` has no `-f`
flag, so a
4       # non-root Dockerfile needs a build CONFIG like this one. Build with:
5       #
6       #   gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project <PROJECT> .
7       #
8       # Override the region/repo/image with --substitutions=_REGION=?,_REPO=?,_IMAGE=? if
needed.
9       # PROJECT_ID is provided by Cloud Build automatically.
10      substitutions:
11        _REGION: asia-southeast1   # L4 GPU is region-limited (asia-southeast1 / europe-west1
/ europe-west4 / us-central1 / us-east4)
12        _REPO: rally
13        _IMAGE: rally-worker
14      steps:
15        - name: 'gcr.io/cloud-builders/docker'
16          args:
17            - build
18            - -t
19            - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
20            - -f
21            - deploy/cloudrun/Dockerfile
22            - .
23      images:
24        - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
25      options:
26        machineType: E2_HIGHCPU_8   # the CUDA + micromamba image is heavy; a bigger builder
keeps it ~8?10 min
27      timeout: 3600s
28
```

666. user (2026-07-09T08:14:28.225Z)

```
deploy\cloudrun\cloudbuild.yaml
deploy\cloudrun\gen_service_yaml.py
deploy\cloudrun\Dockerfile
deploy\cloudrun\post_deploy_smoke.ps1
deploy\cloudrun\README.md
```

667. user (2026-07-09T08:14:39.103Z)

```
=== gcloud reachable? project? ===
gen-lang-client-0306026826
=== CP current (rollback anchor) ===
rally-control-plane-00023-spm   asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-
worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb    2       8Gi
=== worker current image ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb
```

668. assistant (2026-07-09T08:16:03.789Z)

Everything I need to build is confirmed:
- **Rollback anchor:** CP `00023-spm` @ digest `1838c07f`, **2 vCPU / 8 Gi**; worker on the same digest.
- **Build:** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml .` (? `:latest`,

~8?10 min); project `gen-lang-client-0306026826`.

Let me check out the exact release commit and load task tracking, then I'll kick the build off in the background while I run the enablement audit.

669. user (2026-07-09T08:16:08.339Z)

```
HEAD is now at 01d1081 fix(worker): make the cloud-serving L4 infer path observable (#550)
HEAD=01d1081  branch=
=== untracked/dirty (build context cleanliness) ===
?? "SESSION_HANDOFF (1).md"
=== ignore files present? ===
ls: cannot access '.dockerignore': No such file or directory
.gcloudignore
```

670. assistant (2026-07-09T08:17:04.817Z)

Checked out `01d1081` (detached, clean tree; `.gcloudignore` governs the build context so the stray handoff file won't ship). Kicking off the build now in the background and setting up tracking + the enablement audit in parallel.

671. user (2026-07-09T08:17:07.814Z)

Command running in background with ID: b2u1j7ss6. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\b2u1j7ss6.output. You will be notified when it completes. To check interim output, use Read on that file path.

672. user (2026-07-09T08:17:09.798Z)

Task #19 created successfully: Build release image from 01d1081

673. user (2026-07-09T08:17:12.293Z)

Task #20 created successfully: Enablement audit of #550 features + cloud-serving config

674. user (2026-07-09T08:17:13.794Z)

Task #21 created successfully: Deploy worker (image-only bump)

675. user (2026-07-09T08:17:15.793Z)

Task #22 created successfully: Deploy CP (image-only bump, keep 2 vCPU / 8 Gi)

676. user (2026-07-09T08:17:17.298Z)

Task #23 created successfully: Verify live deployment

677. user (2026-07-09T08:17:19.292Z)

Task #24 created successfully: Release Summary + update memory

678. user (2026-07-09T08:17:20.292Z)

```
1       # Cloud Build upload context filter (deploy/cloudrun/cloudbuild.yaml).
2       #
3       # WHY THIS FILE EXISTS: without a .gcloudignore, `gcloud builds submit` falls back to
.gitignore ?
4       # which happens to exclude output/ + scratch/ (~8 GB), so the build "worked" by luck.
But the moment
5       # anyone adds a .gcloudignore for one exclusion, that .gitignore fallback STOPS and the
8 GB would be
6       # uploaded on every build. Make the context explicit here. Verify what uploads with:
```

```
7       #     gcloud meta list-files-for-upload
8       #
9       # Inherit every .gitignore exclusion (output/, scratch/, __pycache__, venvs, ?) ?
10      #!include:.gitignore
11      # ? and add what .gitignore does not cover but the image never needs:
12      .git/
13      # Untracked local dev/eval tools at the repo root (not gitignored, not COPY'd by the
Dockerfile).
14      score_local.py
15      score_golden_gate.py
16      # Docs/tests/CI are not part of the runtime image (the Dockerfile COPYs only
pyproject.toml,
17      # README.md, requirements-trackers.txt, backend/, training/). Dropping them keeps the
context small.
18      docs/
19      tests/
20      .github/
21
```

679. user (2026-07-09T08:17:21.365Z)

```
Found 7 files
backend\worker\__main__.py
backend\reporting\spec.yaml
backend\orchestration.py
backend\config\models.py
backend\config\presets.py
backend\config\placement.py
backend\config\__init__.py
```

680. assistant (2026-07-09T08:18:01.242Z)

Build is running (bg `b2u1j7ss6`); tasks #19?24 created. The `.gcloudignore` confirms the image only COPYs `backend/`, `training/` + 3 root files ? the stray handoff md can't ship. Now the enablement audit ? reading the config preset, placement logic, and the #550 worker path to confirm nothing lands silently disabled.

681. user (2026-07-09T08:18:01.722Z)

Updated task #19 status

682. user (2026-07-09T08:18:02.220Z)

Updated task #20 status

683. user (2026-07-09T08:18:02.739Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
```

```
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {
53                  "profile": "cloud-serving",
54                  "compute_target": "cloud_run",
55                  # #521: run the compile/stitch on the L4 worker (NVENC + 8 vCPU), not the
CPU-only
56                  # 2-vCPU control-plane where the 2026-07-08 CUJ measured ~13 min for an
11-clip 4K reel.
57                  # (validation stays on the control-plane ? the authoritative #512/#513 gate;
segment is
58                  # already cloud_run_l4 via compute_target.)
59                  "phase_placement": {"compile": "cloud_run_l4"},
60              },
61              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
62              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
63              # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
64              # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
65              "indexing": {
66                  "detector_impl": "native",
```

```
                "skip_ai_handoff": False,
                # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
                # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
                # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
                # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
                # nvdec the code-level serving default so it survives an image rebuild; the
worker's
                # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
                "wasb": {
                    "batch_size": 64,
                    "prefetch_batches": 4,
                    "decode_backend": "nvdec_resident",
                },
            },
            "storage": {"backend": "gcs"},
        },
        "cpu-mock": {
            "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
            # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
            "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
    }


    def is_known_profile(profile: str) -> bool:
        """True for a non-empty, recognised profile name."""
        return bool(profile) and profile in PROFILE_PRESETS


    def preset_for(profile: str) -> dict[str, Any]:
        """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
        module-level table), or ``{}`` for ``""`` / an unknown profile."""
        return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


    def _truthy(val: Optional[str]) -> bool:
        """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
        if val is None:
            return False
        return val.strip().lower() in {"1", "true", "yes", "on"}


    def _cuda_available() -> bool:
        """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
        ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
        try:
            # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
            # and optional). Only reached when a caller opts into probe_cuda.
            import torch

            return bool(torch.cuda.is_available())
        except Exception:
            return False

```

```
121
122     def probe_cuda_available() -> bool:
123         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
124
125         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
126         that *wants* the GPU heads-up can pass the result in without the startup module
importing
127         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
128         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
129         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
130         error that could false-fail."""
131         return _cuda_available()
132
133
134     def suggest_profile(
135         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
136     ) -> str:
137         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
138
139         Signals, in priority order:
140
141           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
142           - ``CI`` truthy                        ? ``"cpu-mock"``
143           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
144           - otherwise                            ? ``""`` (no strong suggestion)
145
146         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
147         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
148         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
149         """
150         e = os.environ if env is None else env
151         if e.get("K_SERVICE"):
152             return "cloud-serving"
153         if _truthy(e.get("CI")):
154             return "cpu-mock"
155         if probe_cuda and _cuda_available():
156             return "truly-local"
157         return ""
158
```

684. user (2026-07-09T08:18:03.852Z)

```
1       """Phase ? machine placement resolution (#521).
2
3       A *phase placement* answers one question per pipeline phase ? **where does the heavy
compute for
4       this phase run?** ? with a config knob (``deployment.phase_placement``) instead of code.
The three
5       placeable phases and their values:
6
7           validation  ?  control_plane | cloud_run_l4
8           segment     ?  control_plane | cloud_run_l4   (a.k.a. "detect")
9           compile     ?  control_plane | cloud_run_l4   (a.k.a. "stitch")
10
11      ``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
``cloud_run_l4``
12      dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already
uses). This module
```

```
13      is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the
worker never
14      re-derive the mapping.
15
16      **Default = today's behaviour (pure / additive).** When a phase's knob is unset
(``None``) the
17      resolver reproduces the pre-#521 placement:
18
19          validation  ?  control_plane                                    (the authoritative
gate, #512/#513)
20          segment     ?  cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
21          compile     ?  control_plane                                    (the in-process
ffmpeg stitch)
22
23      so a config with no ``phase_placement`` block behaves byte-identically to before this
module existed.
24      """
25
26      from __future__ import annotations
27
28      from typing import Any
29
30      # Phase identities (kept as constants so callers don't stringly-type the phase names).
31      PHASE_VALIDATION = "validation"
32      PHASE_SEGMENT = "segment"
33      PHASE_COMPILE = "compile"
34      PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
35
36      # Placement targets.
37      PLACE_CONTROL_PLANE = "control_plane"
38      PLACE_CLOUD_L4 = "cloud_run_l4"
39      PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
40
41
42      def resolve_phase_placement(cfg: Any, phase: str) -> str:
43          """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
44
45          Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the
legacy default
46          (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
47          ``deployment`` / ``phase_placement`` degrades to the legacy default.
48          """
49          deployment = getattr(cfg, "deployment", None)
50          placement = getattr(deployment, "phase_placement", None)
51          explicit = getattr(placement, phase, None) if placement is not None else None
52          if explicit:
53              return str(explicit)
54          # Legacy defaults ? reproduce today's placement exactly.
55          if phase == PHASE_SEGMENT:
56              compute_target = (getattr(deployment, "compute_target", "") or "").strip()
57              return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
58          # validation + compile default to the control-plane (their pre-#521 home).
59          return PLACE_CONTROL_PLANE
60
```

685. user (2026-07-09T08:18:04.404Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
```

```
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      import time
28      from typing import Any, List, Optional
29
30      from backend import storage
31      from backend.compute import ComputeJobSpec, get_compute_backend
32      from backend.config import (
33          StartupCheckError,
34          enforce_startup_checks,
35          load_config,
36          probe_cuda_available,
37      )
38      from backend.infrastructure.database import Database
39      from backend.pipeline.segmenters.base import segmenter_registry
40      from backend.pipeline.validators.base import registry as validator_registry
41      from backend.reporting import emit_indexer_report
42      from backend.results import Failure
43      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
44      from backend.utils.hashing import compute_video_id
45      from backend.utils.run_telemetry import RunTelemetry
46
47      logger = logging.getLogger("backend.worker")
48
49
50      def _coerce(v: str) -> Any:
51          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
52          low = v.lower()
53          if low in ("true", "false"):
54              return low == "true"
55          for cast in (int, float):
56              try:
57                  return cast(v)
58              except ValueError:
59                  pass
60          return v
61
62
63      def _apply_overrides(config: Any, sets: List[str]) -> None:
64          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true).
65          Supports dotted paths into plain dict fields (e.g. storage.gcs.mount_root=...) for
#535."""
```

```
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            last = parts[-1]
            coerced = _coerce(val)
            if isinstance(obj, dict):
                obj[last] = coerced
            else:
                setattr(obj, last, coerced)


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str,
        segments: List[dict],
        status: str,
        path: str,
        video_uri: str = "",
        timings: Optional[dict] = None,
        validation: Optional[List[dict]] = None,
        validation_skipped: bool = False,
    ) -> None:
        payload: dict[str, Any] = {
            "video_id": video_id,
            "status": status,
            "segment_count": len(segments),
            # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
            # control plane re-hydrating from outputs/<md5>/ restores the video row
            # with its truthful path ? without this, only the segments recover.
            "video_uri": video_uri,
            "segments": [
                {
                    "start": float(s.get("start_time", 0.0)),
                    "end": float(s.get("end_time", 0.0)),
                }
                for s in segments
            ],
        }
        # Observability (#534 follow-up, 2026-07-09 live CUJ): persist per-stage wall-clock
AND the
        # validator verdicts the worker actually computed. Previously both were invisible on
a PASS ?
        # the results lived only in the ephemeral worker DB and never reached this pushed
```

```
report, so a
128         # passing validation looked skipped and a ~10-min segmentation recorded no timing
anywhere.
129         if timings:
130             payload["timings"] = timings
131         if validation is not None:
132             payload["validation"] = validation
133         # Positive marker for the dispatcher-authoritative path: distinguishes "validation
was skipped
134         # here (the control-plane already gated it)" from a genuine empty validator suite
(validation=[]).
135         if validation_skipped:
136             payload["validation_skipped"] = True
137         with open(path, "w") as f:
138             json.dump(payload, f, indent=2)
139
140
141     def _serialize_and_push_outputs(
142         scratch: str,
143         out_uri: str,
144         video_id: str,
145         segments: List[dict],
146         status: str,
147         storage_cfg: dict,
148         video_uri: str = "",
149         timings: Optional[dict] = None,
150         validation: Optional[List[dict]] = None,
151         validation_skipped: bool = False,
152     ) -> List[str]:
153         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
154         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
155
156         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
157         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
158         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
159         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
160         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
161
162         ``timings`` (per-stage wall-clock), ``validation`` (the per-validator verdicts) and
163         ``validation_skipped`` are folded into report.json for observability (see
164         :func:`_write_report_json`)."""
165         out_local = os.path.join(scratch, "outputs", video_id)
166         os.makedirs(out_local, exist_ok=True)
167         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
168         _write_report_json(
169             video_id,
170             segments,
171             status,
172             os.path.join(out_local, "report.json"),
173             video_uri=video_uri,
174             timings=timings,
175             validation=validation,
176             validation_skipped=validation_skipped,
177         )
178         out_prefix = out_uri.rstrip("/")
179         pushed: List[str] = []
180         for fn in sorted(os.listdir(out_local)):
181             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
182             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
183             pushed.append(uri)
184         return pushed
```

```
185
186
187     def _run_startup_checks(config: Any) -> bool:
188         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
189         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
190         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
191         (returns True, ZERO work) when no deployment.profile is selected.
192
193         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
194         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
195         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
196         no-op of work, not just of output."""
197         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
198         cuda = (
199             probe_cuda_available() if profile else None
200         )  # torch-free on the default-OFF path
201         try:
202             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
203             return True
204         except StartupCheckError:
205             return False  # already logged at ERROR by enforce_startup_checks
206
207
208     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
209         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
210         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
211         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
212
213         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
214         from the local 2/3), never a raw traceback:
215           * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
216             running (the shim best-effort cancels it).
217           * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
218             crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
219             ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
220         from backend.compute import RemoteExecutionFailed
221         from backend.infrastructure.execution_policy import PolicyTimeout
222
223         spec = ComputeJobSpec(
224             video_uri=args.video_uri,
225             out_uri=args.out_uri,
226             segmenter=args.segmenter,
227             config_overrides=tuple(args.set or ()),
228             skip_validation=bool(getattr(args, "skip_validation", False)),
229         )
230         try:
231             result = compute.run_infer(spec)
232         except RemoteExecutionFailed as e:
233             logger.error(
234                 "[worker] remote execution terminated without a completion marker: %s", e
235             )
236             return 5
```

```
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    # Headline metric key per validator (in ``ValidationResult.details``) for the one-line
validation
    # summary log. The FULL details dict is persisted in report.json; this is only the at-a-
glance
    # score for the human-readable ``[worker] validation ?`` line. First key present wins.
    _VALIDATOR_SCORE_KEYS = (
        "avg_laplacian_variance",  # blur_check ? sharpness
        "avg_green_court_ratio",  # relevance ? court coverage
        "cuts_per_minute",  # scene_cut
        "max_keyframe_gap_seconds",  # pts_continuity
        "fps",  # resolution_fps
        "duration",  # format_check
    )


    def _val_token(r: Any) -> str:
        """One ``name=ok`` / ``name=FAIL(score)`` token for the validation-summary log line.

        ``score`` is a best-effort headline metric pulled from ``details`` (see
        :data:`_VALIDATOR_SCORE_KEYS`) and omitted when absent ? so this is robust to a
validator
        with no numeric detail and to the lightweight fakes used in tests (no ``details``
```

```
attribute)."""
297         name = getattr(r, "validator_name", "?")
298         ok = bool(getattr(r, "passed", False))
299         details = getattr(r, "details", None) or {}
300         hint = ""
301         for k in _VALIDATOR_SCORE_KEYS:
302             if k in details:
303                 hint = f"({k}={details[k]})"
304                 break
305         return f"{name}={'ok' if ok else 'FAIL'}{hint}"
306
307
308     def cmd_infer(args: argparse.Namespace) -> int:
309         config = load_config(args.config) if args.config else load_config()
310         _apply_overrides(config, args.set)
311         if not _run_startup_checks(config):
312             return 2
313
314         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
315         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
316         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
317         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
318         compute = get_compute_backend(config)
319         if compute is not None:
320             return _dispatch_remote(compute, args)
321
322         storage_cfg = config.storage.model_dump()
323
324         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
325         os.makedirs(scratch, exist_ok=True)
326         config.output.output_dir = scratch
327
328         # Per-stage wall-clock (#534 follow-up, 2026-07-09 live CUJ): each stage is timed
with
329         # perf_counter and persisted into report.json, so a runlog shows time-per-stage
instead of the
330         # earlier black hole (a ~10-min segmentation recorded nothing anywhere). Built
incrementally so
331         # a mid-run _fail() still reports the stages that DID run.
332         timings: dict[str, float] = {}
333
334         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
335         # For a multi-GB clip this download can dominate cloud wall-clock, so it is timed
too ? the
336         # mid-run _fail() path then still explains latency that accrued before checksum even
started.
337         _t = time.perf_counter()
338         local_video = storage.fetch(
339             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
340         )
341         timings["fetch_s"] = round(time.perf_counter() - _t, 3)
342         print(
343             f"[worker] pulled {args.video_uri} -> {local_video} ({timings['fetch_s']:.1f}s)"
344         )
345
346         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
347         _t = time.perf_counter()
348         video_id = compute_video_id(local_video)
349         timings["checksum_s"] = round(time.perf_counter() - _t, 3)
```

```
        print(f"[worker] checksum: video_id={video_id} ({timings['checksum_s']:.1f}s)")

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
        #
        # When it DOES run, log the verdict loudly (PASS or FAIL): on the cloud-serving path
        # phase_placement.validation=cloud_run_l4 means the worker really does run the full suite, but a
        # PASS previously logged nothing at all ? users reasonably concluded validation was skipped.
        skipped = bool(getattr(args, "skip_validation", False))
        if skipped:
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the authoritative "
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
            timings["validation_s"] = 0.0
        else:
            n_checks = len(validator_registry)
            print(f"[worker] validating ({n_checks} checks)?")
            _t = time.perf_counter()
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
            timings["validation_s"] = round(time.perf_counter() - _t, 3)
            verdict = "FAILED" if any(not r.passed for r in val_results) else "PASSED"
            summary = " ".join(_val_token(r) for r in val_results) or "(no checks ran)"
            print(
                f"[worker] validation {verdict} in {timings['validation_s']:.1f}s ? {summary}"
            )
        # The per-validator verdicts (name/passed/message/details) ? stored in the ephemeral worker DB
        # (below) and, when validation ran here, in the pushed report.json. On the skip path we persist
        # validation_skipped=true INSTEAD of an empty list, so a CP/P3 consumer can distinguish "the
        # dispatcher already validated (skipped here)" from "the worker ran a zero-validator suite".
        validation_payload = [r.model_dump() for r in val_results]
        report_validation = None if skipped else validation_payload
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            validation_payload,
```

```
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                    timings=timings,
                    validation=report_validation,
                    validation_skipped=skipped,
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                # The verdict + per-validator scores were already logged above (the
"validation FAILED
                # in Ys ? ?" line); add the full human-readable reason(s) under a DISTINCT
prefix so the
                # runlog isn't two near-duplicate "validation FAILED" lines.
                print(
                    "[worker] validation rejected by: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar +
```

```
telemetry
            os.makedirs(out_local, exist_ok=True)

            # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM
samples.
            # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
            telem: Optional[RunTelemetry] = None
            try:
                telem = RunTelemetry(
                    os.path.join(out_local, "run_telemetry.jsonl"),
                    label=f"l4-infer-{video_id[:8]}",
                )
            except Exception:  # noqa: BLE001 - telemetry must never break a production run
                telem = None

            out_prefix = getattr(args, "out_uri", None)
            if out_prefix:
                out_prefix = out_prefix.rstrip("/")

            # Configurable sampling for sidecar upload + telemetry snapshots (#534).
            # Local sidecar file is always kept up-to-date (cheap). Only the costly remote
put
            # and nvidia-smi-using snapshot are rate-limited. 0/negative = every callback.
            sample_sec = 15.0
            try:
                idx_cfg = getattr(config, "indexing", None)
                if idx_cfg is not None:
                    raw = getattr(idx_cfg, "detector_progress_sample_sec", 15.0)
                    sample_sec = float(raw) if raw is not None else 15.0
            except Exception:  # noqa: BLE001
                sample_sec = 15.0

            last_sample_t = [0.0]

            def _progress(s: str) -> None:
                print(f"[worker] progress: {s}")
                # Write sidecar for cloud/L4 poll to surface live detector progress in job
status (#534).
                # CP will read outputs/<video_id>/*_detector_progress.json during poll.
                # The *local* file is always written so the latest state is available in
scratch.
                now = time.time()
                do_sample = sample_sec <= 0 or (now - last_sample_t[0] >= sample_sec)

                pfile = os.path.join(out_local, f"{video_id}_detector_progress.json")
                try:
                    with open(pfile, "w") as pf:
                        json.dump({"msg": s, "t": now}, pf)
                except Exception:
                    pass  # best effort

                heavy_done = False
                # LIVE push to storage (the part visible to CP poll during the run).
                if out_prefix and do_sample:
                    try:
                        storage.put(
                            pfile,
                            f"{out_prefix}/outputs/{video_id}/{os.path.basename(pfile)}",
                            config=storage_cfg,
                        )
                        heavy_done = True
                    except Exception:
                        pass  # best-effort live heartbeat

```

```
                # Drive RunTelemetry snapshots (includes nvidia-smi + file rewrite) only on
sample.
                if telem and s and "detector" in (s or "") and do_sample:
                    try:
                        m = re.search(r"(\d+)/(\d+)", s)
                        done = int(m.group(1)) if m else None
                        total = int(m.group(2)) if m else None
                        telem.snapshot(done=done, total=total, phase="detector")
                        heavy_done = True
                    except Exception:  # noqa: BLE001
                        pass

                if heavy_done:
                    last_sample_t[0] = now

            seg_t = time.perf_counter()
            result = segmenter_cls(db, config).process_video(
                video_id,
                local_video,
                max_frames=args.max_frames,
                progress=_progress,
            )
            segmentation_ran = True
            timings["segmentation_s"] = round(time.perf_counter() - seg_t, 3)
            if isinstance(result, Failure):
                print(
                    f"[worker] segmentation ({segmenter_actual}) FAILED after "
                    f"{timings['segmentation_s']:.1f}s ? {result.message}"
                )
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            print(
                f"[worker] segmentation ({segmenter_actual}) done in "
                f"{timings['segmentation_s']:.1f}s ? {len(segments)} rallies"
            )
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
```

```
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
            timings=timings,
            validation=report_validation,
            validation_skipped=skipped,
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
        """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
        resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
        A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
        local = storage.fetch(
            spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
        )
        with open(local, encoding="utf-8") as f:
```

```
            data = json.load(f)
        if not isinstance(data, dict):
            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
        return data


    def _write_compile_marker(
        done_uri: str,
        result: dict,
        video_id: str,
        returncode: int,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "filename": result.get("filename", ""),
                "rendition": result.get("rendition", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
```

```
value as a
680         # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
681         # destination even if the override were dropped.
682         if not (config.storage.output_root or "").strip():
683             config.storage.output_root = args.out_uri
684 
685         spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
686         video_id = str(spec.get("video_id") or "")
687         out_name = str(spec.get("output_name") or "")
688         rendition = str(spec.get("rendition") or "original")
689         source_ref = str(spec.get("video_uri") or "")
690         locale = spec.get("locale")
691         time_pairs = [
692             (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
693         ]
694         stitching = StitchingConfig(**(spec.get("stitching") or {}))
695 
696         print(
697             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
698             f"encoder={stitching.encoder} rendition={rendition} out={out_name} "
699             f"ranges={time_pairs}"
700         )
701 
702         result = orchestration._stitch_reel(
703             config,
704             stitching,
705             video_id,
706             source_ref,
707             time_pairs,
708             out_name,
709             locale,
710             lambda stage: print(f"[worker] compile: {stage}"),
711             rendition,
712             # On the L4 worker the bucket mirror is the ONLY reel delivery ? the worker's
local file is
713             # discarded when the Job exits. A lost mirror MUST fail the stitch (not a
404-on-"success").
714             require_mirror=True,
715         )
716         status = str(result.get("status") or "failed")
717         rc = 0 if status == "success" else 3
718         if status != "success":
719             print(f"[worker] compile FAILED: {result.get('message')}")
720 
721         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
722         if getattr(args, "done_uri", None):
723             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
724         return rc
725 
726 
727     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
728     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
729     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
730 
731     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
732     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
733     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
734     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
735 
```

```
736
737     def _label_clip_name(fname: str) -> str:
738         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
739
740         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
741         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
742
743         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
744         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
745         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
746         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
747
748         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
749         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
750         """
751         name = fname.replace(".rallies.csv", "").replace(".csv", "")
752         return _LABEL_DATE_PREFIX.sub("", name)
753
754
755     def _probe_video_fps_width(path: str):
756         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
757
758         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
759         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
760         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
761         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
762         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
763         rather than guesses.
764         """
765         import json
766         import subprocess
767
768         try:
769             out = subprocess.check_output(
770                 [
771                     ffprobe_bin(),
772                     "-v",
773                     "error",
774                     "-select_streams",
775                     "v:0",
776                     "-show_entries",
777                     "stream=r_frame_rate,width",
778                     "-of",
779                     "json",
780                     path,
781                 ],
782                 stderr=subprocess.DEVNULL,
783             ).decode()
784             st = (json.loads(out).get("streams") or [{}])[0]
785             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
786             fps = float(num) / float(den or "1")
787             width = float(st.get("width") or 0)
```

```
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
```

```
synthetic
847             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
848           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
849             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
850             is the M3.5 "first real training" path; only the trajectory source flips.
851         """
852         import subprocess
853
854         config = load_config(args.config) if args.config else load_config()
855         _apply_overrides(config, args.set)
856         if not _run_startup_checks(config):
857             return 2
858         storage_cfg = config.storage.model_dump()
859
860         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
861         labels_dir = os.path.join(scratch, "labels")
862         traj_dir = os.path.join(scratch, "trajectories")
863         videos_dir = os.path.join(scratch, "videos")
864         os.makedirs(labels_dir, exist_ok=True)
865         os.makedirs(traj_dir, exist_ok=True)
866
867         corpus = args.corpus_uri.rstrip("/")
868         label_uris = sorted(
869             u
870             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
871             if u.lower().endswith(".csv")
872         )
873         if len(label_uris) < 2:
874             print(
875                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
876                 f"under {corpus}/labels/"
877             )
878             return 2
879
880         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
881         runner = None
882         video_by_stem: dict = {}
883         if args.trajectory_source == "detector":
884             os.makedirs(videos_dir, exist_ok=True)
885             runner = _resolve_train_detector(args, config)
886             if runner is None:
887                 return 2
888             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
889                 vname = storage.parse_uri(vuri).name
890                 if not vname.lower().endswith(_VIDEO_EXTS):
891                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
892                 video_by_stem[os.path.splitext(vname)[0]] = vuri
893
894         print(f"[worker] trajectory source: {args.trajectory_source}")
895         specs: List[dict] = []
896         for uri in label_uris:
897             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
898             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
899             local_label = storage.fetch(
900                 uri, os.path.join(labels_dir, fname), config=storage_cfg
901             )
902             if args.trajectory_source == "detector":
903                 vuri = video_by_stem.get(name)
904                 if not vuri:
905                     print(
```

```
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
```

```
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
```

```
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
            "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
            "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
            "`worker infer` CLI still validates.",
        )
        pi.set_defaults(func=cmd_infer)

        pc = sub.add_parser(
            "compile",
            help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
        )
        pc.add_argument(
            "--spec-uri",
            required=True,
            help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
            "config + source uri + video_id)",
        )
        pc.add_argument(
            "--out-uri",
            default="",
            help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
            "Also forwarded via --set storage.output_root; either resolves the reel path.",
        )
        pc.add_argument(
            "--done-uri",
            default=None,
            help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
            "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
```

```
stitch.",
        )
        pc.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pc.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pc.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set storage.output_root=gs://bucket",
        )
        pc.set_defaults(func=cmd_compile)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
```

```
1152        pt.set_defaults(func=cmd_train)
1153        return p
1154
1155
1156    def _setup_logging(verbose: bool) -> None:
1157        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
1158        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
1159        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
1160        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
1161        entry), not at import, so importing the module never reconfigures a caller's
logging."""
1162        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
1163        # configured logging at import), which would leave the level at WARNING and re-
suppress the
1164        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
1165        logging.basicConfig(
1166            level=logging.DEBUG if verbose else logging.INFO,
1167            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
1168            datefmt="%H:%M:%S",
1169            force=True,
1170        )
1171
1172
1173    def main(argv=None) -> int:
1174        args = build_parser().parse_args(argv)
1175        _setup_logging(getattr(args, "verbose", False))
1176        return args.func(args)
1177
1178
1179    if __name__ == "__main__":
1180        sys.exit(main())
1181
```

686. assistant (2026-07-09T08:19:36.595Z)

Worker code read confirms the key point for the audit: **#550's three features have no gating flags** ? validation logging, the `timings` block, and the `_progress`/telemetry heartbeat are all wired unconditionally in `cmd_infer` (lines 332, 373?385, 459?534). Nothing behind a disabled toggle.

The one thing I must verify is that the *baked* config on the live CP genuinely has the cloud-serving profile **explicitly active** ? because `presets.py:16` is emphatic that auto-detect only *suggests* a profile, never applies it. If the live CP relied on `K_SERVICE` auto-detect, the whole cloud-serving bundle (segment?L4, compile?L4, NVENC) would be silently off. Let me verify the live env + how the profile/placement is baked.

687. user (2026-07-09T08:19:43.848Z)

```
=== LIVE CP env (profile / placement / overrides) ===
spec:
  template:
    spec:
      containers:
      - env:
        - name: DEPLOYMENT_PROFILE
          value: cloud-serving
        - name: CLOUD_RUN_REGION
          value: asia-southeast1
        - name: GOOGLE_CLOUD_PROJECT
```

```
          value: gen-lang-client-0306026826
        - name: RALLY_ALLOWED_HOSTS
          value: '*'

=== build progress (peek) ===
submitting build from 01d1081 -> log: /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/deploy/build.log
```

688. user (2026-07-09T08:19:43.883Z)

```
1       """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3       The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4       session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5       clone. It emits a Knative service manifest for::
6
7           python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8           gcloud run services replace service.yaml --region asia-southeast1
9
10      Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11      a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12      cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13      gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15      The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16      it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17      server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18      backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19      identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20      service faces testers again.
21
22      Deliberate manifest choices (learned on the A1 deploy):
23        * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24          on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25        * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26          instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27          once state is shared.
28        * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29          with it; the code default ("rally-worker") is already correct.
30        * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31          committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32          `skip_ai_handoff=true` and never needs it).
33
34      Baked config pins (CONTROL_PLANE_DURABLE_REBUILD P1/D5 ? the live rev-00011 posture,
learned on
35      the 2026-07-05 owner CUJ; before this, regenerating the manifest silently DROPPED both):
36        * `indexing.default_segmenter=wasb_hybrid` ? the engine default is yolo_hybrid, so an
unpinned
```

```
37          cloud job ran the wrong segmenter (#479).
38        * `security.max_part_mb=24` ? upload parts must fit under the ~32 MiB HTTP/1 body cap
at the
39          Google Front End (#480); the engine default (95) gets every part 413'd before
FastAPI runs.
40          NOT fixed via `use-http2`: h2c requires the container itself to speak cleartext
HTTP/2,
41          which uvicorn does not (spec D6) ? the annotation would break the service.
42        * `--memory` defaults to 8Gi (spec D2) ? compile still pulls the gs:// source into the
43          RAM-backed /tmp to stitch, so the instance must hold the largest allowed upload with
44          headroom; >8Gi would force 4 vCPU on Cloud Run for no current need.
45      """
46
47      from __future__ import annotations
48
49      import argparse
50      import base64
51      import json
52      import sys
53
54      DEFAULT_PROJECT = "gen-lang-client-0306026826"
55      DEFAULT_REGION = "asia-southeast1"
56      DEFAULT_SERVICE = "rally-control-plane"
57      DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
58      DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
59      DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
60
61
62      def build_config(args: argparse.Namespace) -> dict:
63          """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
64          bucket = args.serving_bucket.rstrip("/")
65          return {
66              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
67              "indexing": {
68                  # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
69                  "skip_ai_handoff": True,
70                  # rev-00011 pin (#479): the engine default is yolo_hybrid ? pin the
validated one.
71                  "default_segmenter": args.default_segmenter,
72              },
73              "storage": {
74                  "backend": "gcs",
75                  "output_root": bucket,
76                  "input_backend": "gcs",
77                  "input_root": f"{bucket}/videos",
78              },
79              "security": {
80                  "edge_auth_enabled": args.edge_auth == "on",
81                  "cf_team_domain": args.cf_team_domain,
82                  "cf_access_aud": args.cf_access_aud,
83                  "allowed_video_root": "/tmp/apphome/output",
84                  # rev-00011 pin (#480): parts must clear the ~32 MiB HTTP/1 cap at the GFE.
85                  "max_part_mb": args.max_part_mb,
86              },
87          }
88
89
90      def build_yaml(args: argparse.Namespace) -> str:
91          """Render the Knative service manifest for `gcloud run services replace`."""
92          cfg_b64 = base64.b64encode(
93              json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
94          ).decode("ascii")
95          # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
```

```
the
96          # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
97          wrapper = (
98              "mkdir -p /tmp/apphome && "
99              f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
100             "export RALLY_APP_HOME=/tmp/apphome && "
101             "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
102         )
103         return f"""\
104     apiVersion: serving.knative.dev/v1
105     kind: Service
106     metadata:
107       name: {args.service}
108       labels:
109         cloud.googleapis.com/location: {args.region}
110     spec:
111       template:
112         metadata:
113           annotations:
114             autoscaling.knative.dev/minScale: "{args.min_instances}"
115             autoscaling.knative.dev/maxScale: "{args.max_instances}"
116             run.googleapis.com/cpu-throttling: "false"
117         spec:
118           timeoutSeconds: {args.timeout}
119           containers:
120             - image: {args.image}
121               command: ["/bin/bash"]
122               args: ["-c", "{wrapper}"]
123               ports:
124                 - containerPort: 8080
125               resources:
126                 limits:
127                   cpu: "{args.cpu}"
128                   memory: {args.memory}
129               env:
130                 - name: DEPLOYMENT_PROFILE
131                   value: cloud-serving
132                 - name: CLOUD_RUN_REGION
133                   value: {args.region}
134                 - name: GOOGLE_CLOUD_PROJECT
135                   value: {args.project}
136                 - name: RALLY_ALLOWED_HOSTS
137                   value: "*"
138     """
139
140
141     def main(argv: "list[str] | None" = None) -> int:
142         p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
143         p.add_argument(
144             "--edge-auth",
145             choices=("on", "off"),
146             required=True,
147             help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
"
148             "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
149         )
150         p.add_argument("--project", default=DEFAULT_PROJECT)
151         p.add_argument("--region", default=DEFAULT_REGION)
152         p.add_argument("--service", default=DEFAULT_SERVICE)
153         p.add_argument(
154             "--image",
155             default=None,
156             help="Container image; default = the project's rally-worker:latest (post-#470 it
"
157             "carries cloud-run + auth, so the worker image IS the control-plane image).",
```

```
158         )
159         p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
160         p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
161         p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
162         p.add_argument(
163             "--default-segmenter",
164             default="wasb_hybrid",
165             help="rev-00011 pin (#479): the engine default is yolo_hybrid; the alpha serves
the "
166             "validated wasb_hybrid.",
167         )
168         p.add_argument(
169             "--max-part-mb",
170             type=int,
171             default=24,
172             help="rev-00011 pin (#480): upload parts must fit under Cloud Run's ~32 MiB
HTTP/1 "
173             "request-body cap (the engine default 95 gets 413'd at the edge).",
174         )
175         p.add_argument("--min-instances", type=int, default=1)
176         p.add_argument(
177             "--max-instances",
178             type=int,
179             default=1,
180             help="Default 1: per-instance SQLite state splits across instances (issue
#471).",
181         )
182         p.add_argument("--cpu", type=int, default=2)
183         p.add_argument(
184             "--memory",
185             default="8Gi",
186             help="Default 8Gi (CONTROL_PLANE_DURABLE_REBUILD D2): compile pulls the gs://
source "
187             "into RAM-backed /tmp, so size for the largest allowed upload plus headroom.",
188         )
189         p.add_argument("--timeout", type=int, default=300)
190         p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
191         args = p.parse_args(argv)
192         if args.image is None:
193             args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"
194
195         text = build_yaml(args)
196         if args.output == "-":
197             sys.stdout.write(text)
198         else:
199             with open(args.output, "w", encoding="utf-8", newline="\n") as f:
200                 f.write(text)
201         return 0
202
203
204     if __name__ == "__main__":
205         raise SystemExit(main())
206
```

689. user (2026-07-09T08:19:43.978Z)

```
35-    PHASE_COMPILE,
36-    PHASE_SEGMENT,
37:    PHASE_VALIDATION,
38-    PLACE_CLOUD_L4,
39-    AppConfig,
40-    StitchingConfig,
41:    resolve_phase_placement,
```

```
42-)
43-from backend.pipeline.segmenters.base import segmenter_registry
--
226-    """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
227-    ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
``_execute_processing`` can
228:    decide whether to DELEGATE validation to the worker (``phase_placement.validation``). A
per-request
229:    picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is cloud
iff
230-    ``compute_target==cloud_run``) gates it, and cloud must actually be available. A pre-
dispatch reject
231-    (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
remotely."""
--
235-    if choice == "cloud":
236-        return _cloud_available(cfg)
237:    return resolve_phase_placement(
238-        cfg, PHASE_SEGMENT
239-    ) == PLACE_CLOUD_L4 and _cloud_available(cfg)
--
595-        )
596-    else:
597:        # No explicit picker ? consult phase_placement.segment (#521). Its legacy default is
598-        # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's behaviour;
setting
599:        # phase_placement.segment relocates DETECT by config alone (e.g. force it back on
the
600-        # control-plane, or onto the L4 on a default-local box wired for cloud).
601:        if resolve_phase_placement(
602-            cfg, PHASE_SEGMENT
603-        ) == PLACE_CLOUD_L4 and _cloud_available(cfg):
--
734-    # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
735-    # then worker-rejected at its default 20.0 after a successful GPU dispatch).
736:    # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
737-    # the authoritative gate ? it already validated with its resolved config (only a PASS
reaches
738-    # here), so the worker SKIPS its redundant re-validation (#513). If validation is placed
on the
--
740-    # _execute_processing), so the worker must NOT skip: it becomes the gate.
741-    worker_skips_validation = (
742:        resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
743-    )
744-    spec = ComputeJobSpec(
--
747-        segmenter=seg_name,
748-        config_overrides=tuple(overrides),
749:        skip_validation=worker_skips_validation,
750-    )
751-    try:
--
883-    # reports, etc., but we can get it cheaply without a full local copy.
884-    will_detect_on_cloud = _will_detect_on_cloud(config, getattr(req, "compute", None))
885:    validate_on_control_plane = resolve_phase_placement(
886:        config, PHASE_VALIDATION
887-    ) != PLACE_CLOUD_L4 or not will_detect_on_cloud
888-    segment_on_cp = not will_detect_on_cloud
--
1000-         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0 validation
time.
1001-         #
```

```
1002:        # #521 phase_placement.validation: computed early (see above) so we can gate
acquire_local_input
1003-        # for the pure-delegated case (#539). When delegated, CP skips validators; worker
is the gate.
1004-        if not validate_on_control_plane:
1005-            logger.info(
1006-                "[API] validation delegated to the L4 worker "
1007:                "(deployment.phase_placement.validation=cloud_run_l4) ? the control-plane
skips its "
1008-                "validator suite; the worker is the gate for %s.",
1009-                req.video_path,
--
1723-    a clean verdict instead of a worker-crash error.
1724-
1725:    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and
cloud
1726-    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1727-    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
--
1741-        }
1742-
1743:    if resolve_phase_placement(
1744-        cfg, PHASE_COMPILE
1745-    ) == PLACE_CLOUD_L4 and _cloud_available(cfg):
```

690. assistant (2026-07-09T08:21:19.257Z)

Now I have the live enablement picture, and it surfaces the subtle point I need to get right for the summary:

- **`DEPLOYMENT_PROFILE=cloud-serving` is explicitly set** on the live CP (verified in env) ? so the profile is genuinely applied, not left to suggest-only auto-detect. And `gen_service_yaml.py` bakes a `config.json` (profile + `wasb_hybrid` + gcs + edge-auth) that `services update --image` preserves. ?
- But the baked config sets **no `phase_placement.validation`**, and `placement.py:59` defaults validation ? `control_plane`. Per `orchestration.py:741-742`, that means the CP dispatches the worker **with `--skip-validation`** ? so #550's rich validation *PASS* logging would only fire if validation is placed on the L4. I need ground truth on where validation actually ran in the last live run before I write the summary.

691. user (2026-07-09T08:21:30.711Z)

```
=== recent worker executions ===
rally-worker-fkvqp      2026-07-09T06:21:06.775581Z
rally-worker-dpssz      2026-07-09T06:04:20.821211Z
rally-worker-98gqk      2026-07-08T09:30:32.225160Z
rally-worker-gvqzv      2026-07-08T09:17:35.023374Z
rally-worker-8gqkv      2026-07-08T08:53:07.352681Z

=== latest exec: rally-worker-fkvqp ? container args (does it carry --skip-validation?) ===
```

692. user (2026-07-09T08:21:30.851Z)

```
backend\orchestration.py:1437:            # stitching config, which may come from
deployment.cloud_stitch_encoder) and the
backend\orchestration.py:1441:            if requested in ("h264_nvenc", "hevc_nvenc") and
chosen == "libx264":
backend\orchestration.py:1596:    # (deployment.cloud_stitch_encoder, NVENC by default) so the
worker hardware-encodes the reel.
backend\orchestration.py:1599:        getattr(cfg, "deployment", None), "cloud_stitch_encoder",
"h264_nvenc"
backend\config\models.py:408:    # concat stays -c copy). Default ``libx264`` (CPU, works
everywhere incl. CI). ``h264_nvenc`` /
```

```
backend\config\models.py:409:    # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically
faster on the 4K re-encode that
backend\config\models.py:412:    # cloud compile-dispatch forwards
``deployment.cloud_stitch_encoder`` onto this for the L4 run;
backend\config\models.py:414:    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] =
"libx264"
backend\config\models.py:421:    # BOTH libx264 and h264_nvenc, frame-accurate (unlike a stream-
copy concat, which cuts at
backend\config\models.py:570:    # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the
4K re-encode the watermark forces;
backend\config\models.py:573:    cloud_stitch_encoder: Literal["libx264", "h264_nvenc",
"hevc_nvenc"] = "h264_nvenc"
backend\reporting\spec.yaml:63:  # #529: NVENC (or hevc_nvenc) was configured (stitching.encoder
or
backend\reporting\spec.yaml:64:  # deployment.cloud_stitch_encoder) but the compile stitch fell
back to libx264
backend\reporting\spec.yaml:72:      Hardware encoder (h264_nvenc or hevc_nvenc) was configured
for compile but
backend\reporting\spec.yaml:77:      probe size bug fixed in #527. Check `ffmpeg -encoders |
grep nvenc`
backend\pipeline\stitcher\compiler.py:34:#: re-encode win of #521/#525. Empirically (RTX 2070
Super, ffmpeg 8.1.1) h264_nvenc rejects
backend\pipeline\stitcher\compiler.py:99:        # works everywhere; "h264_nvenc"/"hevc_nvenc"
use the L4's HARDWARE encoder for the 4K
backend\pipeline\stitcher\compiler.py:381:        if enc in ("h264_nvenc", "hevc_nvenc") and not
_probe_encoder_available(enc):
```

693. assistant (2026-07-09T08:22:45.229Z)

NVENC enablement is clean: `deployment.cloud_stitch_encoder` **defaults to `h264_nvenc`** (models.py:573) and the cloud compile-dispatch forwards it onto the L4 stitch (orchestration.py:1596-1599), with #529 emitting a signal if it ever silently falls back to libx264. So the hardware encoder is on by default.

But I hit a **contradiction I must resolve before writing the summary**: my verified config analysis says validation defaults to `control_plane` (worker gets `--skip-validation`), yet the chip's own `pr550` memory claims "validation/segment/compile are all `cloud_run_l4`" (worker validates). These can't both be true. The last live worker run (`dpssz` = your GX010155 CUJ) is ground truth ? let me read its logs.

694. user (2026-07-09T08:23:01.803Z)

```
=== dpssz (GX010155 CUJ 06:04) worker logs ? validation/segment narrative ===
06:18:48 [INFO] backend.worker: [worker] wrote completion marker -> gs://khelsutra-rally-
serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/run_telemetry.jsonl
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/report.json
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/intervals.csv
[worker] infer done: 53 segment(s); pushed 3 artifact(s):
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: Phase 3 SKIPPED (fully-local):
emitted 53 candidate windows as rallies (no AI handoff).
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:12:22-00:12:27
(5.3s) | frames=25 peak_v=0.805 mean_v=0.101
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:12:06-00:12:08
(2.9s) | frames=57 peak_v=0.892 mean_v=0.203
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:54-00:11:58
(3.1s) | frames=77 peak_v=2.328 mean_v=0.290
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:37-00:11:46
(8.6s) | frames=232 peak_v=1.631 mean_v=0.208
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:23-00:11:27
(3.7s) | frames=104 peak_v=0.702 mean_v=0.190
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:10-00:11:15
(4.6s) | frames=129 peak_v=2.282 mean_v=0.196
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:01-00:11:04
(2.1s) | frames=49 peak_v=2.231 mean_v=0.185
```

```
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:53-00:10:55
(2.1s) | frames=38 peak_v=0.449 mean_v=0.208
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:23-00:10:34
(10.6s) | frames=269 peak_v=1.094 mean_v=0.219
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:12-00:10:16
(4.0s) | frames=48 peak_v=1.045 mean_v=0.197
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:02-00:10:06
(3.8s) | frames=104 peak_v=1.523 mean_v=0.205
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:09:15-00:09:28
(12.5s) | frames=357 peak_v=1.341 mean_v=0.204
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:08:55-00:09:00
(5.9s) | frames=158 peak_v=0.928 mean_v=0.173
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:08:41-00:08:49
(8.3s) | frames=225 peak_v=1.563 mean_v=0.271
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:08:27-00:08:33
(5.6s) | frames=34 peak_v=0.575 mean_v=0.145
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:07:48-00:07:56
(8.1s) | frames=209 peak_v=1.307 mean_v=0.226
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:07:42-00:07:45
(3.1s) | frames=34 peak_v=0.313 mean_v=0.096
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:07:35-00:07:40
(5.3s) | frames=145 peak_v=1.880 mean_v=0.255
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:07:24-00:07:27
(2.8s) | frames=83 peak_v=1.721 mean_v=0.284
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:07:11-00:07:18
(6.7s) | frames=185 peak_v=1.995 mean_v=0.292
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:53-00:07:01
(8.0s) | frames=217 peak_v=0.885 mean_v=0.228
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:43-00:06:46
(3.0s) | frames=35 peak_v=0.188 mean_v=0.101
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:26-00:06:29
(2.6s) | frames=65 peak_v=0.408 mean_v=0.145
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:17-00:06:23
(6.5s) | frames=179 peak_v=2.008 mean_v=0.225
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:11-00:06:15
(4.0s) | frames=16 peak_v=0.241 mean_v=0.169
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:06:00-00:06:05
(5.7s) | frames=58 peak_v=0.903 mean_v=0.239
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:05:45-00:05:47
(2.0s) | frames=43 peak_v=0.545 mean_v=0.220
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:05:37-00:05:39
(2.1s) | frames=45 peak_v=0.522 mean_v=0.129
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:05:26-00:05:30
(4.0s) | frames=111 peak_v=0.670 mean_v=0.202
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:05:03-00:05:07
(4.8s) | frames=130 peak_v=0.866 mean_v=0.195
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:04:50-00:04:54
(4.1s) | frames=105 peak_v=0.679 mean_v=0.190
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:04:16-00:04:19
(2.7s) | frames=71 peak_v=0.878 mean_v=0.326
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:04:01-00:04:06
(5.5s) | frames=153 peak_v=1.777 mean_v=0.235
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:03:49-00:03:54
(4.5s) | frames=52 peak_v=1.140 mean_v=0.152
=== (if empty above, raw first 20 lines) ===

Container called exit(0).
06:18:48 [INFO] backend.worker: [worker] wrote completion marker -> gs://khelsutra-rally-
serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/run_telemetry.jsonl
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/report.json
    gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/intervals.csv
[worker] infer done: 53 segment(s); pushed 3 artifact(s):
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: Phase 3 SKIPPED (fully-local):
emitted 53 candidate windows as rallies (no AI handoff).
```

```
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:12:22-00:12:27
(5.3s) | frames=25 peak_v=0.805 mean_v=0.101
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:12:06-00:12:08
(2.9s) | frames=57 peak_v=0.892 mean_v=0.203
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:54-00:11:58
(3.1s) | frames=77 peak_v=2.328 mean_v=0.290
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:37-00:11:46
(8.6s) | frames=232 peak_v=1.631 mean_v=0.208
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:23-00:11:27
(3.7s) | frames=104 peak_v=0.702 mean_v=0.190
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:10-00:11:15
(4.6s) | frames=129 peak_v=2.282 mean_v=0.196
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:11:01-00:11:04
(2.1s) | frames=49 peak_v=2.231 mean_v=0.185
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:53-00:10:55
(2.1s) | frames=38 peak_v=0.449 mean_v=0.208
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:23-00:10:34
(10.6s) | frames=269 peak_v=1.094 mean_v=0.219
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:12-00:10:16
(4.0s) | frames=48 peak_v=1.045 mean_v=0.197
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:10:02-00:10:06
(3.8s) | frames=104 peak_v=1.523 mean_v=0.205
06:18:47 [INFO] backend.pipeline.segmenters.trajectory_hybrid: [WASB rally] 00:09:15-00:09:28
(12.5s) | frames=357 peak_v=1.341 mean_v=0.204
```

695. assistant (2026-07-09T08:23:55.020Z)

Ground truth from the `dpssz` (GX010155) worker log resolves it: the worker went **straight to segmentation** (`trajectory_hybrid` ? 53 WASB rallies ? done) with **no validation lines at all**. That confirms my config reading and **contradicts the chip's `pr550` memory** ("validation? all `cloud_run_l4`"). The reality:

- **Validation runs on the control-plane** (the authoritative #512/#513 gate); the worker is dispatched `--skip-validation` and skips its redundant re-validation. That's *exactly* your earlier "straight away says segmenting without validation" observation ? it wasn't a bug, validation happened on the CP.
- After #550 deploys, that same path will now **explicitly log** `[worker] validation SKIPPED (dispatcher already validated)` instead of silently jumping to segment ? so the CUJ will finally *show* the step.

Let me check the build and record the audit outcome.

696. user (2026-07-09T08:23:58.358Z)

```
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading uvicorn-0.51.0-py3-none-any.whl (73 kB)
Downloading click-8.4.2-py3-none-any.whl (119 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Building wheels for collected packages: badminton-highlight-indexer
  Building wheel for badminton-highlight-indexer (pyproject.toml): started
  Building wheel for badminton-highlight-indexer (pyproject.toml): finished with status 'done'
  Created wheel for badminton-highlight-indexer:
filename=badminton_highlight_indexer-0.1.0-py3-none-any.whl size=1953647
sha256=2004c0d3ec841207358abfdde278818bad4e6472b690e96d8e3ef33f926fa10d
  Stored in directory: /tmp/pip-ephem-wheel-cache-
bt_v8p7l/wheels/9b/2c/d1/15e20a2b97f37ccf65a87ba1049c73a9076d0bf0fbaf814e83
Successfully built badminton-highlight-indexer
Installing collected packages: websockets, typing-extensions, tenacity, sniffio, python-dotenv,
pycparser, pyasn1, protobuf, MarkupSafe, h11, google-crc32c, distro, click, annotated-types,
annotated-doc, uvicorn, typing-inspection, pyjwt, pydantic-core, pyasn1-modules, proto-plus,
jinja2, httpcore, grpcio, googleapis-common-protos, google-resumable-media, exceptiongroup,
cffi, pydantic, grpcio-status, cryptography, anyio, starlette, httpx, grpc-google-iam-v1,
google-auth, google-api-core, fastapi, google-genai, google-cloud-core, google-cloud-storage,
google-cloud-run, badminton-highlight-indexer
```

697. assistant (2026-07-09T08:24:53.754Z)

Build is healthy ? in the pip/wheel install phase (no errors), a few minutes from done; I'll be notified when it completes. The audit is complete. Let me correct the chip's inaccurate memory now (independent of the build) and record the finding.

698. user (2026-07-09T08:24:54.263Z)

Updated task #20 status

699. user (2026-07-09T08:24:58.905Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

700. user (2026-07-09T08:25:05.819Z)

```
1       ---
2       name: pr550-worker-l4-observability
3       description: "PR #550 MERGED (squash 01d1081) ? cloud-serving L4 worker infer
observability; NOT yet deployed; live-L4 heartbeat still needs an owner CUJ"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: e663947f-2127-4383-9905-650340ff9863
8       ---
9
10      # PR #550 ? cloud-serving L4 infer observability (MERGED 2026-07-09, squash `01d1081`)
11
12      **MERGED to master `01d1081` 2026-07-09** (squash of b3dff78+e54e4a1+f7f0c82; all 11 CI
checks green;
13      both owner review passes were COMMENT-only, resolved). Remote branch auto-deleted. **NOT
yet deployed**
14      ? the worker/CP image must be rebuilt+redeployed for this observability to go live (the
live worker is
15      still on digest `1838c07f` = master `00e8f92`, pre-#550; see [[live-deployment-
state.md]]).
16
17      Branch `fix/worker-l4-infer-observability` was cut from `origin/master @ 00e8f92`;
commit `b3dff78`.
18      Fixes three linked observability gaps on `backend/worker/__main__.py`'s infer path,
surfaced by the
19      2026-07-09 live CUJ (video GX010155, deployed image `sha256:1838c07f?`, worker `rally-
worker-dpssz`).
20      Because `phase_placement.validation/segment/compile` are all `cloud_run_l4`, the worker
runs the full
21      validator suite + segmentation but none of it was observable.
22
23      - **GAP 1 (validation invisible on PASS):** now logs `[worker] validating (N checks)?` +
24        `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`, and the per-
validator verdicts
25        are folded into the pushed `report.json` under `validation` (previously only in the
ephemeral DB).
26      - **GAP 2 (no per-stage timing):** checksum/validation/segmentation each timed with
27        `time.perf_counter()`, logged, and persisted as a `timings` block in `report.json`
(built
28        incrementally so a mid-run `_fail()` still records the stages that ran).
29      - **GAP 3 (#534 heartbeat never fired on native-L4):** TWO compounding bugs ?
30        (a) `wasb_infer.py` never configured logging so its `logger.info` `done/total (pct%)`
progress sat
31        below root=WARNING and was dropped; (b) even when emitted it goes to **stderr** but
32        `NativeWasbRunner` read only **stdout** (and draining stdout while stderr's ~64 KB
pipe fills risks
33        a deadlock). Fix: `wasb_infer._configure_logging()` ? INFO to **stdout**; runner
```

```
merges
34        **stderr?stdout** (line-buffered) + bounded failure tail. Callback now fires ?
`run_telemetry.jsonl`
35        snapshots + `[worker] progress:` heartbeat + `<video_id>_detector_progress.json` on
GCS.
36
37      `report.json` changes are additive; CP reads it via `report.get(...)`
(`probe_process_cache` /
38      `cached_process_result`) so existing readers are unaffected.
39
40      **Verified locally (Windows):** `run_all_tests.py --cov=backend --cov=training --cov-
fail-under=70`
41      ? 1967 passed / 1 skipped, coverage 84.45% (worker `__main__.py` 91%); ruff 0.15.16 +
mypy 2.1.0 clean.
42      No `test_storage.py` failures ? #548 (already on master) fixed those on Windows.
43
44      **Review round addressed (commit `e54e4a1`)** ? owner review (1 bug, 3 suggestions, 1
nit) all fixed:
45      (1) **BUG:** the streaming (progress) path ? the PRODUCTION native-L4 path, worker
always passes a
46      callback ? enforced NO `timeout_sec`; a wedged child could hang forever. Now a watchdog
47      `threading.Timer` kills the child at the deadline ? EOF ? raise through the same
`TimeoutExpired`
48      handler as `_run` (that handler was dead code for streaming before). (2)
`len(validator_registry)`
49      via a new `ValidationRegistry.__len__` (no more private `_validators`). (3) de-duped
FAIL logs ? the
50      detail line is now `[worker] validation rejected by: ?` (verdict line keeps `validation
FAILED in
51      Ys ? ?`). (4) time the storage pull ? `timings["fetch_s"]` (often the dominant term for
multi-GB
52      clips; mid-run `_fail()` now captures it). (5) skip path writes `validation_skipped:
true` (omits the
53      `validation` array) instead of an ambiguous `validation: []`.
54
55      **2nd-pass review (commit `f7f0c82`)** ? one narrow correctness finding on the timeout
watchdog: a
56      clean `rc=0` finishing at the deadline could catch a late Timer fire and be misreported
as a timeout
57      (the flag, not the no-op `kill()`, was the bug). Fixed with two guards ? the watchdog
only flags a
58      live child (`if proc.poll() is None`), and the post-loop check only raises when the run
did NOT finish
59      cleanly (`timed_out.is_set() and not (res and res.returncode == 0)`), so success wins
the race. Also
60      carry the merged tail into `TimeoutExpired(stderr=?)` so a timeout logs "?last
output:?". Gate: 1968
61      passed / 1 skipped, 84.47%, ruff/mypy clean. Both review passes were COMMENT-only (not
blocking
62      CHANGES_REQUESTED); still awaiting a merge decision + the owner live-L4 CUJ.
63
64      **OPEN follow-up (not verifiable offline):** the live-L4 heartbeat end-to-end (real GPU
+ `wasb` env +
65      Cloud Run L4) needs an **owner-driven CUJ** to confirm the telemetry snapshots /
progress sidecar /
66      stdout heartbeat actually land. The offline tests lock the emission config, the stdout
parse+merge
67      contract, and the report/timing serialization ? but not the real-hardware behavior.
Related:
68      [[ci-windows-coverage-gap]] (why the local Windows run matters), [[live-deployment-
state]] (deploy the
69      image after merge to make this live).
70
```

701. user (2026-07-09T08:25:18.063Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

702. assistant (2026-07-09T08:25:39.784Z)

Memory corrected. Here's where we are ? the build is the only thing gating the deploy:

? Build (`01d1081`) ? healthy, in the dependency-install phase, ~a few minutes out. I'll be notified the moment it finishes, then deploy **worker ? CP** (image-only bump, preserving 2 vCPU/8 Gi + the baked cloud-serving config) and verify.

? Enablement audit ? done. Nothing ships silently disabled. The headline finding (which corrects a wrong note the chip left):

Feature	Live enablement	Silently off?
#550 per-stage `timings`	Unconditional ? always in `report.json`	No
#550 native-L4 heartbeat (#534 revival)	Unconditional; fires during segmentation. **Dead on the live image** (telemetry was a 116-byte header) ? *this deploy is what turns it on*	No
#550 validation logging	Worker is dispatched `--skip-validation` (validation?CP), so it now logs `validation SKIPPED (already validated)`; full PASS-logging available via `phase_placement.validation=cloud_run_l4`	No
segment ? L4	`compute_target=cloud_run` default	No ? active (`dpssz` ran WASB on L4)
compile ? L4 + **NVENC**	preset `phase_placement.compile=cloud_run_l4`; `cloud_stitch_encoder` defaults `h264_nvenc`; #529 flags any silent fallback	No ? on by default
validation ? control-plane	placement default; the authoritative #512/#513 gate	Deliberate (owner-confirmed)

The one nuance you'll want to know for your CUJ: validation runs on the **control-plane**, not the L4 worker ? so the worker "jumps straight to segmenting" *by design* (it's not skipping the gate, the CP already ran it). After this deploy the worker will finally **say so** in its log instead of being silent. If you'd rather the L4 worker be the gate and emit the full `validation PASSED in Ns ? blur=ok?` line, that's a one-knob flip (`deployment.phase_placement.validation=cloud_run_l4`) ? say the word and I'll bake it into this deploy.

Standing by for the build to finish.

703. user (2026-07-09T08:29:43.200Z)

So the entire validation still runs fully on CP?

704. assistant (2026-07-09T08:30:49.234Z)

Good question to pin down ? and worth being precise about, because I've now caught *two* memory notes that overstated where validation runs. Let me verify the CP-side mechanism from the code and the actual CP logs of your GX010155 run, rather than trust memory.

705. user (2026-07-09T08:30:50.680Z)

```
860         import backend.main as _main  # call-time seam: main's globals/model stay patchable
861
862         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
863         # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
864         if config is not None and not isinstance(config, AppConfig):
865             # Old signature: _execute_processing(req, progress=None)
866             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
867                 req = db
868                 db = None
869             elif req is None and isinstance(config, _main.ProcessRequest):
870                 req = config
```

```
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        # Early placement decision for gating local acquire (#539): when validation+segment
are
        # both delegated to cloud_run_l4 (and compute cloud), the CP is a pure thin
dispatcher and
        # does not need the source bytes at all. We still need a correct video_id for
caches,
        # reports, etc., but we can get it cheaply without a full local copy.
        will_detect_on_cloud = _will_detect_on_cloud(config, getattr(req, "compute", None))
        validate_on_control_plane = resolve_phase_placement(
            config, PHASE_VALIDATION
        ) != PLACE_CLOUD_L4 or not will_detect_on_cloud
        segment_on_cp = not will_detect_on_cloud
        needs_local_file_for_cp_work = validate_on_control_plane or segment_on_cp

        notify("hashing")
        # M2 (updated for #539): resolve the input. We gate the (potentially expensive)
acquire_local_input
        # so that a pure dispatcher CP (validation+segment on L4 + cheap video_id available)
does not
        # download the whole video just to hash it. The original req.video_path is kept for
DB/report.
        # Only hash/validate/segment consumers on the CP get a local path.
        local_video = ""
        try:
            # Prefer cheap video_id (submit-time metadata or trusted) without touching
bytes.
            video_id = None
            trust_submitted = bool(
                getattr(getattr(config, "deployment", None), "trust_submit_video_id", True)
            )
            candidate = trusted_video_id or ""
            if trust_submitted and _looks_like_video_id(candidate):
                video_id = candidate
                logger.info(
                    "[API] trusting submit-time video_id=%s ? skipped acquire for hash
(#524/#539).",
                    video_id,
                )
            else:
                input_ref = storage.resolve_input_ref(req.video_path, getattr(config,
"storage", None))
                video_id = submit_time_video_id(input_ref, config)
                if video_id and _looks_like_video_id(video_id):
                    logger.info(
                        "[API] video_id from object metadata (no full download) for %s
(#539).",
                        req.video_path,
                    )
                else:
                    video_id = None

            # Only acquire a local copy if CP-side work needs it OR we still must compute
the hash.
            if needs_local_file_for_cp_work or video_id is None:
                local_video = storage.acquire_local_input(
                    req.video_path, getattr(config, "storage", None)
```

```
                )
                if video_id is None:
                    video_id = _main.compute_video_id(local_video)
                    logger.info(
                        "[API] video_id computed after local acquire (no cheap id) for %s
(#539).",
                        req.video_path,
                    )
            else:
                # Pure delegated + cheap id: CP stays thin dispatcher, no full download.
                local_video = ""
                logger.info(
                    "[API] skipping CP acquire for fully delegated phases with cheap
video_id (#539).",
                )
        except Exception:
            if local_video:
                storage.cleanup_acquired_local_input(
                    req.video_path, local_video, getattr(config, "storage", None)
                )
            raise
        try:
            existing_video = db.get_video(video_id)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        was_reingest = existing_video is not None

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(
            getattr(config, "indexing", None), "default_segmenter", "motion"
        )
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        val_results: List[Any] = []
        val_context: Dict[str, Any] = {}
        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
        compute_actual: Optional[str] = None
        response: Optional[Dict[str, Any]] = None
        try:
            if (
                existing_video
                and existing_video.get("status") == "processed"
                and not req.force
            ):
                cached_segments = db.query_play_segments(video_id, {})
                if cached_segments:
                    logger.info(
                        "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
                        video_id,
                    )
                    response = _cached_process_payload(
                        video_id,
                        existing_video.get("validation_results", []),
                        cached_segments,
                        getattr(req, "compute", None),
```

```
                    )
                    return response

            # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
            #    UNLESS this exact content already has a cached verdict under the current
validator config.
            notify("validating")
            # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
            # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
            # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
            # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
            # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
            # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
            # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
            # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
            # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
            # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
            # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
            #
            # #521 phase_placement.validation: computed early (see above) so we can gate
acquire_local_input
            # for the pure-delegated case (#539). When delegated, CP skips validators;
worker is the gate.
            if not validate_on_control_plane:
                logger.info(
                    "[API] validation delegated to the L4 worker "
                    "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
                    "validator suite; the worker is the gate for %s.",
                    req.video_path,
                )
            val_fingerprint = (
                registry.config_fingerprint(config) if validate_on_control_plane else ""
            )
            # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
            # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
            # failing the job. The cache is a pure speedup ? never a new failure mode.
            reused_validation = False
            if validate_on_control_plane and not req.force:
                try:
                    cached = db.get_validation_cache(video_id)
                    if cached is not None and cached.get("fingerprint") == val_fingerprint:
                        # Replay the stored ValidationResult list; val_context stays {} (no
fresh
                        # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
                        # deliberate cost of skipping the decode). Segmentation reads
neither of these.
                        val_results = [
                            ValidationResult(**r) for r in cached.get("results", [])
                        ]
```

```
1028                         reused_validation = True
1029                         logger.info(
1030                             "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
1031                             "validator config unchanged ? skipping the validator suite. "
1032                             "force=true re-validates.",
1033                             cached.get("passed"),
1034                             video_id,
1035                         )
1036                 except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
1037                     reused_validation = False
1038                     logger.warning(
1039                         "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
1040                         video_id,
1041                         exc_info=True,
1042                     )
1043             if validate_on_control_plane and not reused_validation:
1044                 logger.info(f"Running validations for {req.video_path}...")
1045                 val_results, val_context = registry.run_all_with_context(
1046                     local_video, config
1047                 )
1048             failures = [r for r in val_results if not r.passed]
1049             if validate_on_control_plane and not reused_validation:
1050                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
1051                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
1052                 db.set_validation_cache(
1053                     video_id,
1054                     val_fingerprint,
1055                     not failures,
1056                     [r.model_dump() for r in val_results],
1057                 )
1058
1059             if failures:
1060                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1061                 # status; it no longer destroys the video's prior good segments.
1062                 db.add_video(
1063                     video_id,
1064                     req.video_path,
1065                     "failed",
1066                     [r.model_dump() for r in val_results],
1067                 )
1068                 response = {
1069                     "video_id": video_id,
1070                     "status": "failed",
1071                     "message": _validation_failure_message(failures),
1072                     "results": [r.model_dump() for r in val_results],
1073                 }
1074                 return response
1075
1076             # 2. Run segmenter & indexer
1077             logger.info("[API] Video passed checks. Segmenting and indexing...")
1078             db.add_video(
1079                 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
```

706. user (2026-07-09T08:30:59.078Z)

```
=== CP (rally-control-plane) logs during GX010155 CUJ (06:03-06:25) ? validation/acquire
narrative ===
2026-07-09T06:21:06.969673Z     06:21:06 [INFO] compute.cloud_run: cloud-run: dispatched COMPILE
job=rally-worker exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-fkvqp; bucket-polling gs://khelsutra-rally-
```

serving/_compile/f333a400b33b42e699ec89c8543aa568.done
2026-07-09T06:21:06.660589Z 06:21:06 [INFO] backend.orchestration: [compile] dispatching stitch for video_id=e2e144175c8d05451903ae44a9ce0aa0 rendition=original -> Cloud Run Job (out=gs://khelsutra-rally-serving, 19 clip(s))
2026-07-09T06:18:49.286582Z 06:18:49 [INFO] backend.orchestration: [COMPUTE] video_id=e2e144175c8d05451903ae44a9ce0aa0 requested=cloud actual=cloud_run dispatched=None status=success
2026-07-09T06:18:49.286530Z 06:18:49 [INFO] backend.orchestration: [COMPUTE] cloud_run dispatched: execution=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-worker/executions/rally-worker-dpssz job=rally-worker region=asia-southeast1 out=gs://khelsutra-rally-serving/outputs/e2e144175c8d05451903ae44a9ce0aa0/ video_id=e2e144175c8d05451903ae44a9ce0aa0
2026-07-09T06:04:21.023925Z 06:04:21 [INFO] compute.cloud_run: cloud-run: dispatched job=rally-worker exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-worker/executions/rally-worker-dpssz; bucket-polling gs://khelsutra-rally-serving/_dispatch/9d6f3ca2145e42459014f3362208866d.done
2026-07-09T06:04:19.401995Z 06:04:19 [INFO] backend.orchestration: [API] cloud-serving: dispatching gs://khelsutra-rally-serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4 -> Cloud Run Job (out=gs://khelsutra-rally-serving)
2026-07-09T06:04:19.399551Z 06:04:19 [INFO] backend.orchestration: [API] validation delegated to the L4 worker (deployment.phase_placement.validation=cloud_run_l4) ? the control-plane skips its validator suite; the worker is the gate for gs://khelsutra-rally-serving/videos/uploads/avidullu-cuj-run-1/GX010155.MP4.

707. assistant (2026-07-09T08:32:00.672Z)

The CP log is the ground truth and it **overturns my earlier reading** ? I need to correct myself:

```
06:04:19 [API] validation delegated to the L4 worker
  (deployment.phase_placement.validation=cloud_run_l4) ?
  the control-plane skips its validator suite; the worker is the gate for ?GX010155.MP4
```

So on your live run, **the CP did NOT validate** ? `phase_placement.validation` *is* `cloud_run_l4` in the live baked config, meaning validation was **delegated to the L4 worker** (which is why the CP could skip the download and survive on 2 vCPU/8 Gi ? #539). The `dpssz` worker log showed no validation lines because the **pre-#550 worker validates silently** ? exactly the bug #550 fixes. The chip's memory was right and *my* correction was wrong. Let me decode the actual live config to confirm, revert my bad memory edit, and grab the build digest (build just finished, exit 0).

708. user (2026-07-09T08:32:10.198Z)

=== LIVE CP baked config.json (decoded from the container args) ===


```
{
  "deployment": {
    "profile": "cloud-serving",
    "compute_target": "cloud_run",
    "phase_placement": {
      "validation": "cloud_run_l4",
      "segment": "cloud_run_l4",
      "compile": "cloud_run_l4"
    }
  },
  "indexing": {
    "skip_ai_handoff": true,
    "default_segmenter": "wasb_hybrid"
  },
  "storage": {
    "backend": "gcs",
    "output_root": "gs://khelsutra-rally-serving",
    "input_backend": "gcs",
    "input_root": "gs://khelsutra-rally-serving/videos"
```

```
  },
  "security": {
    "edge_auth_enabled": true,
    "cf_team_domain": "https://khelsutra.cloudflareaccess.com",
    "cf_access_aud": "c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b",
    "allowed_video_root": "/tmp/apphome/output",
    "max_part_mb": 24
  }
}

--- phase_placement ---
{
  "profile": "cloud-serving",
  "compute_target": "cloud_run",
  "phase_placement": {
    "validation": "cloud_run_l4",
    "segment": "cloud_run_l4",
    "compile": "cloud_run_l4"
  }
}
```

709. user (2026-07-09T08:32:15.172Z)

```
=== new image digest (from build) ===
latest: digest: sha256:261ccf3a63218e068fd5218e7ecff1e8ca1af8654932dbf5e9852f113ce16df7 size:
4309
DONE
5b0b41b9-ec13-4768-bcbc-3abfe5c9df90  2026-07-09T08:17:20+00:00  11M32S    gs://gen-lang-
client-0306026826_cloudbuild/source/1783585030.607411-1a6590afc71b4c04a34b619f07d786ea.tgz
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker (+1 more)  SUCCESS
--- authoritative digest via artifacts ---
sha256:261ccf3a63218e068fd5218e7ecff1e8ca1af8654932dbf5e9852f113ce16df7
```

710. assistant (2026-07-09T08:33:38.945Z)

Confirmed by decoding the live CP's baked `config.json` ? I owe you a correction. The live config is:

```json
"phase_placement": { "validation": "cloud_run_l4", "segment": "cloud_run_l4", "compile":
"cloud_run_l4" }
```

So no ? validation does NOT run on the CP. It runs fully on the L4 worker. All three heavy phases are delegated; the CP is a pure thin dispatcher that skips its validator suite *and* skips the download (#539) ? which is the actual reason 2 vCPU/8 Gi survives a 7.84 GiB clip. My earlier "validation stays on the control-plane" was wrong ? I read the `gen_service_yaml.py` defaults instead of the *actual baked* config, which carries an explicit full `phase_placement`. The chip's memory was right; my "correction" to it was the error. Reverting that now.

This also makes #550 **more** relevant than I said: since the worker *is* the validation gate, #550's validation logging is exercised on the live path (pre-#550 it validated silently). Build succeeded (`sha256:261ccf3a?`, 11m32s) ? fixing the memory and deploying the worker now.

711. user (2026-07-09T08:33:39.451Z)

Updated task #19 status

712. user (2026-07-09T08:33:39.954Z)

Updated task #21 status

713. user (2026-07-09T08:33:47.955Z)

<tool_use_error>String to replace not found in file.
String: ? **Correction (VERIFIED live 2026-07-09):** `phase_placement.segment/compile` are
`cloud_run_l4` but
validation STAYS on the control-plane (the worker is dispatched `--skip-validation` ?
VERIFIED 2026-07-09
from the live baked config + the `dpssz` worker log, which segments with ZERO validation lines),
the worker
runs SEGMENTATION on the L4; its own validation is SKIPPED (and was silently so). #550 makes the
worker log
its state EITHER way: `validation SKIPPED (dispatcher already validated)` on the default path,
or `validation
PASSED/FAILED in Ys` only if validation is explicitly placed on the L4
(`deployment.phase_placement.validation=cloud_run_l4`). (An earlier draft wrongly said
validation was on `cloud_run_l4`.)
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so
the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding
text.)</tool_use_error>

714. user (2026-07-09T08:33:58.880Z)

```
=== deploy WORKER (job image update) -> 261ccf3a ===
Job [rally-worker] has successfully been updated.

To execute this job, use:
gcloud run jobs execute rally-worker
=== verify worker now points at new digest ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:261ccf3a63218e068fd5218e7ecff1e8ca1af8654932dbf5e9852f113ce16df7
```

715. assistant (2026-07-09T08:34:18.572Z)

Worker is live on `261ccf3a` (verified ? job image updated). ? Now the CP (image-only bump,
preserving the full `phase_placement` + 2 vCPU/8 Gi), and re-reading the memory since a reformat
broke my revert.

716. user (2026-07-09T08:34:19.014Z)

Updated task #21 status

717. user (2026-07-09T08:34:19.548Z)

Updated task #22 status

718. user (2026-07-09T08:36:23.839Z)

Command running in background with ID: bb3flbnjb. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\744270aa-6a38-4011-b018-cdd9aae94849\tasks\bb3flbnjb.output. You will be notified when
it completes. To check interim output, use Read on that file path.

719. user (2026-07-09T08:36:23.879Z)

```
18      Fixes three linked observability gaps on `backend/worker/__main__.py`'s infer path,
surfaced by the
19      2026-07-09 live CUJ (video GX010155, deployed image `sha256:1838c07f?`, worker `rally-
worker-dpssz`).
20      ? **Correction (VERIFIED live 2026-07-09):** `phase_placement.segment/compile` are
`cloud_run_l4` but
21      **validation STAYS on the control-plane** ? the CP is the authoritative #512/#513 gate
and dispatches the
22      worker with `--skip-validation` (`orchestration.py:741`; the live baked config sets no
`phase_placement.validation`,
23      so `placement.py` defaults it to `control_plane`). Ground truth: the `dpssz` GX010155
worker log segments with
```

```
24      ZERO validation lines. So on the L4 worker only SEGMENTATION runs; its validation is
SKIPPED (silently, pre-#550).
25      #550 makes the worker log its state EITHER way ? `validation SKIPPED (dispatcher already
validated)` on the
26      default path, or the full `validation PASSED/FAILED in Ys` only if validation is
explicitly placed on the L4
27      (`deployment.phase_placement.validation=cloud_run_l4`). (An earlier draft wrongly said
validation was on `cloud_run_l4`.)
28
29      - **GAP 1 (validation invisible on PASS):** now logs `[worker] validating (N checks)?` +
30        `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`, and the per-
validator verdicts
31        are folded into the pushed `report.json` under `validation` (previously only in the
ephemeral DB).
```

720. assistant (2026-07-09T08:36:52.033Z)

The CP `services update` is running (it blocks until the new revision is Ready, so it went to background `bb3flbnjb` ? I'll be notified). Now I have the exact text to fix the memory ? reverting my wrong "validation on CP" note to the verified truth:

721. user (2026-07-09T08:36:59.078Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

722. assistant (2026-07-09T08:37:03.188Z)

CP deploy completed (exit 0), and the memory is corrected. Let me read the deploy result and verify the CP is healthy.

723. user (2026-07-09T08:37:05.089Z)

```
=== deploy CP (image-only bump) -> 261ccf3a ===
Deploying...
Creating Revision.....................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................done
Routing traffic.....done
Done.
Service [rally-control-plane] revision [rally-control-plane-00024-k5n] has been
deployed and is serving 100 percent of traffic.
Service URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
=== CP state after update (rev / image / cpu / mem) ===
rally-control-plane-00024-k5n   asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-
worker@sha256:261ccf3a63218e068fd5218e7ecff1e8ca1af8654932dbf5e9852f113ce16df7    2       8Gi
```

724. assistant (2026-07-09T08:37:32.337Z)

CP is live: **revision `rally-control-plane-00024-k5n`**, serving 100%, image `261ccf3a`, and **2 vCPU / 8 Gi preserved**. ? Now the health verification ? `/api/config` reachability + clean boot (profile active, no OOM on the small box with the new image).

725. user (2026-07-09T08:37:32.877Z)

Updated task #22 status

726. user (2026-07-09T08:37:33.434Z)

Updated task #23 status

727. user (2026-07-09T08:37:42.117Z)

```
=== /api/config HTTP status (run.app direct) ===
run.app /api/config -> HTTP 000
=== edge (api.khelsutra.guru) /api/config ===
edge /api/config -> HTTP 000
=== new revision boot logs (00024-k5n) ? startup + any error/OOM ===
Default STARTUP TCP probe succeeded after 1 attempt for container "rally-worker-1" on port 8080.
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO:     Application startup complete.
INFO:     Waiting for application startup.
08:36:38 [INFO] backend.config.loader: [CONFIG] deployment profile ACTIVE: cloud-serving
(compute_target=cloud_run)
```

728. assistant (2026-07-09T08:38:18.087Z)

CP boot is **clean and verified**: STARTUP TCP probe succeeded, `Uvicorn running`, `Application startup complete`, and `deployment profile ACTIVE: cloud-serving` ? no errors, no OOM on the 2 vCPU/8 Gi box with the new image. The `curl` HTTP 000 is a sandbox egress limit in git-bash (gcloud reaches `googleapis.com`, but arbitrary hosts like `run.app`/the CF edge are blocked here), not a CP fault ? Google's own TCP probe + "startup complete" already prove it's serving. Let me confirm via PowerShell (different egress path):

729. user (2026-07-09T08:38:25.076Z)

```
https://rally-control-plane-492532266377.asia-southeast1.run.app/api/config -> HTTP 200 (3957
bytes)
https://api.khelsutra.guru/api/config -> HTTP 200 (37467 bytes)
```

730. assistant (2026-07-09T08:39:09.822Z)

Both surfaces return HTTP 200 ? run.app (3957 bytes) and the CF edge `api.khelsutra.guru` (37467 bytes). The deploy is fully verified. Let me record state and prep the summary.

731. user (2026-07-09T08:39:10.464Z)

Updated task #23 status

732. user (2026-07-09T08:39:10.829Z)

Updated task #24 status

733. user (2026-07-09T08:39:12.505Z)

```
10      # Live cloud-serving deployment state (as of 2026-07-09 release)
11
12      **LIVE (2026-07-09):** control-plane rev **`rally-control-plane-00023-spm`** (was
00022-29w) + worker Job both on image
13      **`sha256:1838c07f6cb1182ce5bb25892739715956e4bb1093c1ea969ff89327bcaca3cb`** (built
from `origin/master @ 00e8f92`, Cloud Build `f93a0dd7`). Ships the 11-commit delta over 304af7b:
**#531** single-pass stitch, **#535** GCSFuse worker-mount, **#537** SD rendition, **#539** CP
```

acquire-gating, **#538** cancel propagation, **#529/#534** telemetry, **#548** GCSFuse OS-native path, **#547** gates reorg (scripts/docs).
14   ? **CP DOWNGRADED to 2 vCPU / 8 GiB** (from 00022's 4/16) ? justified by **#539 gating CP `acquire_local_input`** when validation+segment are on L4 (CP no longer RAM-downloads the full input), with val/seg/**compile(stitch)** all on the L4. Cloud Run needs ?4 vCPU for 16 GiB, so 2 vCPU ? 8 GiB (the pre-drift 00020 default). **Deploy method = image-only bump** (`gcloud run services/jobs update --image [+ --cpu/--memory for CP]`), so the baked cloud-serving config (phase_placement `validation/segment/compile=cloud_run_l4`, edge-auth+CF AUD, GCS roots) is PRESERVED ? only image (+CP resources) changed. Worker-first. Verified: CP boots clean (profile ACTIVE, uvicorn up), `GET /api/config` **200** on the edge + run.app.
15   **Rollback anchors (prior = 00022-29w / `sha256:7b62a8df?` @ 4vCPU/16Gi):** worker `gcloud run jobs update rally-worker --region asia-southeast1 --image ?/rally-worker@sha256:7b62a8df2dc6097aa228a695de73d4c0bc4fe8008a08a8024c02a5376cda6943`; CP `gcloud run services update rally-control-plane --region asia-southeast1 --image ?@sha256:7b62a8df? --cpu=4 --memory=16Gi` (or `services update-traffic --to-revisions rally-control-plane-00022-29w=100`).
16
17   **Cancel gap (found + filed #538):** canceling a run in the SPA does NOT cancel the dispatched L4 worker exec ? it runs to completion, burning GPU (repro: `rally-worker-8gqkv` on third-ksg, killed manually). #538 has a detailed validation matrix (detect+compile cancel, edge cases).
18   Carries #525 (compile?L4 + `phase_placement`), #527/#528 (#521 follow-ups), #530 (#524 trust submit-time video_id).
19   Region `asia-southeast1`, project `gen-lang-client-0306026826`. CP booted clean (cloud-serving profile ACTIVE, edge-auth ON).
20
21   **Wins ON via the cloud-serving preset** (no extra config): `phase_placement.compile=cloud_run_l4` + `cloud_stitch_encoder=h264_nvenc`,
22   `segment?cloud_run_l4` (via `compute_target=cloud_run`), `trust_submit_video_id` (default on). **`validation` deliberately stays on the
23   control-plane** (the authoritative #512/#513 gate) ? owner-confirmed 2026-07-08, NOT an oversight.

734. assistant (2026-07-09T08:40:06.296Z)

The memory is internally contradictory ? line 14 already says `validation/segment/compile=cloud_run_l4` (correct, matches the decoded config), while lines 22-23 claim validation stays on the CP. Line 14 was right. Let me update the live-deployment-state to the new revision and fix that contradiction (flagging the discrepancy for you).

735. user (2026-07-09T08:40:12.848Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\live-deployment-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

736. user (2026-07-09T08:40:17.878Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\live-deployment-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

737. user (2026-07-09T08:40:23.223Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\live-deployment-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

738. assistant (2026-07-09T08:40:36.211Z)

Live-deployment-state updated and the validation-placement contradiction resolved. Now the index + the pr550 "deployed" status ? reading them fresh to avoid another reformat mismatch.

739. user (2026-07-09T08:40:37.243Z)

```
1       # Memory index
2
3       - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-
handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip
golden gate.
4       - ? **LIVE deploy (2026-07-09):** [live-deployment-state.md](live-deployment-state.md) ?
CP rev **00023-spm** + worker on digest **1838c07f** (master **00e8f92**; ships
#531/#535/#537/#538/#539/#529/#534/#548/#547 over 304af7b). **CP DOWNGRADED 4/16 ? 2 vCPU/8
GiB** (safe via #539 acquire-gating; val/seg/compile all on L4). Image-only bump preserves the
baked cloud-serving/phase_placement config. Verified 200 on /api/config. Rollback ? digest
**7b62a8df** @ 4/16 or rev **00022-29w**. Release-process gaps ? **#532**. Runlog: **PR #549
MERGED 2026-07-09** (squash `6cfcb37c`) = `DEPLOY_REPORT_cloud-serving_cp-2vcpu-
downgrade_2026-07-09.md` ? the 7.84 GiB GX010155 CUJ + cross-run E2E table (bigger clip ~20m10s
vs 5.81 GiB ~21m34s).
5       - ? **MERGED PR #550 (2026-07-09, squash `01d1081`):**
[pr550-worker-l4-observability.md](pr550-worker-l4-observability.md) ? makes the cloud-serving
L4 worker infer path observable (validation PASS logging + verdicts in report.json, per-stage
`timings`, and the #534 heartbeat FINALLY firing: `wasb_infer` logs INFO?stdout + runner merges
stderr?stdout). **NOT yet deployed** (rebuild+redeploy worker/CP image to go live; live worker
still on `1838c07f`=`00e8f92`). **Owner review addressed (2 passes):** `e54e4a1` = streaming-
path `timeout_sec` watchdog (the real BUG ? production path had no kill) +
`len(validator_registry)` + de-duped FAIL logs + `timings["fetch_s"]` + `validation_skipped`;
`f7f0c82` = closed the watchdog race so a clean rc=0 at the deadline can't be misreported as a
timeout. Local gate green (1968 pass, 84.47%, ruff/mypy clean); both passes COMMENT-only.
**Live-L4 heartbeat still needs an owner CUJ.**
6       - ? **CLOSED 2026-07-08:** [followup-validation-latency-live-cuj.md](followup-
validation-latency-live-cuj.md) ? live SPA CUJ filled both CF-gated cells on rev **00019**
(validation wall **183 s** PASS + reel compiled; fresh clip, corroborates not re-measures
Video-1). Docs **PR #522**. Surfaced stitch bottleneck ? issues **#521** (compile?L4 + config
placement) & **#520** (debug tracing).
```

740. user (2026-07-09T08:40:38.309Z)

```
1       ---
2       name: pr550-worker-l4-observability
3       description: "PR #550 MERGED (squash 01d1081) ? cloud-serving L4 worker infer
observability; NOT yet deployed; live-L4 heartbeat still needs an owner CUJ"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: e663947f-2127-4383-9905-650340ff9863
8       ---
9
10      # PR #550 ? cloud-serving L4 infer observability (MERGED 2026-07-09, squash `01d1081`)
11
12      **MERGED to master `01d1081` 2026-07-09** (squash of b3dff78+e54e4a1+f7f0c82; all 11 CI
checks green;
13      both owner review passes were COMMENT-only, resolved). Remote branch auto-deleted. **NOT
yet deployed**
14      ? the worker/CP image must be rebuilt+redeployed for this observability to go live (the
live worker is
15      still on digest `1838c07f` = master `00e8f92`, pre-#550; see [[live-deployment-
state.md]]).
16
```

741. user (2026-07-09T08:41:10.448Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

742. user (2026-07-09T08:41:12.534Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
```

indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

743. user (2026-07-09T08:41:15.423Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

744. user (2026-07-09T08:41:18.795Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

745. assistant (2026-07-09T08:41:48.386Z)

Deploy complete and all state recorded. Marking done and handing you the Release Summary.

746. user (2026-07-09T08:41:48.533Z)

Updated task #24 status

747. assistant (2026-07-09T08:42:13.261Z)

? Release Summary ? cloud-serving cut `01d1081` (2026-07-09)

Deployed & verified

	value
Image	`sha256:261ccf3a?` (Cloud Build `5b0b41b9`, 11m32s, from `origin/master @ 01d1081`)
Worker Job	`rally-worker` ? `261ccf3a` ?
Control-plane	rev **`rally-control-plane-00024-k5n`**, 100% traffic, **2 vCPU / 8 Gi** preserved ?
Health	boot clean (`profile ACTIVE: cloud-serving`, uvicorn up, no OOM); `/api/config` **200** on both run.app + `api.khelsutra.guru`
Rollback	worker+CP ? digest `1838c07f` @ rev `00023-spm` (both already 2/8)

Deployed **worker-first**, then CP (image-only bump ? baked `phase_placement`/edge-auth config untouched).

What's in this cut

Only **one code PR** landed since the live image (`00e8f92`): **#550** (worker L4 infer observability). #549 was docs-only (the runlog). Everything else (#531/#535/#537/#538/#539/#529/#534/#548/#547) was already live ? this deploy carries it forward unchanged.

Enablement audit ? nothing ships silently disabled

Feature	How it's enabled	On live now?
#550 per-stage `timings` in report.json	Unconditional in `cmd_infer`	? default
#550 validation PASS logging + verdicts?report.json	Unconditional; exercised because **the L4 worker IS the validation gate**	? default
#550 #534 native-L4 heartbeat (progress + telemetry)	Unconditional; the fix that revives it. **Was dead** on the old image (116-byte telemetry) ? *this deploy turns it on*	? default
segment ? L4	`compute_target=cloud_run`	?
compile ? L4 + **h264_nvenc**	`phase_placement.compile=cloud_run_l4`; `cloud_stitch_encoder` defaults `h264_nvenc`; #529 signals any silent libx264 fallback	? default
validation ? L4	baked `phase_placement.validation=cloud_run_l4`	?
`trust_submit_video_id` (#524)	default on	?

Every #550 feature is wired **unconditionally** (no gating flag), so nothing is held back. The cloud-serving profile is **explicitly** set (`DEPLOYMENT_PROFILE=cloud-serving` env + baked `config.json`), so the whole bundle genuinely applies rather than relying on suggest-only auto-detect.

?? One thing for you to confirm (not a blocker)

The live baked config runs **validation, segment, AND compile all on the L4** ? the CP is a pure thin dispatcher (that's *why* 2 vCPU/8 Gi survives a 7.84 GiB clip: #539 skips the download). But a memory note claimed validation was **"owner-confirmed on the control-plane, 2026-07-08."** The running config contradicts that ? validation is on the L4 today. If that move was intentional, great (it's what makes the tiny CP viable). If you meant validation to gate cheaply on the CP *before* burning a GPU dispatch, that's a one-knob change (`phase_placement.validation`) ? say the word. I've flagged the discrepancy in memory either way.

What to watch in your CUJ (this is the payoff of #550)

On the new cut the **worker log** should now show what was invisible before:
- `[worker] validating (N checks)?` ? `[worker] validation PASSED in Xs ? blur_check=ok relevance=ok ?`
- `[worker] progress: ? detector N/M (pct%)` heartbeat + a populated `run_telemetry.jsonl` (not a bare header)
- per-stage `timings` block in `report.json`

You're clear to run the CUJ. I'll harvest the logs afterward to confirm the live-L4 heartbeat + validation logging actually fire (the one thing offline tests couldn't prove) and drop it in a runlog.